

full_project_pt2

March 31, 2024

1 Final project pt 2 jupyter notebook

Group members: Michael Casey, Luke Denoncourt, Durgesh Tiwari Team Lead: Luke Denoncourt

The following sections details contributions of team members. Each name denotes contributing the entirety of that work

Project part	Team members contributed
1. Time series decomposition	Michael Casey
2. Time series visualization	Michael Casey
3. Description of time series	Michael Casey
4. TS models - LSTM	Luke Denoncourt
4. TS models - regression	Luke Denoncourt
4. TS models - ARIMA	Durgesh Tiwari
5. Predictions on models developed	Luke Denoncourt and Durgesh Tiwari
5. Predictions comparing prediction results	Luke Denoncourt
6. Collate jupyter notebooks to final output	Luke Denoncourt

```
[9]: from random import gauss
from random import seed
import random
import numpy as np
import pandas as pd
from pandas.plotting import autocorrelation_plot
from matplotlib import pyplot as plt
import seaborn as sn
import datetime as dt
import IPython.display
import matplotlib as mpl
import IPython

import tensorflow as tf
import keras
from keras.models import Sequential
from keras.layers import LSTM, Dense

from sklearn.linear_model import ElasticNet
```

```

from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
import statsmodels.tsa.stattools as smt

```

- 2 1. Time Series decomposition, 2. Time series visualization, and
3. Description of time series

3 MICHAEL CASEY CONTRIBUTION

Organized as parts 1 through 3 for each stock in the magnificent 7

4 Stock Price and Sales Analysis of Magnificent 7 Stocks

4.1 The seven stocks in the Mag7 are:

GOOG

TSLA

AMZN

AAPL

MSFT

META

NVDA

4.1.1 Analysis of GOOG Stock and Fundamentals over last 10 years

```

[40]: #goog = pd.read_csv('GOOGL.csv', header=0, index_col=0,
    ↪ parse_dates=True, squeeze=True)
goog = pd.read_csv('./Aux_Data/GOOGL.csv')

```

```

[41]: goog.columns

```

```

[41]: Index(['Date', 'Sales', 'FreeCashFlow', 'Cash', 'Long Term Debt', 'Price'],
dtype='object')

```

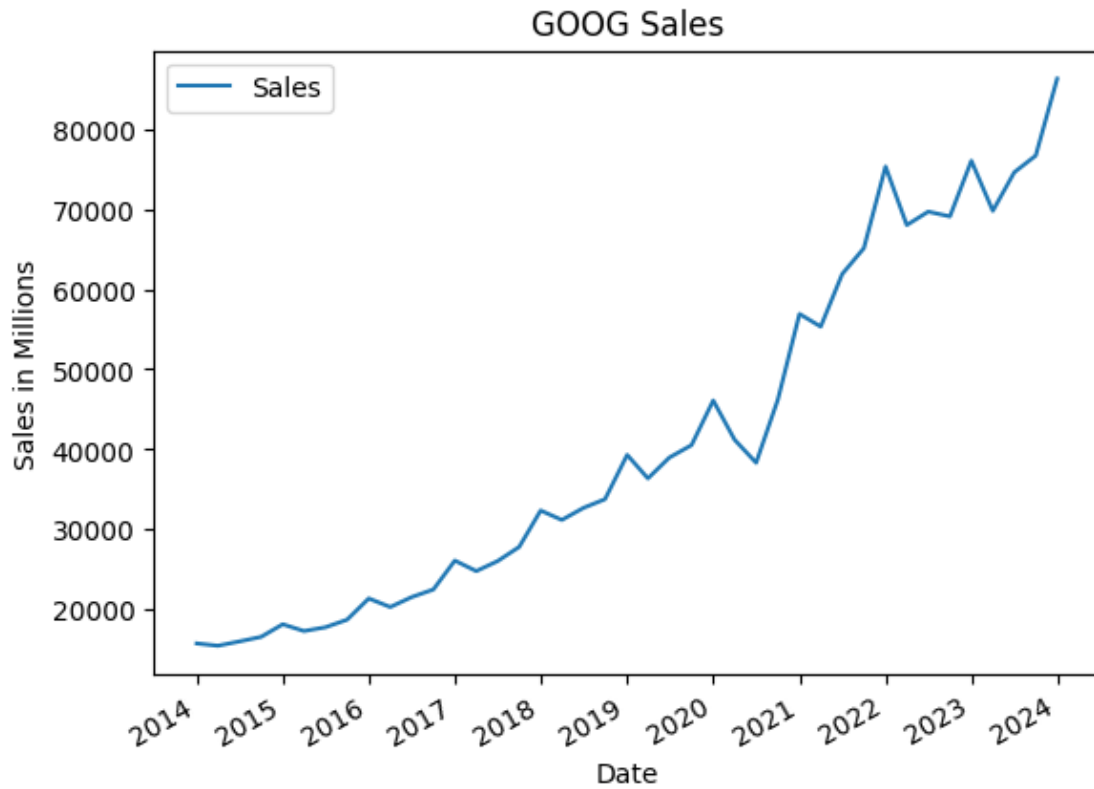
```

[42]: goog['Date'] = pd.to_datetime(goog['Date'], format='%m/%d/%Y')
    goog['Year'] = goog['Date'].dt.year

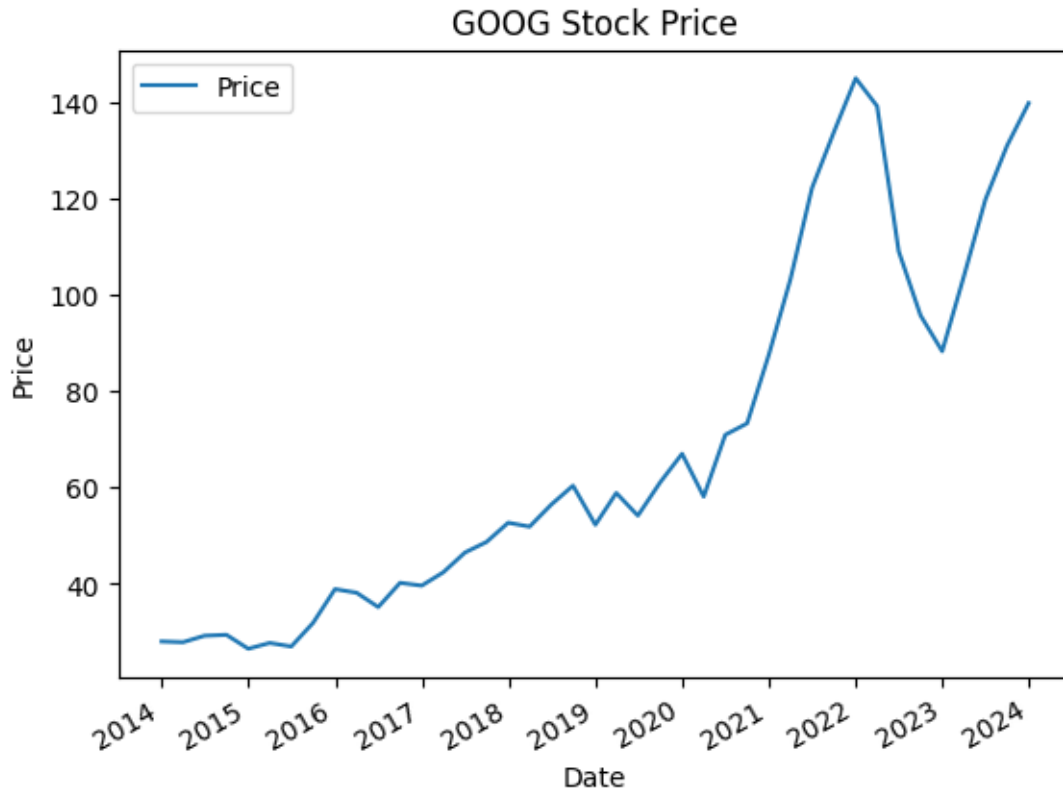
```

```
goog['Quarter'] = goog['Date'].dt.quarter
```

```
[43]: goog.plot('Date', 'Sales')  
plt.title('GOOG Sales')  
plt.ylabel('Sales in Millions')  
plt.show()
```

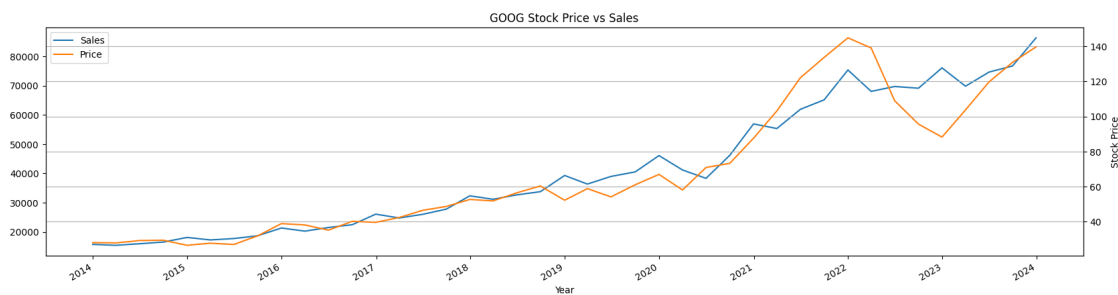


```
[44]: goog.plot('Date', 'Price')  
plt.title('GOOG Stock Price')  
plt.ylabel('Price')  
plt.show()
```



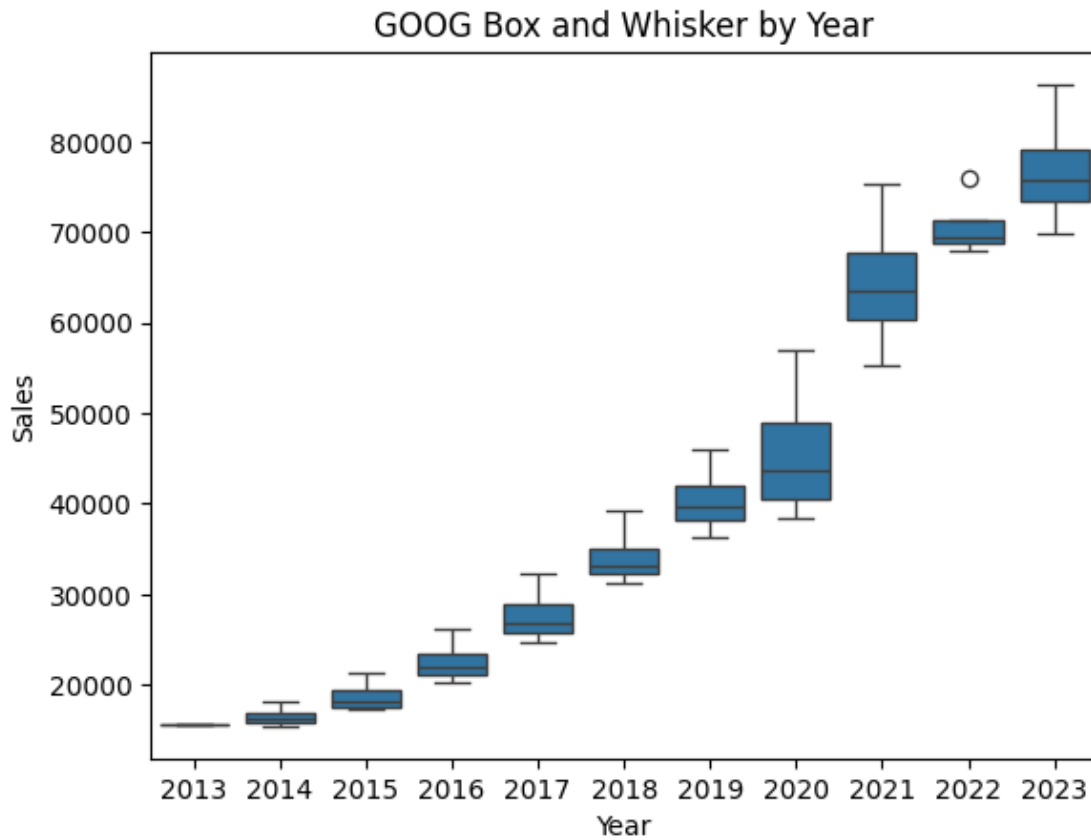
```
[45]: goog.loc[:,['Date','Sales','Price']].plot(x = 'Date',secondary_y=['Price'],
        ↳ xlabel = 'Year', ylabel = 'Stock Price',sharex = True,mark_right=False,
        ↳ figsize = (20,5), grid=True)
plt.title('GOOG Stock Price vs Sales')
```

[45]: Text(0.5, 1.0, 'GOOG Stock Price vs Sales')



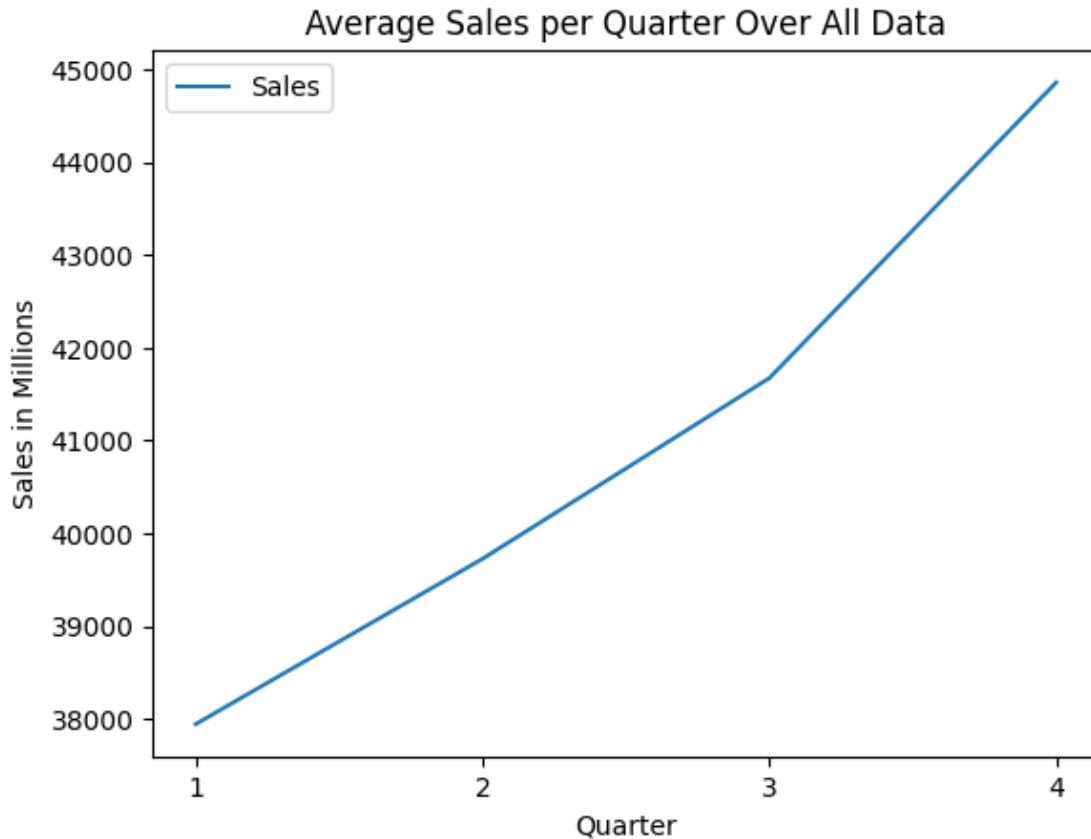
```
[46]: sn.boxplot(x='Year',y='Sales', data = goog).set(title = 'GOOG Box and Whisker_
        ↳ by Year')
```

```
[46]: [Text(0.5, 1.0, 'GOOG Box and Whisker by Year')]
```



```
[47]: avgperqtr = goog[['Quarter', 'Sales']].groupby('Quarter').mean().reset_index()
```

```
[48]: avgperqtr.plot('Quarter', 'Sales')
plt.ylabel('Sales in Millions')
plt.title('Average Sales per Quarter Over All Data')
plt.xticks(goog.Quarter.unique())
plt.show()
```



```
[49]: goog1 = pd.DataFrame(goog, columns=['Year', 'Quarter', 'Sales'])
```

```
[50]: goog2 = goog1[goog1['Year'] > 2013]
```

```
[51]: dictyear = {}
saleslist = []
for year in goog1['Year'].unique():
    #print(year)
    #byyear = goog1[goog1['Year'] == year]
    #saleslist = saleslist.append(byyear.Sales)
    dictyear[year] = list(goog1[goog1['Year'] == year].Sales)
```

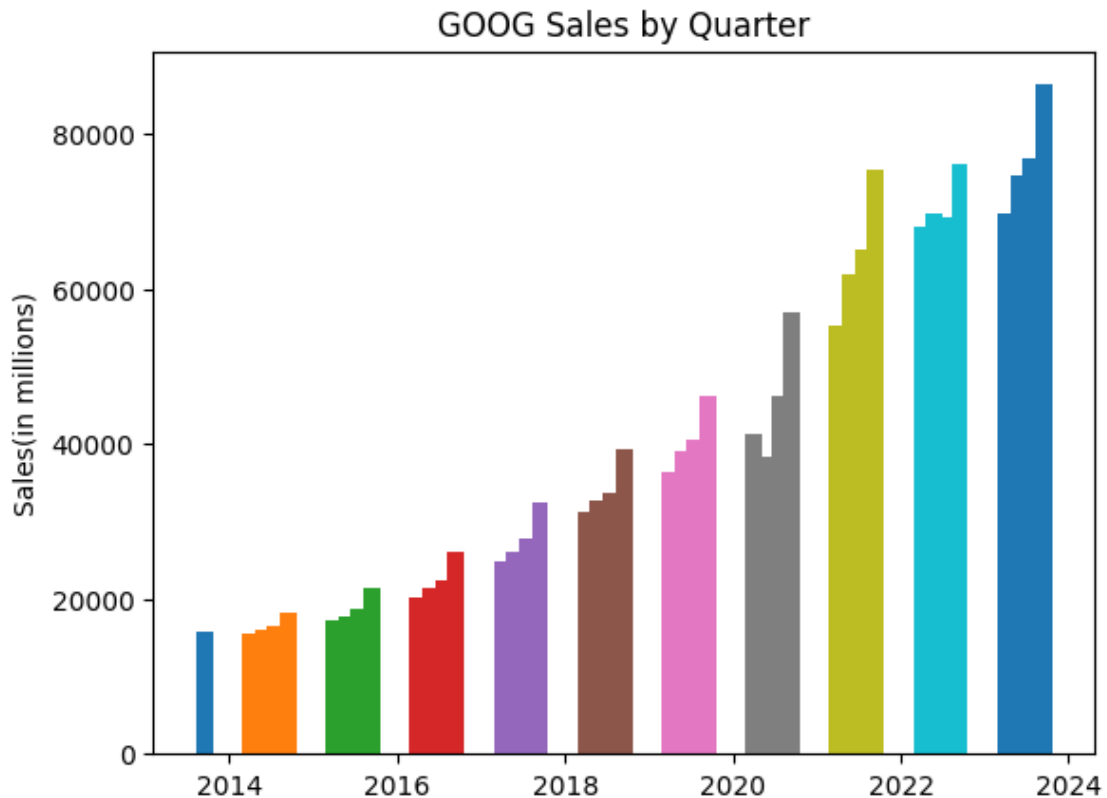
```
[52]: #adopted from https://medium.com/@BetterEverything/
↳graph-quarterly-data-with-python-6f38bcc6b7fe
for year, data in dictyear.items():
    quarters = list(goog1.loc[(goog1['Year'] == year)].Quarter)
    x_coordinates = [year + 0.15 * q for q in quarters]
```

```

    #print(x_coordinates, data)
    plt.bar(x_coordinates,data,width=0.2,align='edge')
plt.title('GOOG Sales by Quarter')
plt.ylabel('Sales(in millions)')

#plt.xticks(labels=[np.unique(goog1['Year'].values)], ticks = [])
#print(plt.xticks())
plt.show()

```

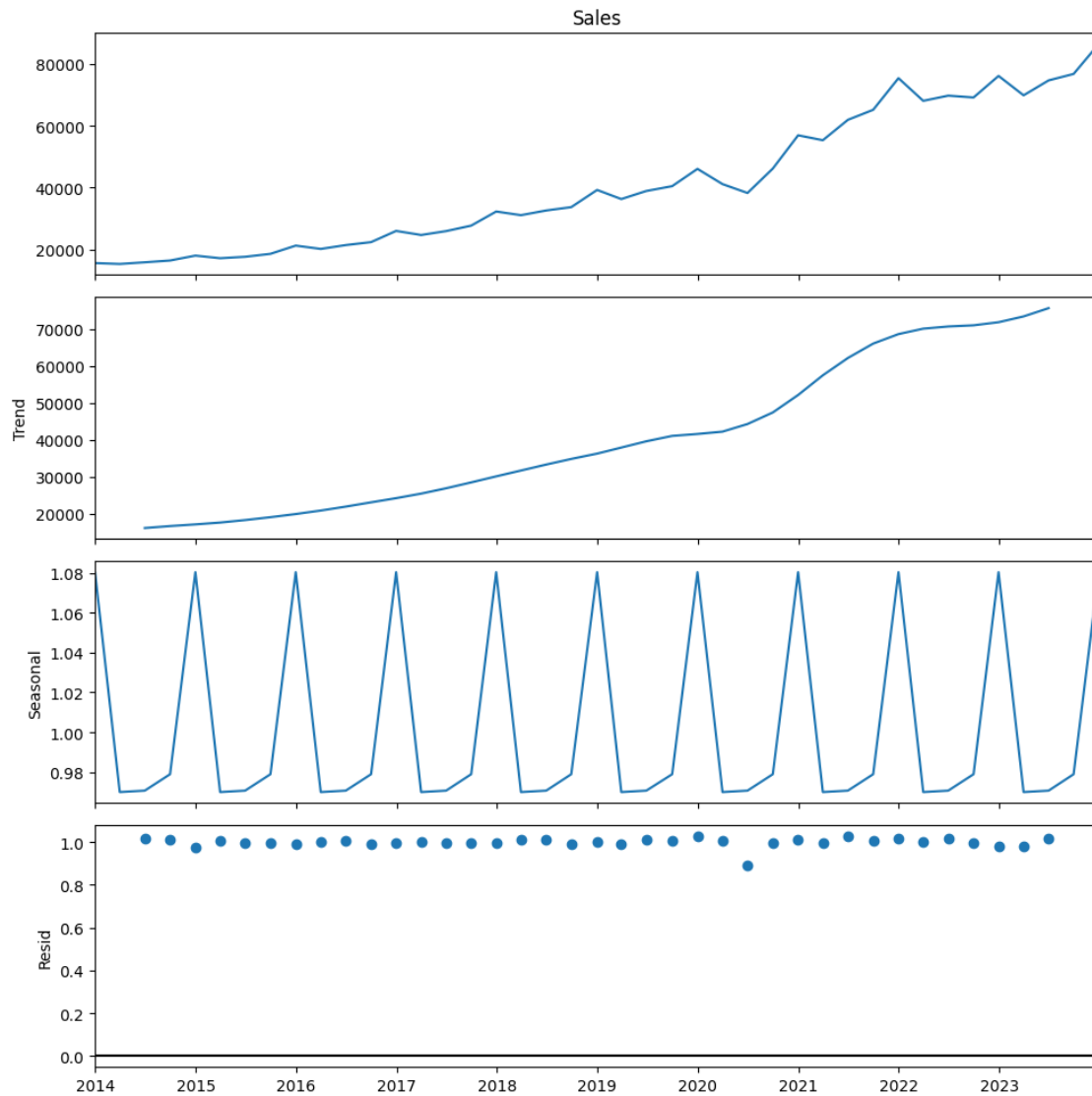


```
[53]: goog.set_index('Date', inplace=True)
```

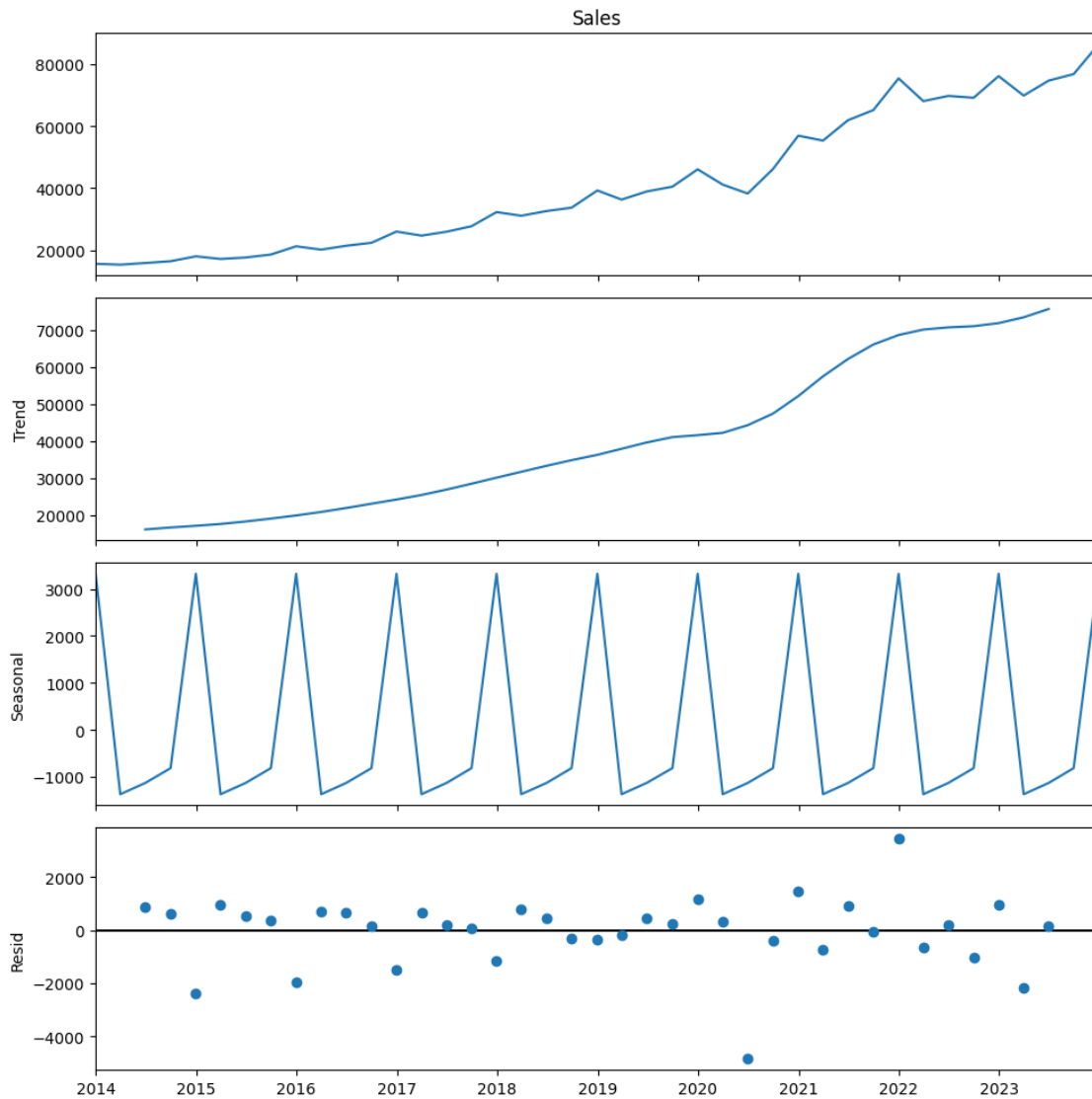
```

[54]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
    ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Sales'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()

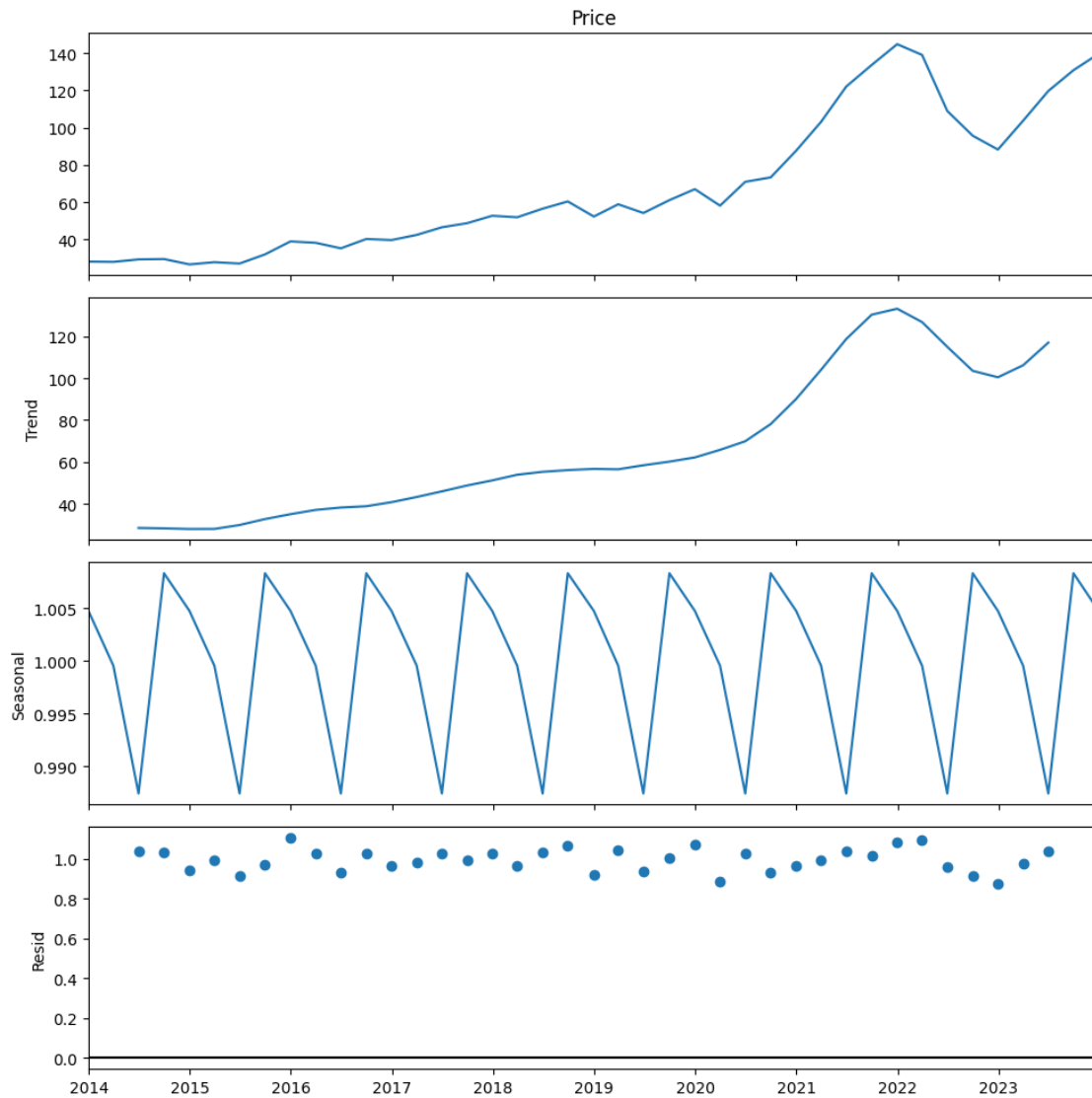
```



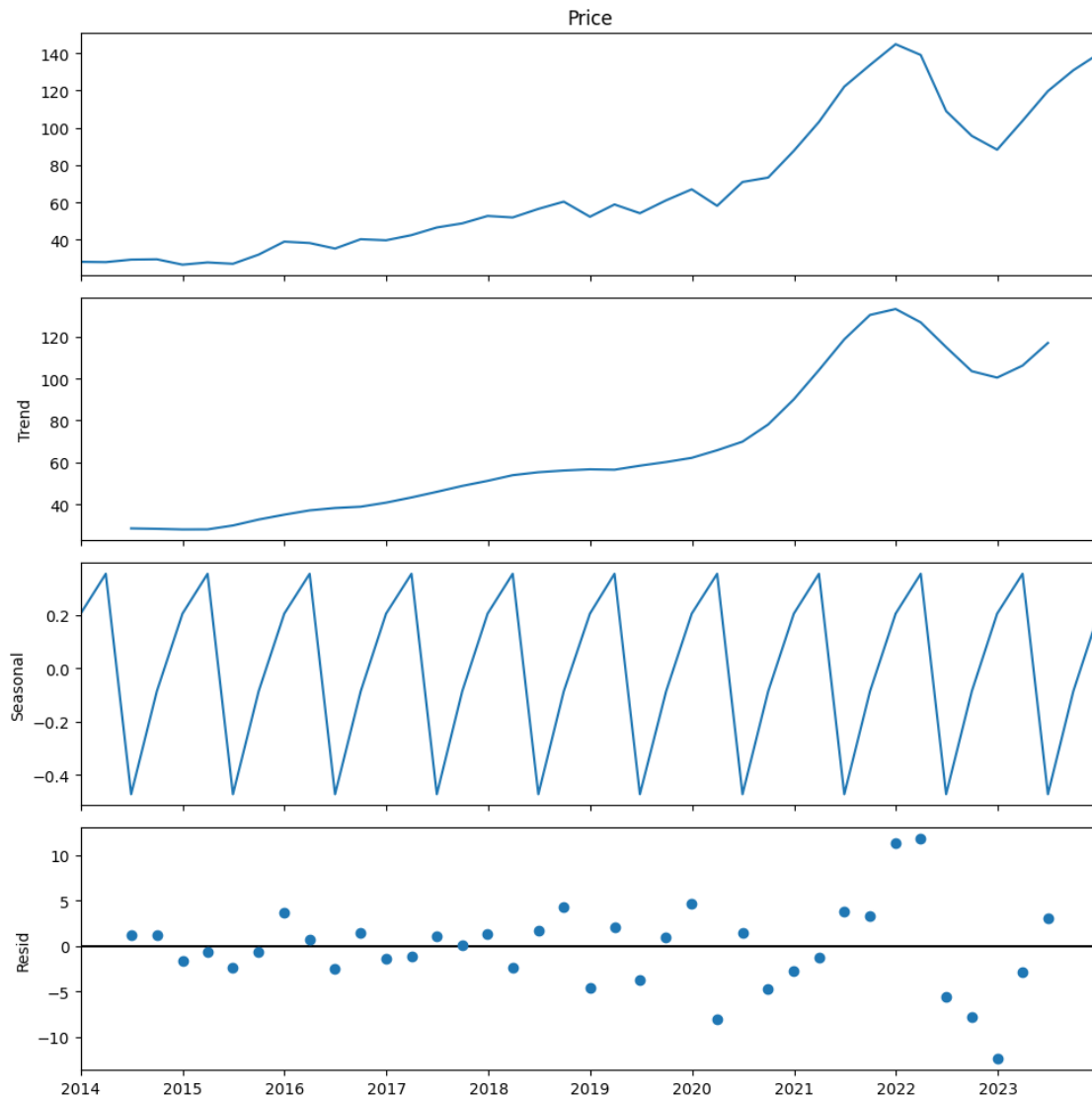
```
[55]: result = seasonal_decompose(goog['Sales'],model='additive',period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
#plt.title('Time Series Decomposition of Sales')
plt.show()
```

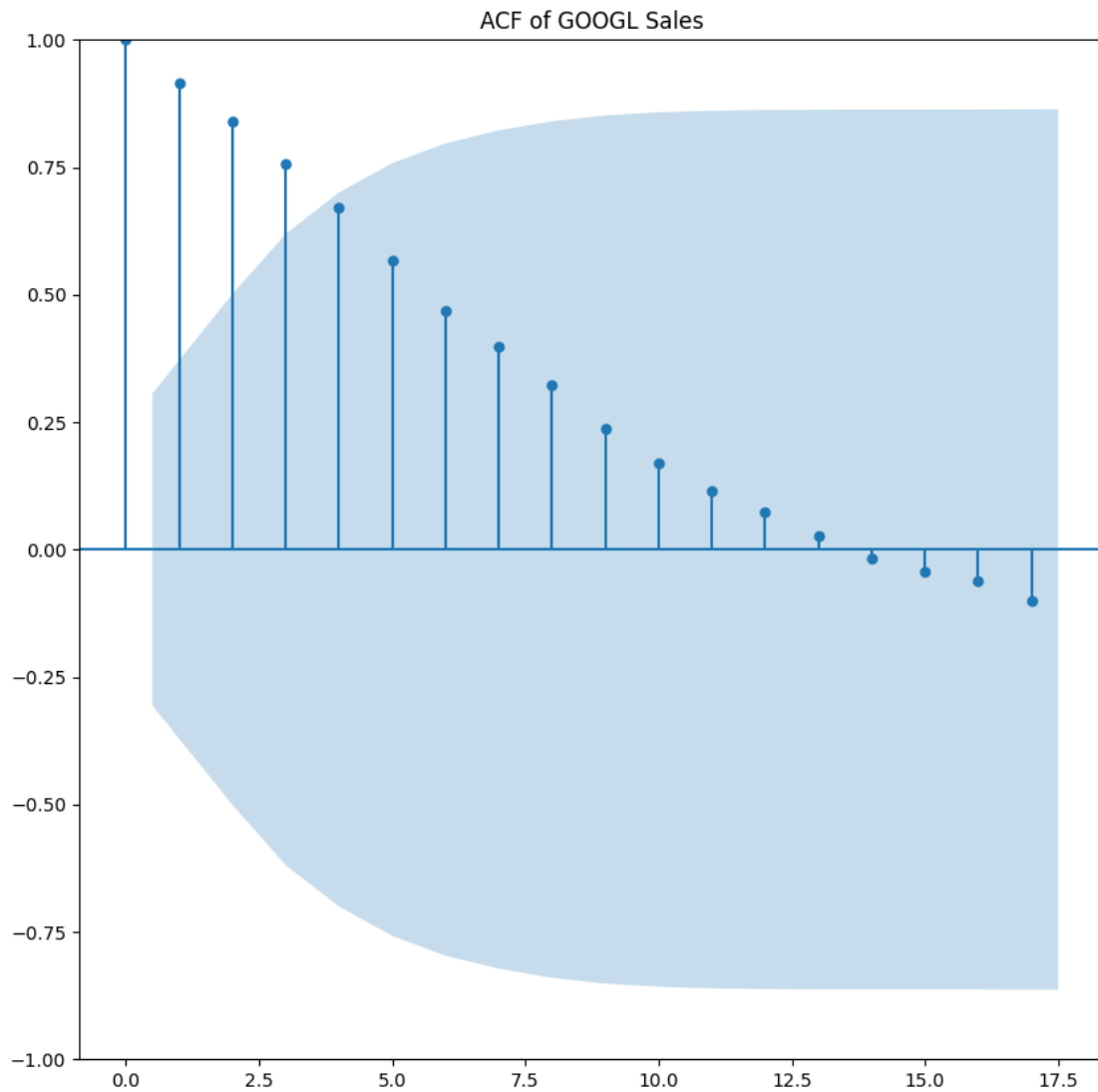
```
[56]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
      ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Price'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



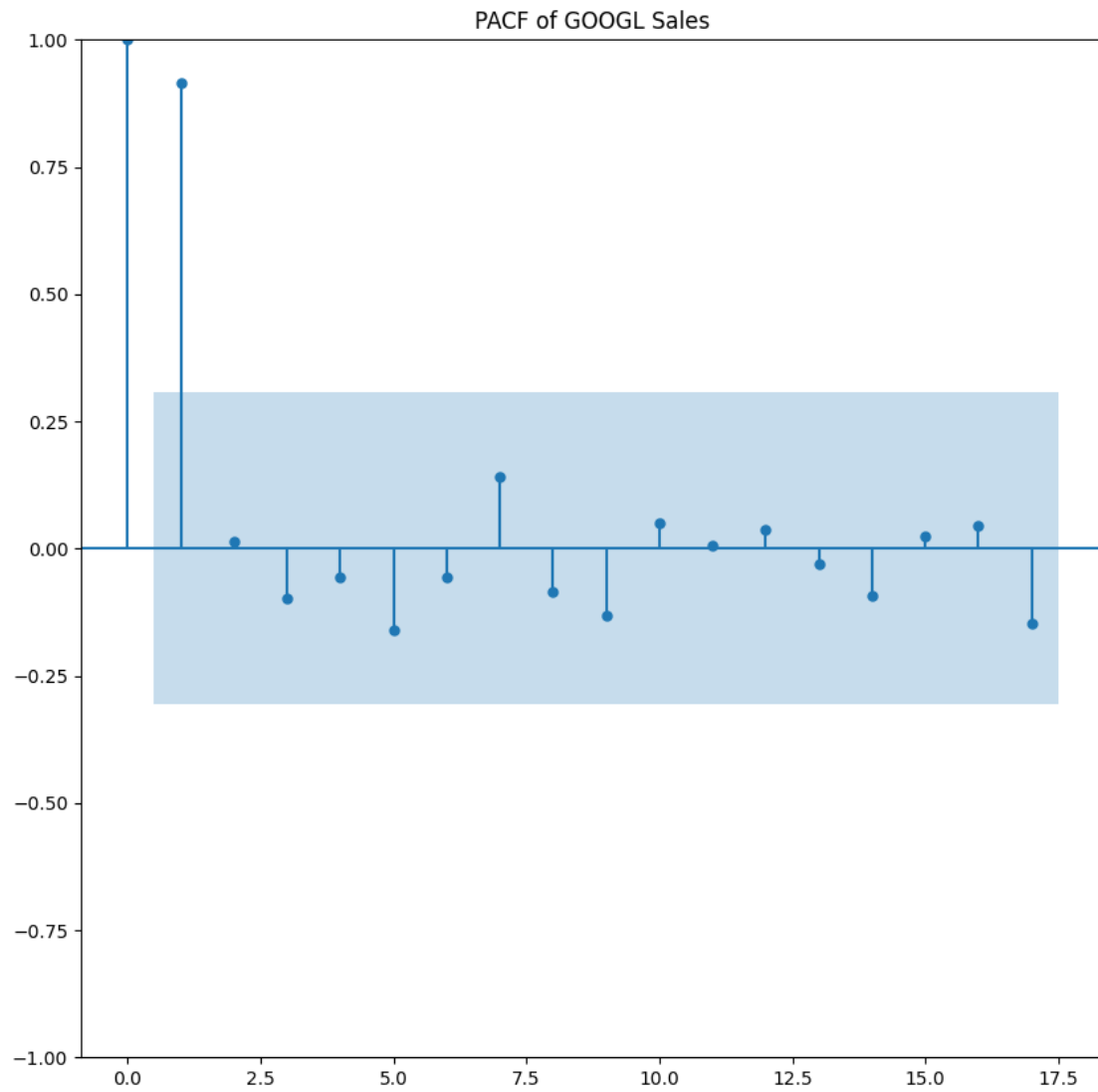
```
[57]: result = seasonal_decompose(goog['Price'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



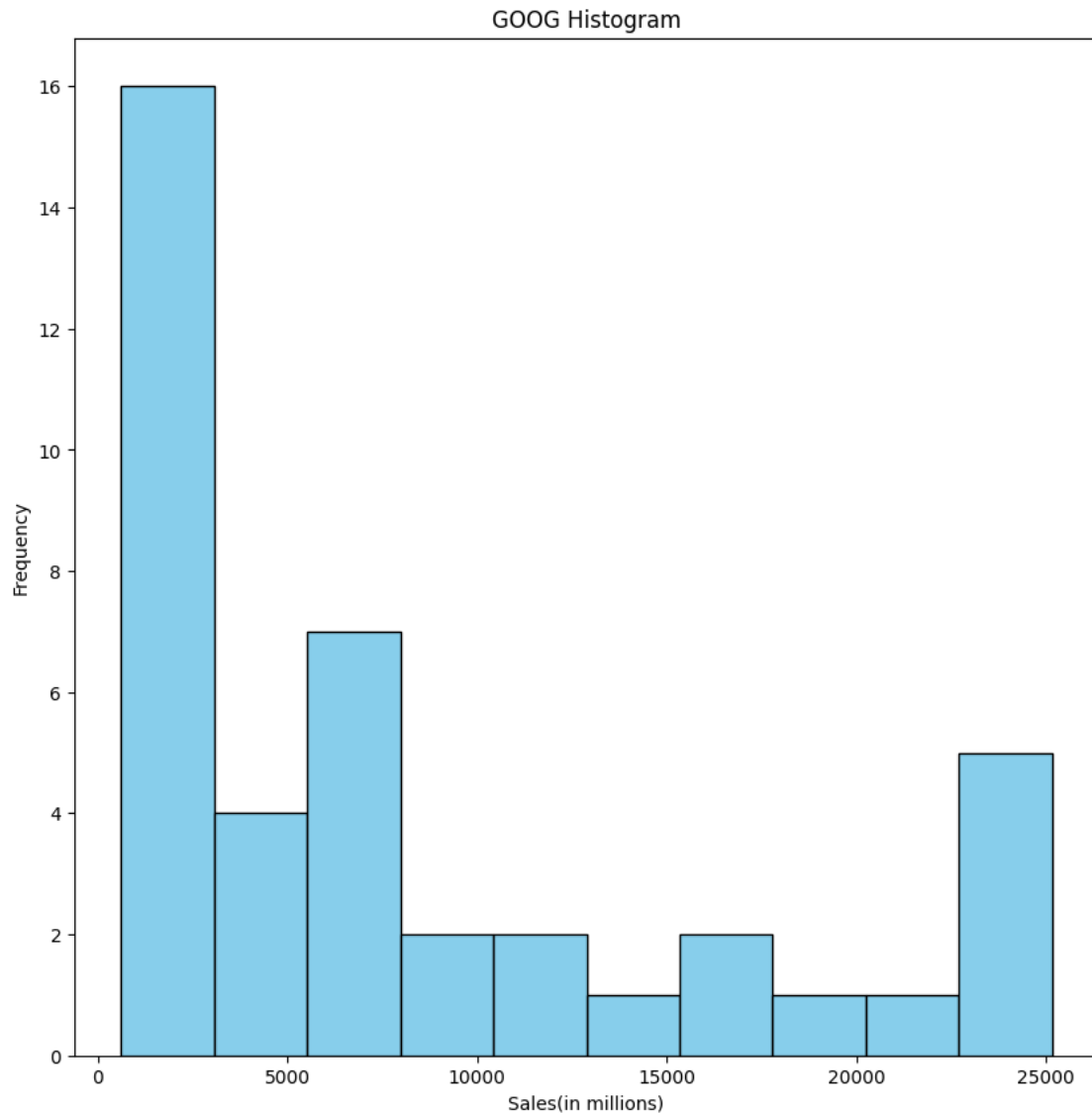
```
[91]: #https://machinelearningmastery.com/
      ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_acf(goog['Sales'], title='ACF of GOOGL Sales')
pyplot.show()
```



```
[92]: #https://machinelearningmastery.com/  
      ↪gentle-introduction-autocorrelation-partial-autocorrelation/  
from matplotlib import pyplot  
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf  
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)  
plot_pacf(goog['Sales'], title='PACF of GOOGL Sales')  
pyplot.show()
```



```
[94]: plt.hist(goog['Sales'], color='skyblue', edgecolor='black')
plt.xlabel('Sales(in millions)')
plt.ylabel('Frequency')
plt.title('GOOG Histogram')
pyplot.show()
```



```
[95]: # statistical test
result = adfuller(goog['Sales'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 0.939400

p-value: 0.993584

Critical Values:

1%: -3.646

5%: -2.954

10%: -2.616

```
[96]: # statistical test
result = adfuller(goog['Price'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: -1.382937

p-value: 0.590428

Critical Values:

1%: -3.670

5%: -2.964

10%: -2.621

4.2 GOOG Conclusion

```
[97]: #The null hypothesis of the ADF test is the time series is Non Stationary. We
      ↪conclude GOOG is Non Stationary since the test
      #statistic is larger than the critical values.
```

Description of Time series (is it stationary, are there any trends, seasonality etc)

For GOOGL, you can see there is clearly a trend higher in both price and sales over the last 10 years. There does appear to be seasonality mainly in the 4th quarter of the year. The additive model seems to be more appropriate for the Sales and Stock price data as the residuals in each case appear like white noise in the additive model but have more of a pattern in the multiplicative model. This is shown in both time series decomposition charts.

The null hypothesis of the ADF test is the time series is Non Stationary. We conclude GOOG is Non Stationary both when looking at Sales and Price since the test statistic is larger than the critical values.

GOOGL does show autocorrelation out to about 5 lags.

```
[ ]:
```

4.2.1 Analysis of TSLA Stock and Fundamentals over last 10 years

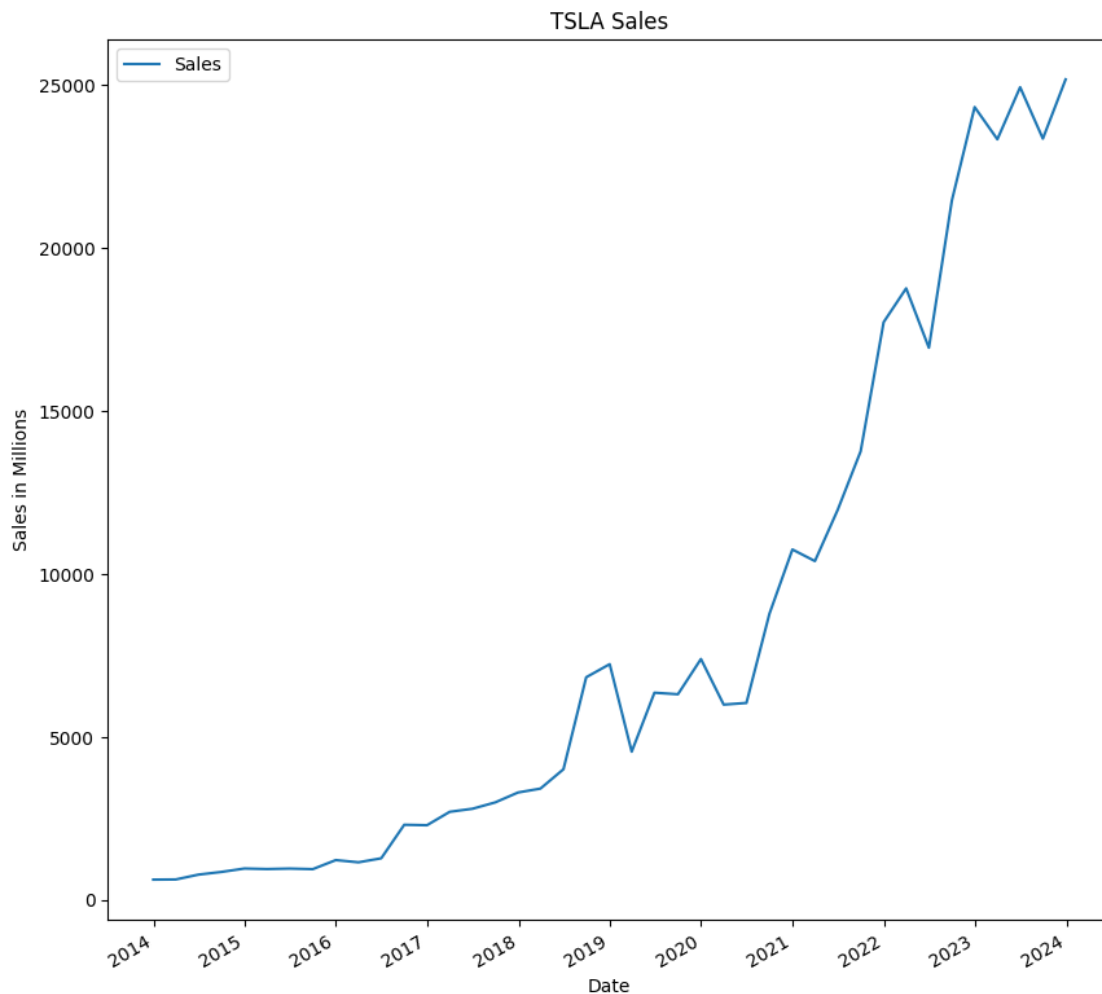
```
[98]: #goog = pd.read_csv('GOOGL.csv', header=0, index_col=0,
      ↪parse_dates=True, squeeze=True)
goog = pd.read_csv('./Aux_Data/TSLA.csv')
```

```
[99]: goog.columns
```

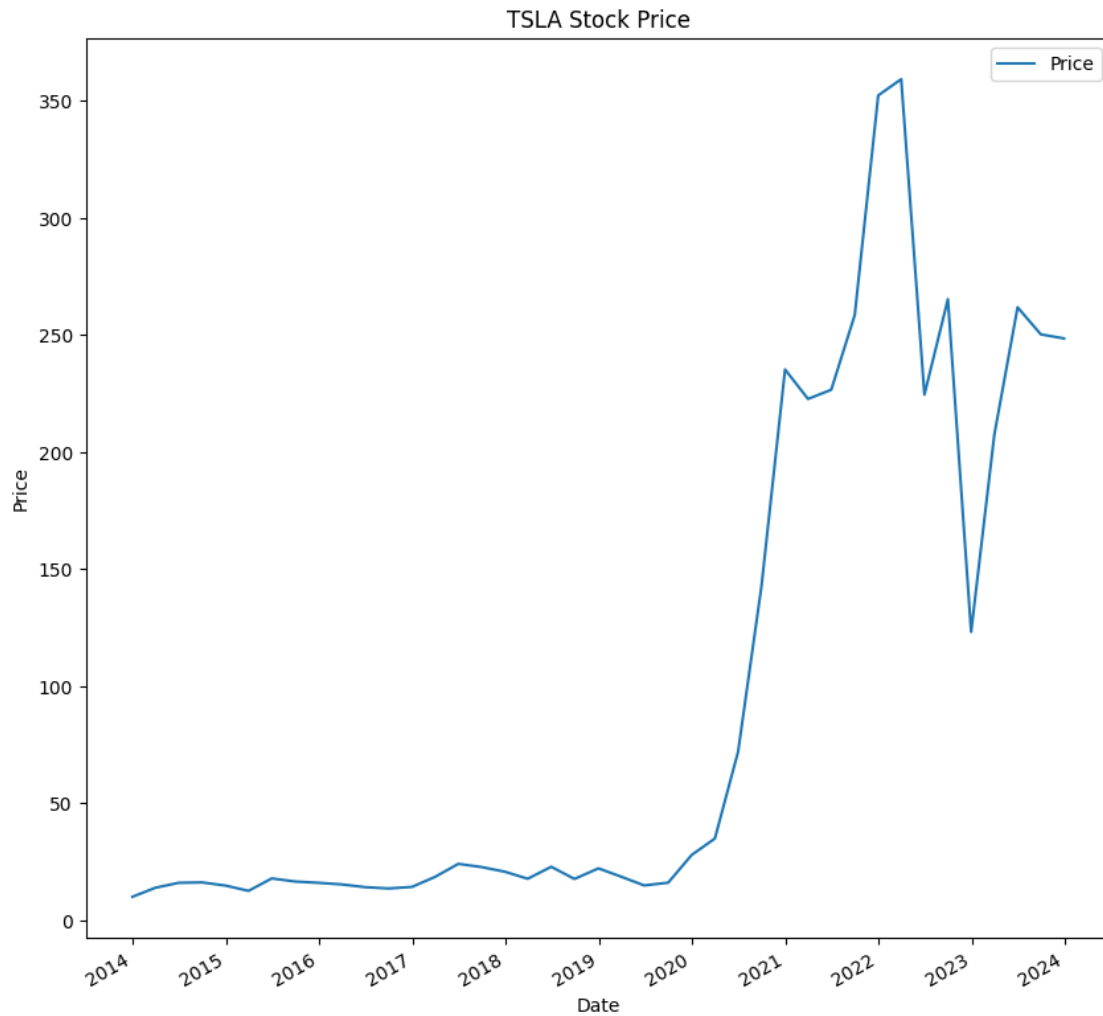
```
[99]: Index(['Date', 'Sales', 'FreeCashFlow', 'Cash', 'Long Term Debt', 'Price'],
      dtype='object')
```

```
[100]: goog['Date'] = pd.to_datetime(goog['Date'], format='%m/%d/%Y')
goog['Year'] = goog['Date'].dt.year
goog['Quarter'] = goog['Date'].dt.quarter
```

```
[101]: goog.plot('Date', 'Sales')
plt.title('TSLA Sales')
plt.ylabel('Sales in Millions')
plt.show()
```

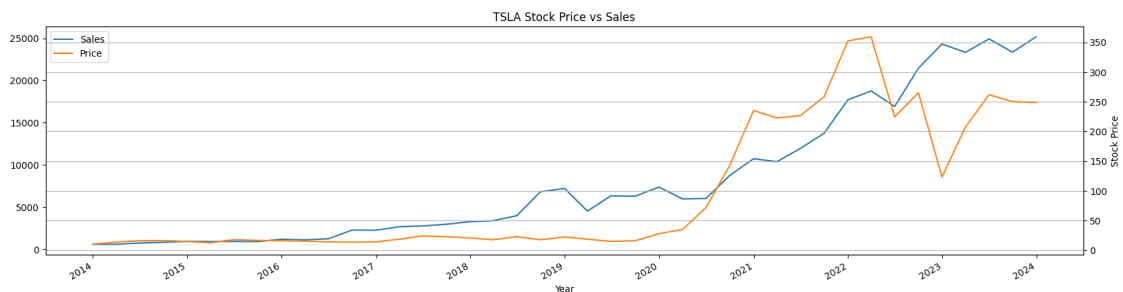


```
[102]: goog.plot('Date', 'Price')
plt.title('TSLA Stock Price')
plt.ylabel('Price')
plt.show()
```

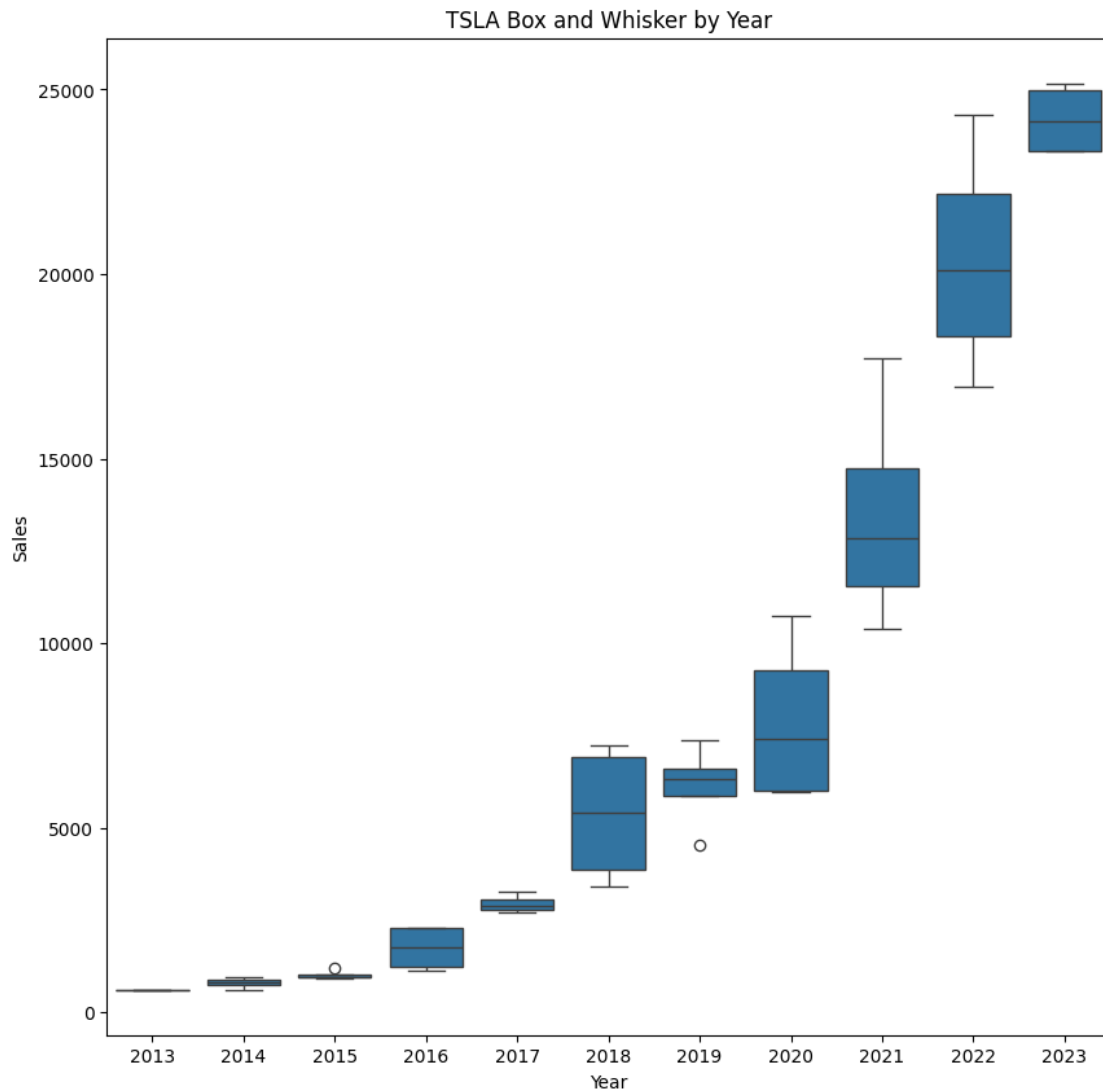
```
[103]: goog.loc[:,['Date','Sales','Price']].plot(x = 'Date',secondary_y=['Price'],
↪xlabel = 'Year', ylabel = 'Stock Price',sharex = True,mark_right=False,
↪figsize = (20,5), grid=True)
plt.title('TSLA Stock Price vs Sales')
```

[103]: Text(0.5, 1.0, 'TSLA Stock Price vs Sales')



```
[104]: sn.boxplot(x='Year',y='Sales', data = goog).set(title = 'TSLA Box and Whisker_
↳by Year')
```

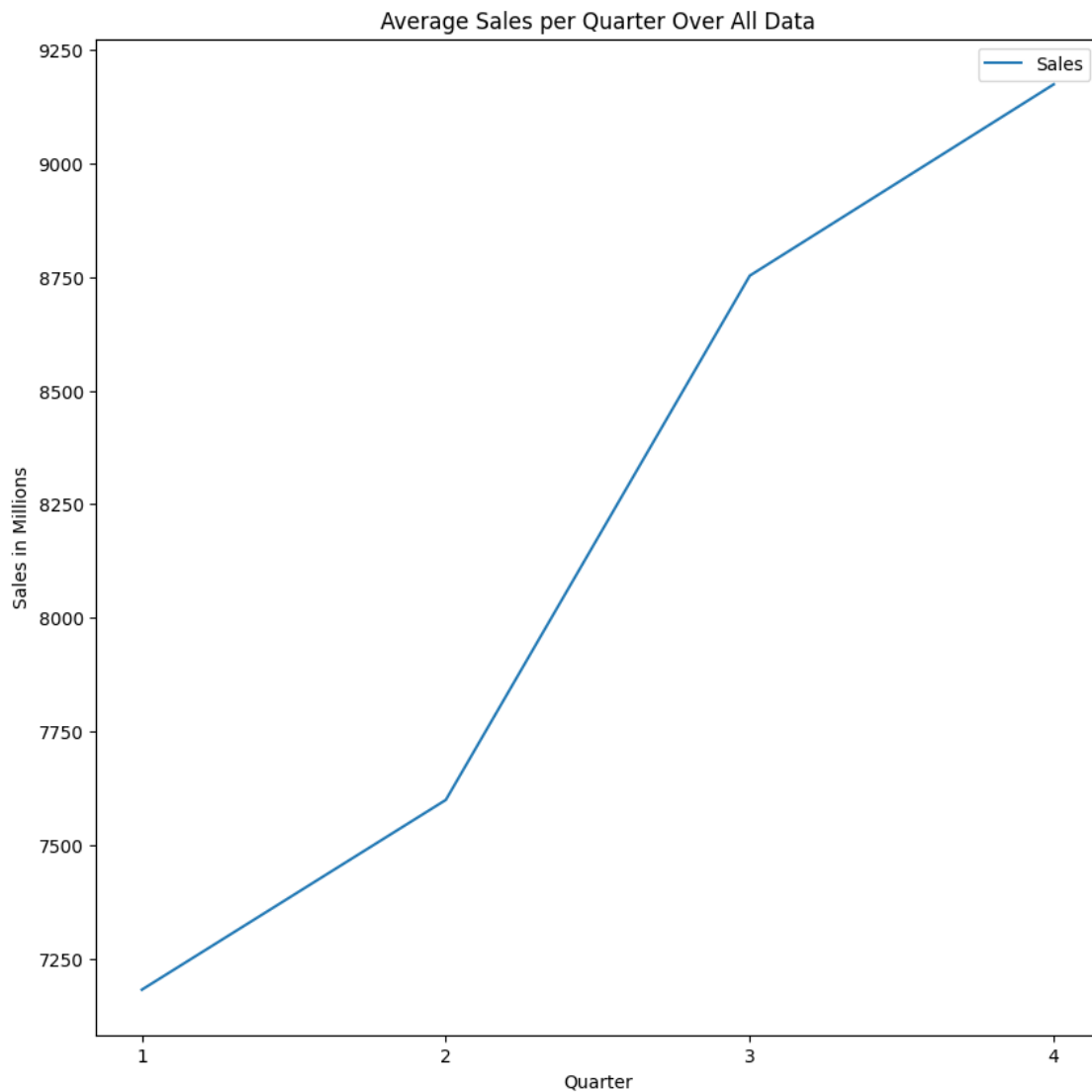
```
[104]: [Text(0.5, 1.0, 'TSLA Box and Whisker by Year')]
```



```
[105]: avgperqtr = goog[['Quarter', 'Sales']].groupby('Quarter').mean().reset_index()
```

```
[106]: avgperqtr.plot('Quarter','Sales')
plt.ylabel('Sales in Millions')
plt.title('Average Sales per Quarter Over All Data')
plt.xticks(goog.Quarter.unique())
```

```
plt.show()
```

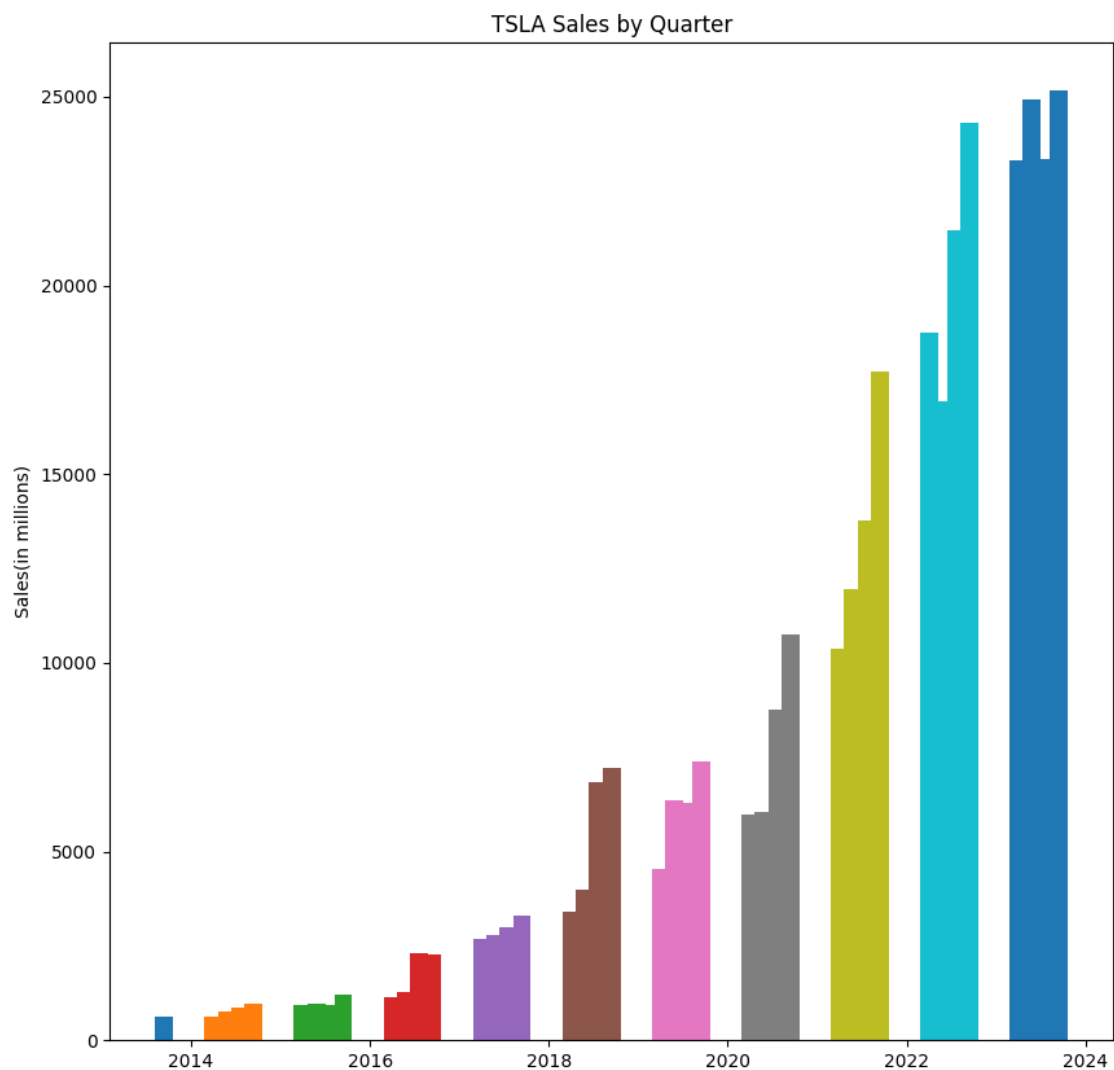


```
[107]: goog1 = pd.DataFrame(goog, columns=['Year', 'Quarter', 'Sales'])
```

```
[108]: dictyear = {}  
saleslist = []  
for year in goog1['Year'].unique():  
    #print(year)  
    #byyear = goog1[goog1['Year'] == year]  
    #saleslist = saleslist.append(byyear.Sales)  
    dictyear[year] = list(goog1[goog1['Year'] == year].Sales)
```

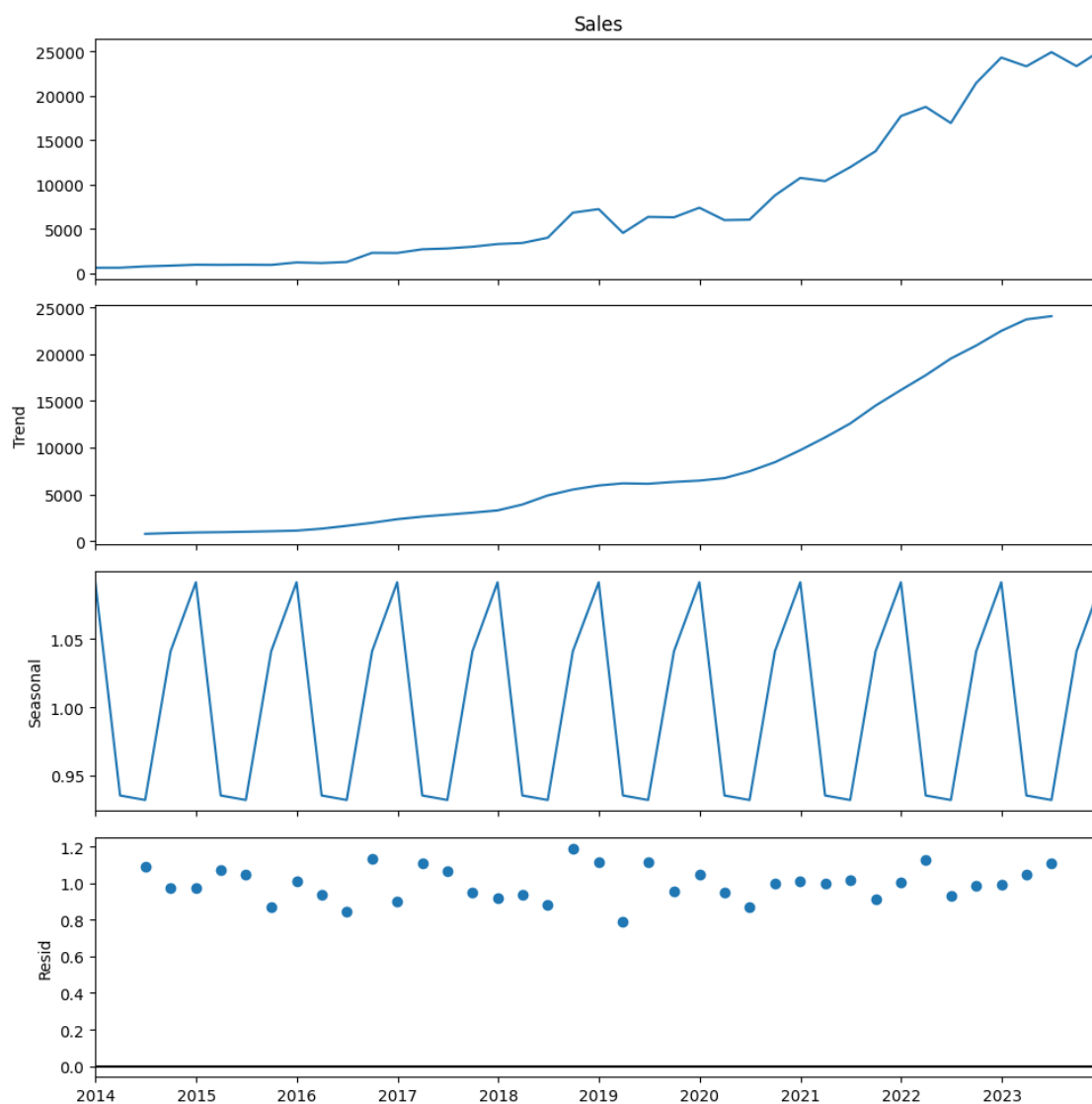
```
[109]: #adopted from https://medium.com/@BetterEverything/
↳graph-quarterly-data-with-python-6f38bcc6b7fe
for year, data in dictyear.items():
    quarters = list(goog1.loc[(goog1['Year'] == year)].Quarter)
    x_coordinates = [year + 0.15 * q for q in quarters]
    #print(x_coordinates, data)
    plt.bar(x_coordinates, data, width=0.2, align='edge')
plt.title('Tesla Sales by Quarter')
plt.ylabel('Sales(in millions)')

#plt.xticks(labels=[np.unique(goog1['Year'].values)], ticks = [])
#print(plt.xticks())
plt.show()
```



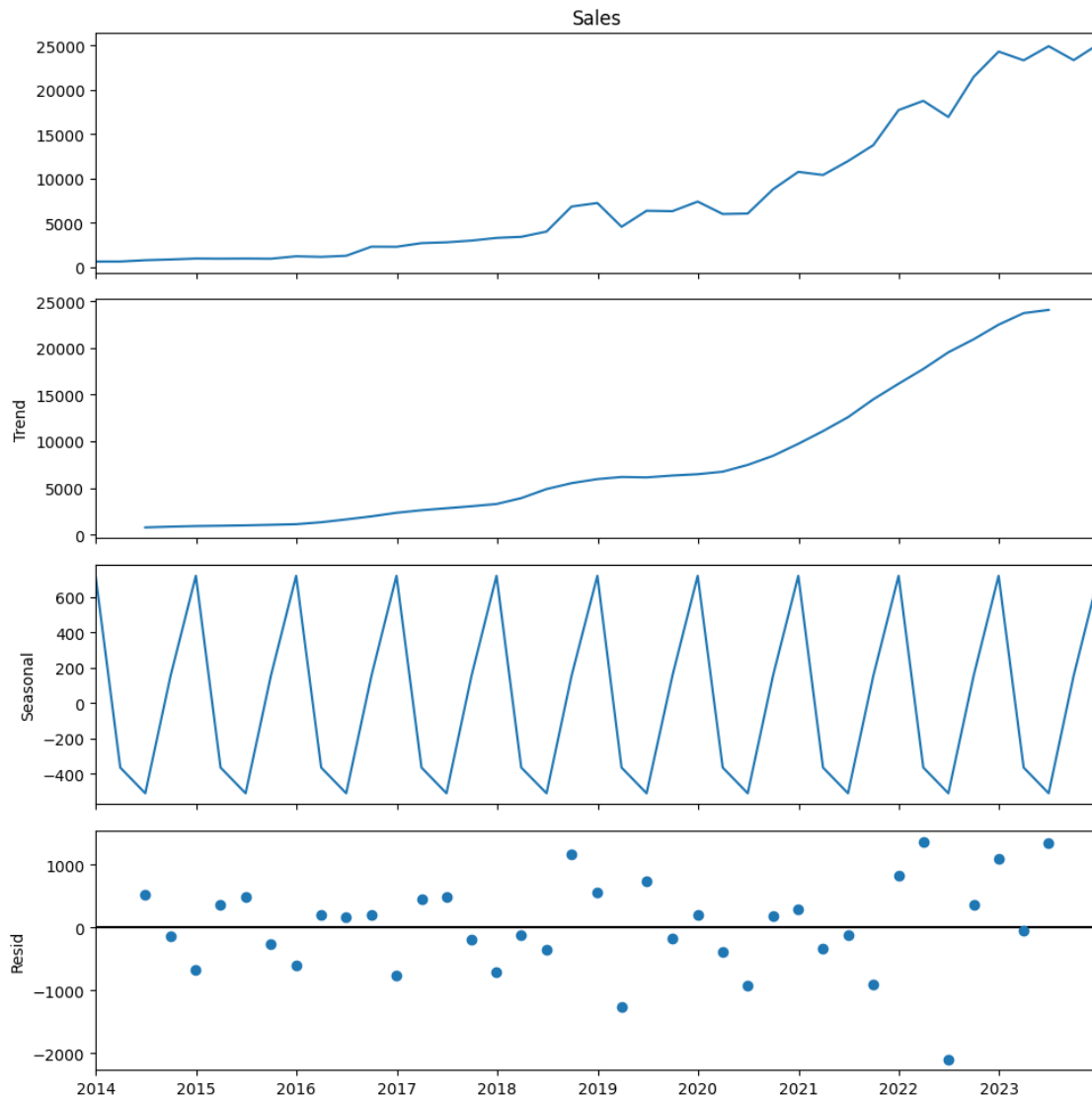
```
[110]: goog.set_index('Date', inplace=True)
```

```
[111]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
    ↪parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Sales'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

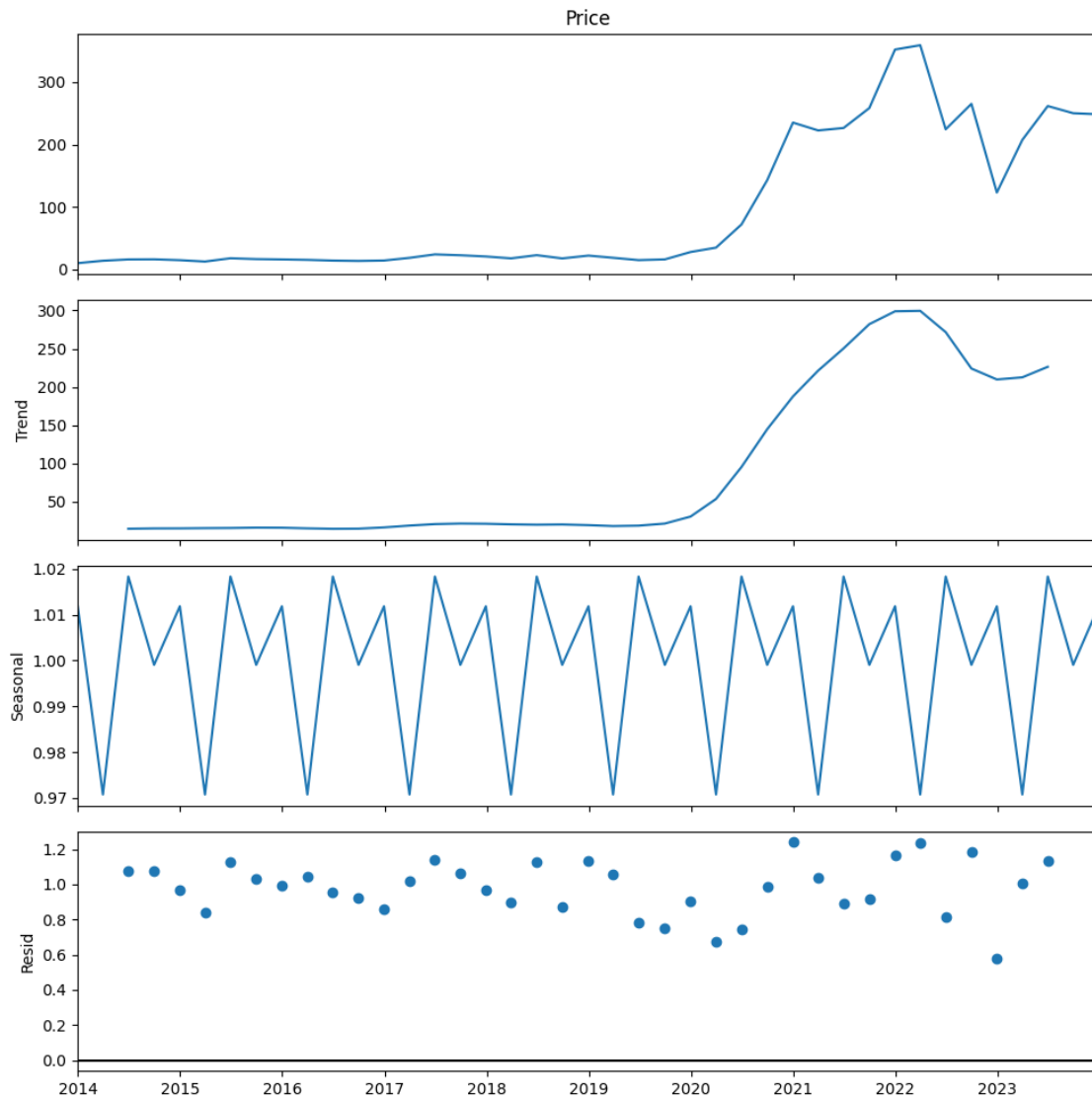


```
[112]: result = seasonal_decompose(goog['Sales'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
```

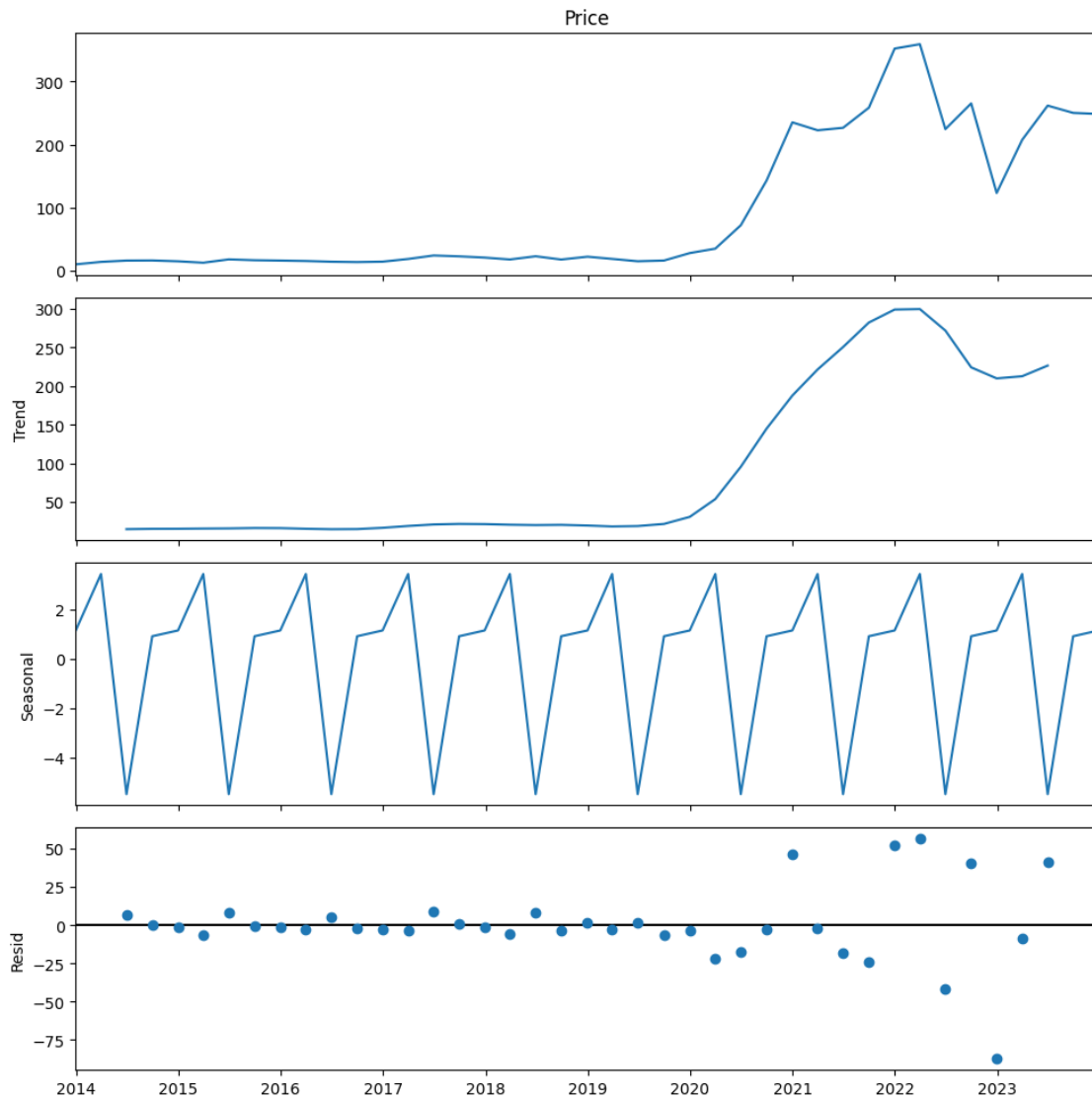
```
result.plot()
#plt.title('Time Series Decomposition of Sales')
plt.show()
```



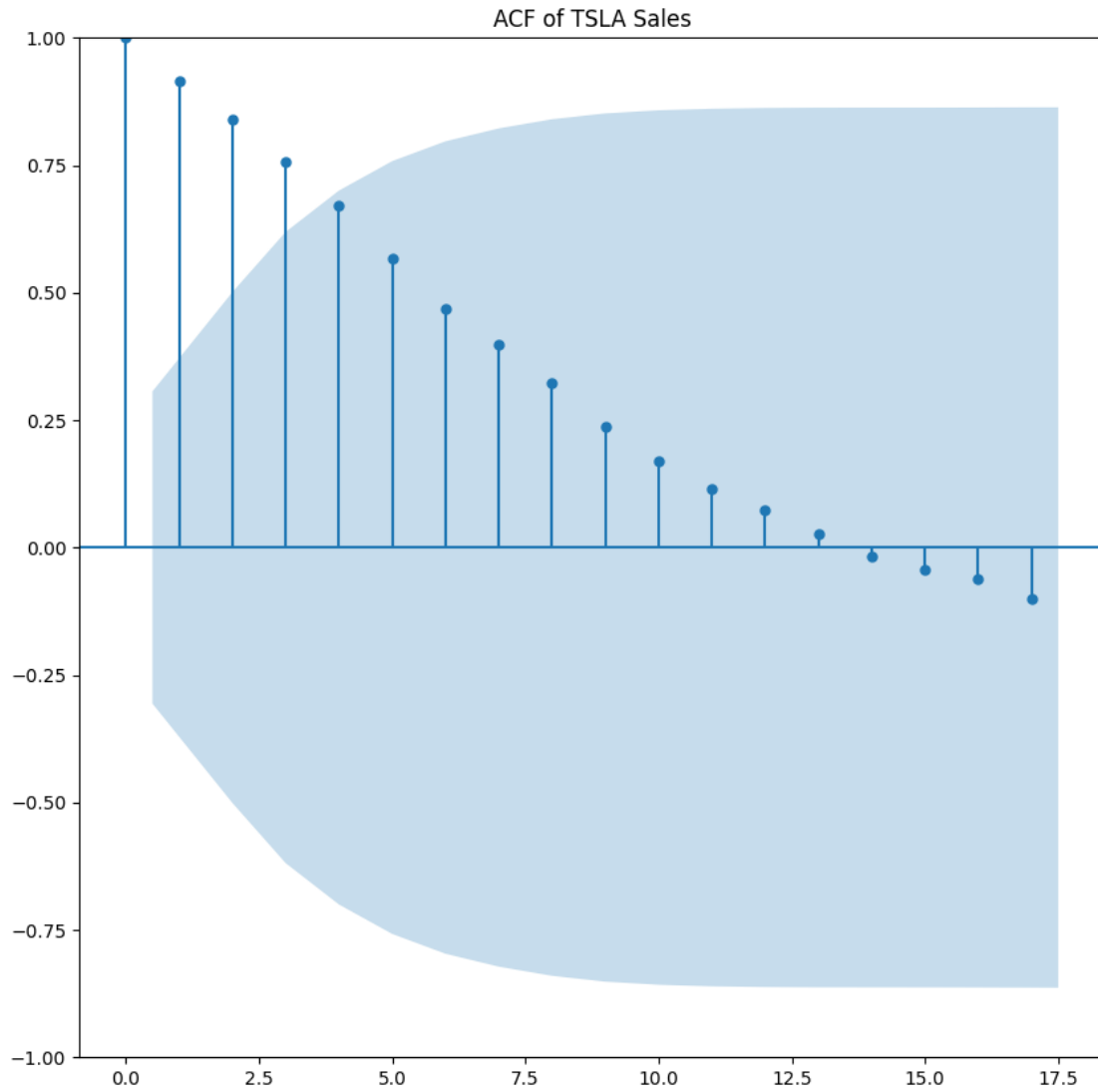
```
[113]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
        ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Price'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



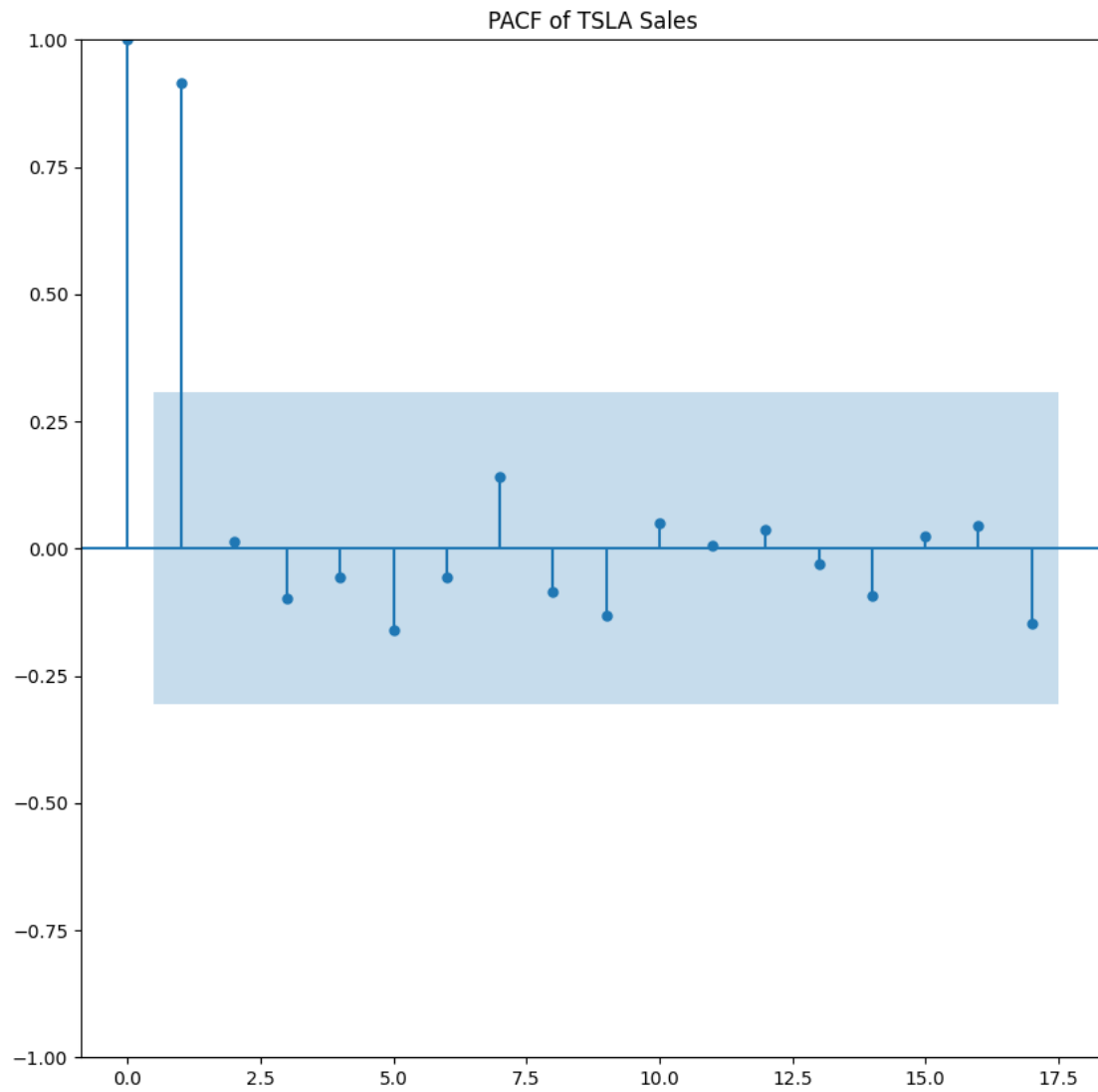
```
[114]: result = seasonal_decompose(goog['Price'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



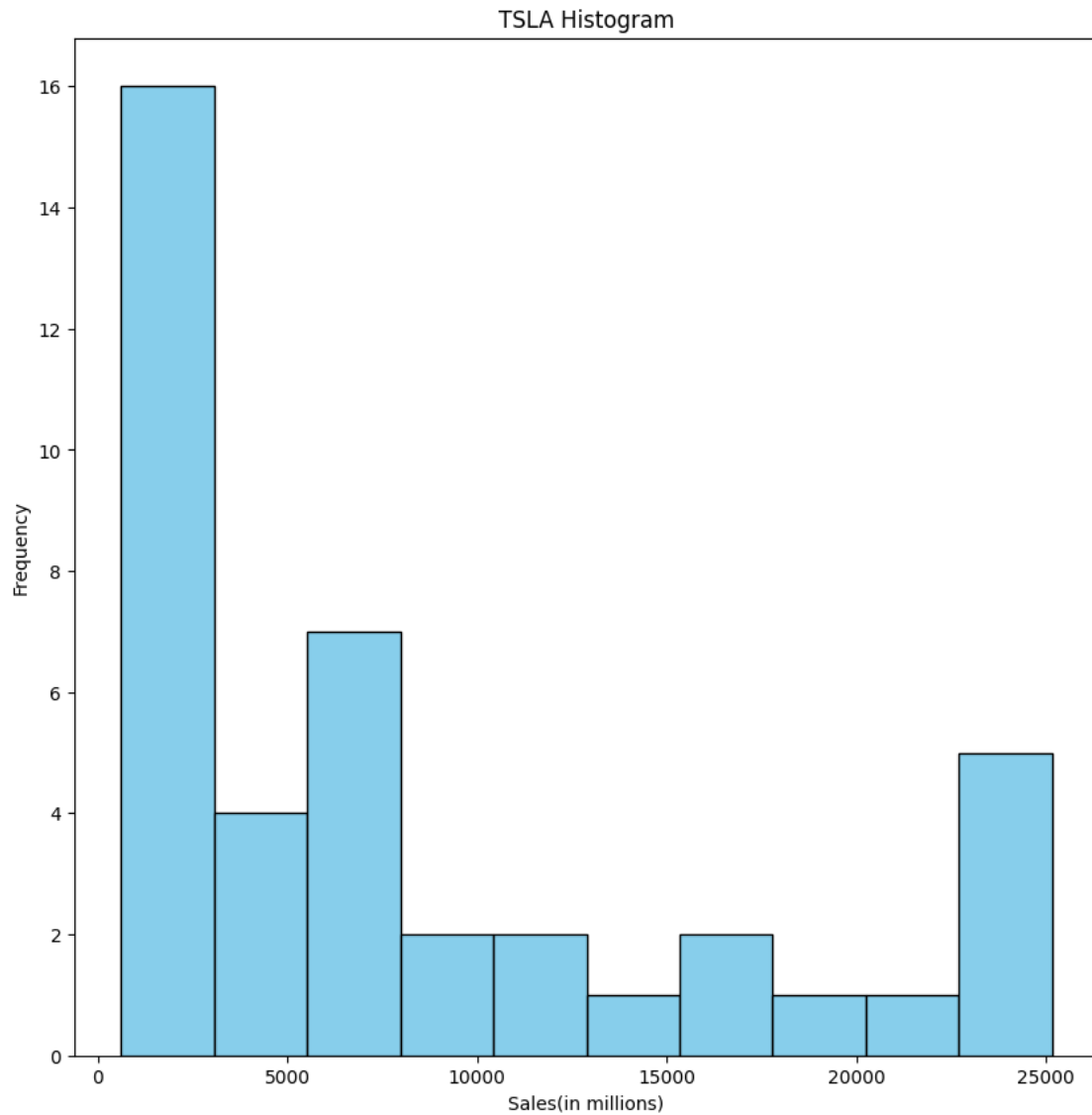
```
[115]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_acf(goog['Sales'], title='ACF of TSLA Sales')
pyplot.show()
```

```
[116]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_pacf(goog['Sales'], title='PACF of Tesla Sales')
pyplot.show()
```



```
[117]: plt.hist(goog['Sales'], color='skyblue', edgecolor='black')
plt.xlabel('Sales(in millions)')
plt.ylabel('Frequency')
plt.title('Tesla Histogram')
pyplot.show()
```



```
[118]: # statistical test
result = adfuller(goog['Sales'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 0.939400

p-value: 0.993584

Critical Values:

1%: -3.646

5%: -2.954

10%: -2.616

```
[119]: # statistical test
result = adfuller(goog['Price'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: -1.382937

p-value: 0.590428

Critical Values:

1%: -3.670

5%: -2.964

10%: -2.621

4.3 TSLA Conclusion

Description of Time series (is it stationary, are there any trends, seasonality etc)

For TSLA, you can see there is clearly a trend higher in both price and sales over the last 10 years. More recently, TSLA's stock price is declining on concerns about low priced EV's coming from China as well as general concern of the amount of TSLA's manufacturing exposure. There does appear to be seasonality mainly in the 4th quarter of the year. The additive model seems to be more appropriate for the Sales data while the multiplicative and additive approach have similar patterns in the residuals. It's unclear to me which model is best for the stock price data since the residuals in both models appear to have somewhat of a pattern. This is shown in both time series decomposition charts.

The null hypothesis of the ADF test is the time series is Non Stationary. We conclude TSLA is Non Stationary both when looking at Sales and Price since the test statistic is larger than the critical values.

TSLA does show autocorrelation out to about 4 lags.

```
[ ]:
```

4.3.1 Analysis of AMZN Stock and Fundamentals over last 10 years

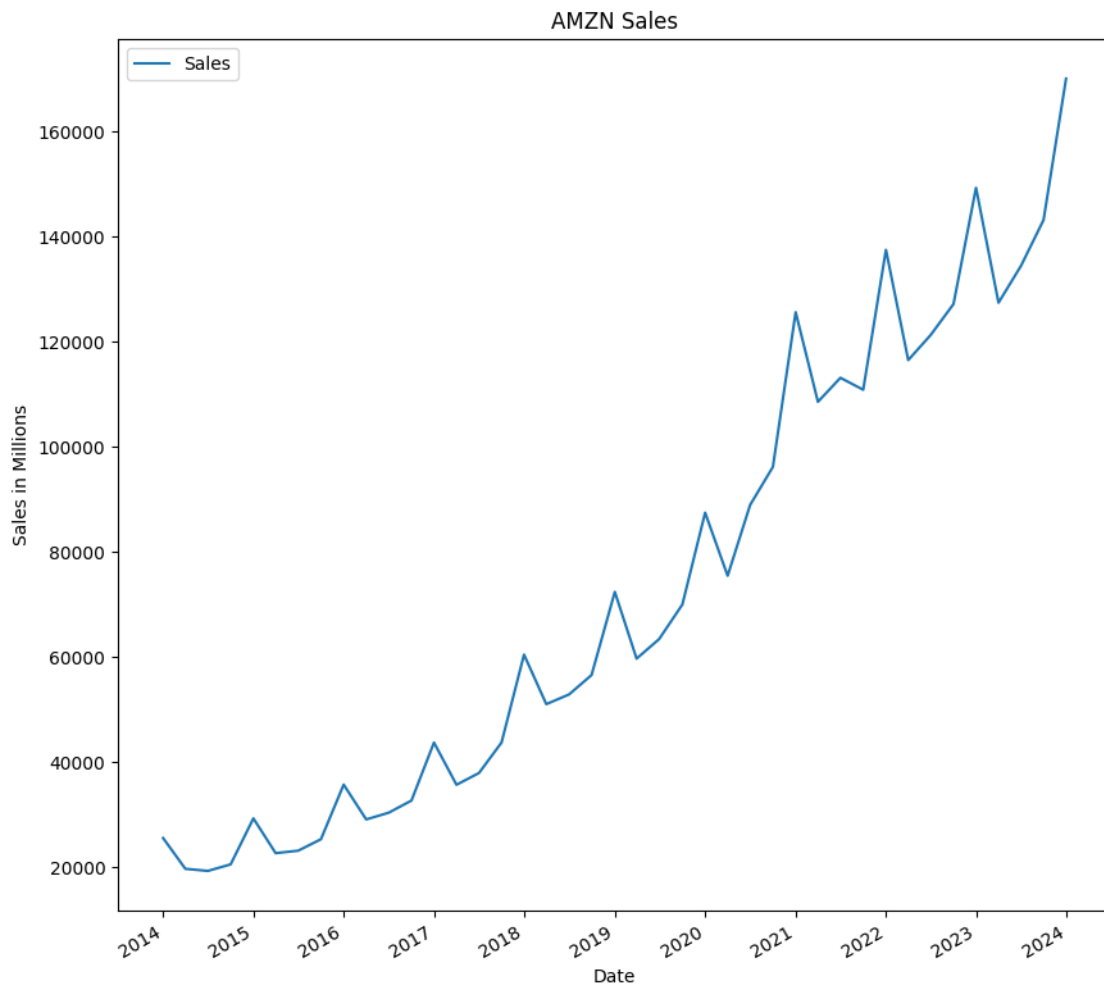
```
[120]: #goog = pd.read_csv('GOOGL.csv', header=0, index_col=0,
    ↪ parse_dates=True, squeeze=True)
goog = pd.read_csv('./Aux_Data/AMZN.csv')
```

```
[121]: goog.columns
```

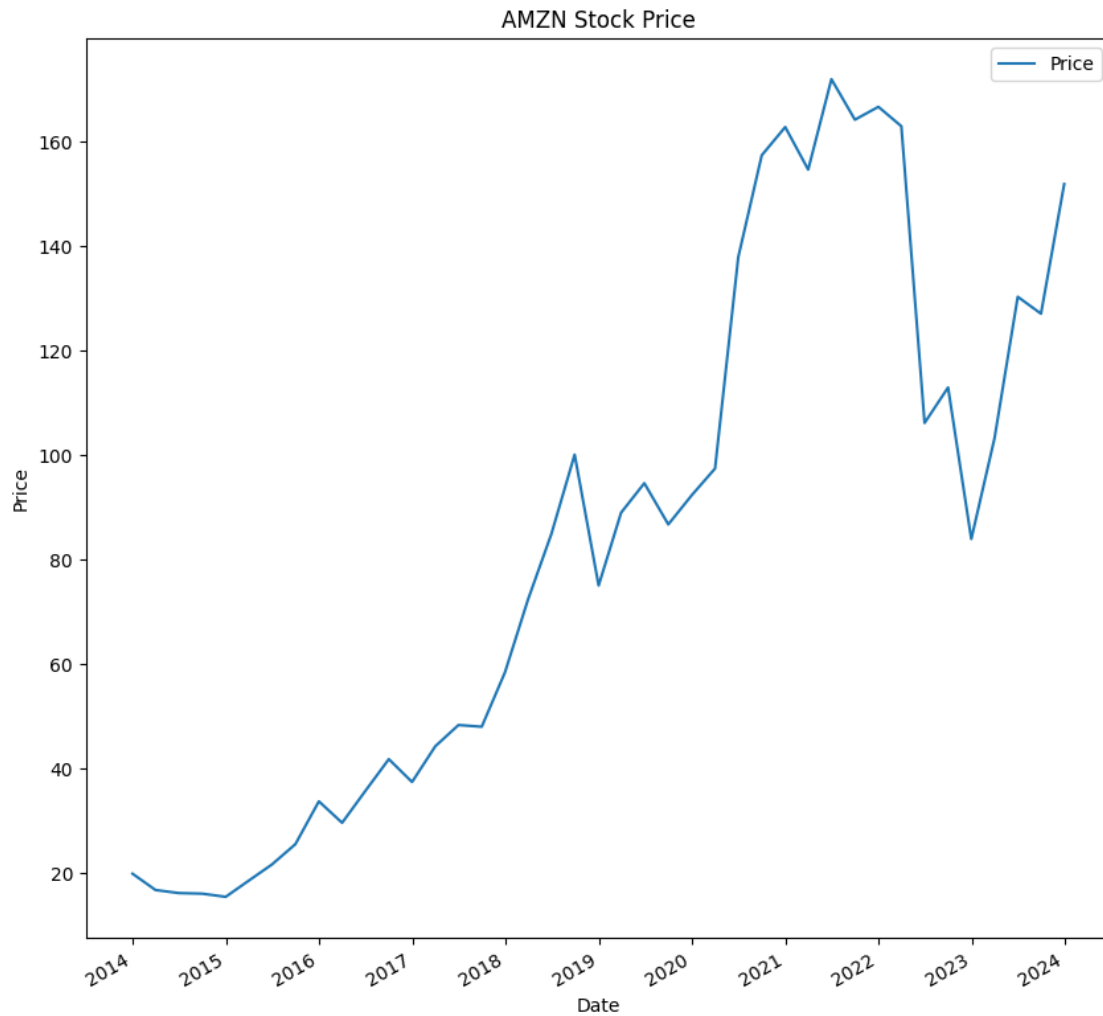
```
[121]: Index(['Date', 'Sales', 'FreeCashFlow', 'Cash', 'Long Term Debt', 'Price'],
dtype='object')
```

```
[122]: goog['Date'] = pd.to_datetime(goog['Date'], format='%m/%d/%Y')
goog['Year'] = goog['Date'].dt.year
goog['Quarter'] = goog['Date'].dt.quarter
```

```
[123]: goog.plot('Date', 'Sales')
plt.title('AMZN Sales')
plt.ylabel('Sales in Millions')
plt.show()
```

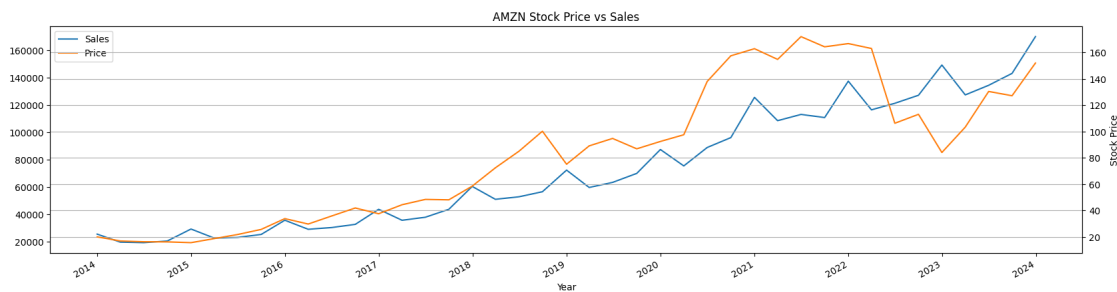


```
[124]: goog.plot('Date', 'Price')
plt.title('AMZN Stock Price')
plt.ylabel('Price')
plt.show()
```



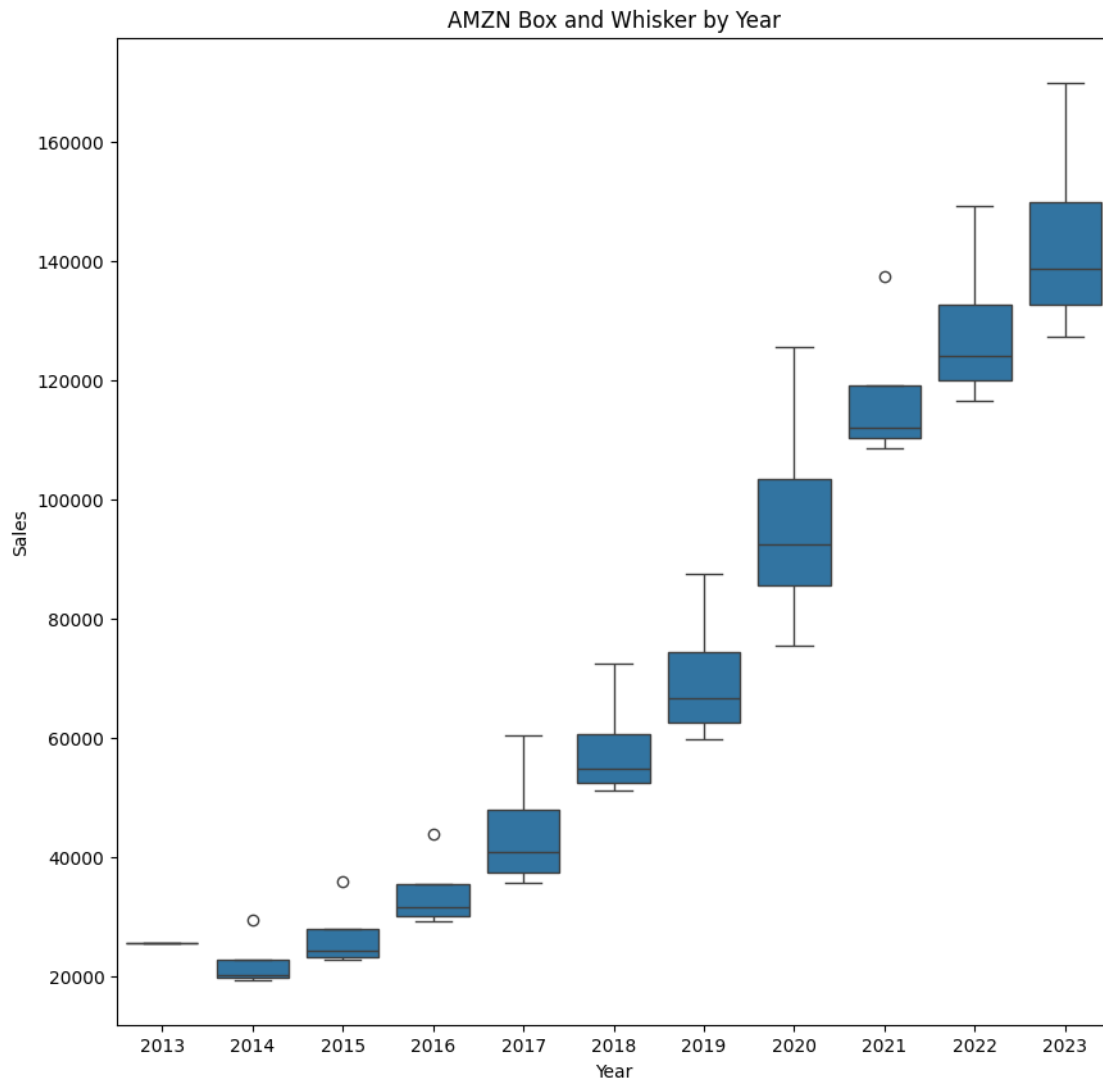
```
[125]: goog.loc[:,['Date','Sales','Price']].plot(x = 'Date',secondary_y=['Price'],
↪xlabel = 'Year', ylabel = 'Stock Price',sharex = True,mark_right=False,
↪figsize = (20,5), grid=True)
plt.title('AMZN Stock Price vs Sales')
```

[125]: Text(0.5, 1.0, 'AMZN Stock Price vs Sales')



```
[126]: sn.boxplot(x='Year',y='Sales', data = goog).set(title = 'AMZN Box and Whisker_
↳by Year')
```

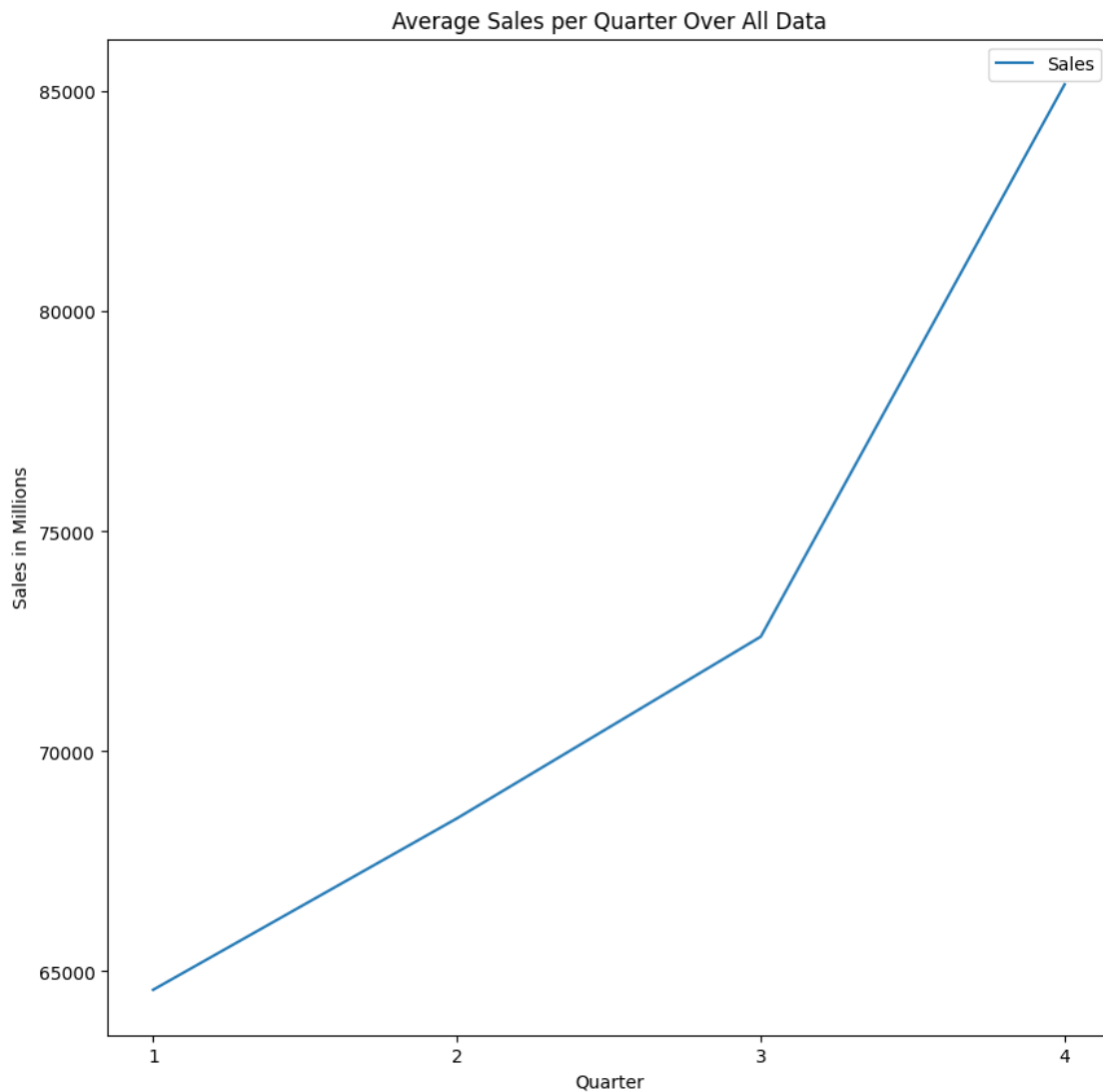
```
[126]: [Text(0.5, 1.0, 'AMZN Box and Whisker by Year')]
```



```
[127]: avgperqtr = goog[['Quarter', 'Sales']].groupby('Quarter').mean().reset_index()
```

```
[128]: avgperqtr.plot('Quarter','Sales')
plt.ylabel('Sales in Millions')
plt.title('Average Sales per Quarter Over All Data')
plt.xticks(goog.Quarter.unique())
```

```
plt.show()
```



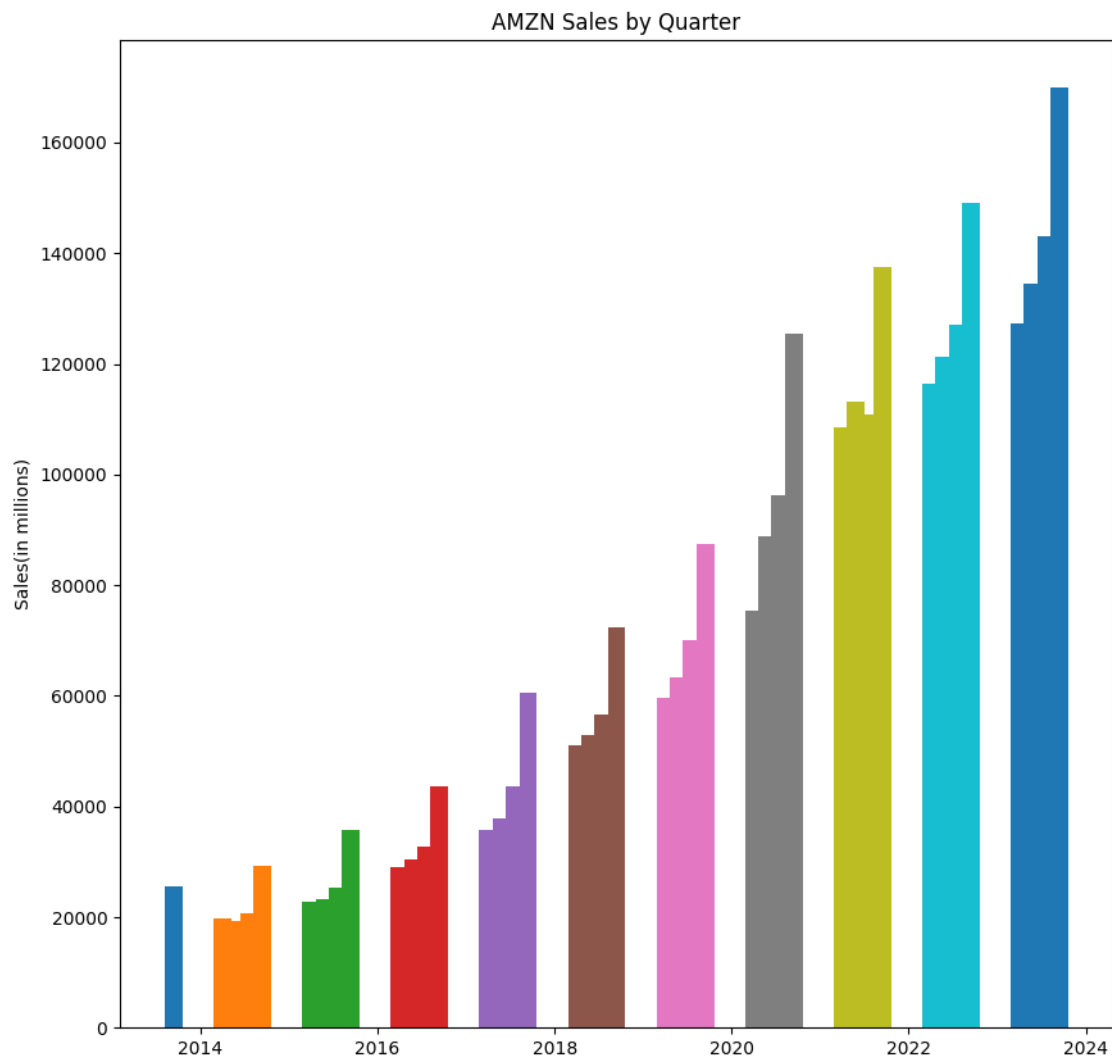
```
[129]: goog1 = pd.DataFrame(goog, columns=['Year', 'Quarter', 'Sales'])
```

```
[130]: dictyear = {}  
saleslist = []  
for year in goog1['Year'].unique():  
    #print(year)  
    #byyear = goog1[goog1['Year'] == year]  
    #saleslist = saleslist.append(byyear.Sales)  
    dictyear[year] = list(goog1[goog1['Year'] == year].Sales)
```



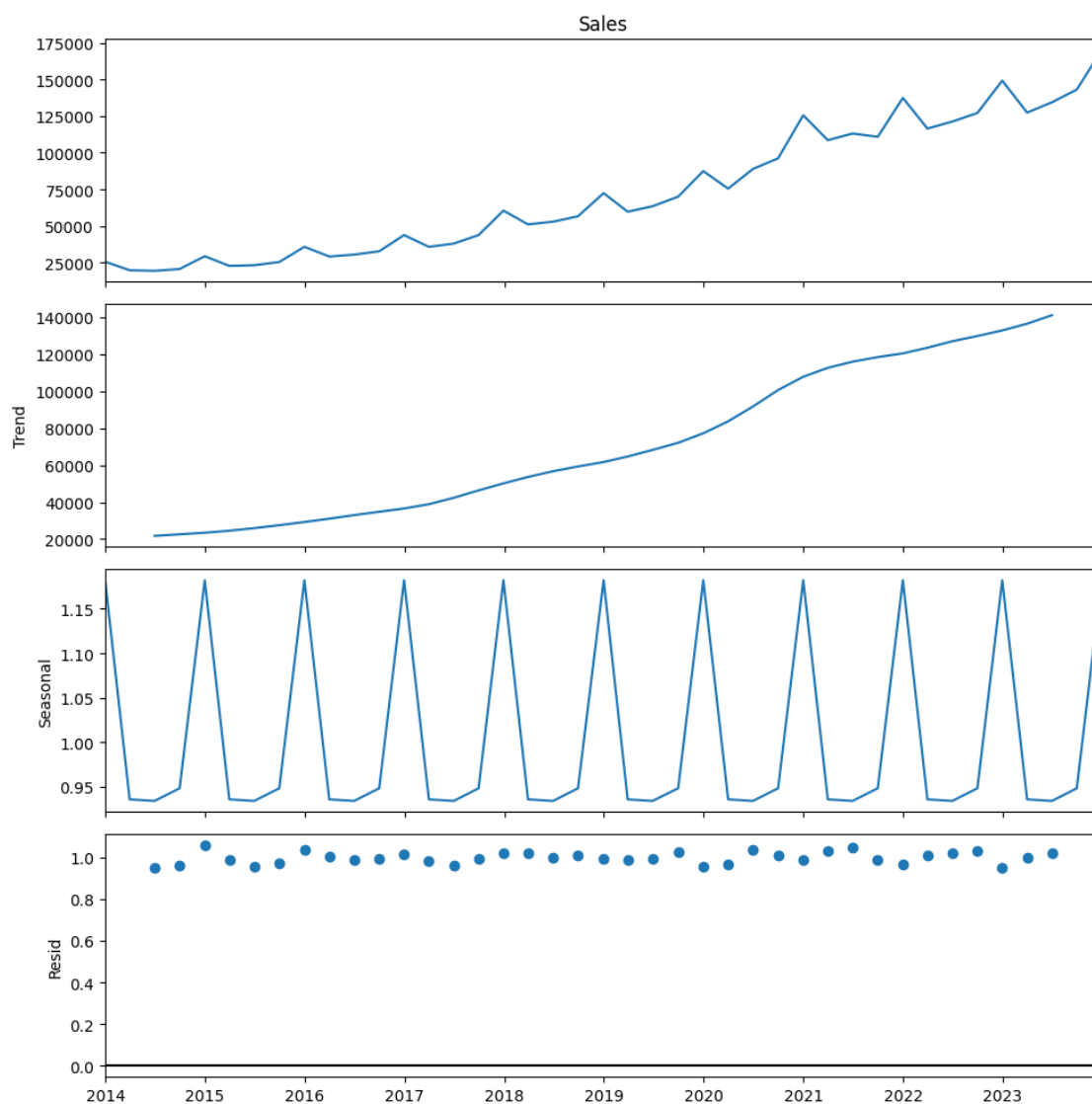
```
[131]: #adopted from https://medium.com/@BetterEverything/
↳graph-quarterly-data-with-python-6f38bcc6b7fe
for year, data in dictyear.items():
    quarters = list(goog1.loc[(goog1['Year'] == year)].Quarter)
    x_coordinates = [year + 0.15 * q for q in quarters]
    #print(x_coordinates, data)
    plt.bar(x_coordinates, data, width=0.2, align='edge')
plt.title('AMZN Sales by Quarter')
plt.ylabel('Sales(in millions)')

#plt.xticks(labels=[np.unique(goog1['Year'].values)], ticks = [])
#print(plt.xticks())
plt.show()
```



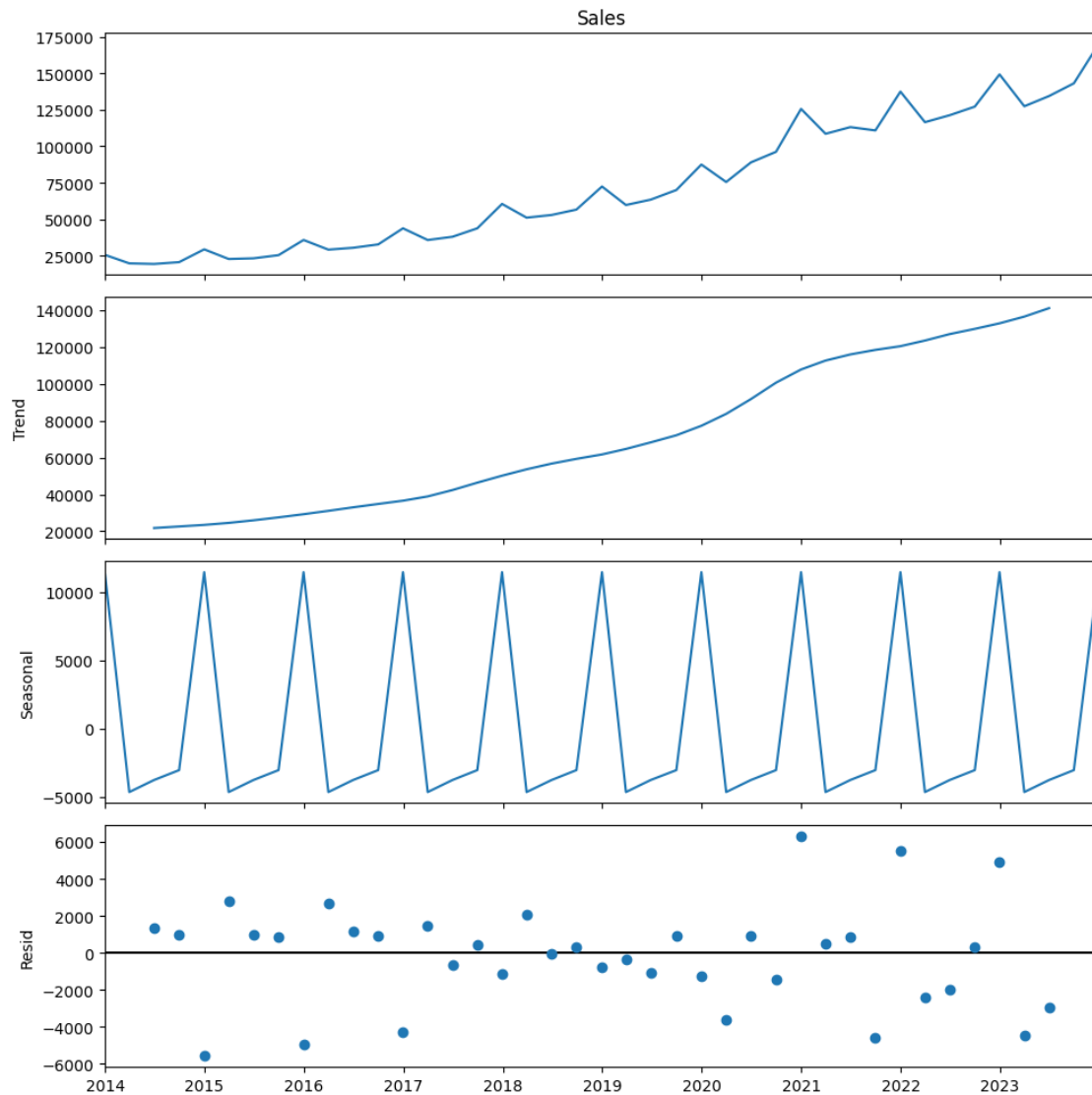
```
[132]: goog.set_index('Date', inplace=True)
```

```
[133]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
        ↪parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Sales'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

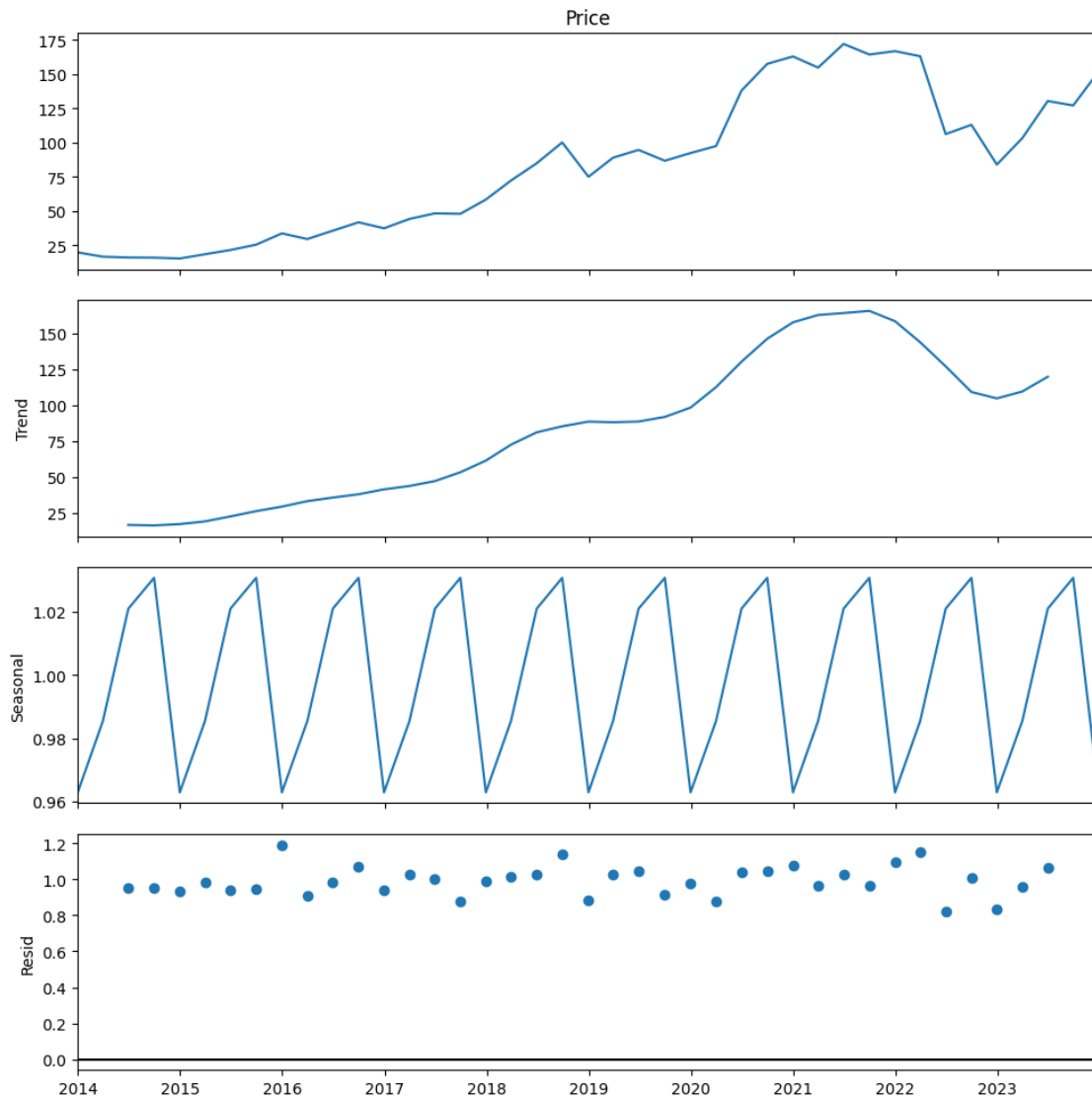


```
[134]: result = seasonal_decompose(goog['Sales'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
```

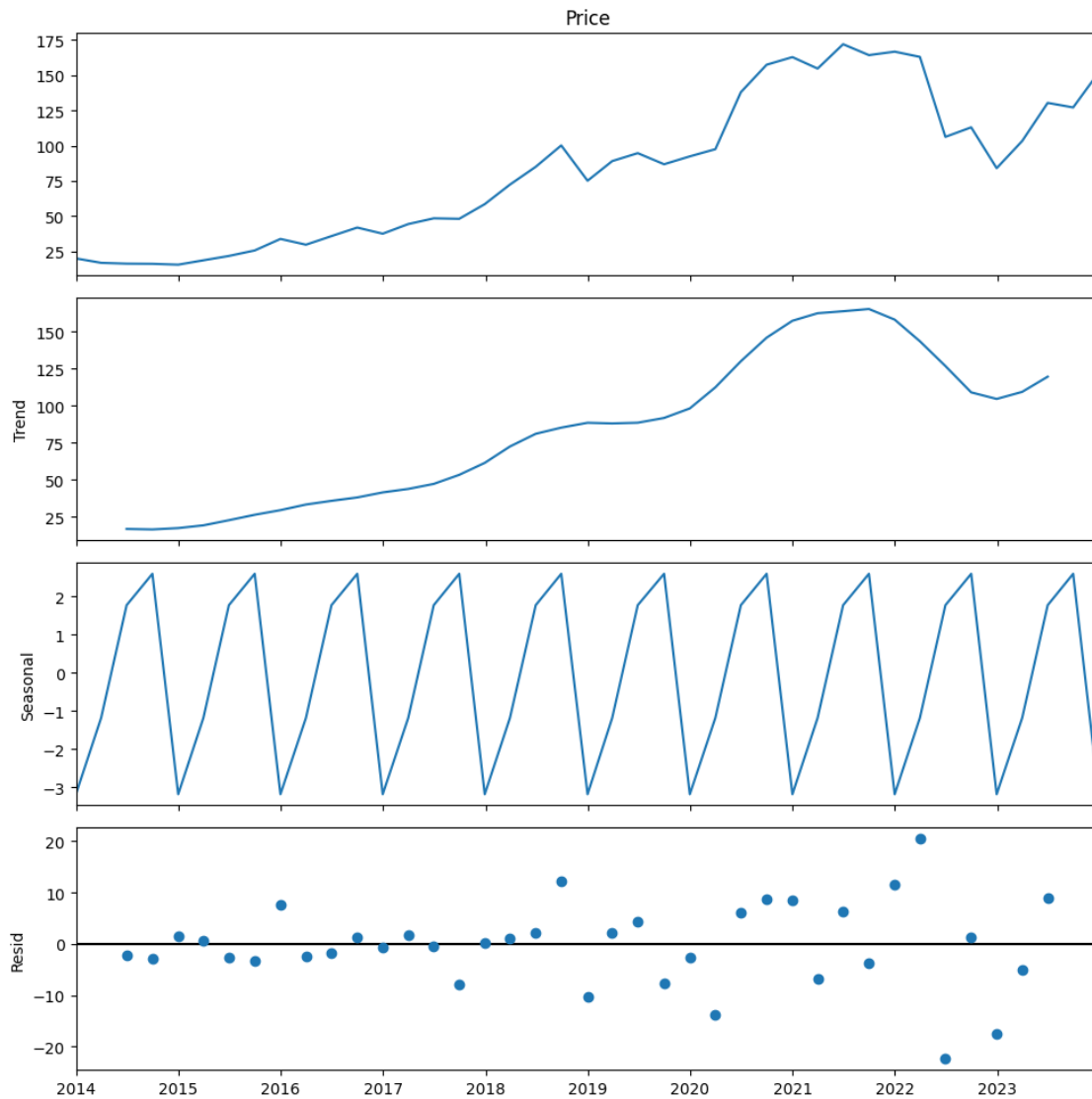
```
result.plot()
#plt.title('Time Series Decomposition of Sales')
plt.show()
```



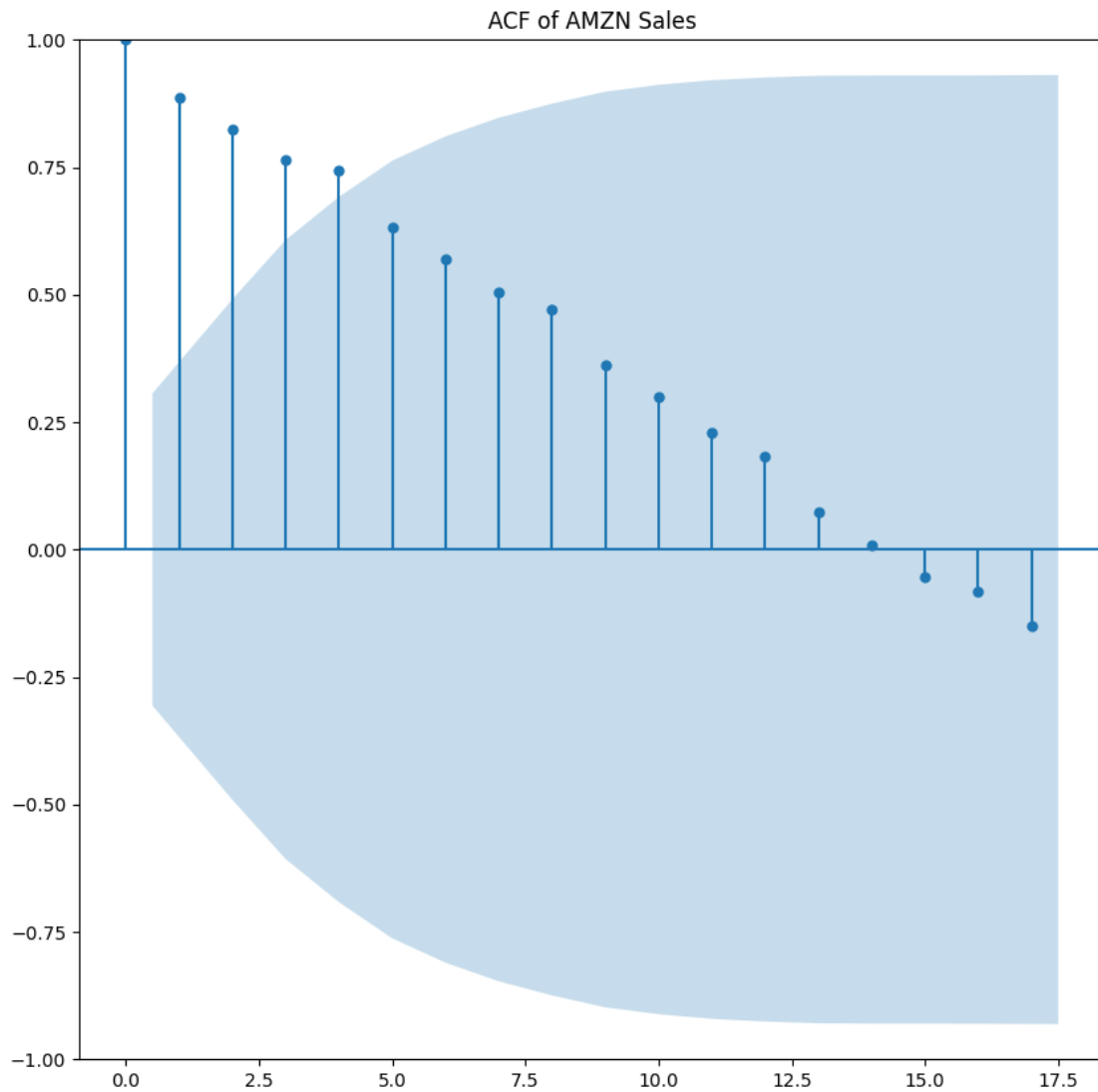
```
[135]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
        ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Price'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



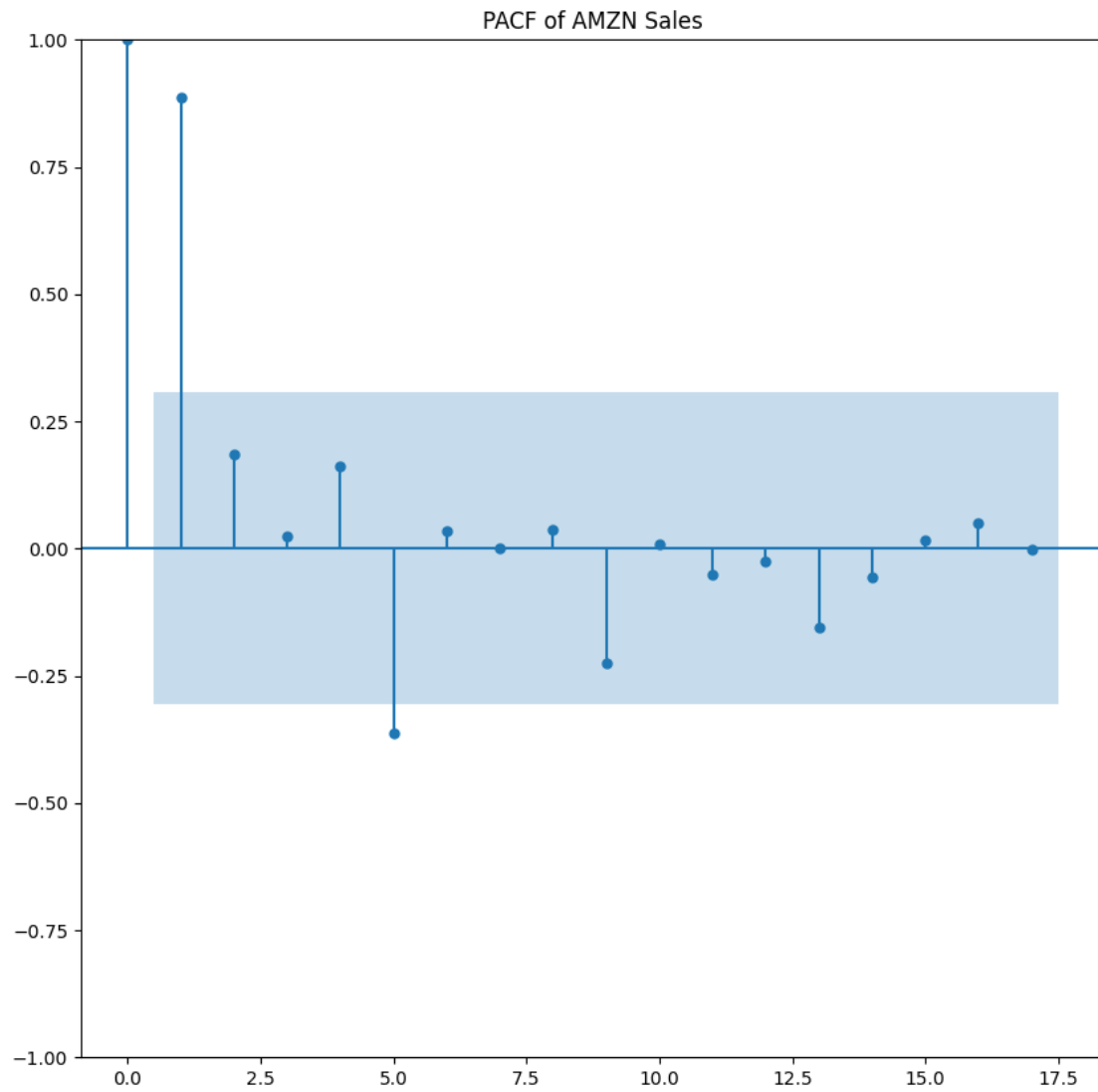
```
[136]: result = seasonal_decompose(goog['Price'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



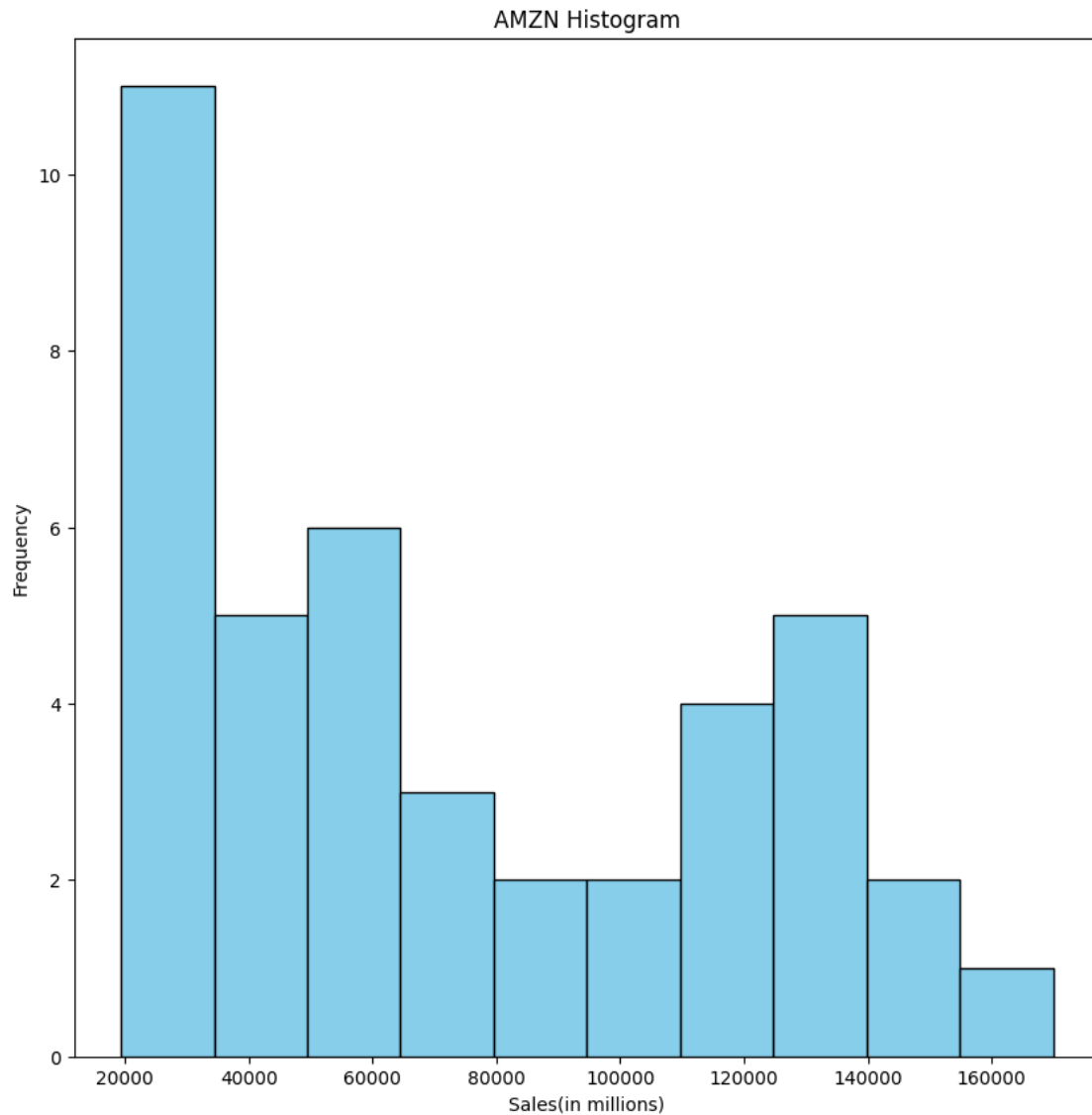
```
[137]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_acf(goog['Sales'], title='ACF of AMZN Sales')
pyplot.show()
```



```
[138]: #https://machinelearningmastery.com/  
       ↪gentle-introduction-autocorrelation-partial-autocorrelation/  
from matplotlib import pyplot  
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf  
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)  
plot_pacf(goog['Sales'], title='PACF of AMZN Sales')  
pyplot.show()
```



```
[139]: plt.hist(goog['Sales'], color='skyblue', edgecolor='black')
plt.xlabel('Sales(in millions)')
plt.ylabel('Frequency')
plt.title('AMZN Histogram')
pyplot.show()
```



```
[140]: # statistical test
result = adfuller(goog['Sales'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 1.213185

p-value: 0.996076

Critical Values:

1%: -3.639

5%: -2.951

10%: -2.614

```
[141]: # statistical test
result = adfuller(goog['Price'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: -0.890552

p-value: 0.791068

Critical Values:

1%: -3.606

5%: -2.937

10%: -2.607

4.4 AMZN Conclusion

Description of Time series (is it stationary, are there any trends, seasonality etc)

For AMZN, you can see there is clearly a trend higher in both price and sales over the last 10 years. It's interesting to note the higher level of volatility in Sales vs Stock price. Seems like investors might be more nervous about AMZN while Sales continue on a steady trend higher. There does appear to be seasonality mainly in the 4th quarter of the year. The additive model seems to be more appropriate for the Sales and Stock price data as the residuals in each case appear like white noise in the additive model but have more of a pattern in the multiplicative model. This is shown in both time series decomposition charts.

The null hypothesis of the ADF test is the time series is Non Stationary. We conclude AMZN is Non Stationary both when looking at Sales and Price since the test statistic is larger than the critical values.

AMZN does show autocorrelation out to about 5 lags.

```
[ ]:
```

```
[ ]:
```

```
[145]: #goog = pd.read_csv('GOOGL.csv', header=0, index_col=0,
    ↪ parse_dates=True, squeeze=True)
goog = pd.read_csv('./Aux_Data/AAPL.csv')
```

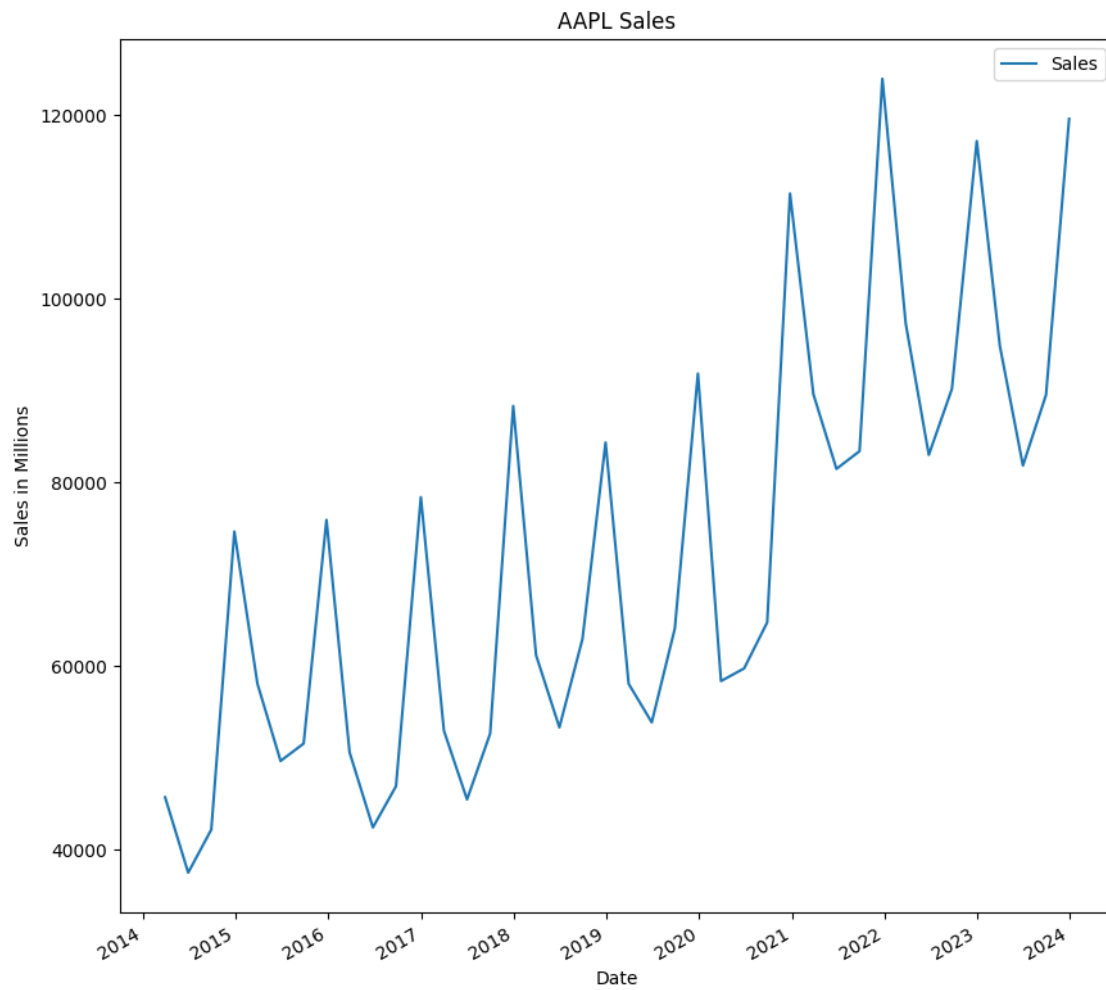
```
[146]: goog.columns
```

```
[146]: Index(['Date', 'Sales', 'FreeCashFlow', 'Cash', 'Long Term Debt', 'Price'],
dtype='object')
```

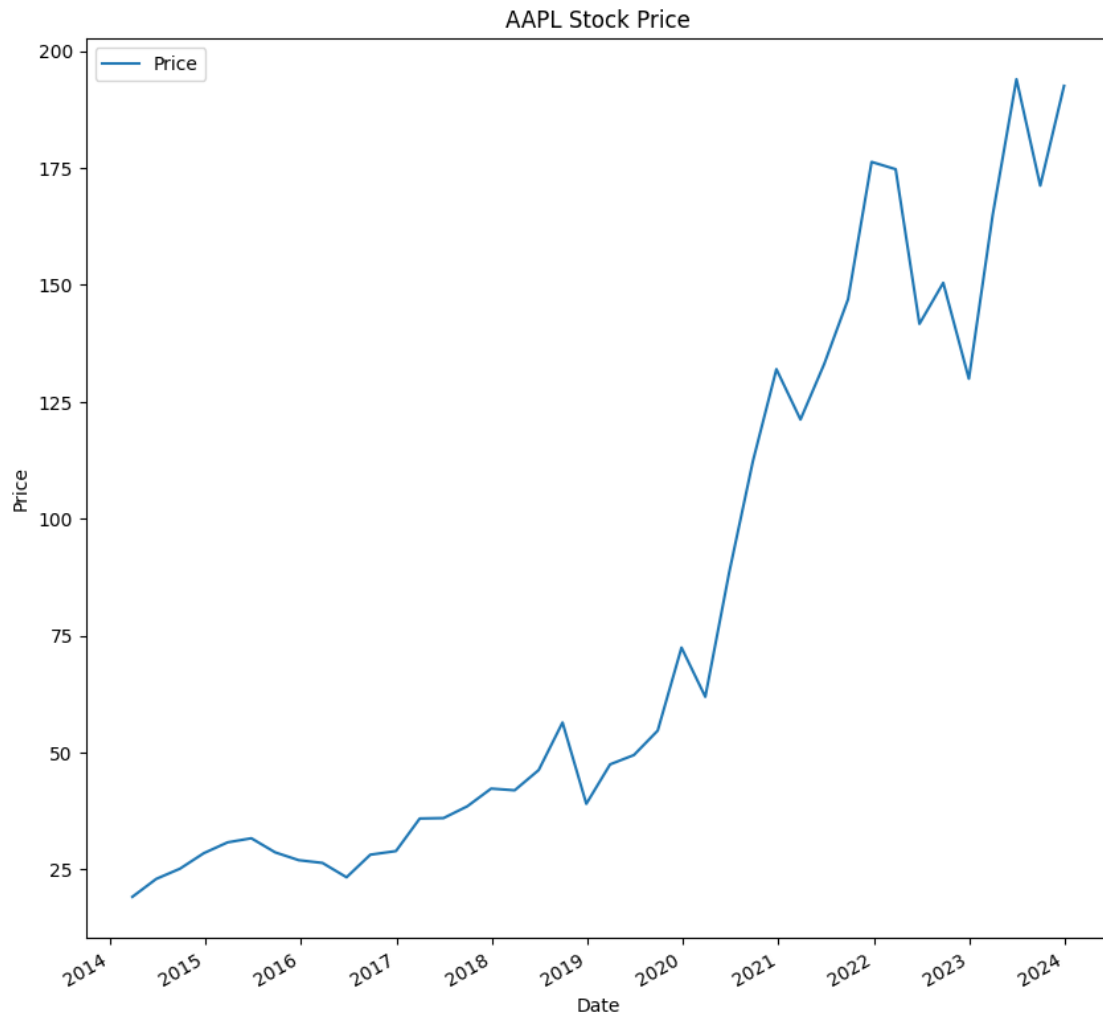
```
[147]: goog['Date'] = pd.to_datetime(goog['Date'], format='%m/%d/%Y')
goog['Year'] = goog['Date'].dt.year
```

```
goog['Quarter'] = goog['Date'].dt.quarter
```

```
[148]: goog.plot('Date', 'Sales')  
plt.title('AAPL Sales')  
plt.ylabel('Sales in Millions')  
plt.show()
```

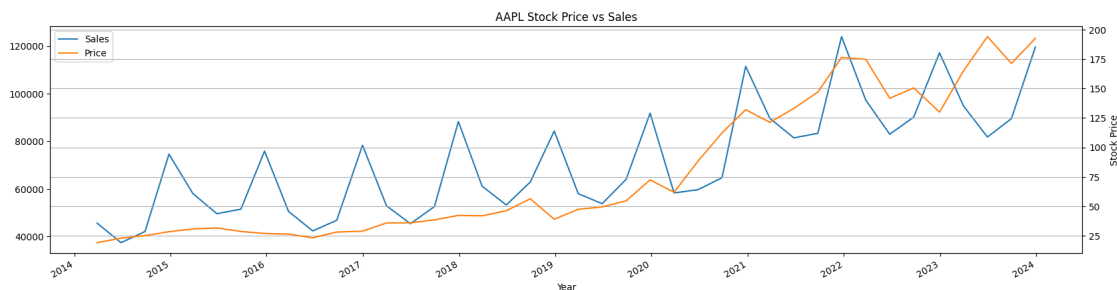


```
[149]: goog.plot('Date', 'Price')  
plt.title('AAPL Stock Price')  
plt.ylabel('Price')  
plt.show()
```



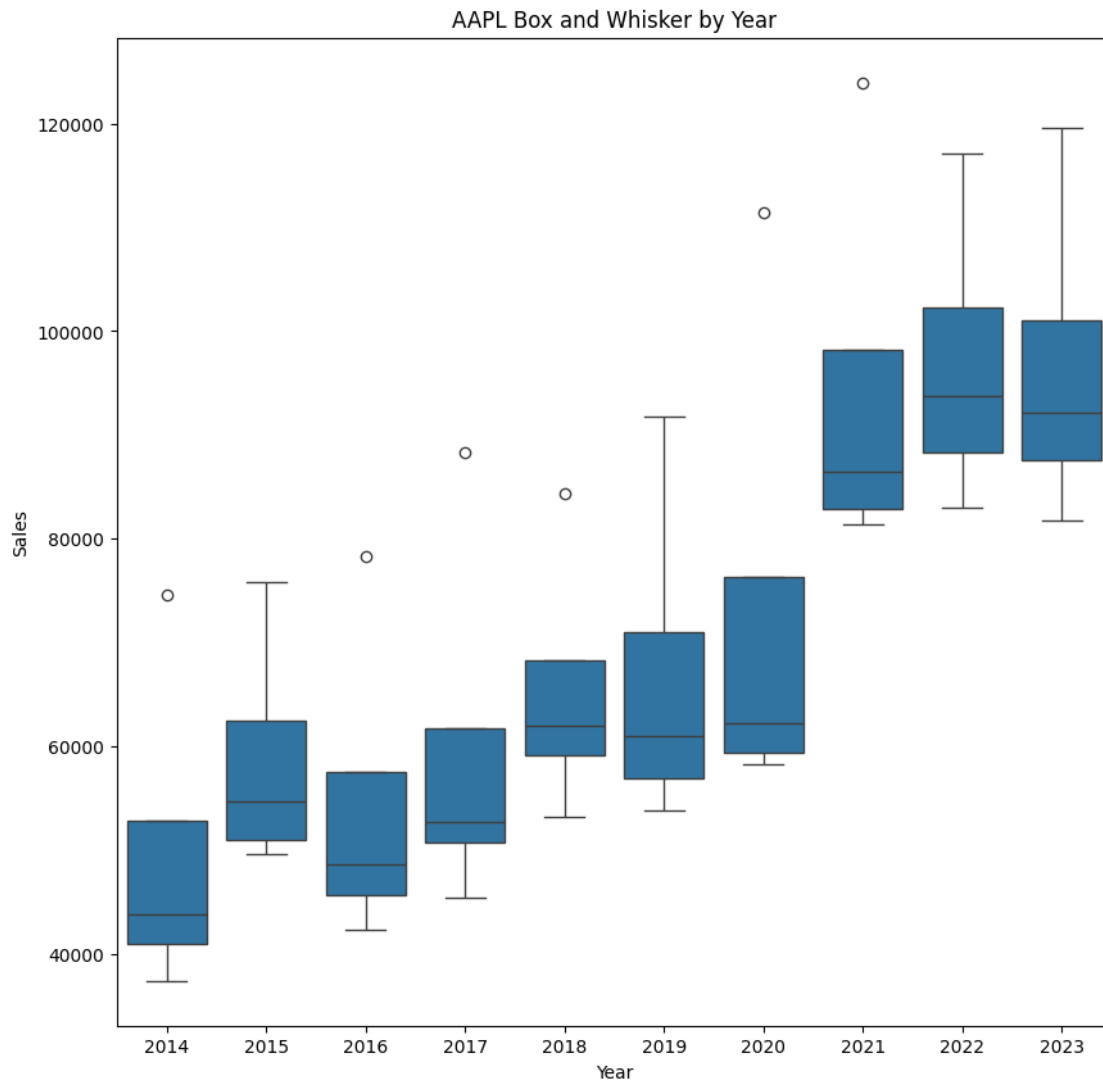
```
[150]: goog.loc[:,['Date','Sales','Price']].plot(x = 'Date',secondary_y=['Price'],
↪xlabel = 'Year', ylabel = 'Stock Price',sharex = True,mark_right=False,
↪figsize = (20,5), grid=True)
plt.title('AAPL Stock Price vs Sales')
```

```
[150]: Text(0.5, 1.0, 'AAPL Stock Price vs Sales')
```



```
[151]: sn.boxplot(x='Year',y='Sales', data = goog).set(title = 'AAPL Box and Whisker_
↳by Year')
```

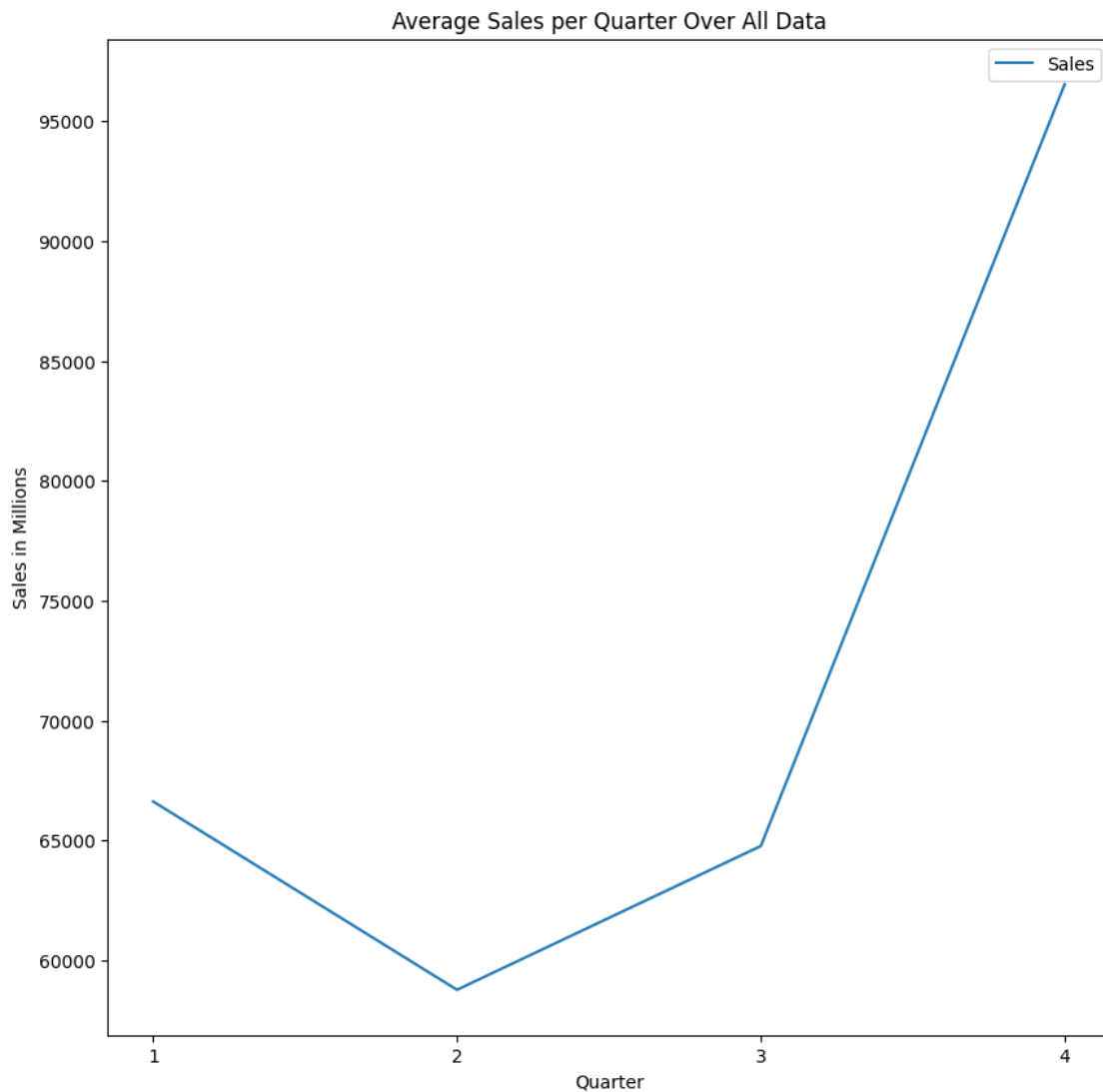
```
[151]: [Text(0.5, 1.0, 'AAPL Box and Whisker by Year')]
```



```
[152]: avgperqtr = goog[['Quarter', 'Sales']].groupby('Quarter').mean().reset_index()
```

```
[153]: avgperqtr.plot('Quarter','Sales')
plt.ylabel('Sales in Millions')
plt.title('Average Sales per Quarter Over All Data')
plt.xticks(goog.Quarter.unique())
```

```
plt.show()
```

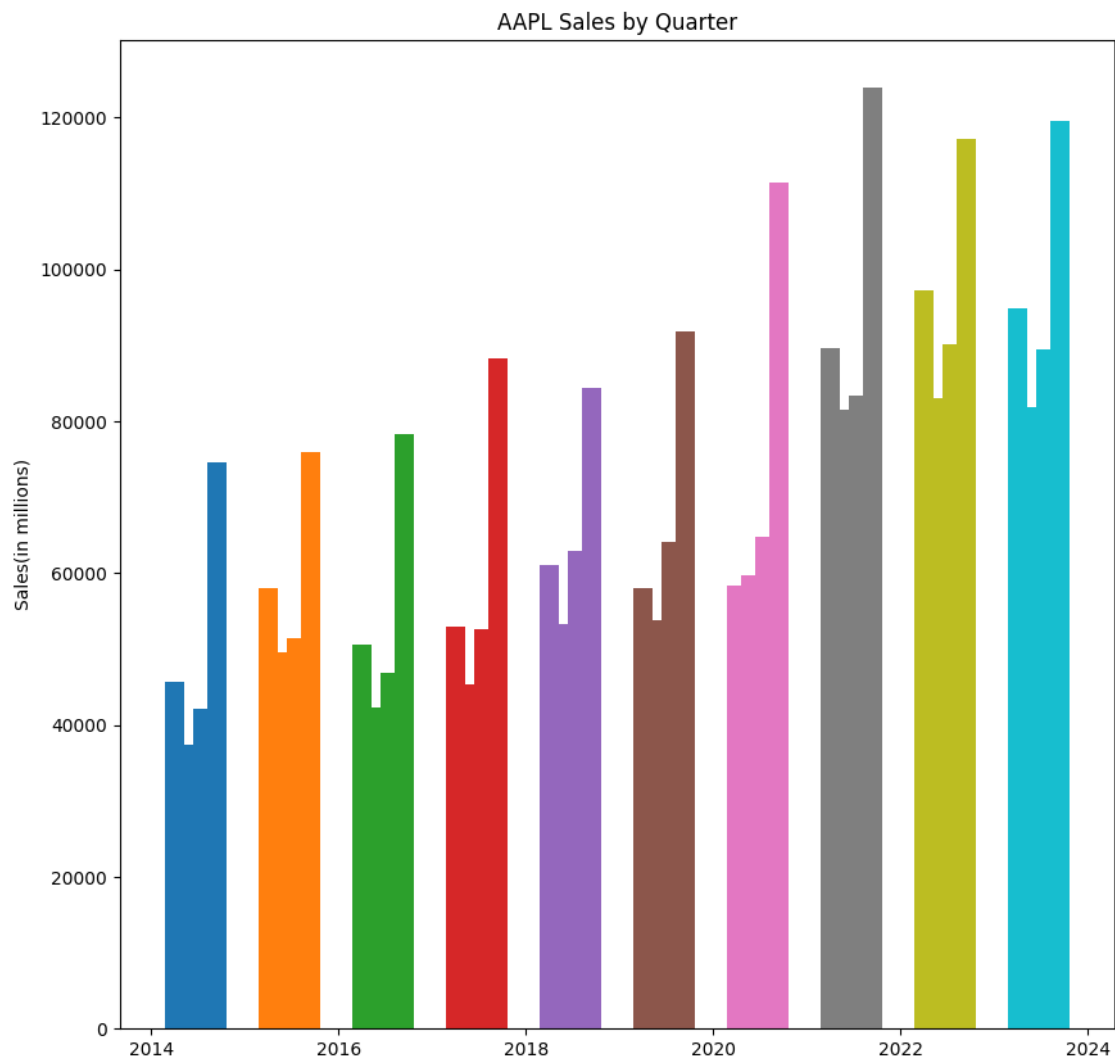


```
[154]: goog1 = pd.DataFrame(goog, columns=['Year', 'Quarter', 'Sales'])
```

```
[155]: dictyear = {}  
saleslist = []  
for year in goog1['Year'].unique():  
    #print(year)  
    #byyear = goog1[goog1['Year'] == year]  
    #saleslist = saleslist.append(byyear.Sales)  
    dictyear[year] = list(goog1[goog1['Year'] == year].Sales)
```

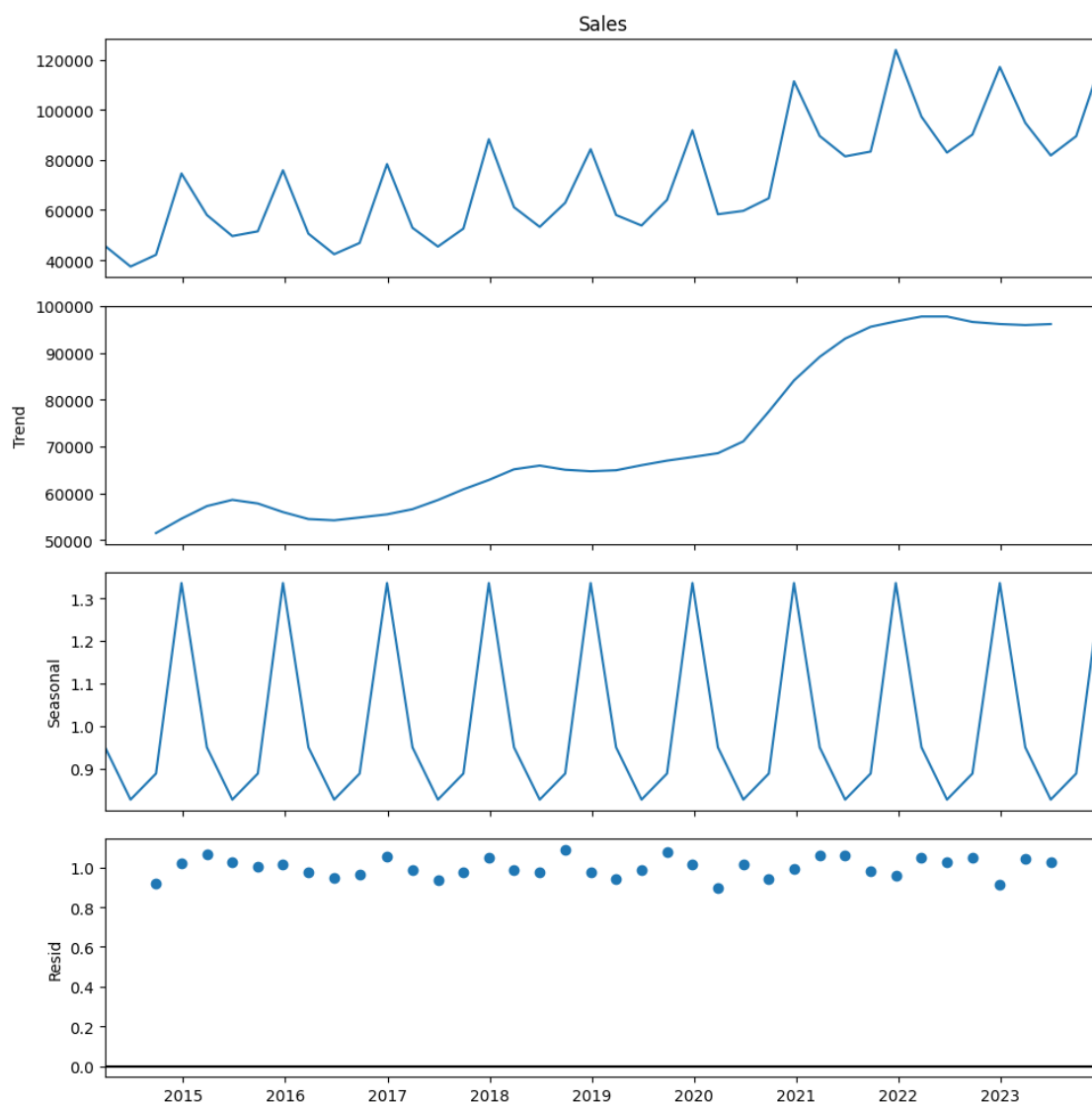
```
[156]: #adopted from https://medium.com/@BetterEverything/
↳graph-quarterly-data-with-python-6f38bcc6b7fe
for year, data in dictyear.items():
    quarters = list(goog1.loc[(goog1['Year'] == year)].Quarter)
    x_coordinates = [year + 0.15 * q for q in quarters]
    #print(x_coordinates, data)
    plt.bar(x_coordinates, data, width=0.2, align='edge')
plt.title('AAPL Sales by Quarter')
plt.ylabel('Sales(in millions)')

#plt.xticks(labels=[np.unique(goog1['Year'].values)], ticks = [])
#print(plt.xticks())
plt.show()
```



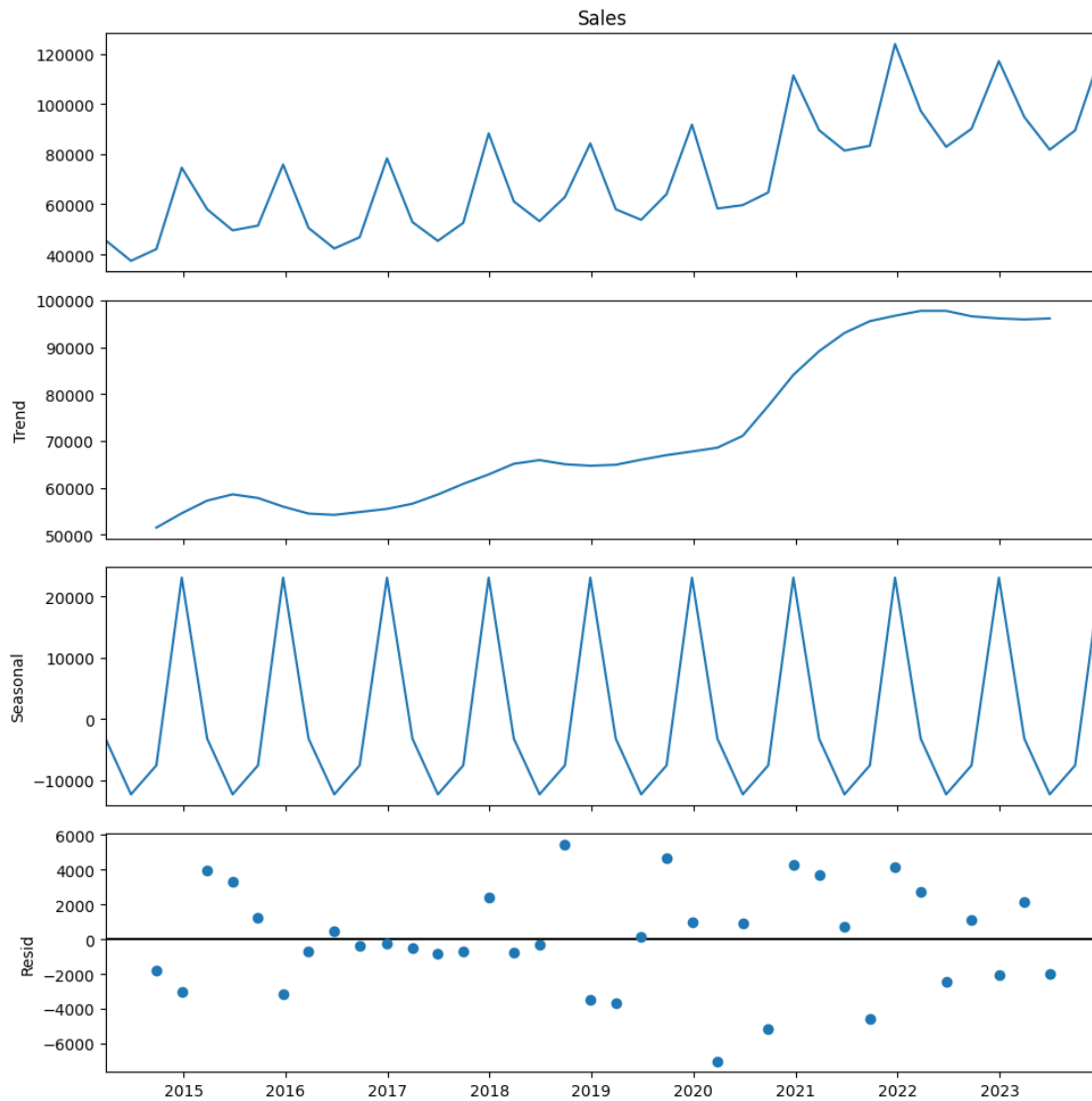
```
[157]: goog.set_index('Date', inplace=True)
```

```
[158]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
    ↪parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Sales'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

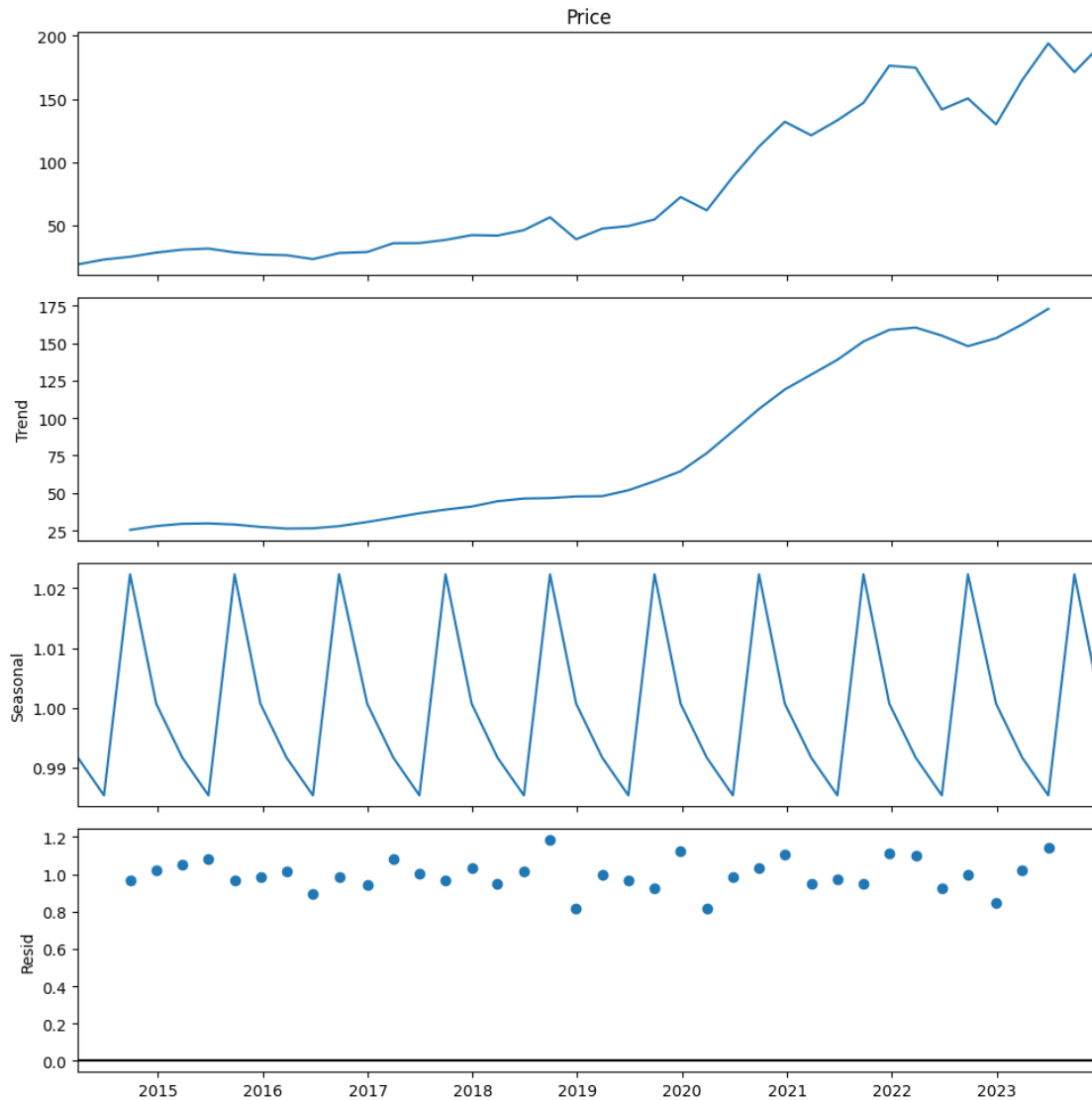


```
[159]: result = seasonal_decompose(goog['Sales'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
```

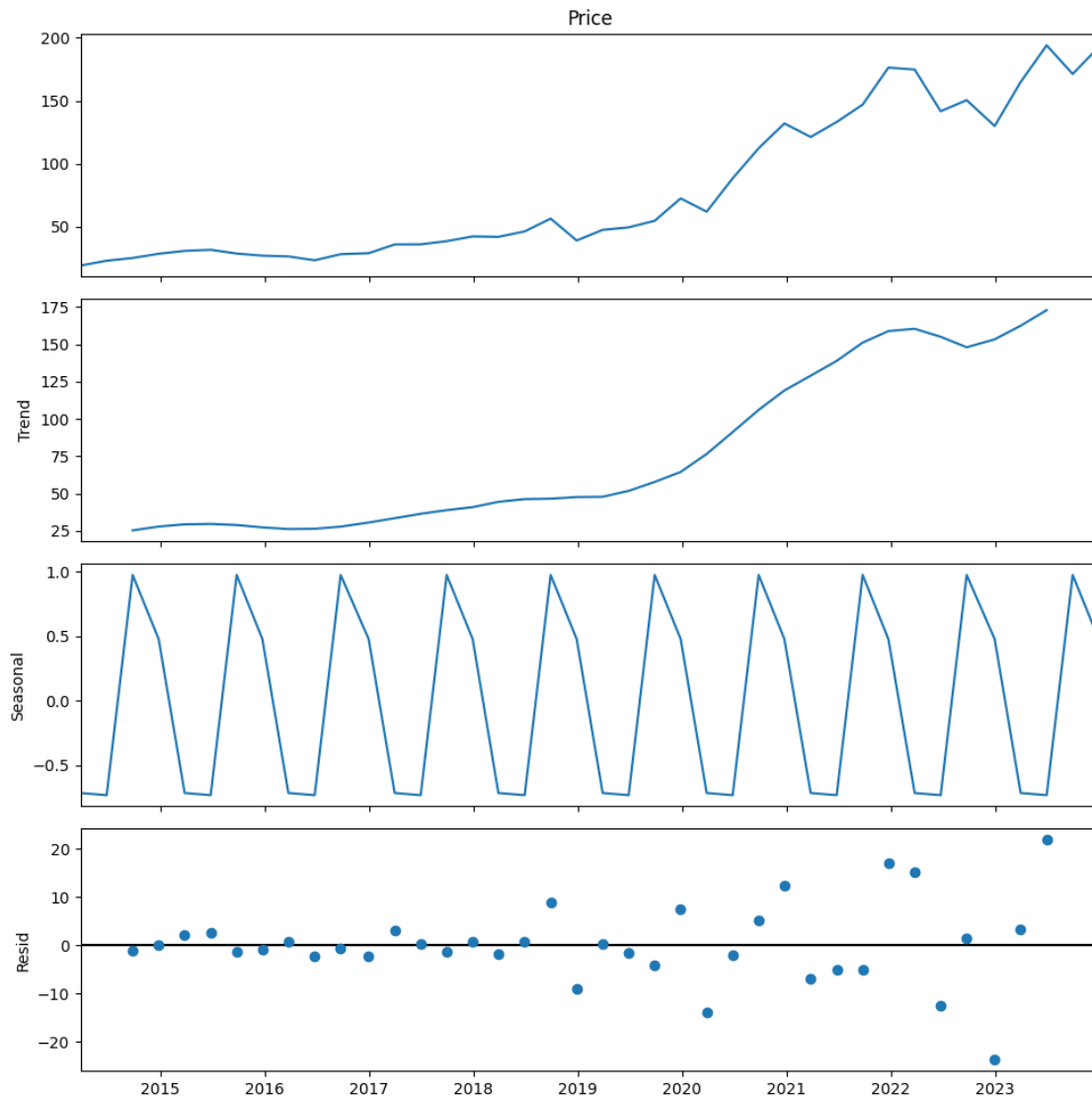
```
result.plot()
#plt.title('Time Series Decomposition of Sales')
plt.show()
```



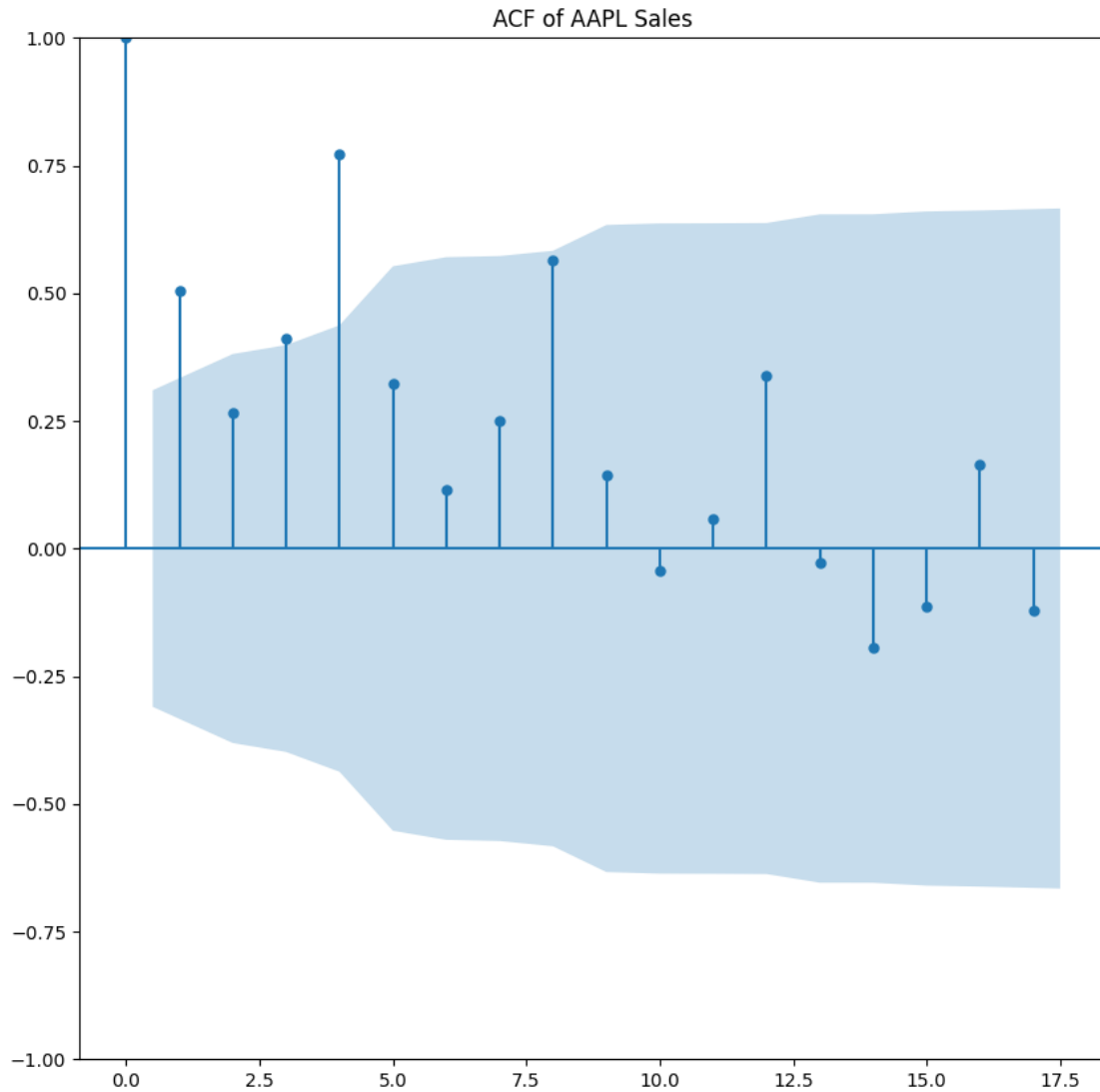
```
[160]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
        ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Price'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

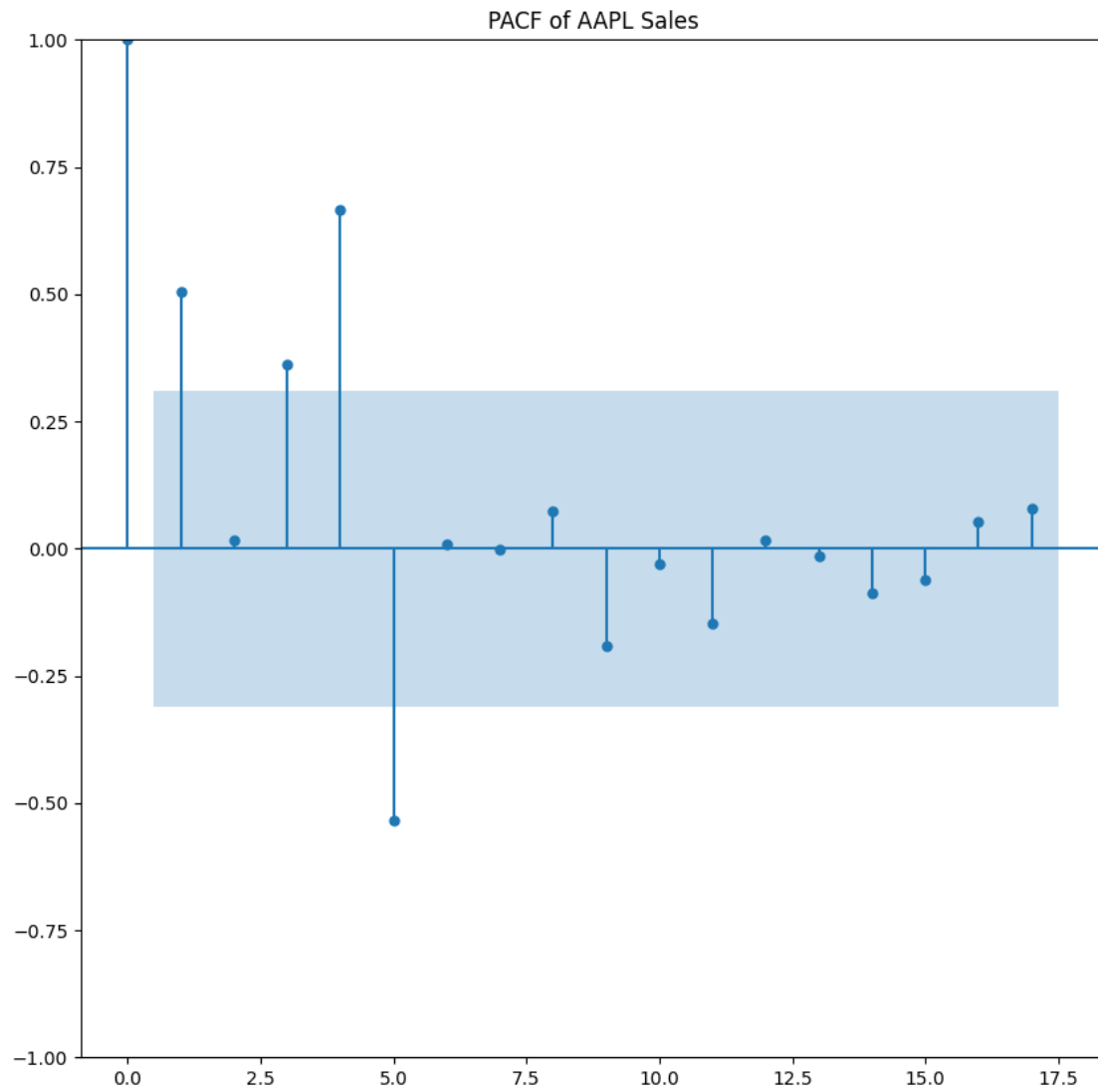
```
[161]: result = seasonal_decompose(goog['Price'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



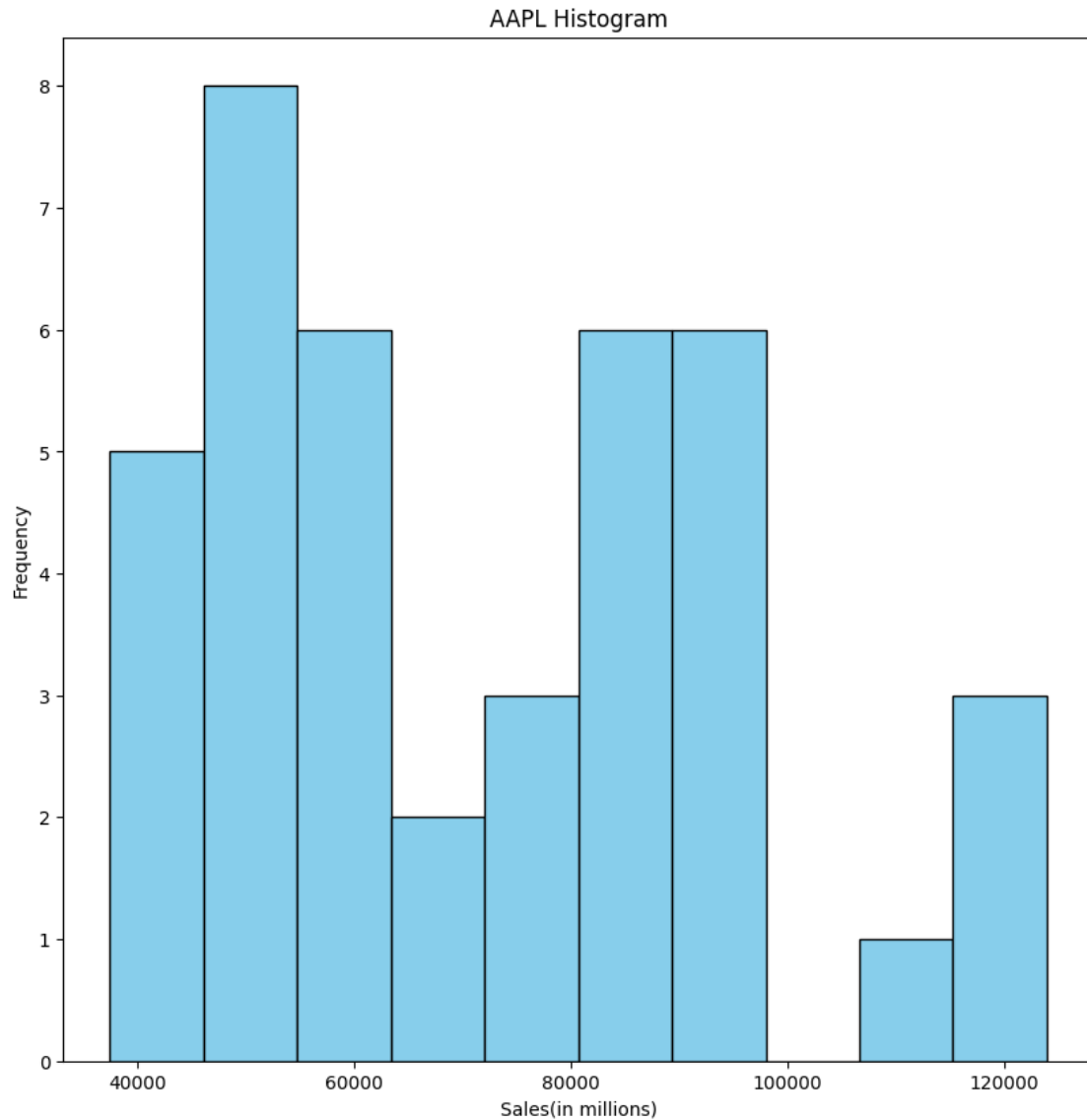
```
[162]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_acf(goog['Sales'], title='ACF of AAPL Sales')
pyplot.show()
```



```
[163]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_pacf(goog['Sales'], title='PACF of AAPL Sales')
pyplot.show()
```



```
[164]: plt.hist(goog['Sales'], color='skyblue', edgecolor='black')
plt.xlabel('Sales(in millions)')
plt.ylabel('Frequency')
plt.title('AAPL Histogram')
pyplot.show()
```



```
[165]: # statistical test
result = adfuller(goog['Sales'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

```
ADF Statistic: -0.518770
p-value: 0.888301
Critical Values:
1%: -3.633
```

```
5%: -2.949
10%: -2.613
```

```
[166]: # statistical test
result = adfuller(goog['Price'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

```
ADF Statistic: 2.614015
p-value: 0.999078
Critical Values:
1%: -3.679
5%: -2.968
10%: -2.623
```

4.5 AAPL Conclusion

Description of Time series (is it stationary, are there any trends, seasonality etc)

For AAPL, the results are interesting compared to the first three we've analyzed. You can see there is clearly a trend higher in both price and sales over the last 10 years. It's interesting to note seasonality in Sales. Second quarter is lowest quarter possibly due to sales mostly coming around the holidays, back to school and Chinese New Year. This can be seen in both the Average Sales per Quarter chart and the time series Decomposition. The additive model seems to be more appropriate for the Sales data while the multiplicative approach seems to be more appropriate for the stock price data.

This is shown in both time series decomposition charts. The null hypothesis of the ADF test is the time series is Non Stationary. We conclude AAPL is Non Stationary both when looking at Sales and Price since the test statistic is larger than the critical values.

AAPL does show autocorrelation at 1, 2 and 5 lags.

```
[ ]:
```

4.5.1 Analysis of MSFT Stock and Fundamentals over last 10 years

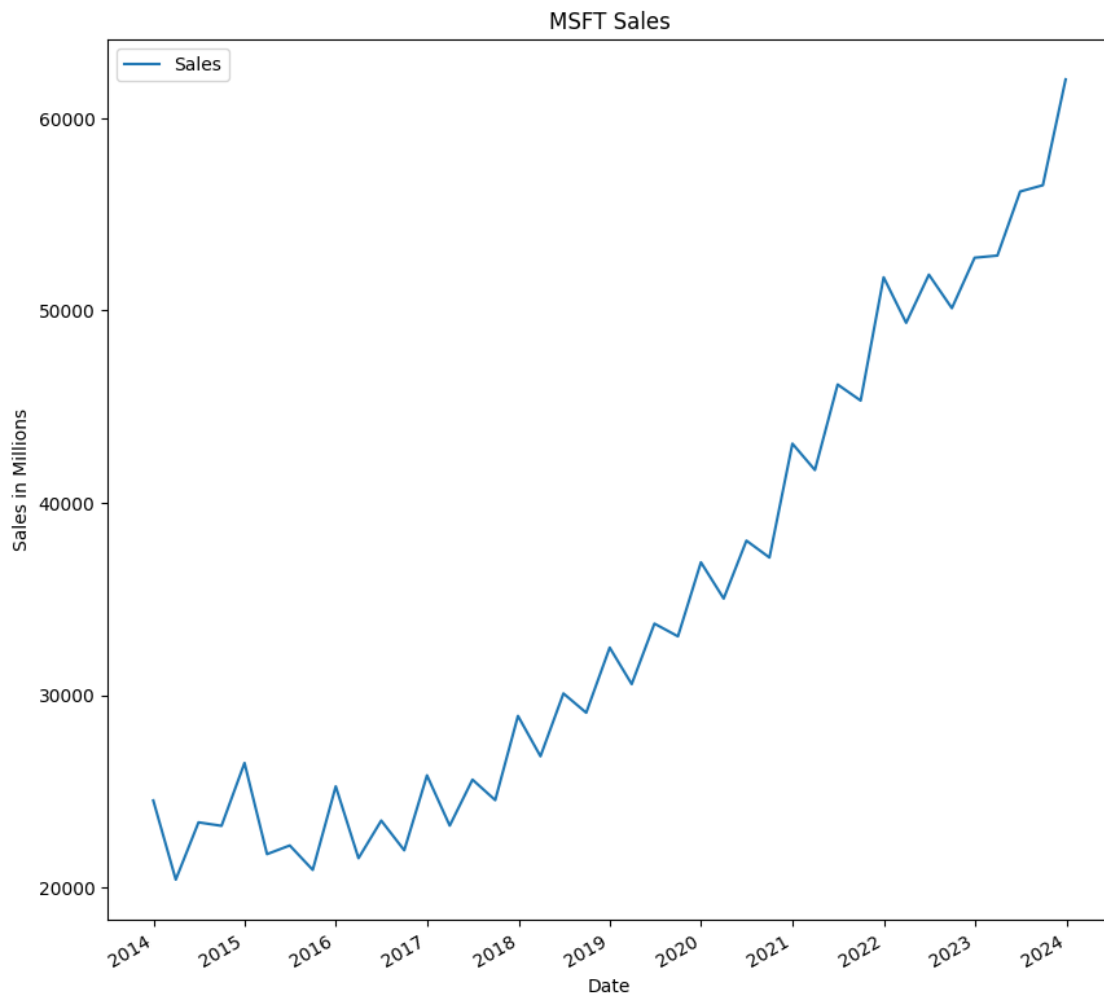
```
[168]: #goog = pd.read_csv('GOOGL.csv', header=0, index_col=0,
    ↪ parse_dates=True, squeeze=True)
goog = pd.read_csv('./Aux_data/msft.csv')
```

```
[169]: goog.columns
```

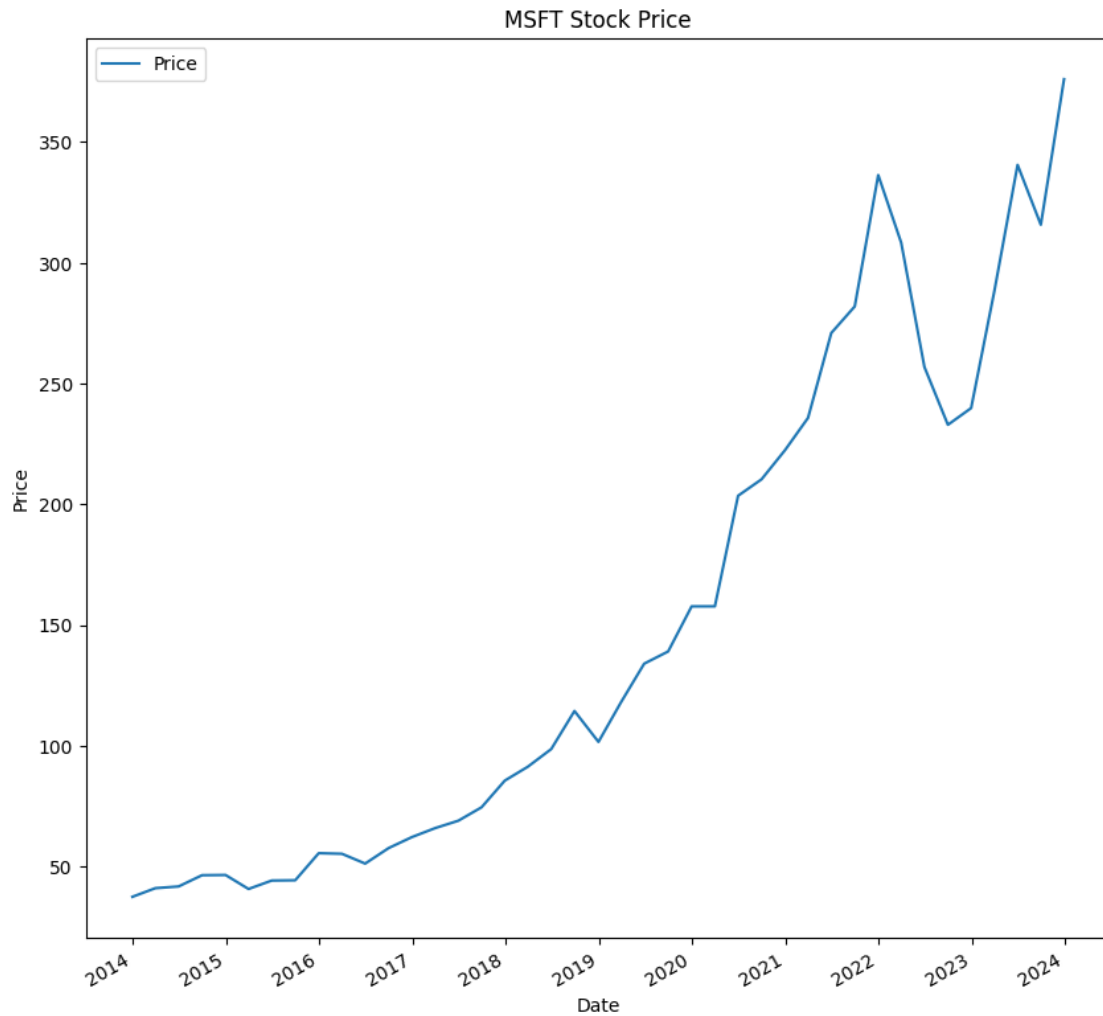
```
[169]: Index(['Date', 'Sales', 'FreeCashFlow', 'Cash', 'Long Term Debt', 'Price'],
dtype='object')
```

```
[170]: goog['Date'] = pd.to_datetime(goog['Date'], format='%m/%d/%Y')
goog['Year'] = goog['Date'].dt.year
goog['Quarter'] = goog['Date'].dt.quarter
```

```
[171]: goog.plot('Date', 'Sales')
plt.title('MSFT Sales')
plt.ylabel('Sales in Millions')
plt.show()
```

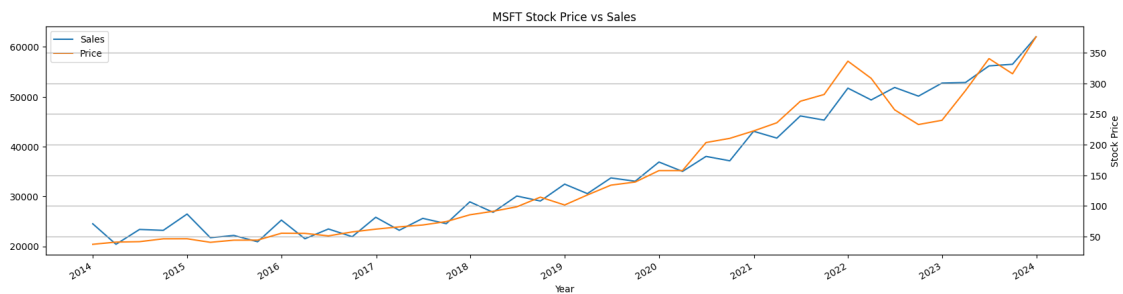


```
[172]: goog.plot('Date', 'Price')
plt.title('MSFT Stock Price')
plt.ylabel('Price')
plt.show()
```



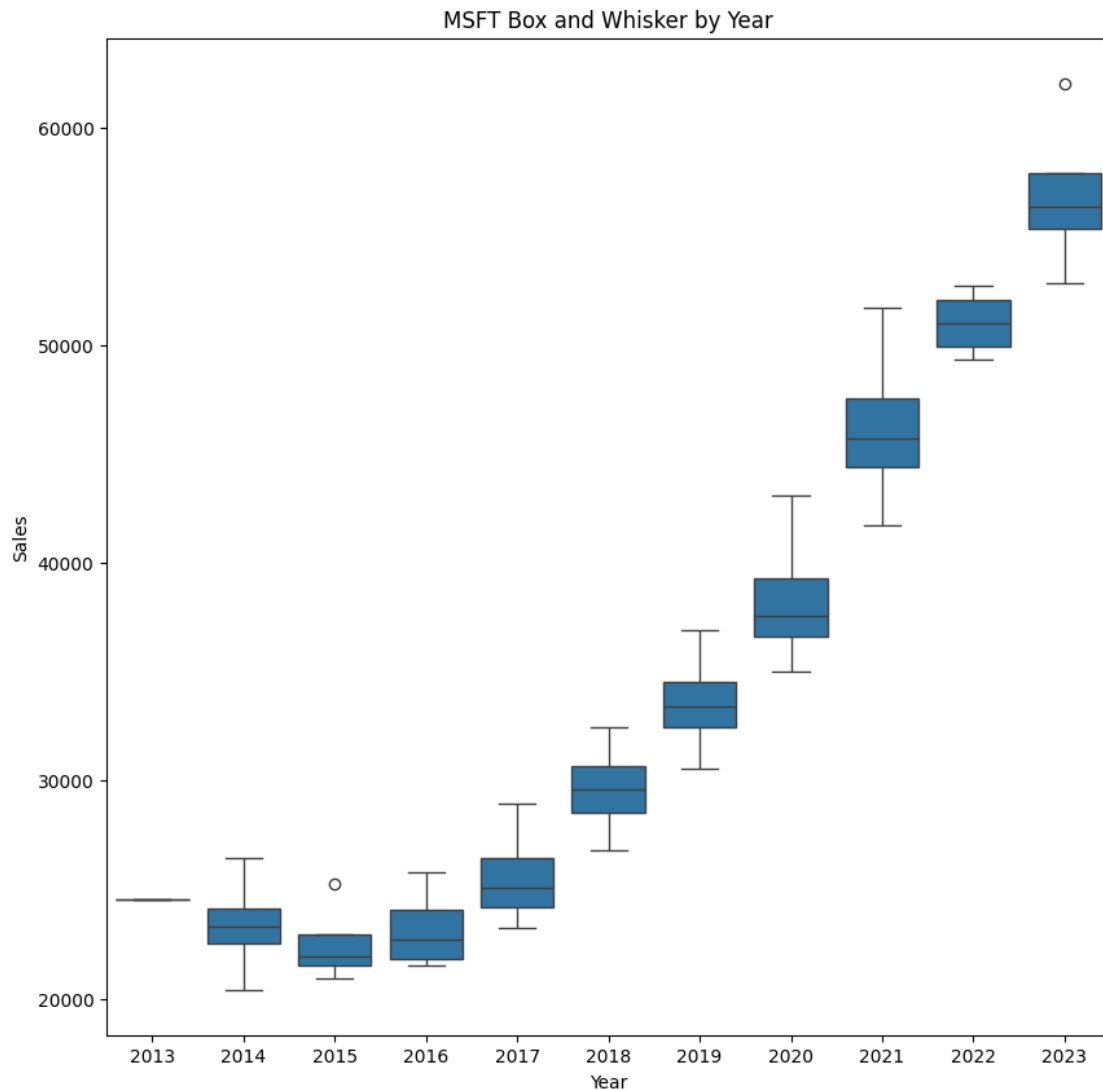
```
[173]: goog.loc[:,['Date','Sales','Price']].plot(x = 'Date',secondary_y=['Price'],
↪xlabel = 'Year', ylabel = 'Stock Price',sharex = True,mark_right=False,
↪figsize = (20,5), grid=True)
plt.title('MSFT Stock Price vs Sales')
```

```
[173]: Text(0.5, 1.0, 'MSFT Stock Price vs Sales')
```




```
[174]: sn.boxplot(x='Year',y='Sales', data = goog).set(title = 'MSFT Box and Whisker_
↳by Year')
```

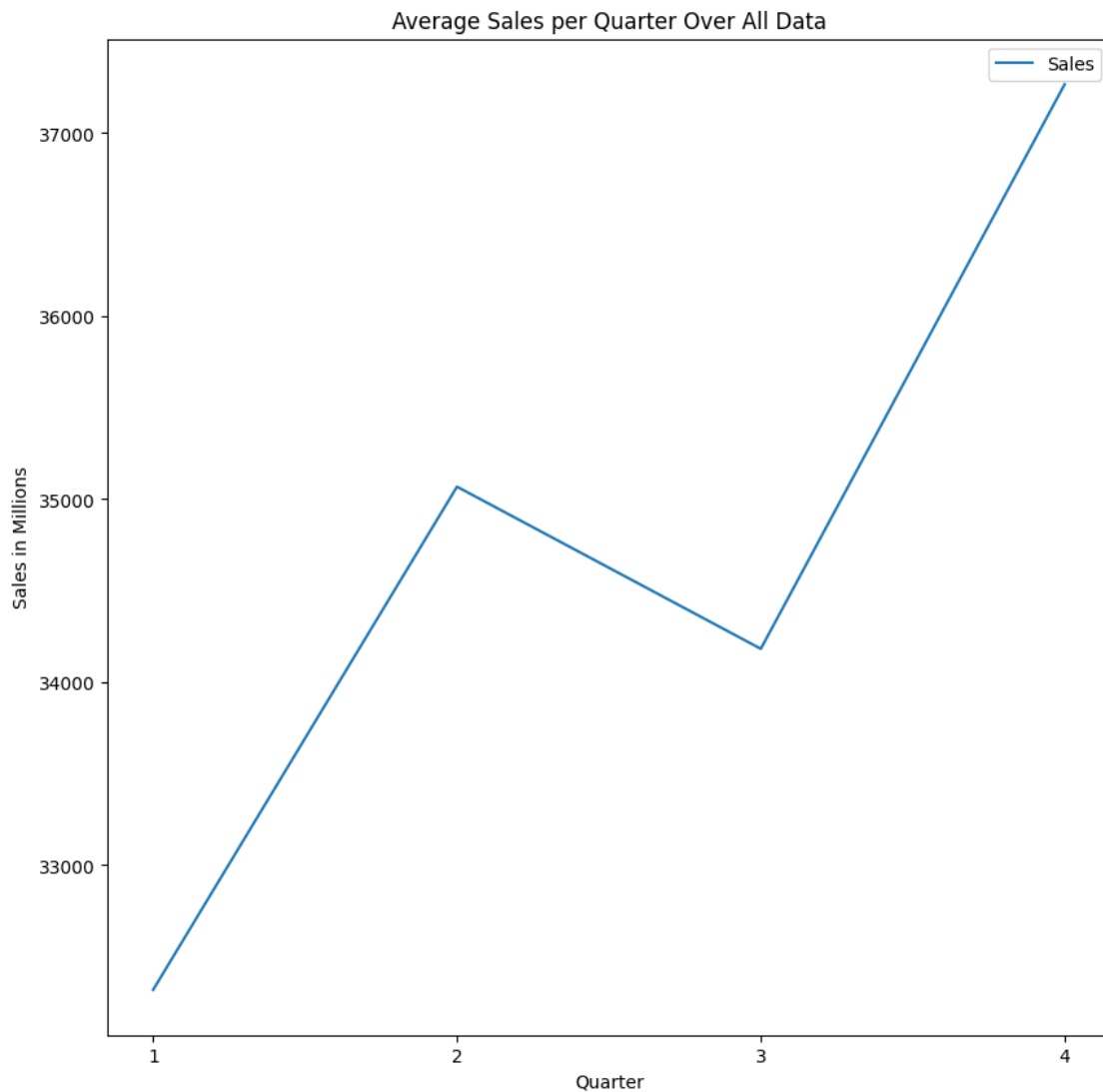
```
[174]: [Text(0.5, 1.0, 'MSFT Box and Whisker by Year')]
```



```
[175]: avgperqtr = goog[['Quarter', 'Sales']].groupby('Quarter').mean().reset_index()
```

```
[176]: avgperqtr.plot('Quarter','Sales')
plt.ylabel('Sales in Millions')
plt.title('Average Sales per Quarter Over All Data')
plt.xticks(goog.Quarter.unique())
```

```
plt.show()
```

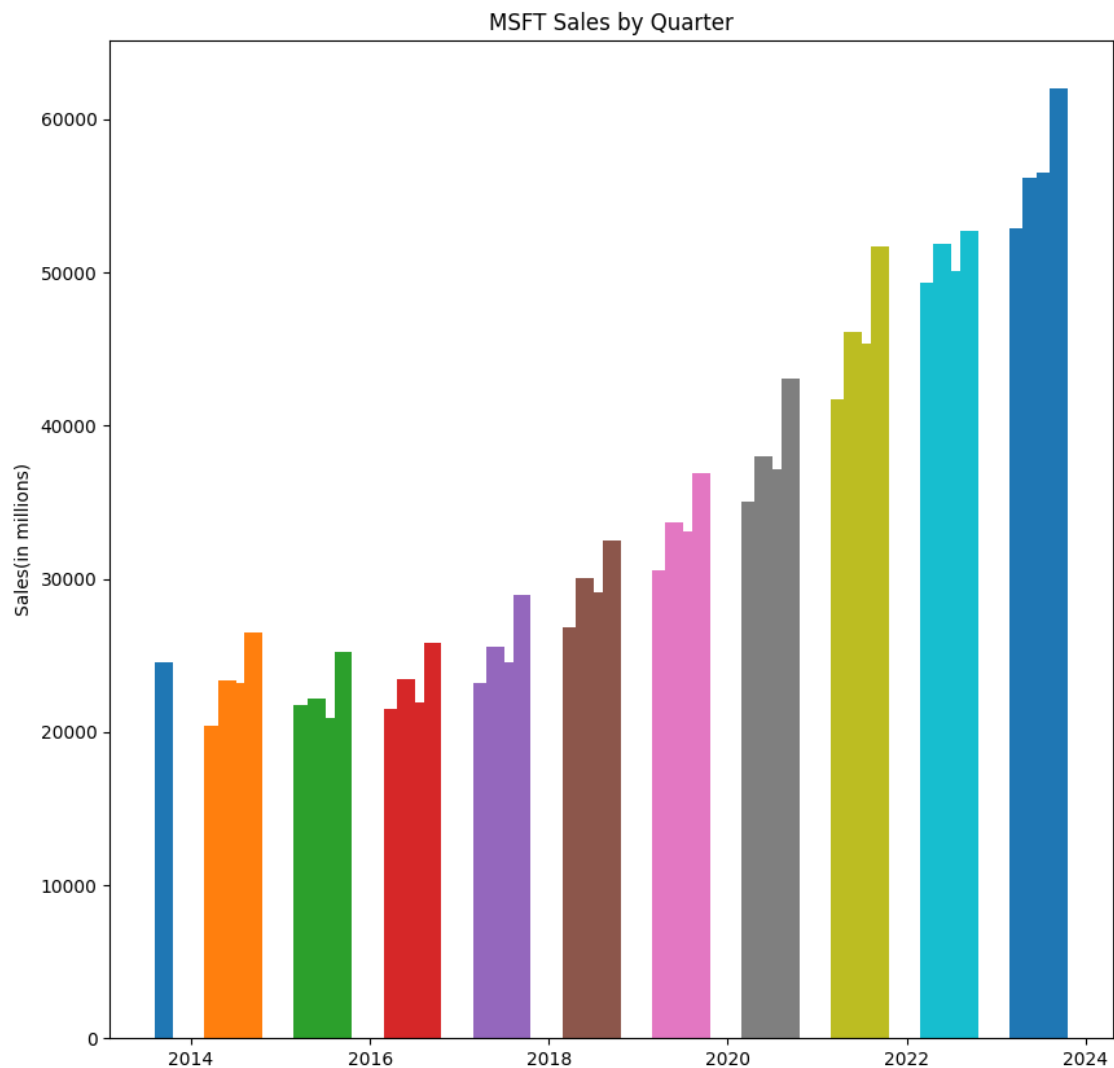


```
[177]: goog1 = pd.DataFrame(goog, columns=['Year', 'Quarter', 'Sales'])
```

```
[178]: dictyear = {}  
saleslist = []  
for year in goog1['Year'].unique():  
    #print(year)  
    #byyear = goog1[goog1['Year'] == year]  
    #saleslist = saleslist.append(byyear.Sales)  
    dictyear[year] = list(goog1[goog1['Year'] == year].Sales)
```

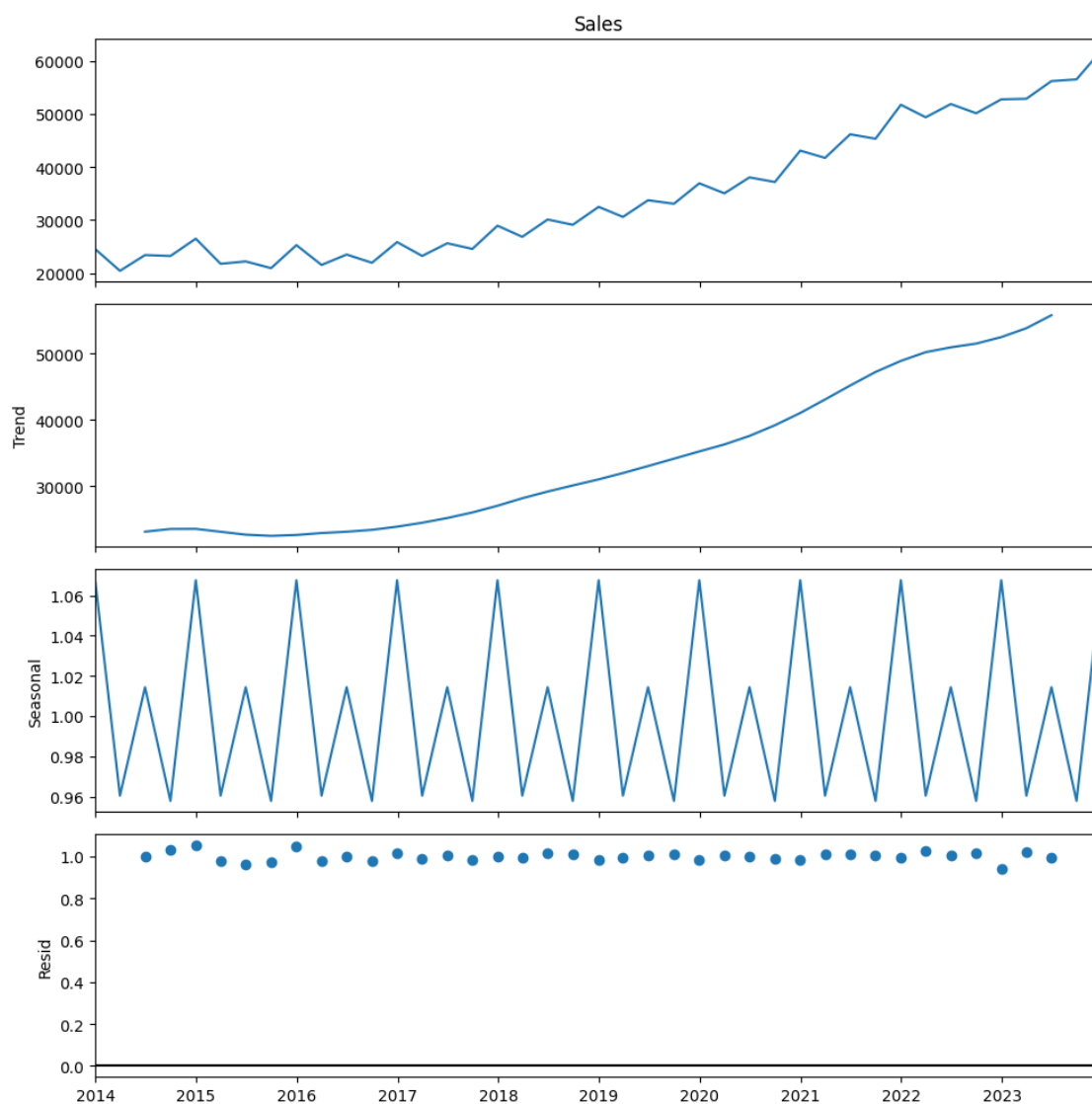
```
[179]: #adopted from https://medium.com/@BetterEverything/
↳graph-quarterly-data-with-python-6f38bcc6b7fe
for year, data in dictyear.items():
    quarters = list(goog1.loc[(goog1['Year'] == year)].Quarter)
    x_coordinates = [year + 0.15 * q for q in quarters]
    #print(x_coordinates, data)
    plt.bar(x_coordinates, data, width=0.2, align='edge')
plt.title('MSFT Sales by Quarter')
plt.ylabel('Sales(in millions)')

#plt.xticks(labels=[np.unique(goog1['Year'].values)], ticks = [])
#print(plt.xticks())
plt.show()
```



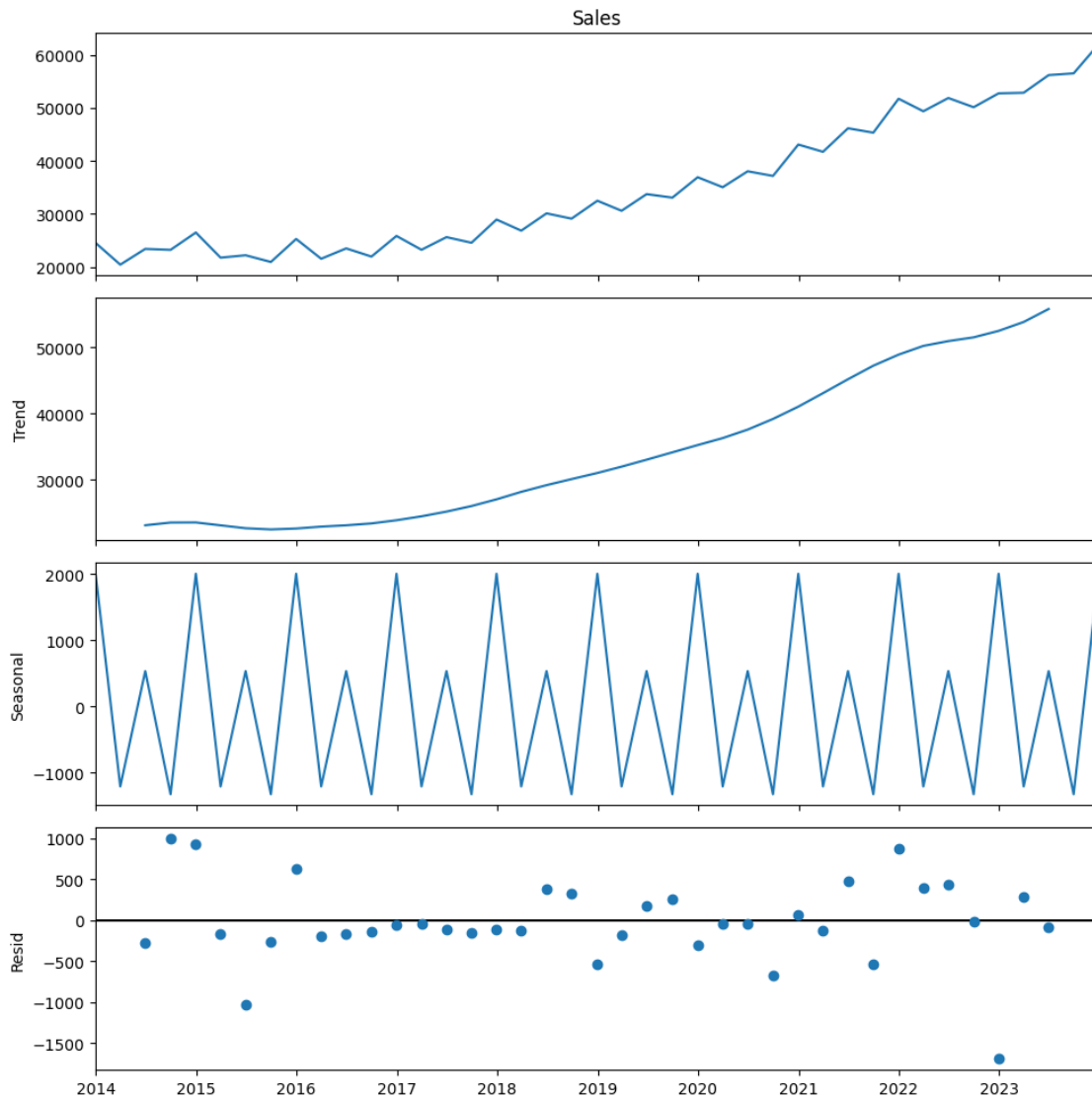
```
[180]: goog.set_index('Date', inplace=True)
```

```
[181]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
    ↪parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Sales'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

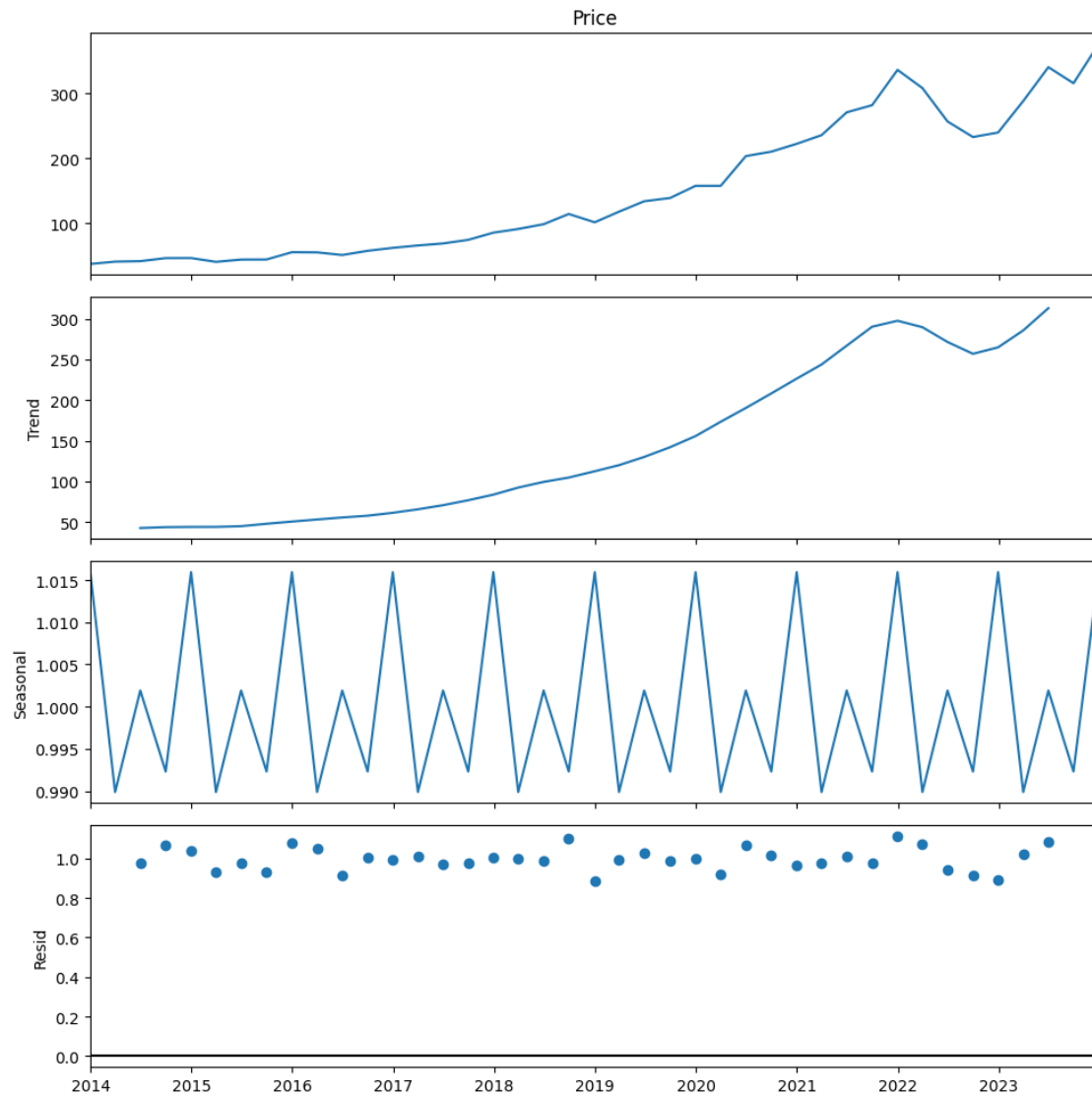


```
[182]: result = seasonal_decompose(goog['Sales'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
```

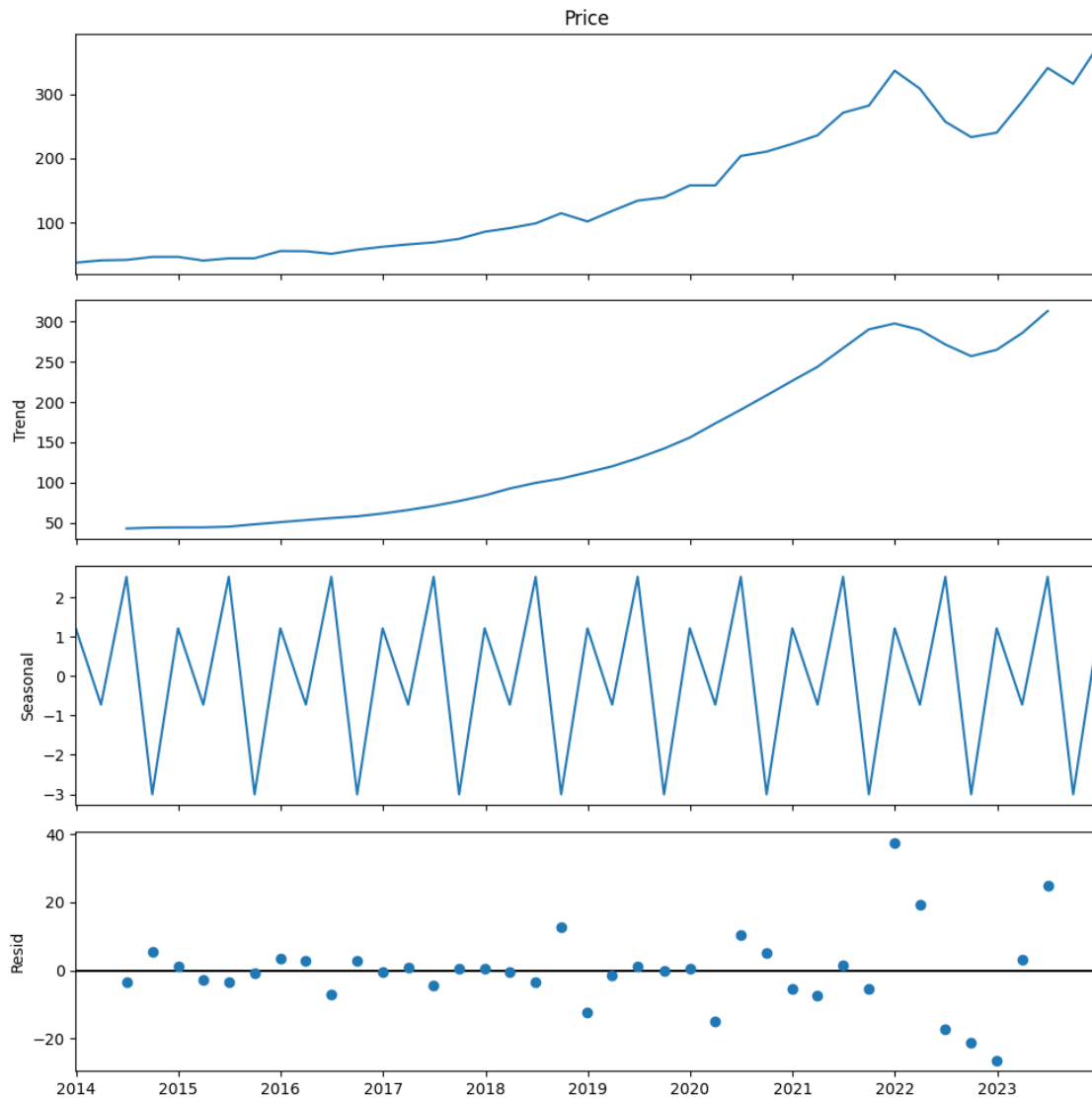
```
result.plot()
#plt.title('Time Series Decomposition of Sales')
plt.show()
```



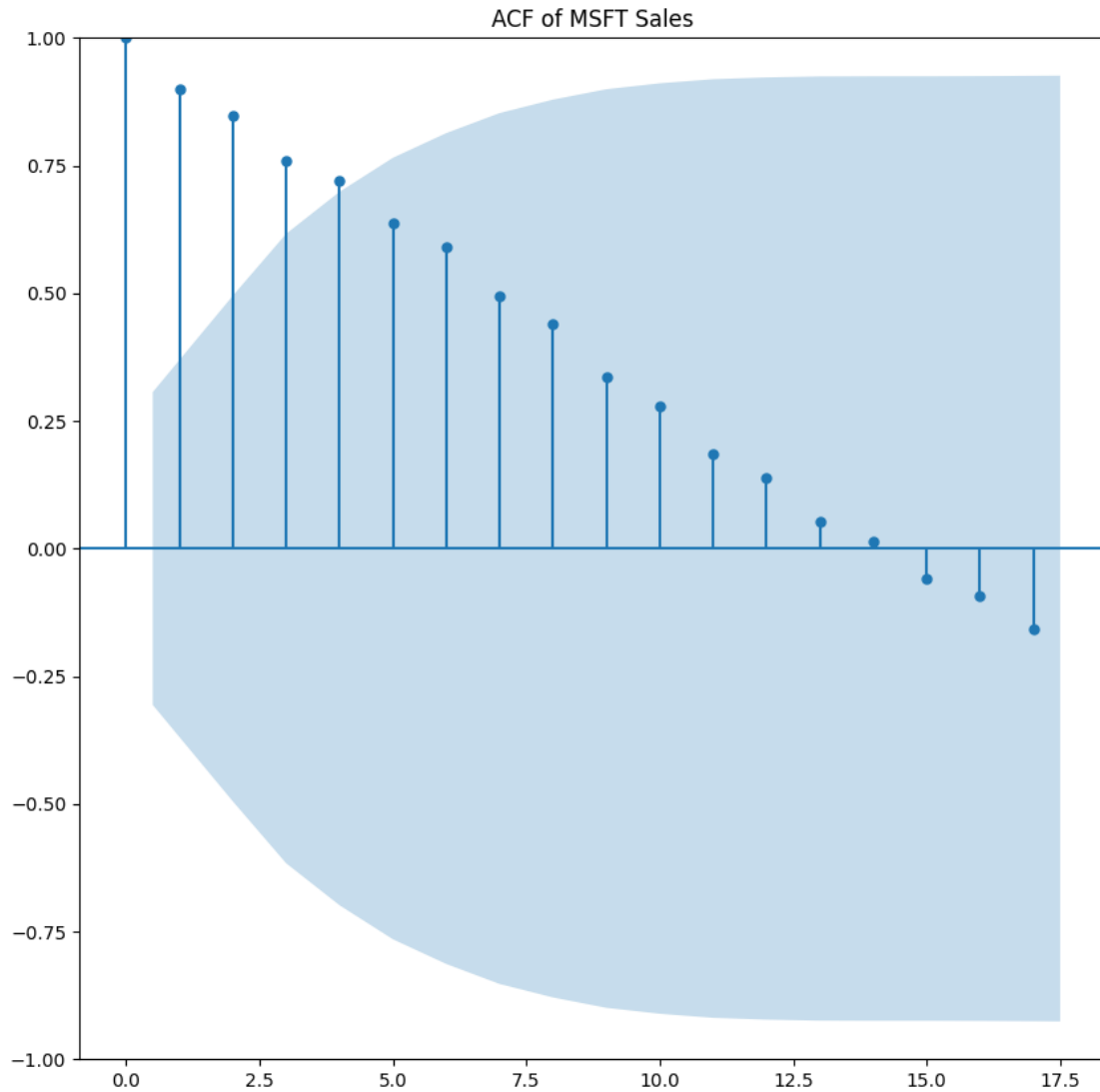
```
[183]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
        ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Price'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



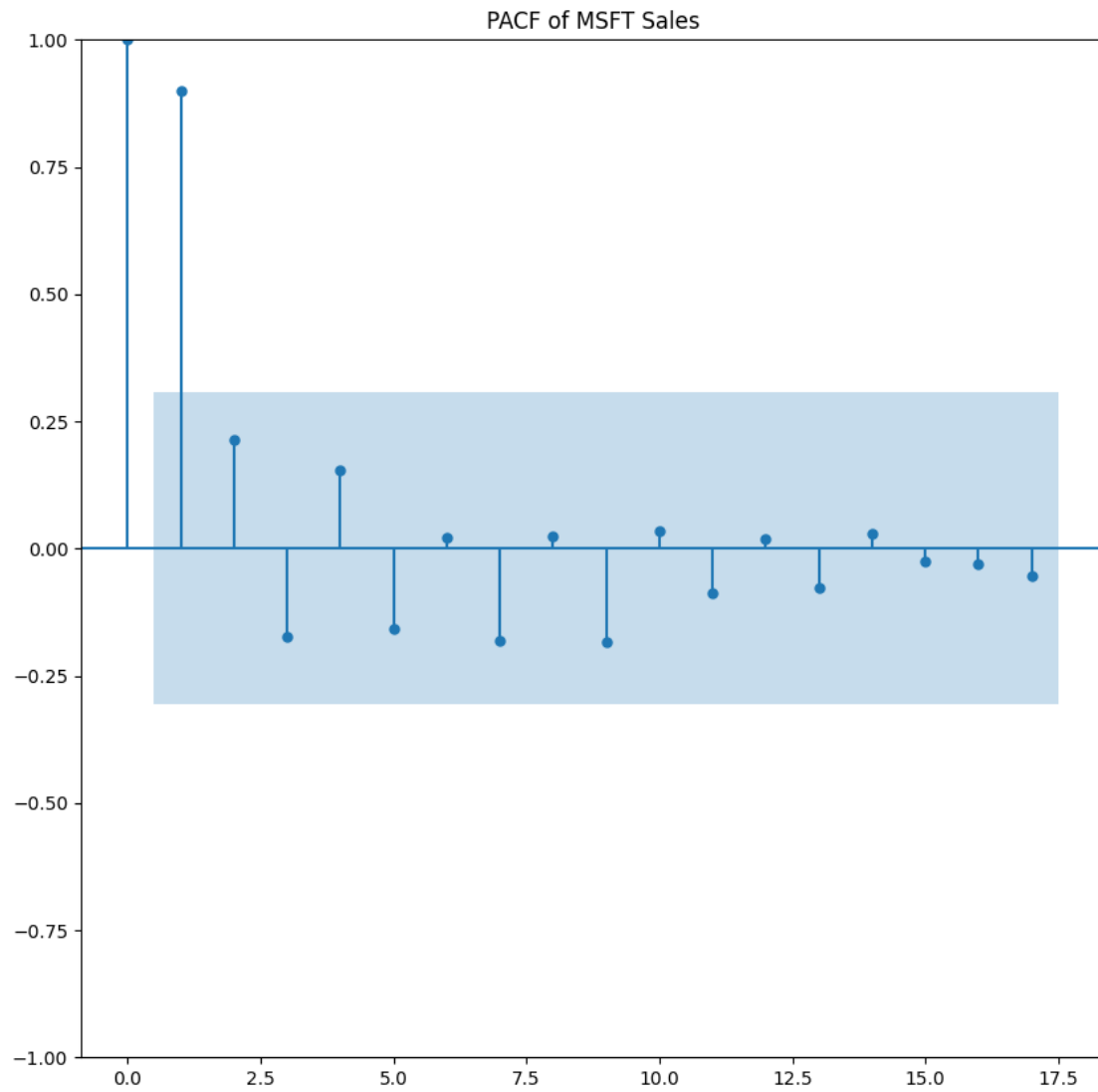
```
[184]: result = seasonal_decompose(goog['Price'], model='additive',period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



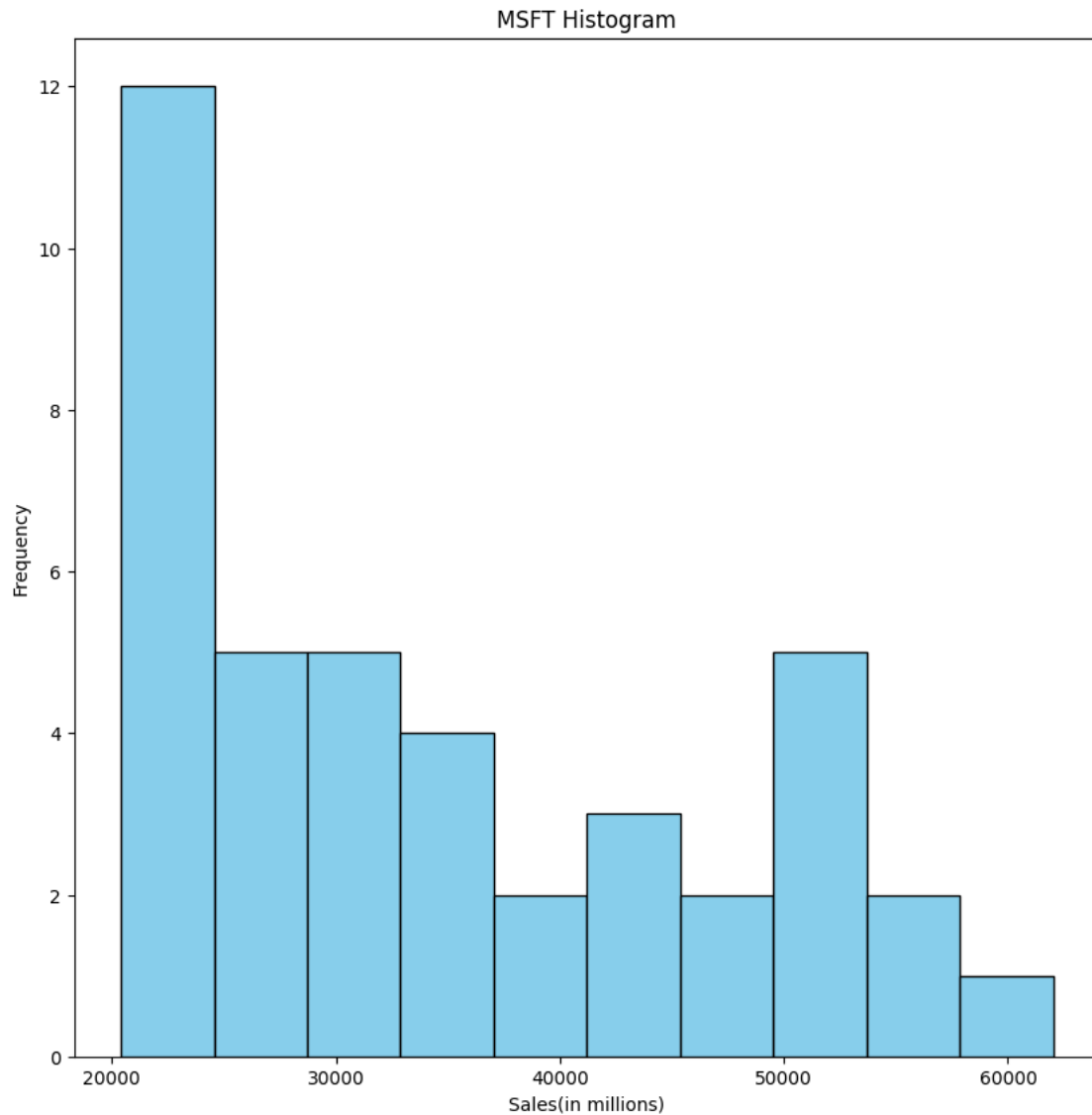
```
[185]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_acf(goog['Sales'], title='ACF of MSFT Sales')
pyplot.show()
```



```
[186]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_pacf(goog['Sales'], title='PACF of MSFT Sales')
pyplot.show()
```

```
[187]: plt.hist(goog['Sales'], color='skyblue', edgecolor='black')
plt.xlabel('Sales(in millions)')
plt.ylabel('Frequency')
plt.title('MSFT Histogram')
pyplot.show()
```



```
[188]: # statistical test
result = adfuller(goog['Sales'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 1.150560

p-value: 0.995626

Critical Values:

1%: -3.654

5%: -2.957

10%: -2.618

```
[189]: # statistical test
result = adfuller(goog['Price'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 2.103584

p-value: 0.998793

Critical Values:

1%: -3.646

5%: -2.954

10%: -2.616

4.6 MSFT Conclusion

Description of Time series (is it stationary, are there any trends, seasonality etc)

For MSFT, you can see there is clearly a very steady trend higher in Sales over the last 10 years. Price trending higher as well. The seasonality is interesting. 1st quarter definitely the lowest but a stall of sales growth between the 2nd and 4th quarters. The fourth quarter is massively higher compared to the first 3 quarters. The additive model seems to be more appropriate for the Sales and Stock price data as the residuals in each case appear like white noise in the additive model but have more of a pattern in the multiplicative model. This is shown in both time series decomposition charts.

The null hypothesis of the ADF test is the time series is Non Stationary. We conclude MSFT is Non Stationary both when looking at Sales and Price since the test statistic is larger than the critical values.

MSFT does show autocorrelation out to about 5 lags.

[]:

4.6.1 Analysis of META Stock and Fundamentals over last 10 years

```
[190]: #goog = pd.read_csv('GOOGL.csv', header=0, index_col=0,
    ↪ parse_dates=True, squeeze=True)
goog = pd.read_csv('./Aux_data/meta.csv')
```

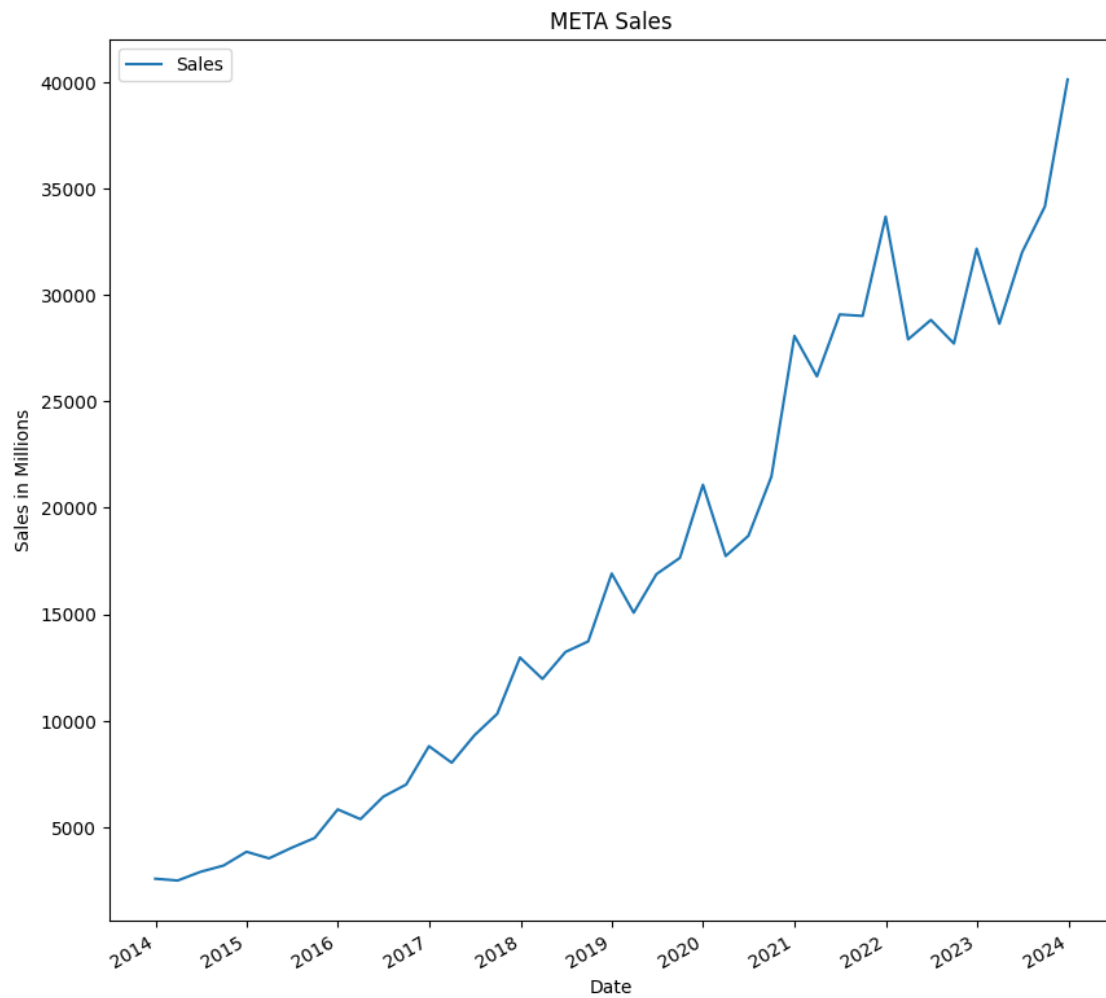
```
[191]: goog.columns
```

```
[191]: Index(['Date', 'Sales', 'FreeCashFlow', 'Cash', ' Long Term Debt ', 'Price'],
      dtype='object')
```

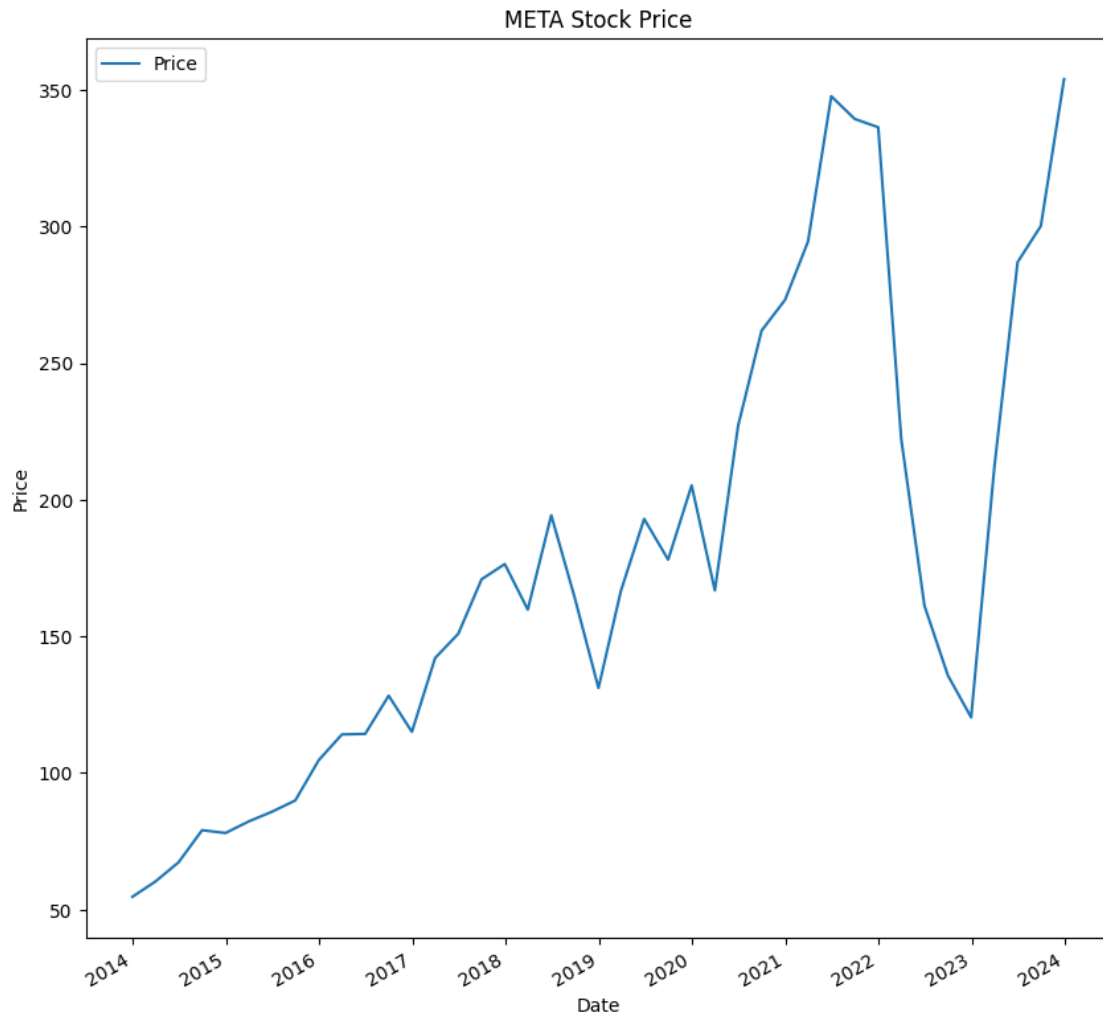
```
[192]: goog['Date'] = pd.to_datetime(goog['Date'], format='%m/%d/%Y')
goog['Year'] = goog['Date'].dt.year
```

```
goog['Quarter'] = goog['Date'].dt.quarter
```

```
[193]: goog.plot('Date', 'Sales')  
plt.title('META Sales')  
plt.ylabel('Sales in Millions')  
plt.show()
```

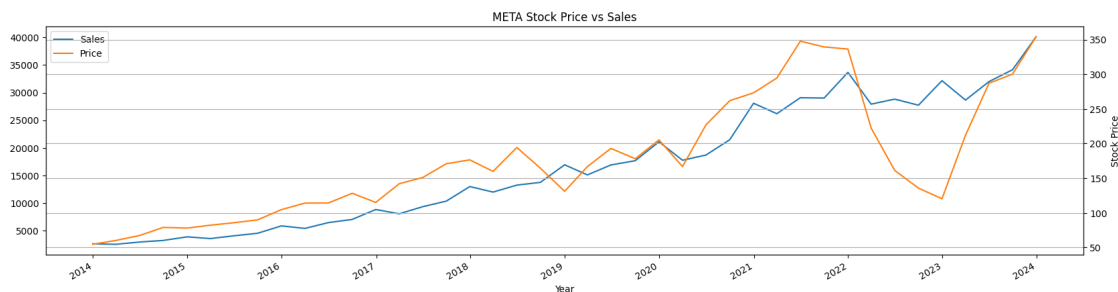


```
[194]: goog.plot('Date', 'Price')  
plt.title('META Stock Price')  
plt.ylabel('Price')  
plt.show()
```



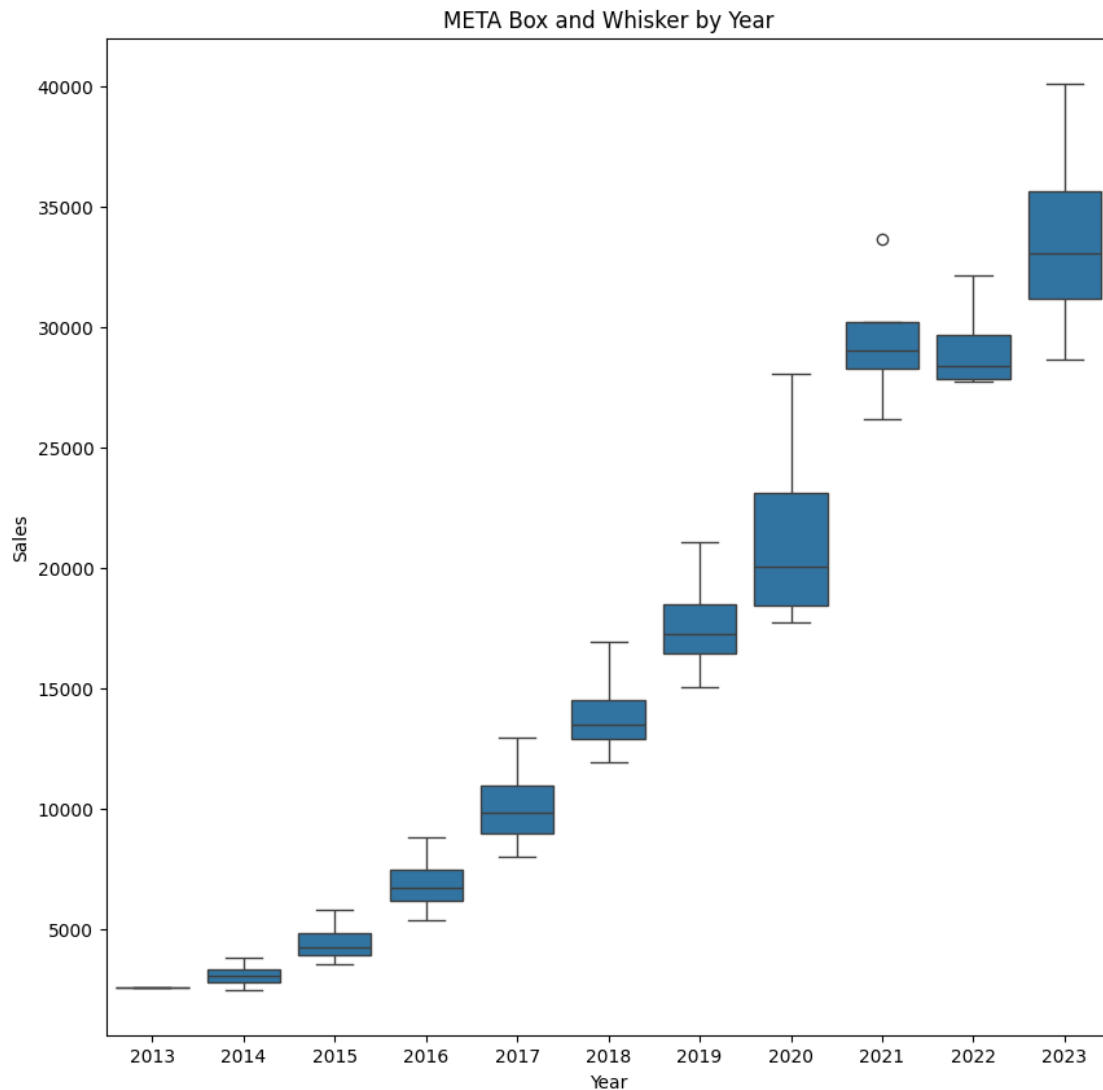
```
[195]: goog.loc[:,['Date','Sales','Price']].plot(x = 'Date',secondary_y=['Price'],
↪xlabel = 'Year', ylabel = 'Stock Price',sharex = True,mark_right=False,
↪figsize = (20,5), grid=True)
plt.title('META Stock Price vs Sales')
```

```
[195]: Text(0.5, 1.0, 'META Stock Price vs Sales')
```



```
[196]: sn.boxplot(x='Year',y='Sales', data = goog).set(title = 'META Box and Whisker_
↳by Year')
```

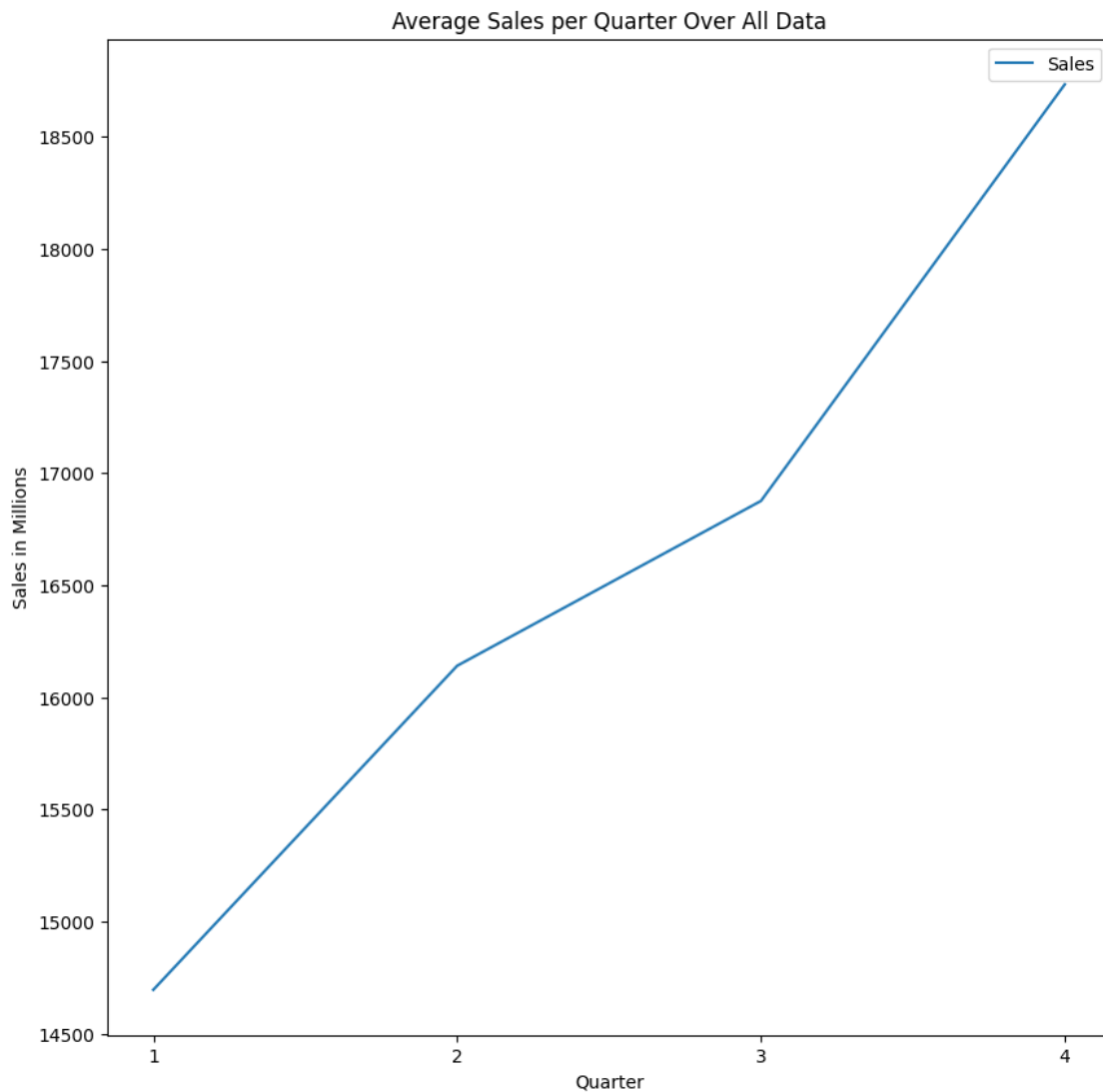
```
[196]: [Text(0.5, 1.0, 'META Box and Whisker by Year')]
```



```
[197]: avgperqtr = goog[['Quarter', 'Sales']].groupby('Quarter').mean().reset_index()
```

```
[198]: avgperqtr.plot('Quarter','Sales')
plt.ylabel('Sales in Millions')
plt.title('Average Sales per Quarter Over All Data')
plt.xticks(goog.Quarter.unique())
```

```
plt.show()
```

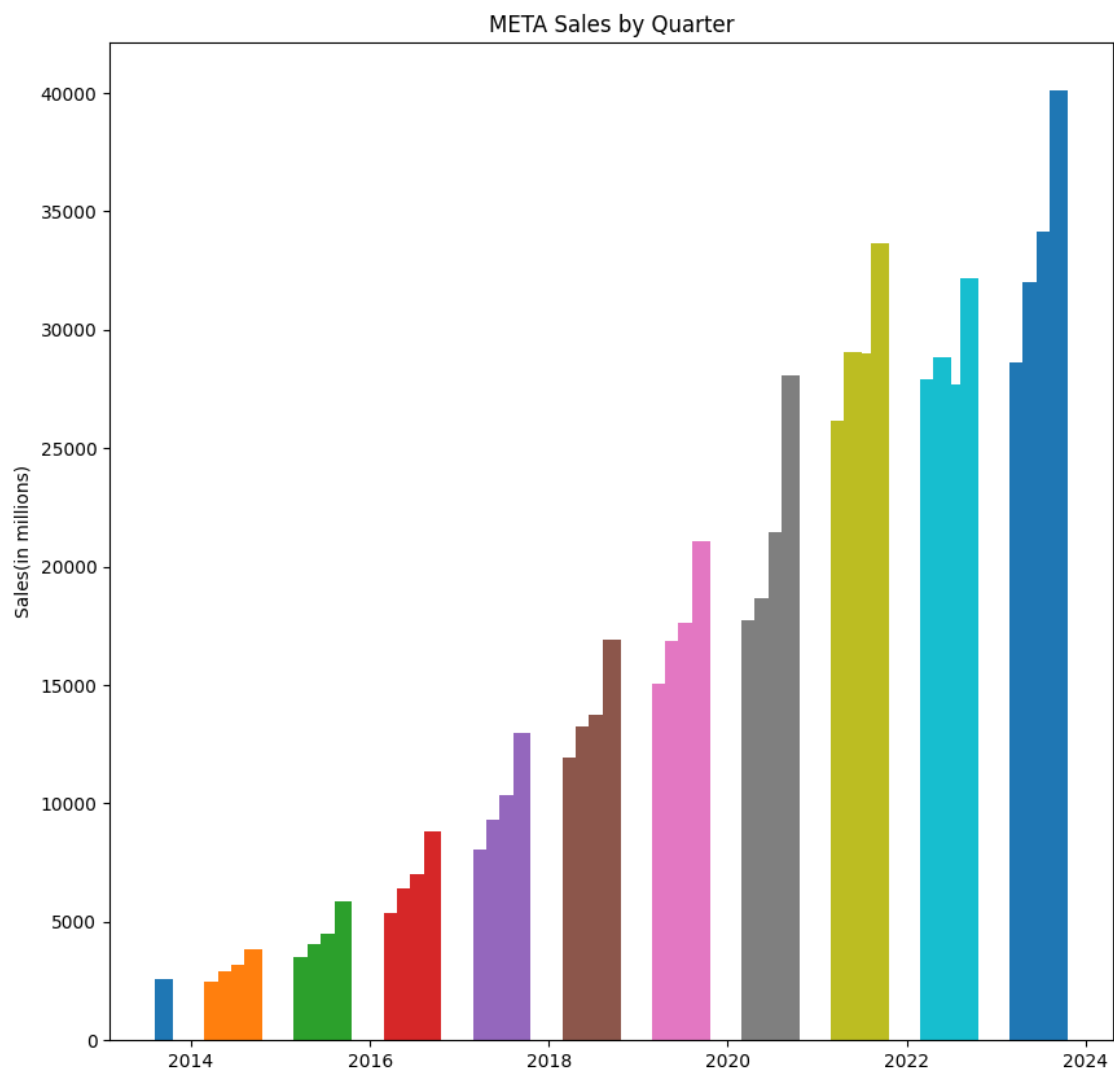


```
[199]: goog1 = pd.DataFrame(goog, columns=['Year', 'Quarter', 'Sales'])
```

```
[200]: dictyear = {}  
saleslist = []  
for year in goog1['Year'].unique():  
    #print(year)  
    #byyear = goog1[goog1['Year'] == year]  
    #saleslist = saleslist.append(byyear.Sales)  
    dictyear[year] = list(goog1[goog1['Year'] == year].Sales)
```

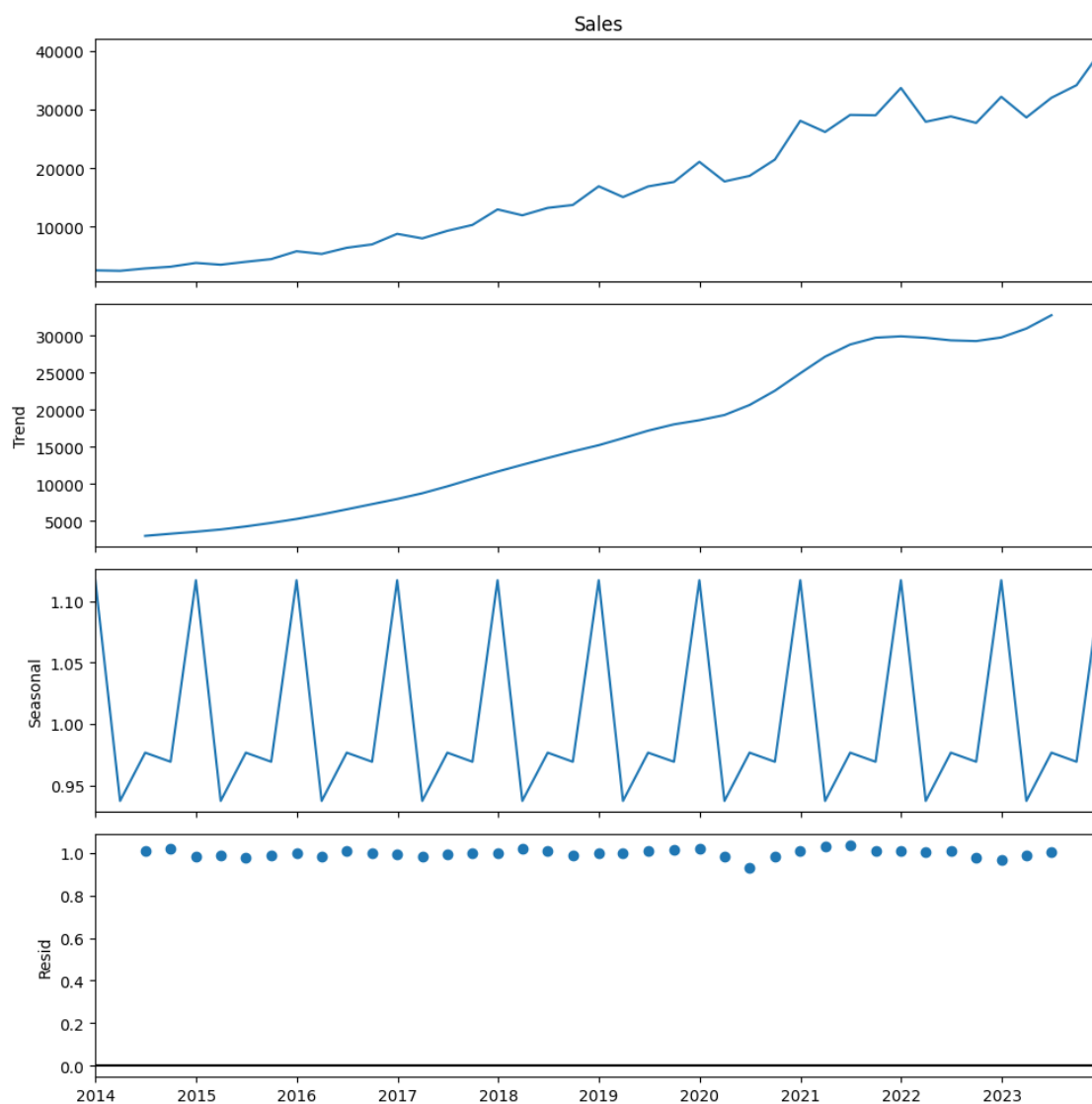
```
[201]: #adopted from https://medium.com/@BetterEverything/
↳graph-quarterly-data-with-python-6f38bcc6b7fe
for year, data in dictyear.items():
    quarters = list(goog1.loc[(goog1['Year'] == year)].Quarter)
    x_coordinates = [year + 0.15 * q for q in quarters]
    #print(x_coordinates, data)
    plt.bar(x_coordinates, data, width=0.2, align='edge')
plt.title('META Sales by Quarter')
plt.ylabel('Sales(in millions)')

#plt.xticks(labels=[np.unique(goog1['Year'].values)], ticks = [])
#print(plt.xticks())
plt.show()
```



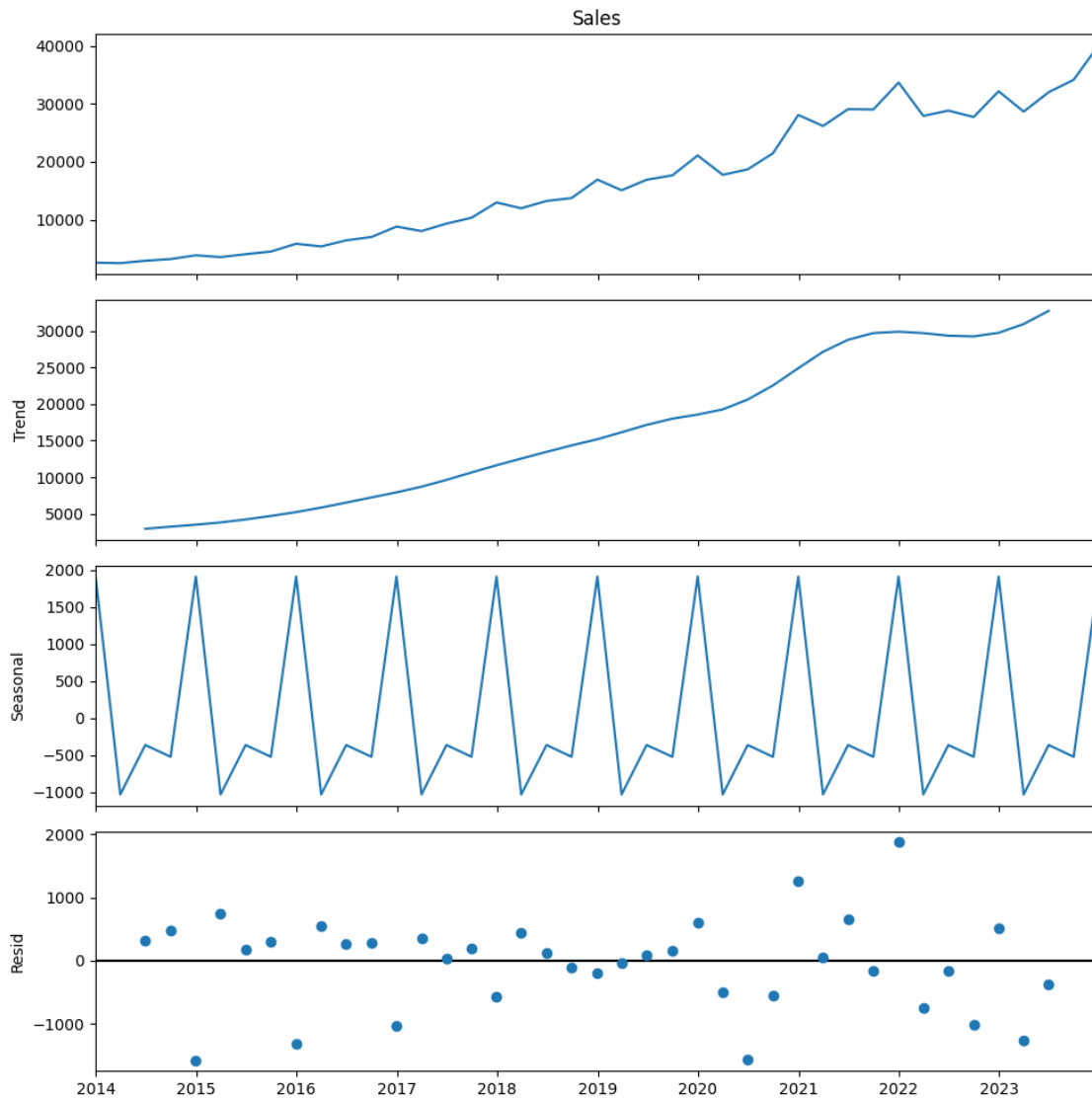

```
[202]: goog.set_index('Date', inplace=True)
```

```
[203]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
#                    ↪parse_dates=True,squeeze=True)
result = seasonal_decompose(goog['Sales'], model='multiplicative',period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

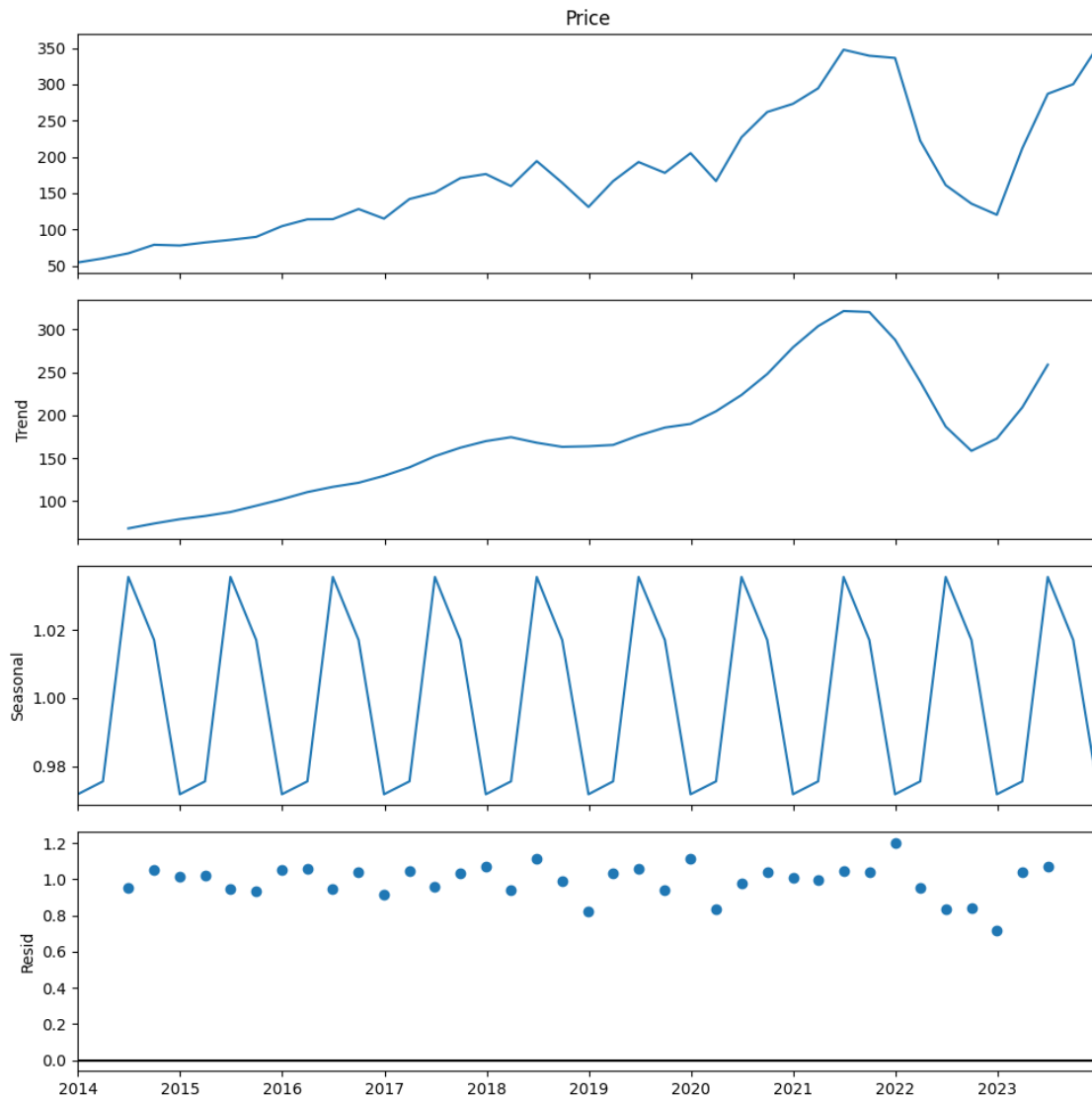


```
[204]: result = seasonal_decompose(goog['Sales'],model='additive',period = 4)
plt.rcParams["figure.figsize"] = (10,10)
```

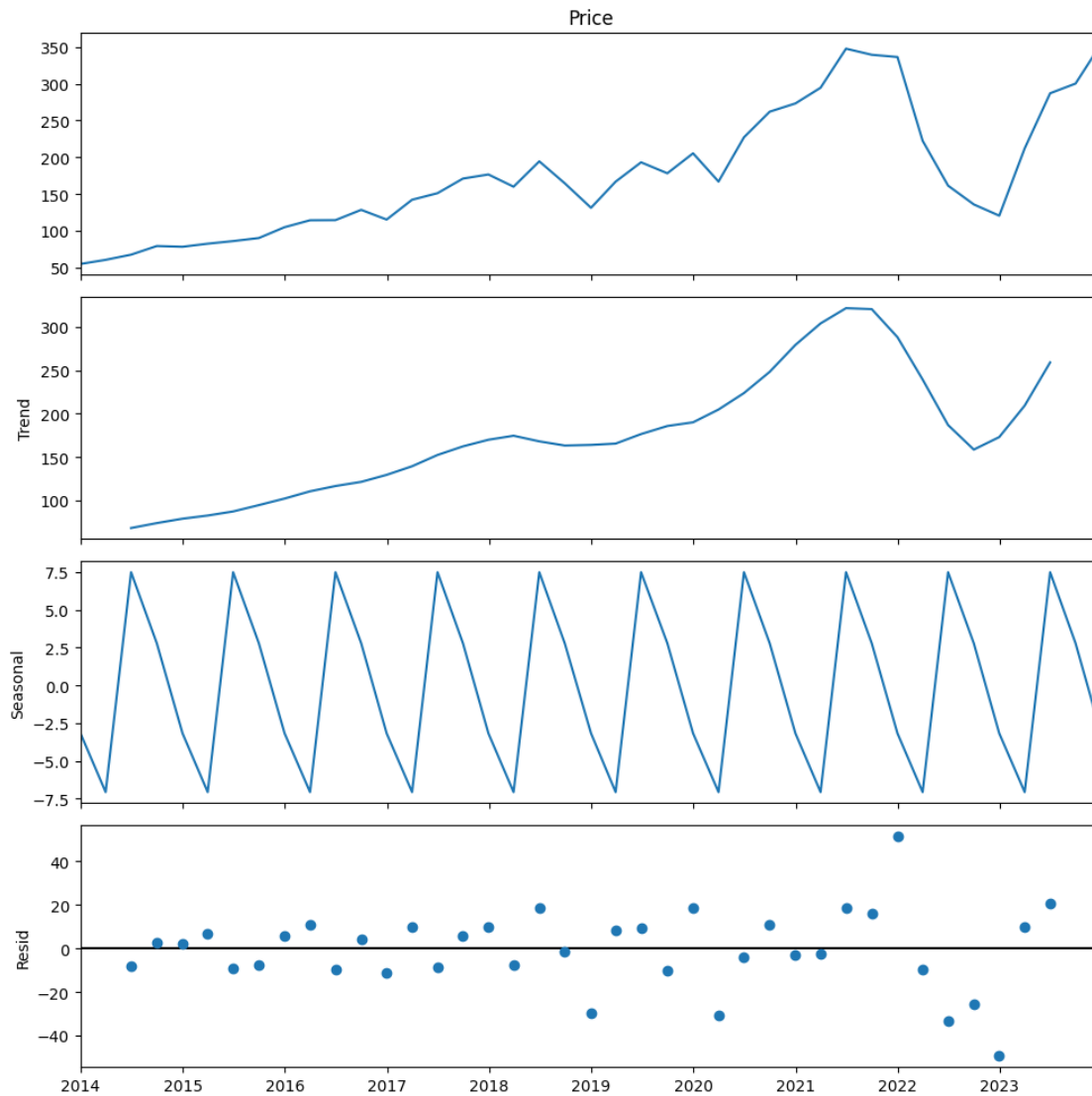
```
result.plot()
#plt.title('Time Series Decomposition of Sales')
plt.show()
```



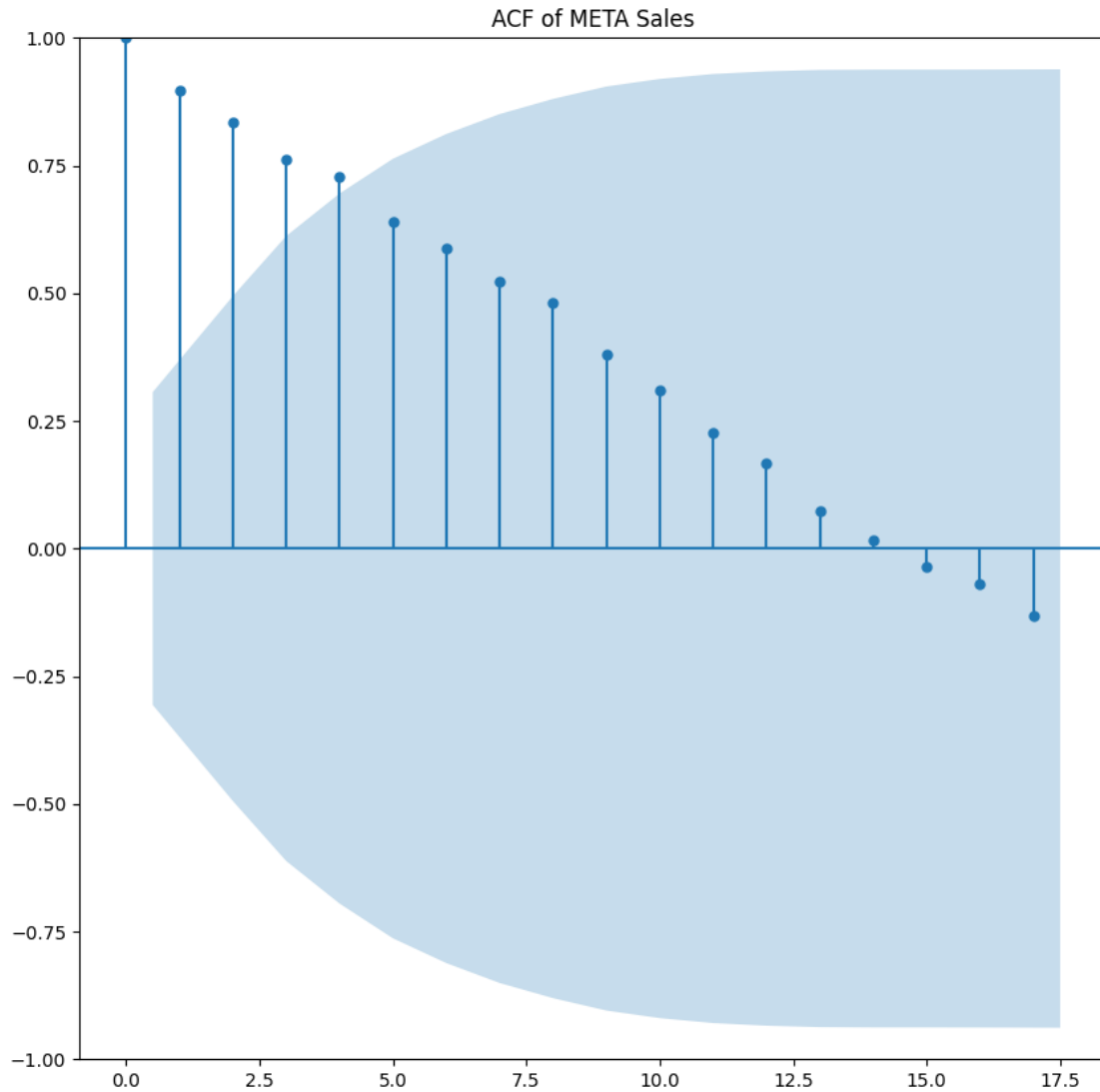
```
[205]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
        ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Price'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



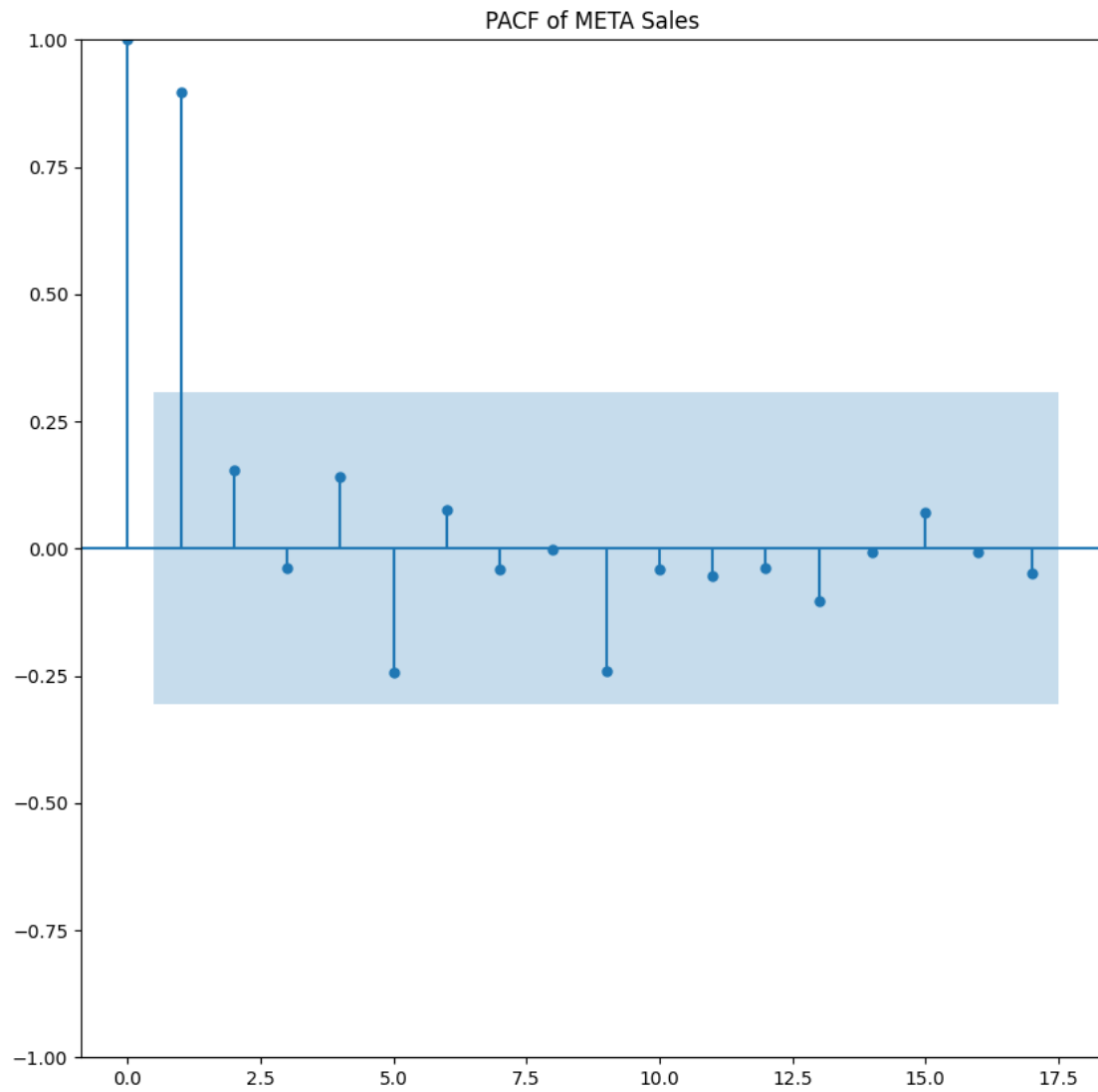
```
[206]: result = seasonal_decompose(goog['Price'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



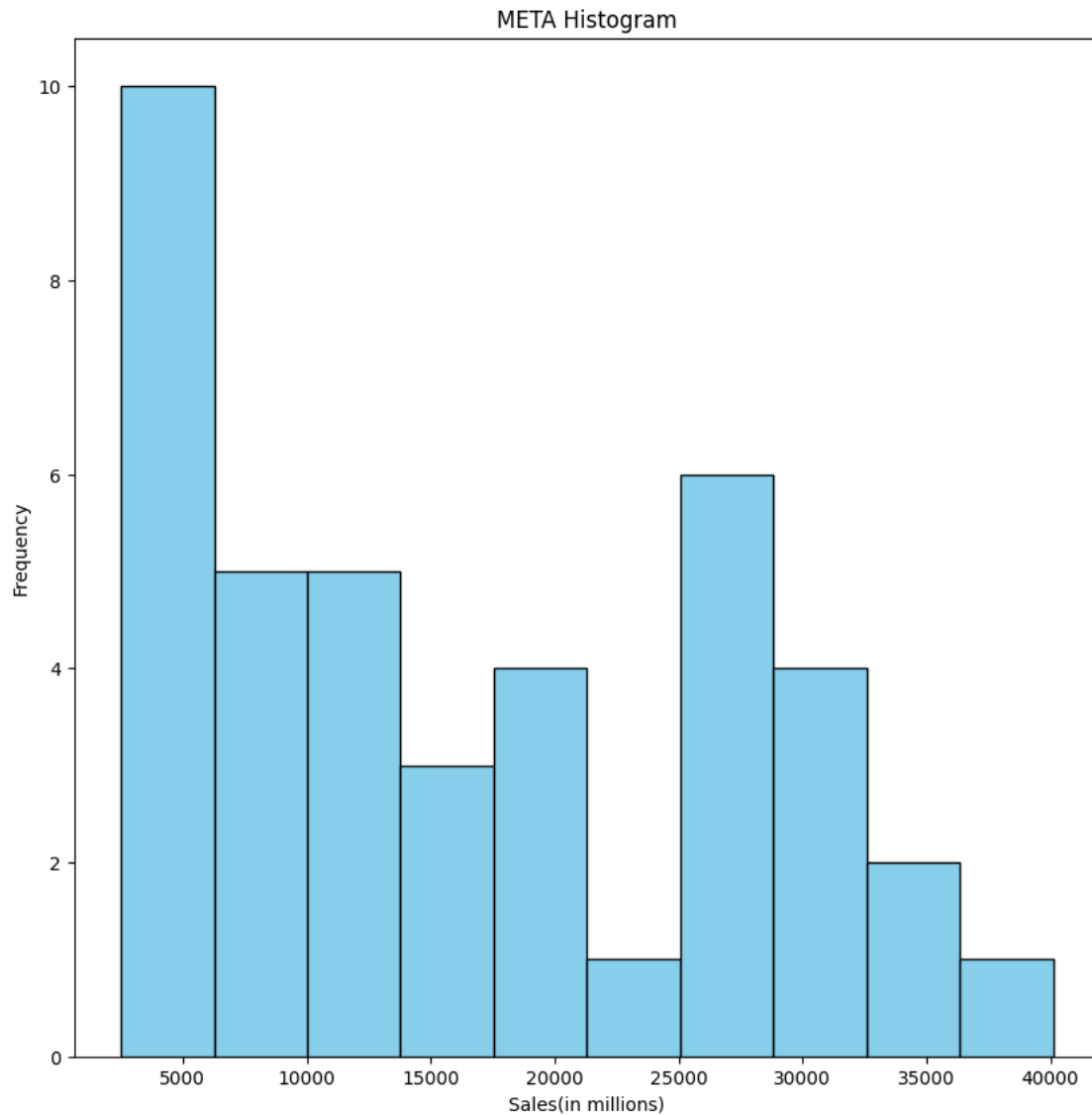
```
[207]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_acf(goog['Sales'], title='ACF of META Sales')
pyplot.show()
```



```
[208]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_pacf(goog['Sales'], title='PACF of META Sales')
pyplot.show()
```



```
[209]: plt.hist(goog['Sales'], color='skyblue', edgecolor='black')
plt.xlabel('Sales(in millions)')
plt.ylabel('Frequency')
plt.title('META Histogram')
pyplot.show()
```



```
[210]: # statistical test
result = adfuller(goog['Sales'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 1.346103

p-value: 0.996855

Critical Values:

1%: -3.639

5%: -2.951

10%: -2.614

```
[211]: # statistical test
result = adfuller(goog['Price'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: -1.355435

p-value: 0.603445

Critical Values:

1%: -3.654

5%: -2.957

10%: -2.618

4.7 META Conclusion

Description of Time Series (is it stationary, are there any trends, seasonality, etc)

For META, you can see there is clearly a very steady trend higher in Sales and Stock Price over the last 10 years. The seasonality of the Sales and Stock Price is really interesting. Sales has seasonality where sales bottom in the first quarter and peak in the 4th quarter. Seasonality for stock price is different. The multiplicative model indicates stock price seasonality is the strongest in the middle quarters(2nd and 3rd quarters) and weakest in the first and last quarter. The additive model seems to be more appropriate for the Sales and Stock price data as the residuals in each case appear like white noise in the additive model but have more of a pattern in the multiplicative model. This is shown in both time series decomposition charts. The null hypothesis of the ADF test is the time series is Non Stationary.

We conclude META is Non Stationary both when looking at Sales and Price since the test statistic is larger than the critical values.

META does show autocorrelation out to about 5 lags.

```
[ ]:
```

4.7.1 Analysis of NVDA Stock and Fundamentals over last 10 years

```
[212]: #goog = pd.read_csv('GOOGL.csv',header=0, index_col=0,
    ↪parse_dates=True,squeeze=True)
goog = pd.read_csv('./Aux_data/nvda.csv')
```

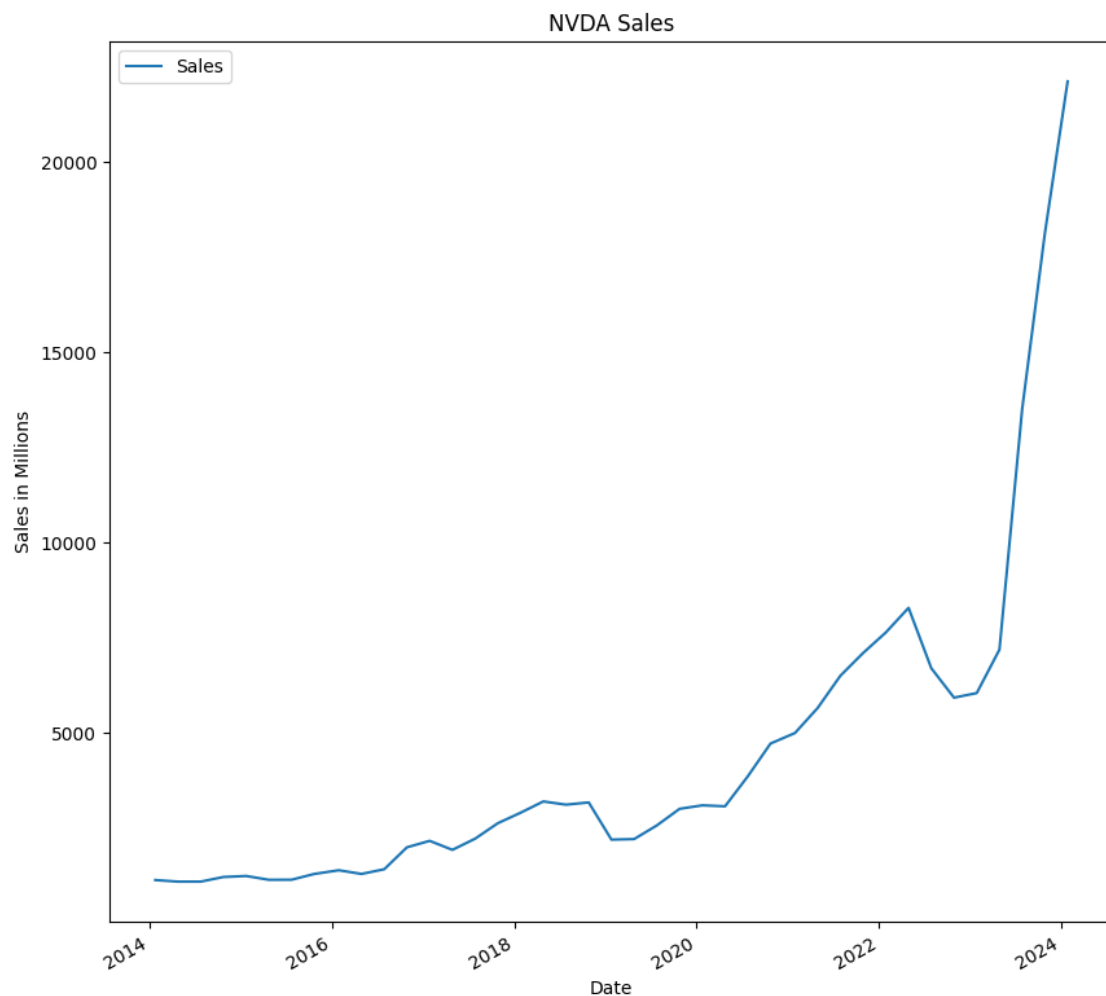
```
[213]: goog.columns
```

```
[213]: Index(['Date', 'Sales', 'FreeCashFlow', 'Cash', 'Long Term Debt', 'Price'],
dtype='object')
```

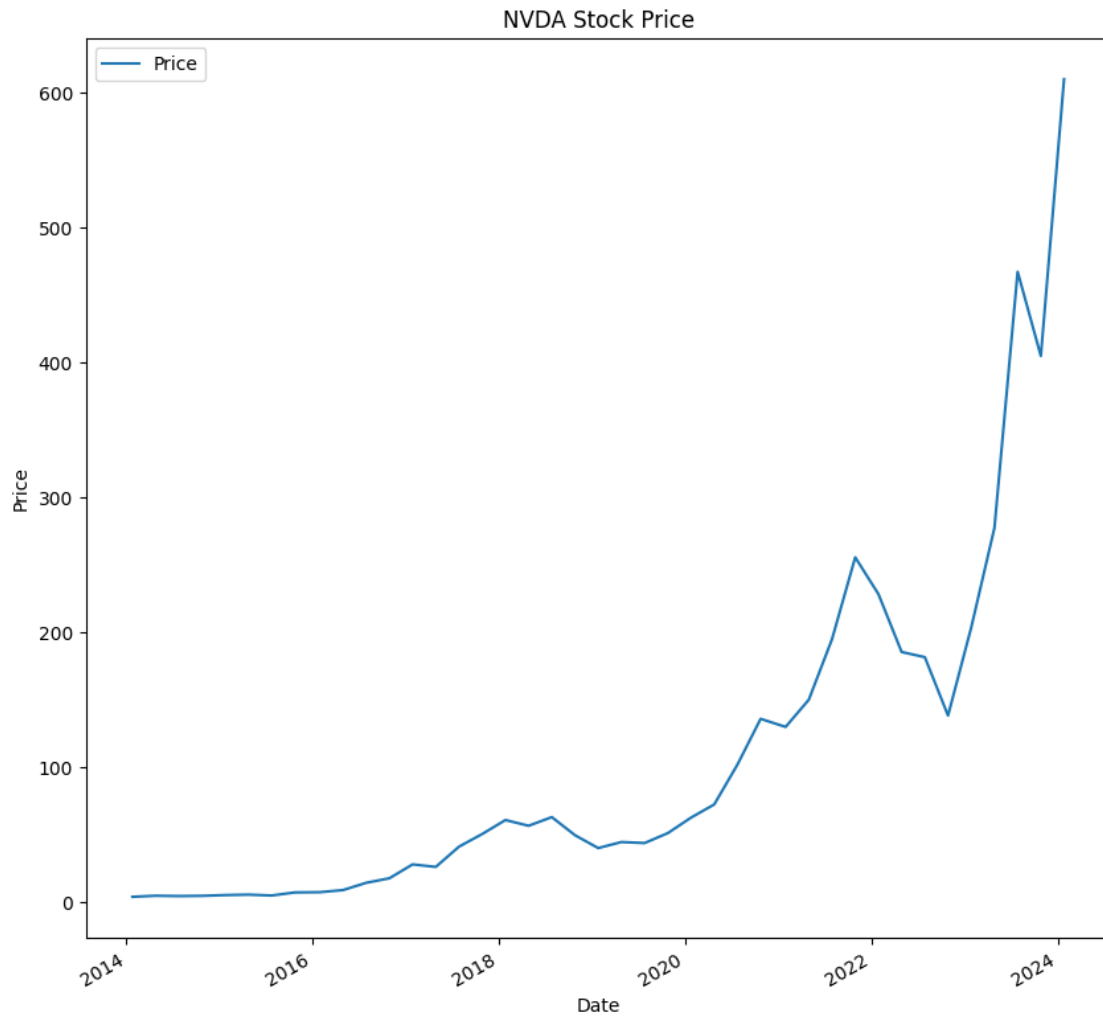


```
[214]: goog['Date'] = pd.to_datetime(goog['Date'], format='%m/%d/%Y')
goog['Year'] = goog['Date'].dt.year
goog['Quarter'] = goog['Date'].dt.quarter
```

```
[215]: goog.plot('Date', 'Sales')
plt.title('NVDA Sales')
plt.ylabel('Sales in Millions')
plt.show()
```

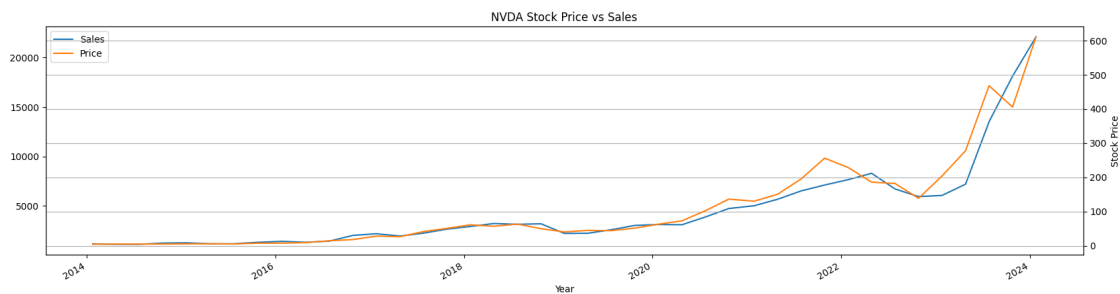


```
[216]: goog.plot('Date', 'Price')
plt.title('NVDA Stock Price')
plt.ylabel('Price')
plt.show()
```



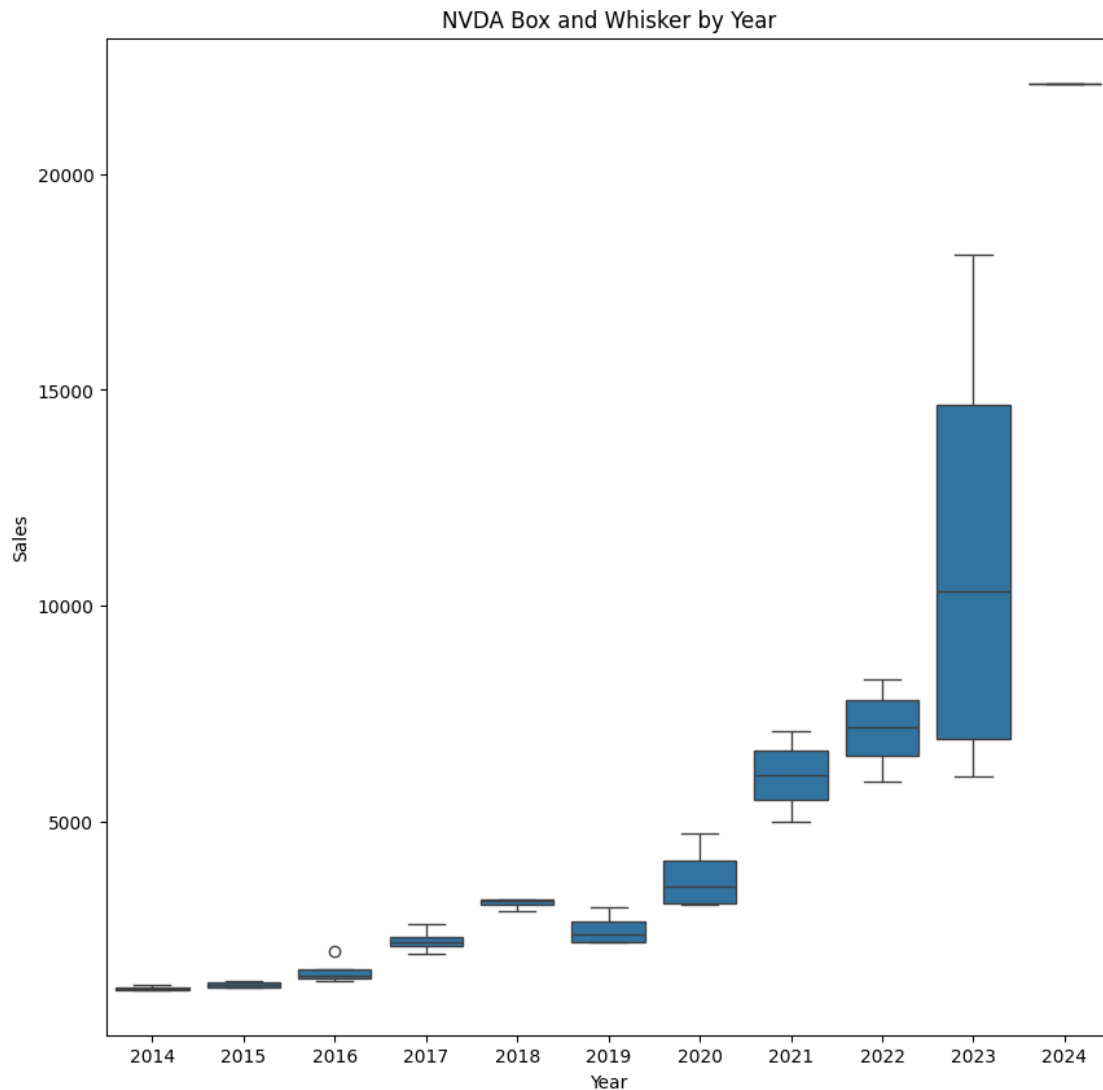
```
[217]: goog.loc[:,['Date','Sales','Price']].plot(x = 'Date',secondary_y=['Price'],
↪xlabel = 'Year', ylabel = 'Stock Price',sharex = True,mark_right=False,
↪figsize = (20,5), grid=True)
plt.title('NVDA Stock Price vs Sales')
```

```
[217]: Text(0.5, 1.0, 'NVDA Stock Price vs Sales')
```



```
[218]: sn.boxplot(x='Year',y='Sales', data = goog).set(title = 'NVDA Box and Whisker_
↳by Year')
```

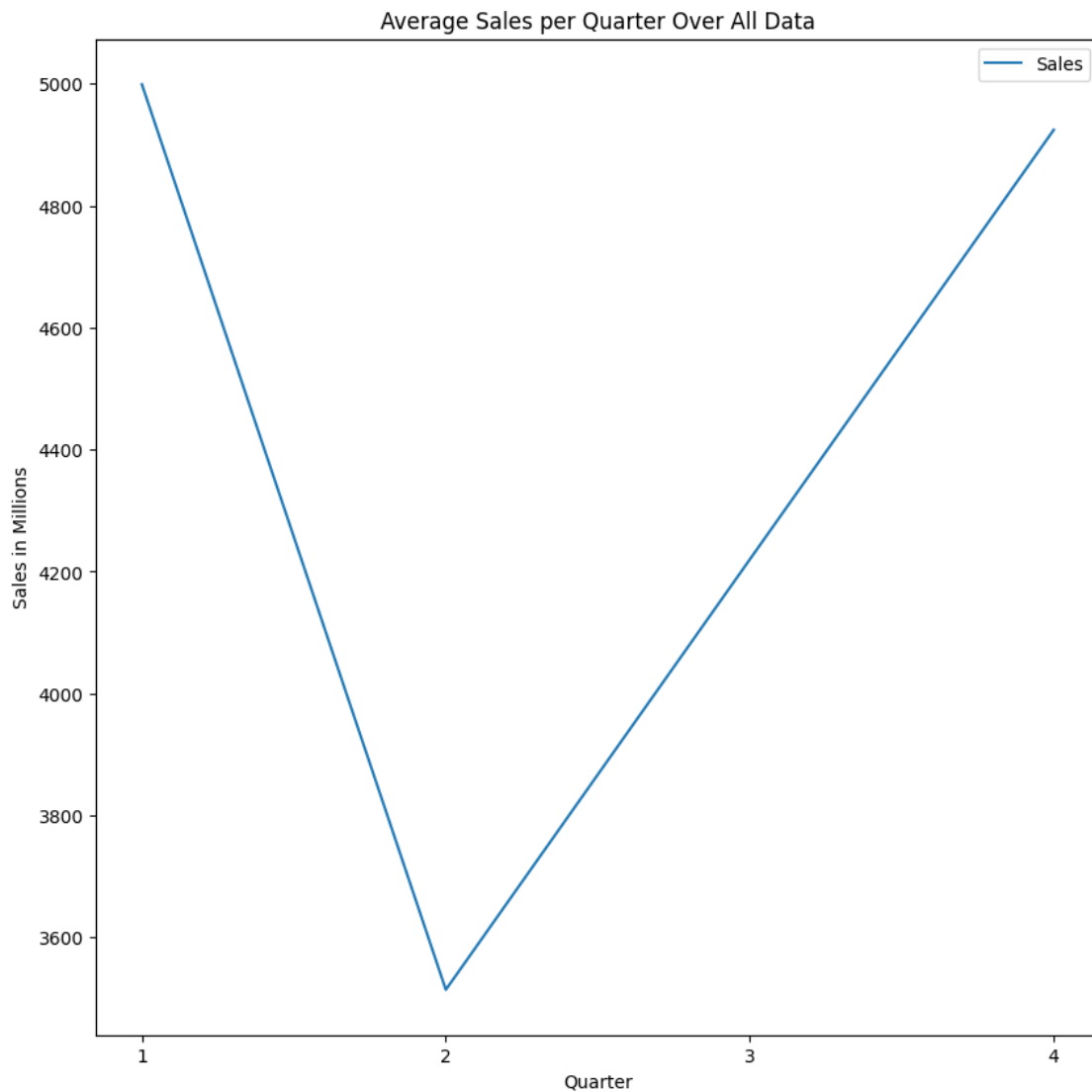
```
[218]: [Text(0.5, 1.0, 'NVDA Box and Whisker by Year')]
```



```
[219]: avgperqtr = goog[['Quarter', 'Sales']].groupby('Quarter').mean().reset_index()
```

```
[220]: avgperqtr.plot('Quarter','Sales')
plt.ylabel('Sales in Millions')
plt.title('Average Sales per Quarter Over All Data')
plt.xticks(goog.Quarter.unique())
```

```
plt.show()
```

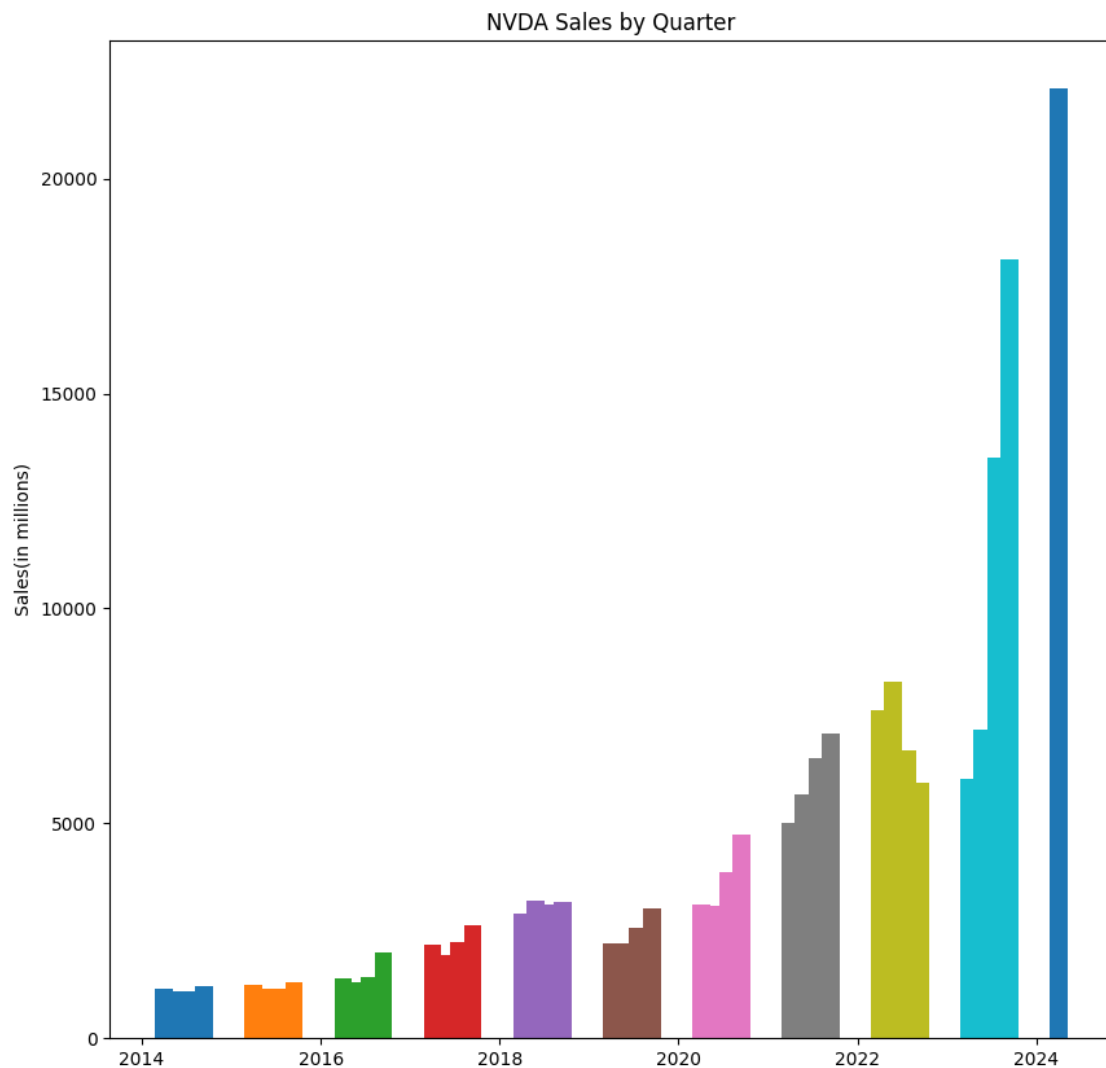


```
[221]: goog1 = pd.DataFrame(goog, columns=['Year', 'Quarter', 'Sales'])
```

```
[222]: dictyear = {}  
saleslist = []  
for year in goog1['Year'].unique():  
    #print(year)  
    #byyear = goog1[goog1['Year'] == year]  
    #saleslist = saleslist.append(byyear.Sales)  
    dictyear[year] = list(goog1[goog1['Year'] == year].Sales)
```

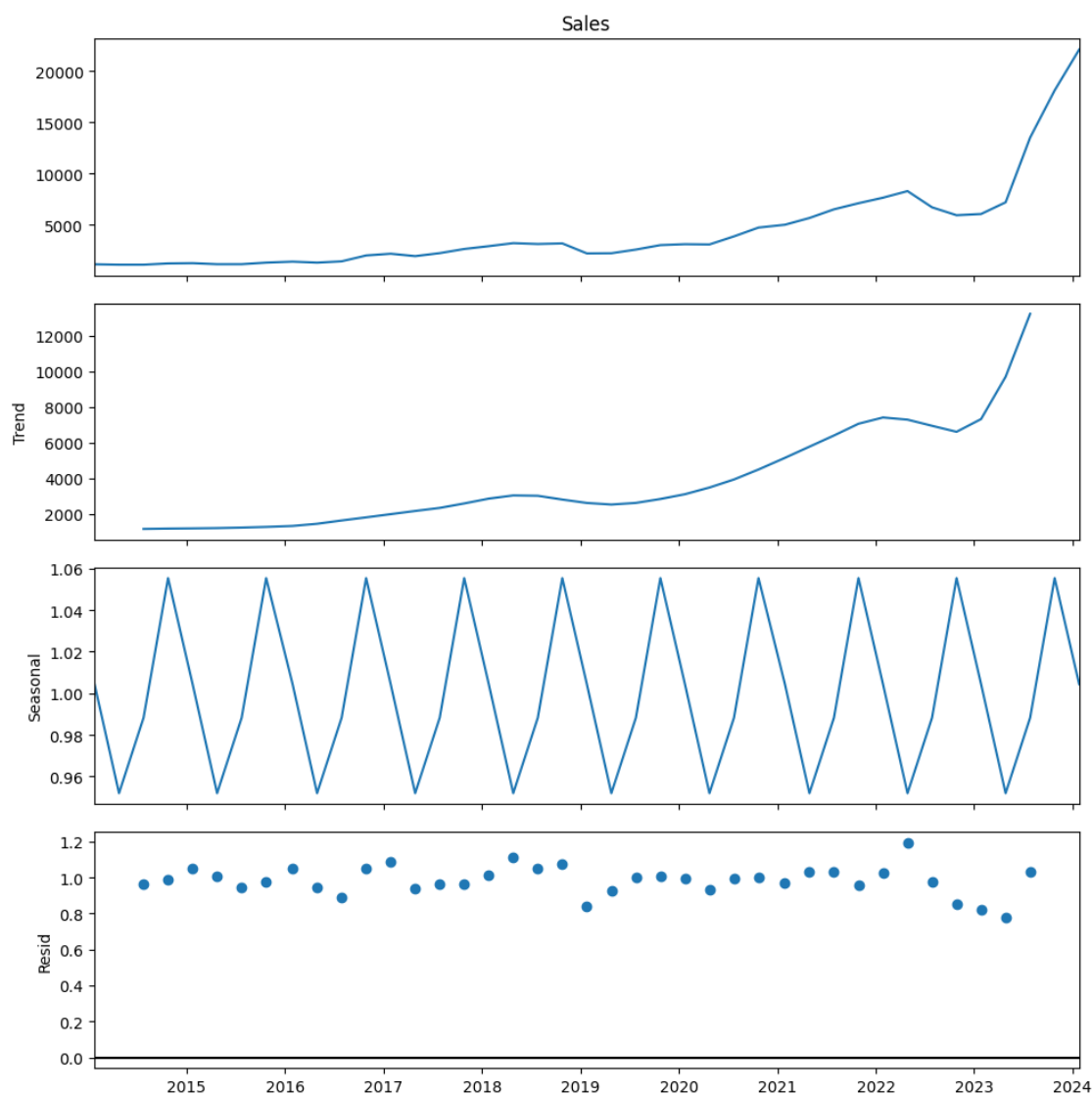
```
[223]: #adopted from https://medium.com/@BetterEverything/
↳graph-quarterly-data-with-python-6f38bcc6b7fe
for year, data in dictyear.items():
    quarters = list(goog1.loc[(goog1['Year'] == year)].Quarter)
    x_coordinates = [year + 0.15 * q for q in quarters]
    #print(x_coordinates, data)
    plt.bar(x_coordinates, data, width=0.2, align='edge')
plt.title('NVDA Sales by Quarter')
plt.ylabel('Sales(in millions)')

#plt.xticks(labels=[np.unique(goog1['Year'].values)], ticks = [])
#print(plt.xticks())
plt.show()
```



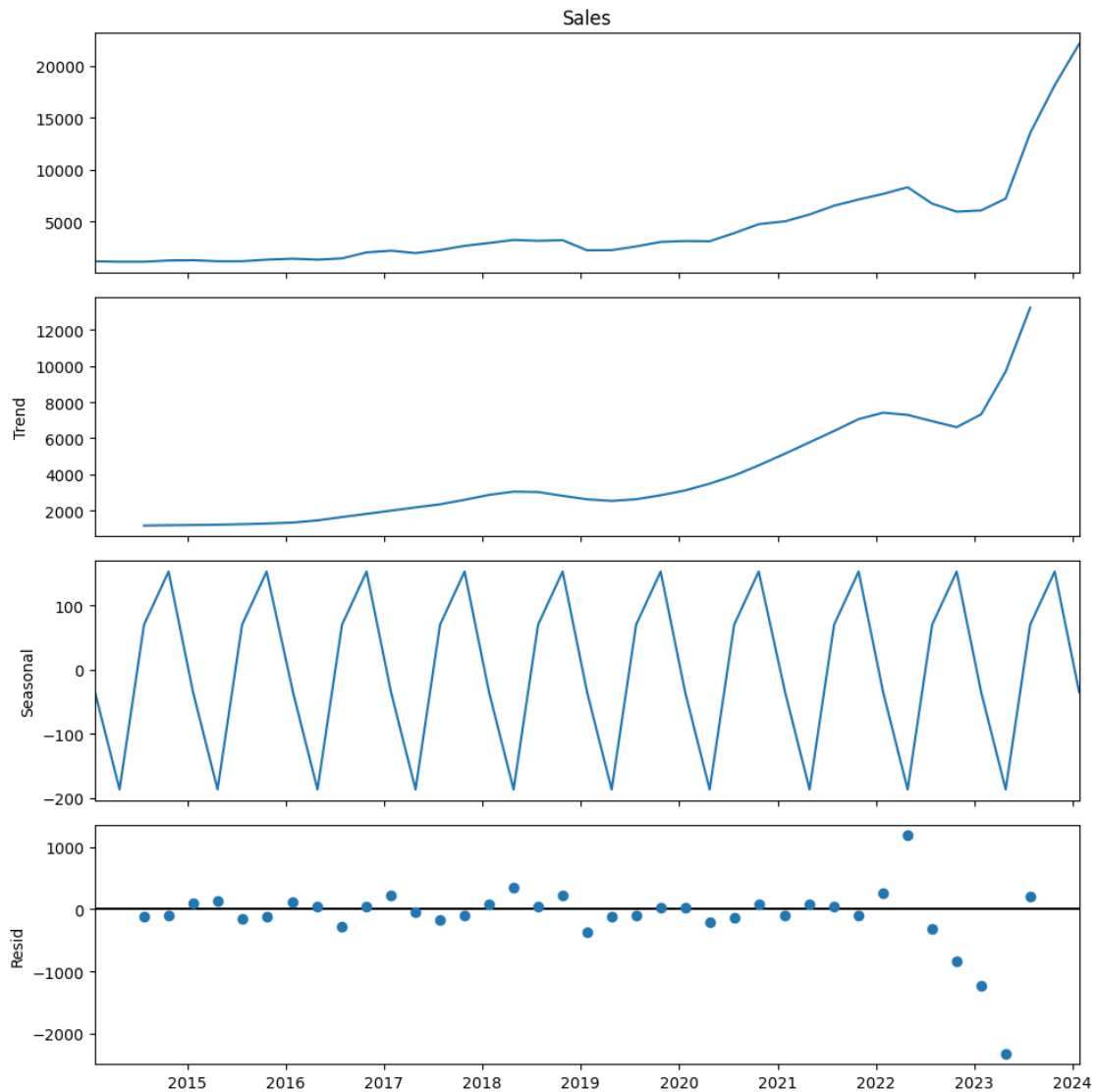
```
[224]: goog.set_index('Date', inplace=True)
```

```
[225]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
↳parse_dates=True,squeeze=True)
result = seasonal_decompose(goog['Sales'], model='multiplicative',period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

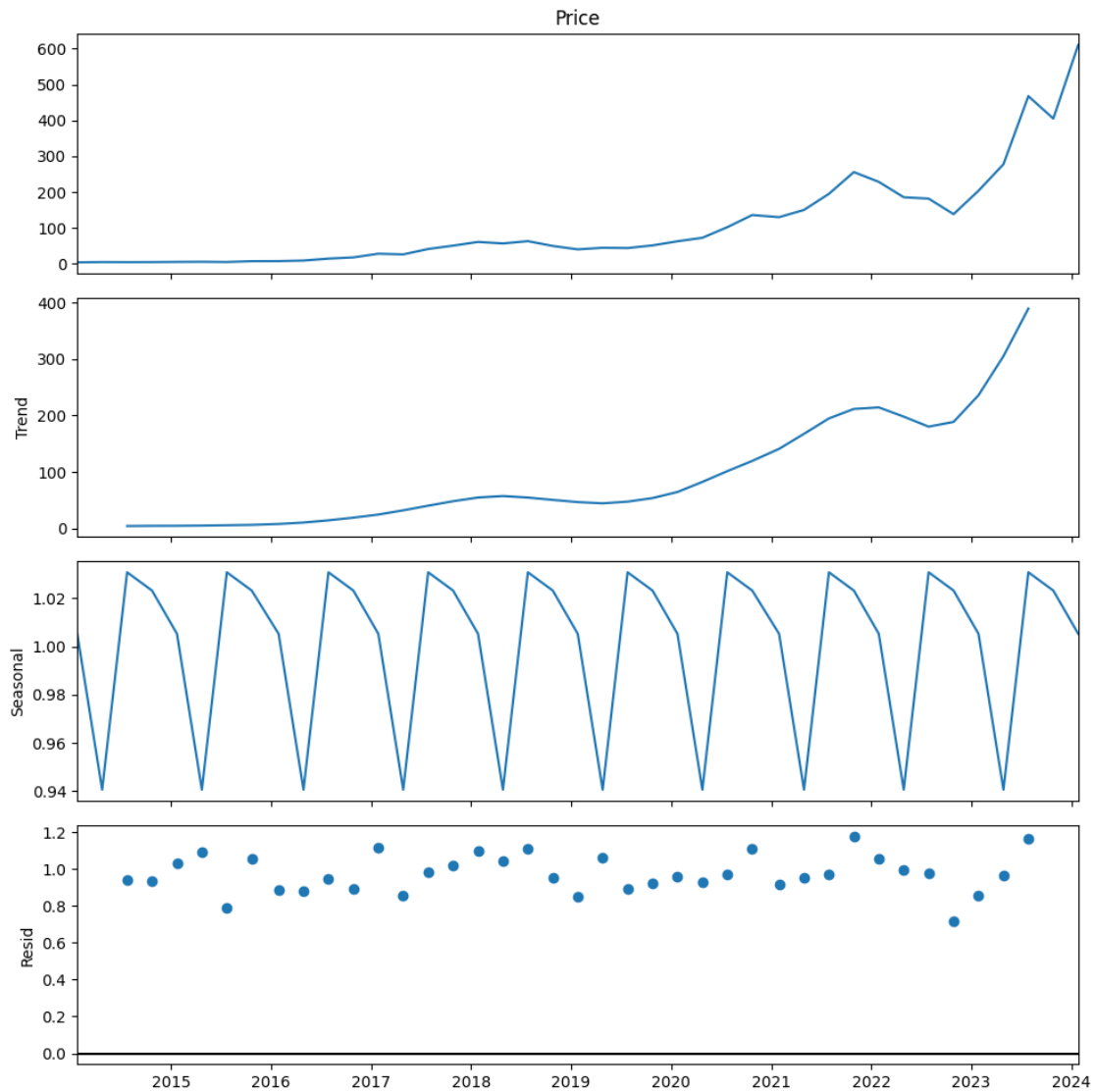


```
[226]: result = seasonal_decompose(goog['Sales'],model='additive',period = 4)
plt.rcParams["figure.figsize"] = (10,10)
```

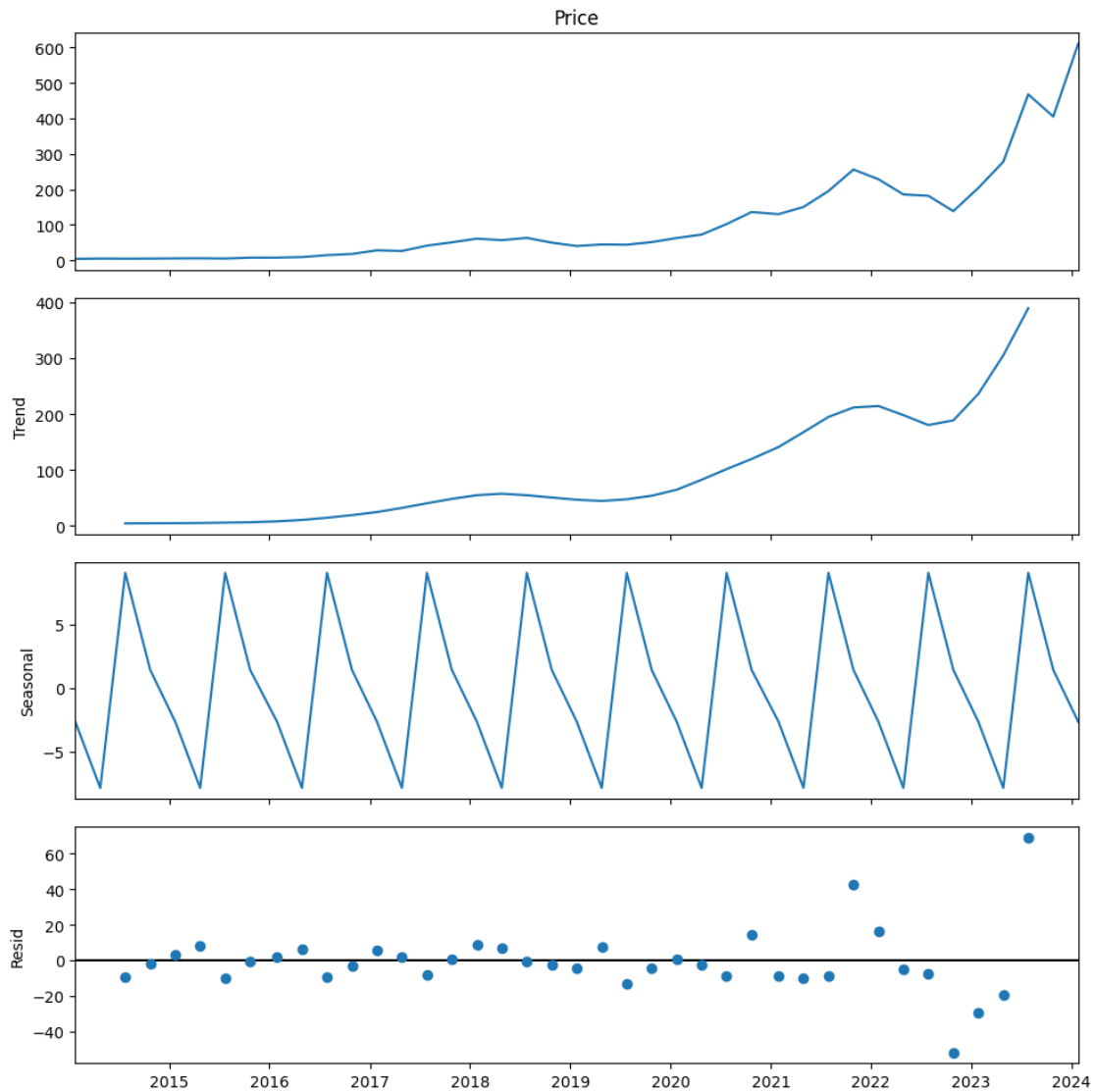
```
result.plot()
#plt.title('Time Series Decomposition of Sales')
plt.show()
```



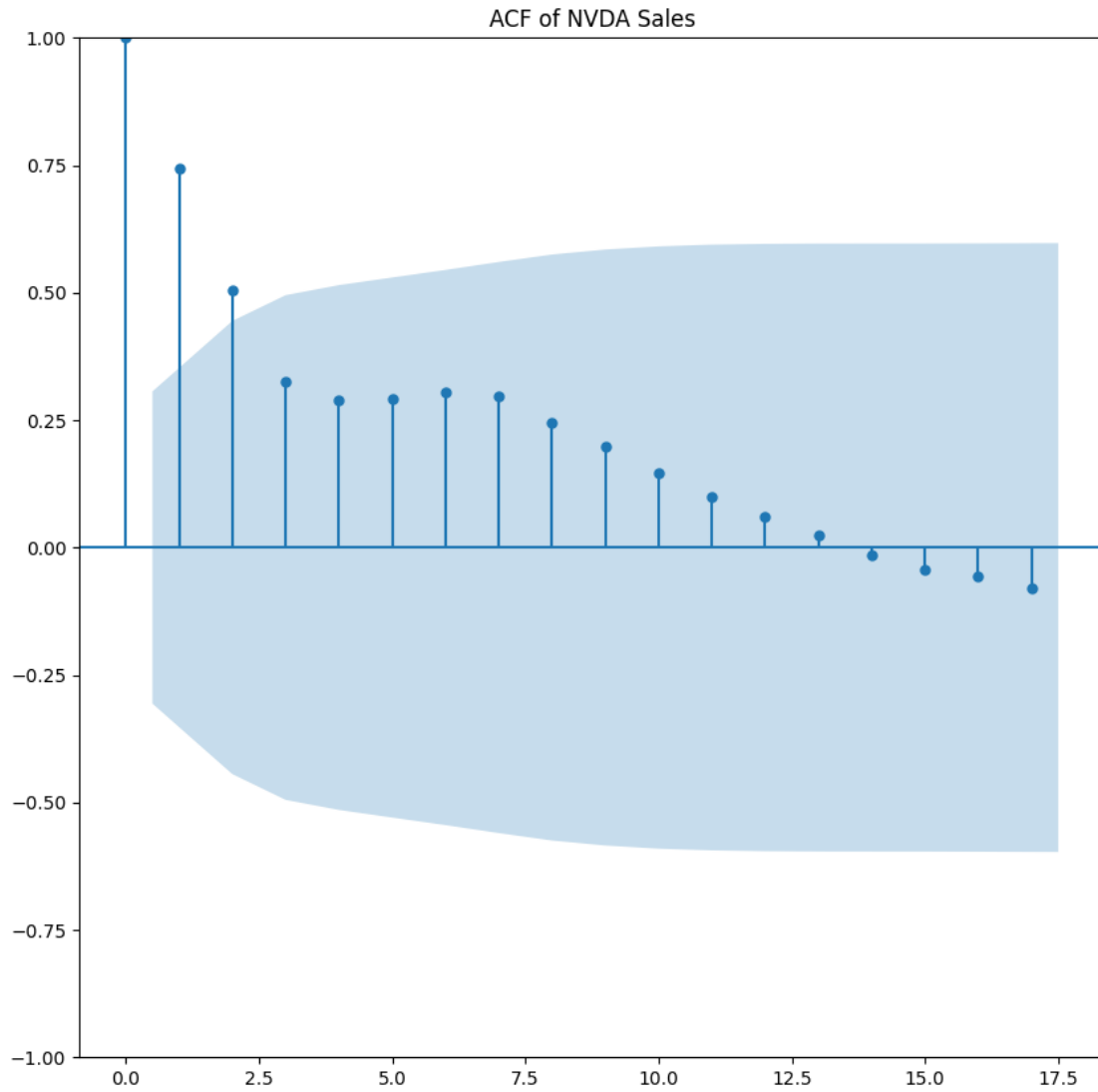
```
[227]: #series = read_csv('airline-passengers.csv', header=0, index_col=0,
        ↪ parse_dates=True, squeeze=True)
result = seasonal_decompose(goog['Price'], model='multiplicative', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```



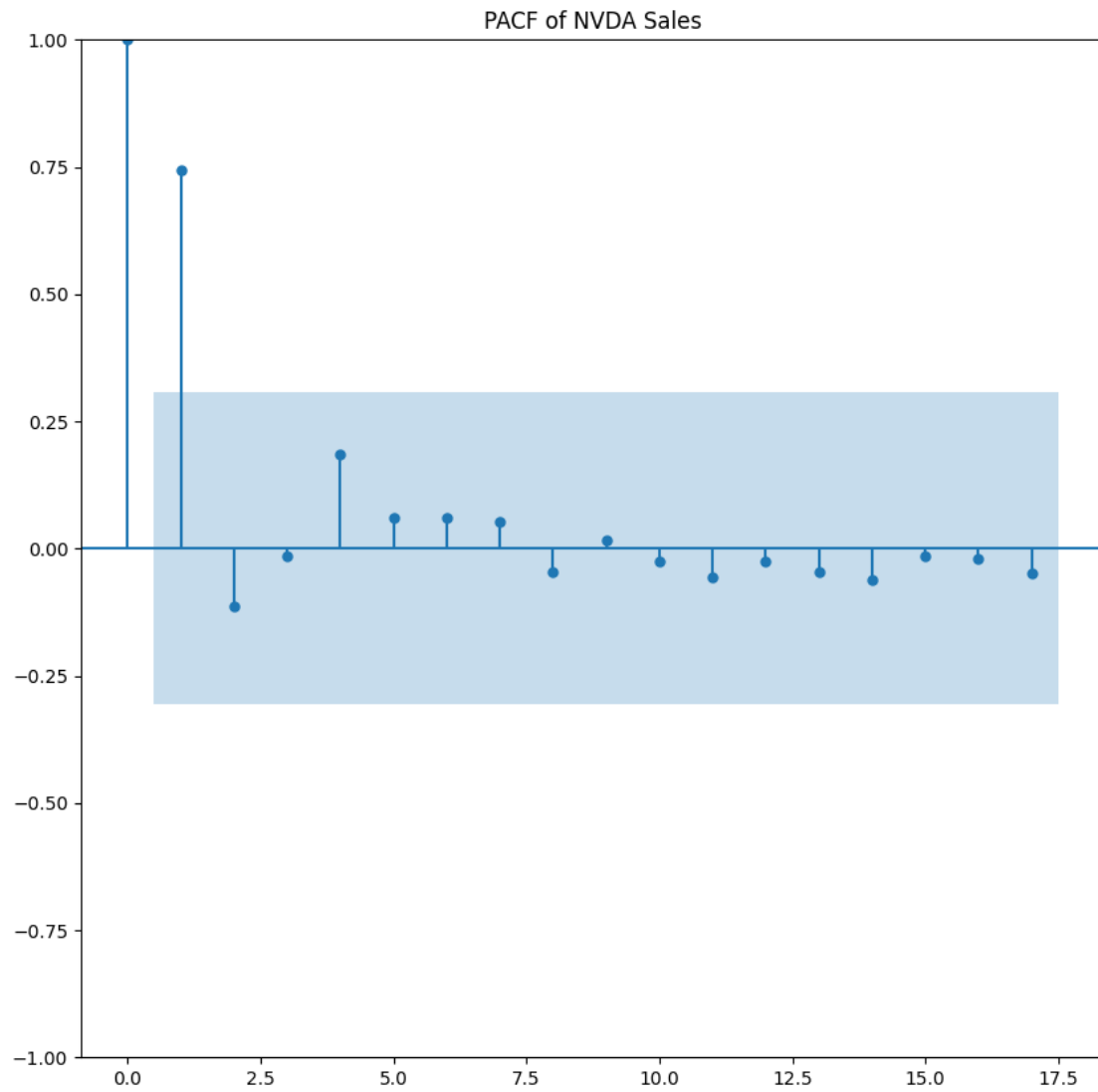
```
[228]: result = seasonal_decompose(goog['Price'], model='additive', period = 4)
plt.rcParams["figure.figsize"] = (10,10)
result.plot()
plt.show()
```

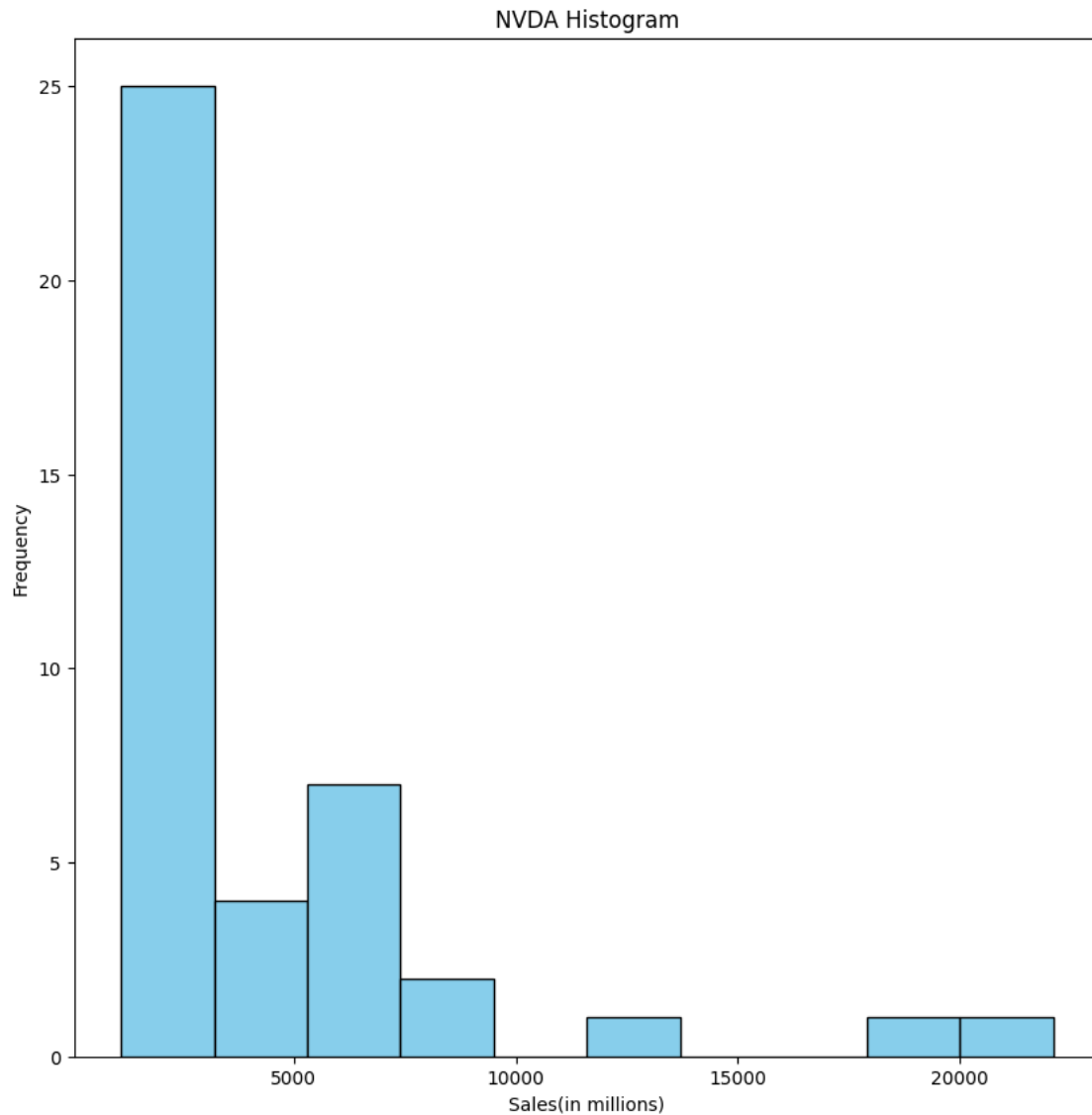
```
[229]: #https://machinelearningmastery.com/
        ↪gentle-introduction-autocorrelation-partial-autocorrelation/
from matplotlib import pyplot
from statsmodels.graphics.tsaplots import plot_acf
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)
plot_acf(goog['Sales'], title='ACF of NVDA Sales')
pyplot.show()
```



```
[230]: #https://machinelearningmastery.com/  
       ↪gentle-introduction-autocorrelation-partial-autocorrelation/  
from matplotlib import pyplot  
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf  
#series = read_csv('daily-minimum-temperatures.csv', header=0, index_col=0)  
plot_pacf(goog['Sales'], title='PACF of NVDA Sales')  
pyplot.show()
```



```
[231]: plt.hist(goog['Sales'], color='skyblue', edgecolor='black')
plt.xlabel('Sales(in millions)')
plt.ylabel('Frequency')
plt.title('NVDA Histogram')
pyplot.show()
```



```
[232]: # statistical test
result = adfuller(goog['Sales'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 3.369466

p-value: 1.000000

Critical Values:

1%: -3.627

5%: -2.946

10%: -2.612

```
[233]: # statistical test
result = adfuller(goog['Price'])
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

ADF Statistic: 4.251282

p-value: 1.000000

Critical Values:

1%: -3.654

5%: -2.957

10%: -2.618

4.8 NVDA Conclusion

Description of Time Series (is it stationary, are there any trends, seasonality, etc)

For NVDA, you can see there is clearly a very steady trend higher in Sales and Stock Price over the last 10 years. In particular, the most recent quarter of sales reported is literally almost off the charts. There is seasonality with the 2nd quarter being the lowest compared to the other three quarters. The additive model seems to be more appropriate for the Sales and Stock price data as the residuals in each case appear like white noise in the additive model but have more of a pattern in the multiplicative model. This is shown in both time series decomposition charts.

The null hypothesis of the ADF test is the time series is Non Stationary. We conclude NVDA is Non Stationary both when looking at Sales and Price since the test statistic is larger than the critical values.

NVDA does show autocorrelation out to about 3 lags.

```
[ ]:
```

```
[ ]:
```

5 4. Modeling - LSTM

6 LUKE DENONCOURT

```
[234]: data = pd.read_csv('dataframe_full.csv')
```

```
[235]: data = data.set_index('Date')
data.shape
```

```
[235]: (1260, 14373)
```

```
[236]: data.columns
```

```
[236]: Index(['AAPL_Open', 'AAPL_High', 'AAPL_Low', 'AAPL_Close', 'AAPL_Volume',  
        'AAPL_Dividends', 'AAPL_Stock_Splits', 'A_Open', 'A_High', 'A_Low',  
        ...  
        'ZUO_Volume', 'ZUO_Dividends', 'ZUO_Stock Splits', 'ZWS_Open',  
        'ZWS_High', 'ZWS_Low', 'ZWS_Close', 'ZWS_Volume', 'ZWS_Dividends',  
        'ZWS_Stock Splits'],  
        dtype='object', length=14373)
```

```
[237]: ### Get unique list of stock symbols still in dataframe
```

```
stock_symbol_list = []  
for a in data.columns:  
    symbol = a.split('_')[0]  
    if symbol not in stock_symbol_list:  
        stock_symbol_list.append(symbol)
```

```
[238]: len(stock_symbol_list)
```

```
[238]: 2053
```

```
[239]: ### Get open price for all stocks
```

```
stock_open_price_col_list = []  
for a in data.columns:  
    symbol = a.split('_')[0]  
    col_name = symbol + '_Open'  
    if col_name not in stock_open_price_col_list:  
        stock_open_price_col_list.append(col_name)  
  
data_open = data[stock_open_price_col_list]
```

```
[240]: ### Get volume for all stocks
```

```
stock_volume_col_list = []  
for a in data.columns:  
    symbol = a.split('_')[0]  
    col_name = symbol + '_Volume'  
    if col_name not in stock_volume_col_list:  
        stock_volume_col_list.append(col_name)  
  
data_volume = data[stock_volume_col_list]
```

```
[241]: ### Get HIGH for all stocks
```

```

stock_high_col_list = []
for a in data.columns:
    symbol = a.split('_')[0]
    col_name = symbol + '_High'
    if col_name not in stock_high_col_list:
        stock_high_col_list.append(col_name)

data_high = data[stock_high_col_list]

```

[242]: *### Get low for all stocks*

```

stock_low_col_list = []
for a in data.columns:
    symbol = a.split('_')[0]
    col_name = symbol + '_Low'
    if col_name not in stock_low_col_list:
        stock_low_col_list.append(col_name)

data_low = data[stock_low_col_list]

```

[243]: *### Get close for all stocks*

```

stock_close_col_list = []
for a in data.columns:
    symbol = a.split('_')[0]
    col_name = symbol + '_Close'
    if col_name not in stock_close_col_list:
        stock_close_col_list.append(col_name)

data_close = data[stock_close_col_list]

```

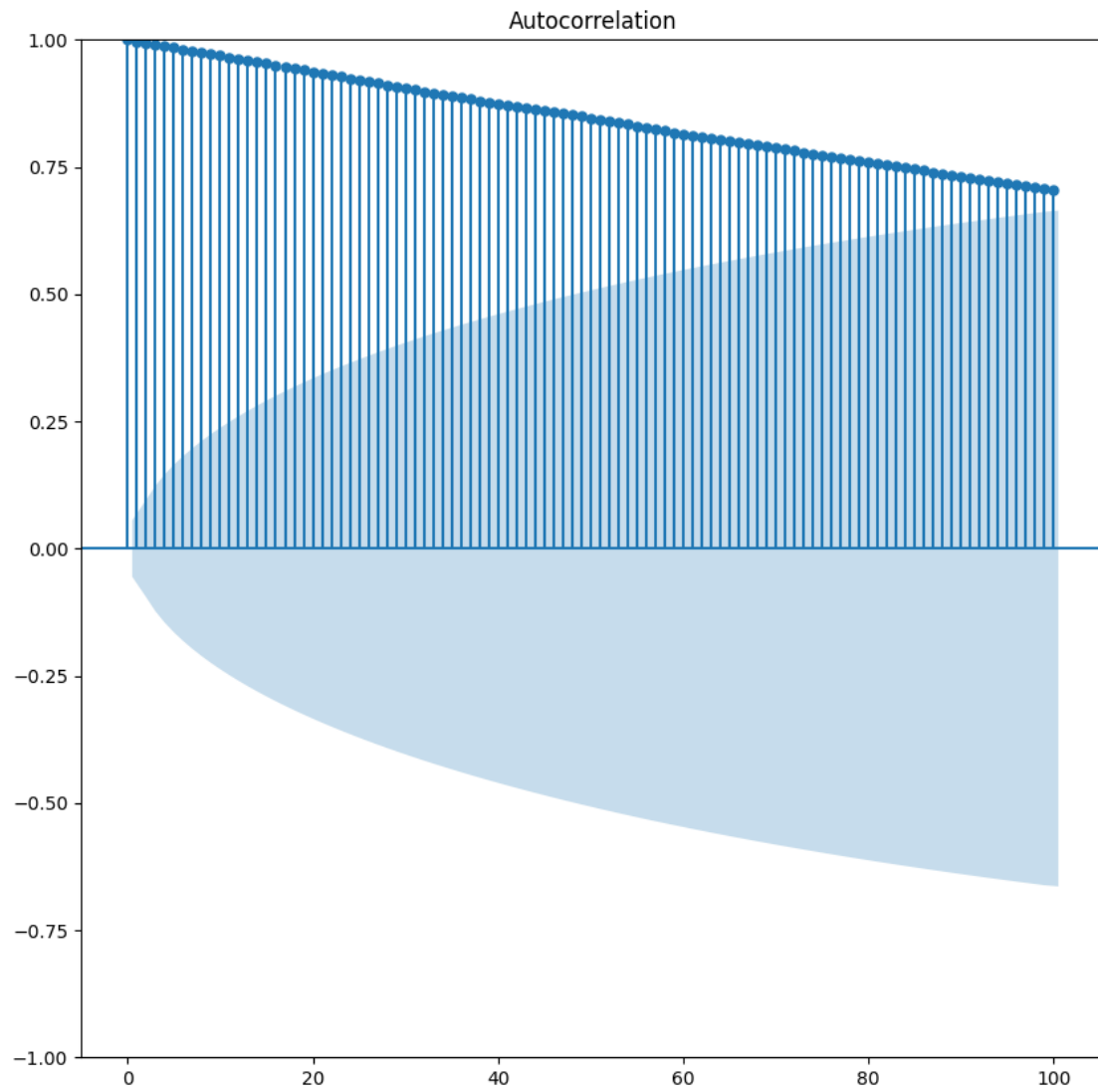
[244]: `aux_data = pd.read_csv('./Aux_Data/AAPL.csv')`

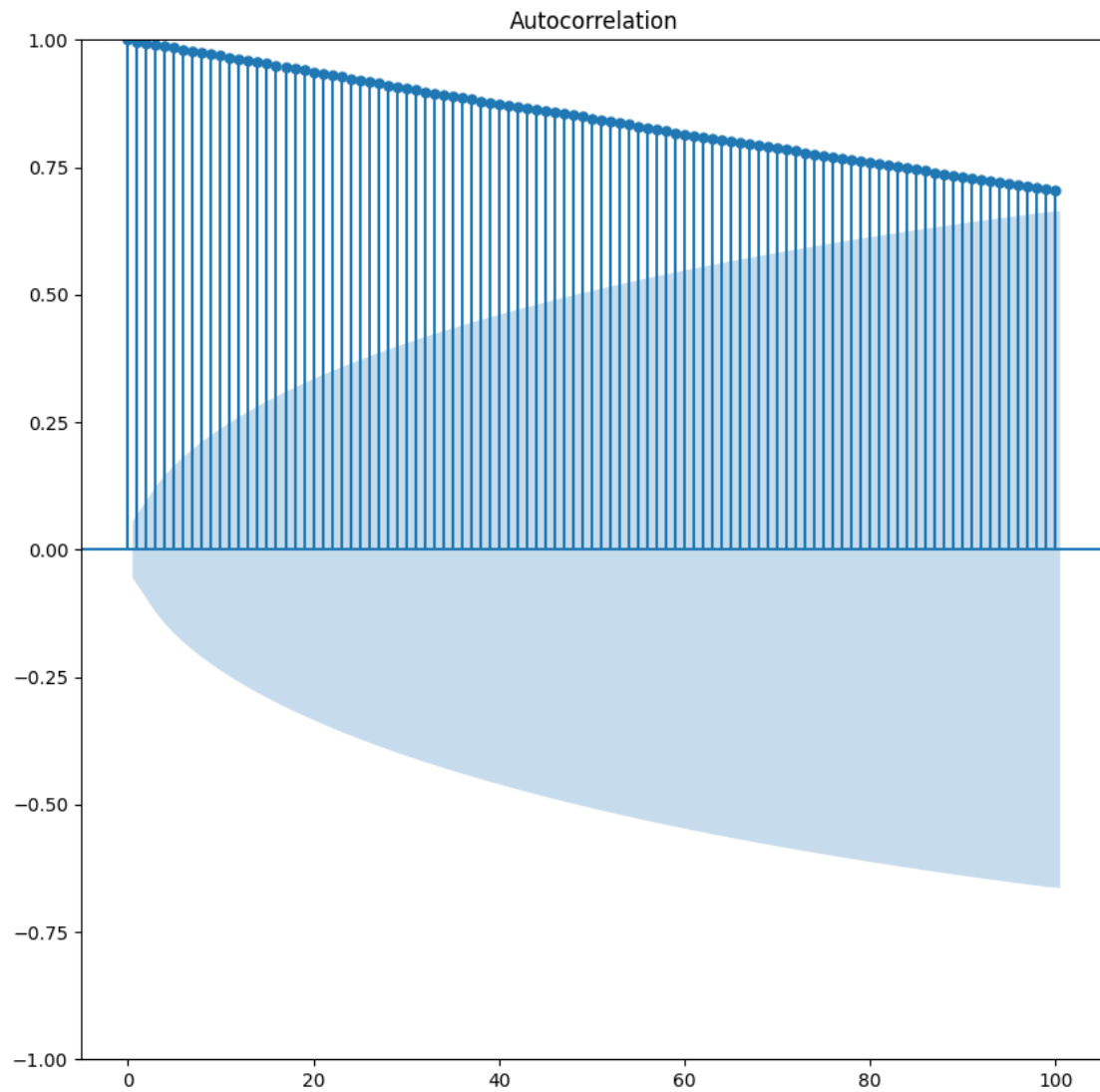
7 Visualization of data

[245]: `apple_open = data_open['AAPL_Open']`

[246]: `plot_acf(apple_open, lags=100)`

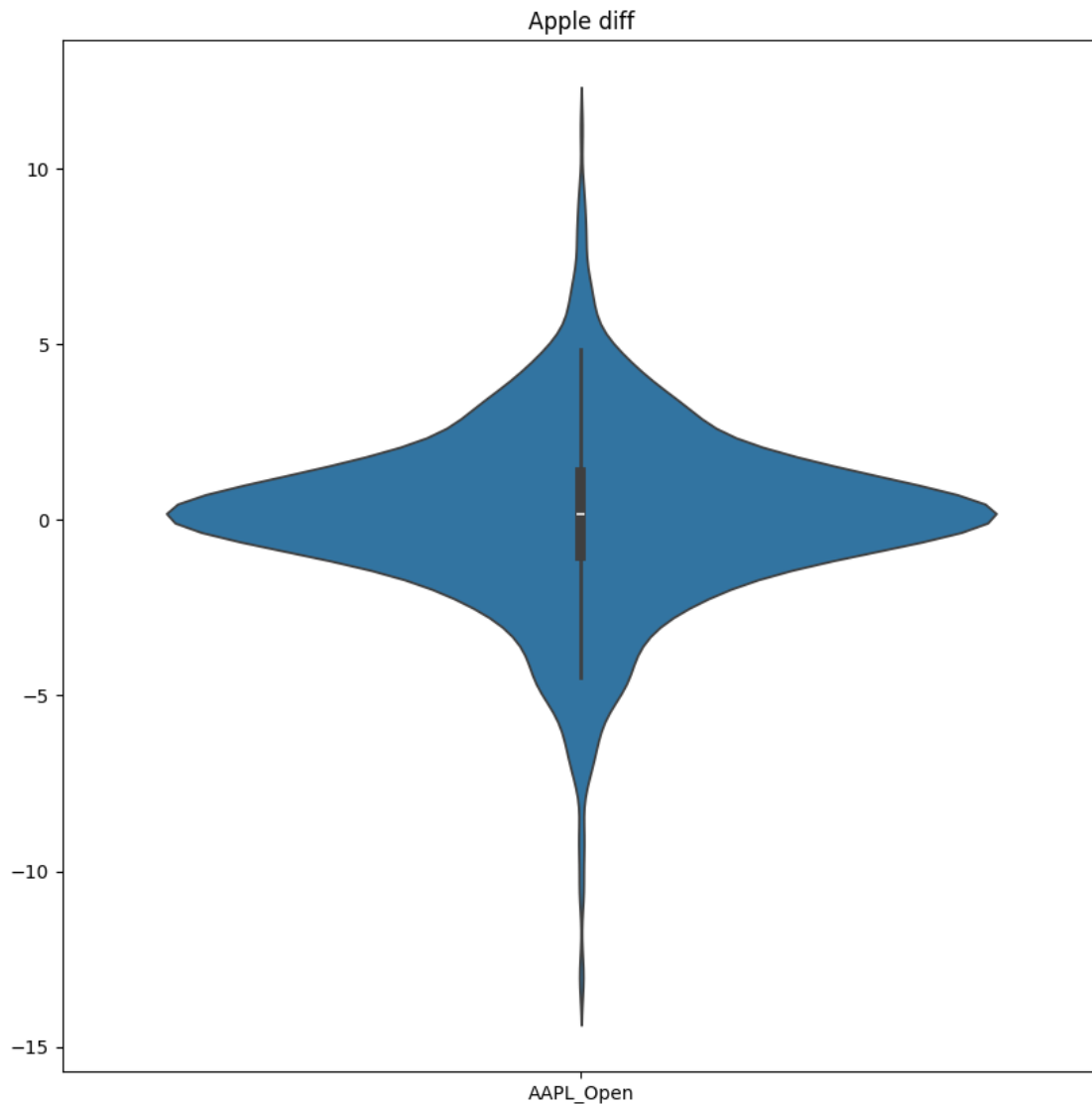
[246]:





```
[249]: import seaborn as sns
apple_diff = pd.DataFrame(data_open['AAPL_Open']).diff().dropna()
sns.violinplot(apple_diff)
print(np.mean(apple_diff), np.median(apple_diff))
plt.title('Apple diff')
plt.show()
```

```
0.10247922620179216 0.16787962553436842
```



[]:

8 Multivariate analysis

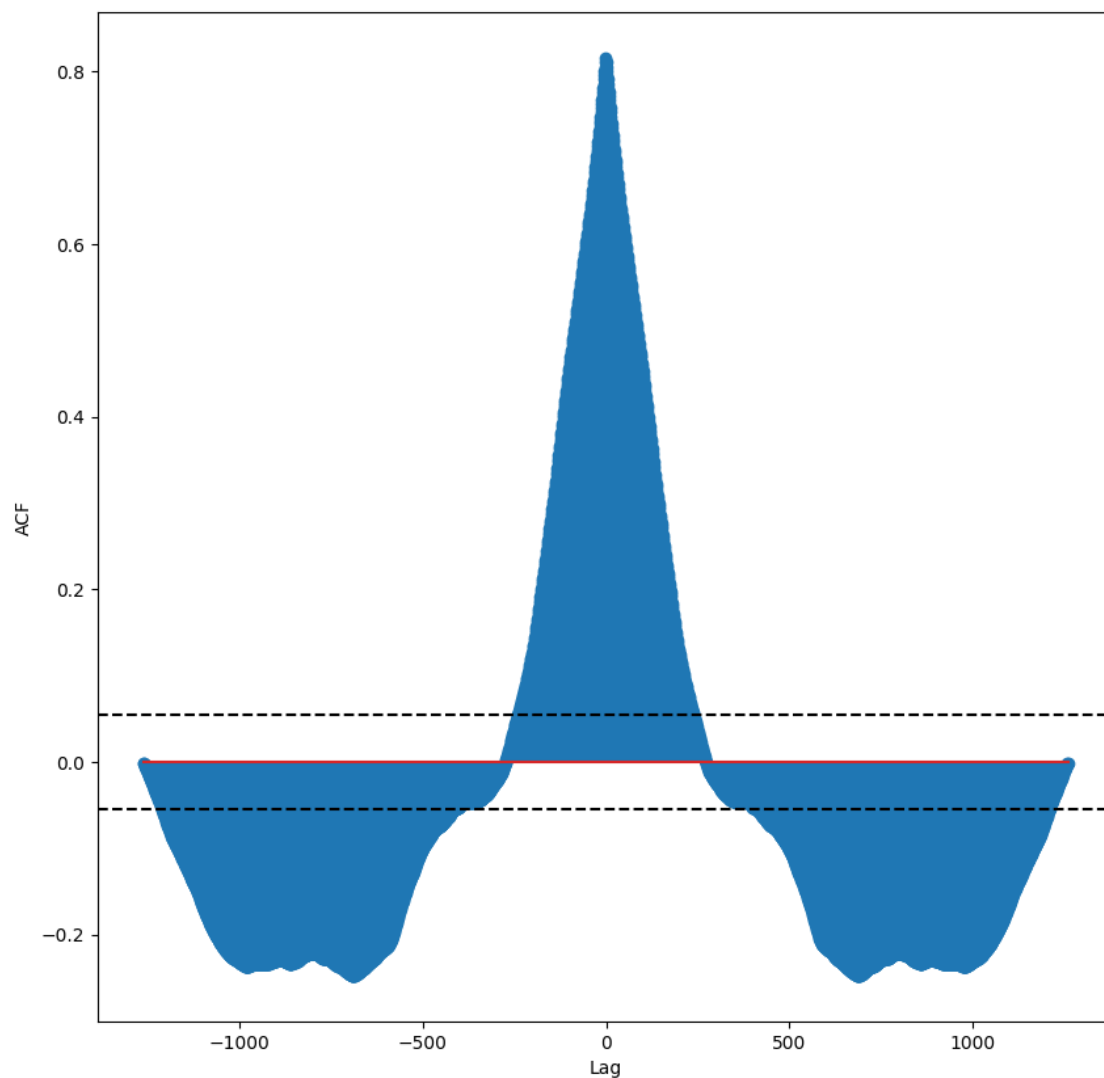
first getting cross correlation of data to aapl

```
[250]: forwards = smt.ccf(data_open['AAPL_Open'], data_open['NVDA_Open'],  
    ↪adjusted=False)  
    backwards = smt.ccf(data_open['AAPL_Open'], data_open['NVDA_Open'],  
    ↪adjusted=False)[::-1]
```

```
[251]: ccf_output = np.r_[backwards[::-1], forwards]
```

```
[252]: plt.stem(range(-len(ccf_output)//2, len(ccf_output)//2), ccf_output)
plt.xlabel('Lag')
plt.ylabel('ACF')
plt.axhline(-1.96/np.sqrt(len(data)), color='k', ls='--')
plt.axhline(1.96/np.sqrt(len(data)), color='k', ls='--')
```

```
[252]: <matplotlib.lines.Line2D at 0x1c8ee2ad720>
```



```
[253]: def ccf_finder(series1_symbol, series2_symbol, ccf_df):
    series1 = data_open[series1_symbol + '_Open']
    series2 = data_open[series2_symbol + '_Open']
    forwards = smt.ccf(series1, series2, adjusted=False)
    backwards = smt.ccf(series1, series2, adjusted=False)[::-1]
    output = np.r_[backwards[:-1], forwards]
```

```

lag_range = range(-len(output)//2, len(output)//2)

max_output = output[np.argmax(output)]
max_lag = lag_range[np.argmax(output)]
min_output = output[np.argmin(output)]
min_lag = lag_range[np.argmin(output)]
ccf_df.loc[len(ccf_df)] = (series1_symbol, series2_symbol, max_output,
↪ max_lag, min_output, min_lag)
return ccf_df

```

```

[254]: ccf_apple_df = pd.
↪ DataFrame(columns=['stock1', 'stock2', 'max_ccf', 'max_ccf_lag', 'min_ccf', 'min_ccf_lag'])

for stock in stock_symbol_list:
    if stock != 'AAPL':
        ccf_apple_df = ccf_finder('AAPL', stock, ccf_apple_df)

```

```

[255]: ccf_apple_df.sort_values(by='max_ccf', ascending=False)

```

```

[255]:
   stock1 stock2  max_ccf  max_ccf_lag  min_ccf  min_ccf_lag
692   AAPL  FAST  0.958722         -1 -0.353687         -980
1243  AAPL  MSFT  0.957450         -1 -0.326265         -986
1396  AAPL  ODFL  0.950275         -1 -0.325775         -982
1225  AAPL  MPWR  0.947544         -1 -0.336482         -979
499   AAPL  CWST  0.945919         -1 -0.326350         -980
...
21    AAPL   AC  0.125866        -373 -0.148634         -605
1641  AAPL  RYTM  0.125223        -723 -0.304895         -400
1293  AAPL   NBR  0.118941       -1169 -0.292174         -797
972   AAPL   IOR  0.118379         -1 -0.106290         -162
2007  AAPL   WTI  0.111411       -1037 -0.251978         -794

```

[2052 rows x 6 columns]

```

[256]: stock_list_greater_point_five_corr_apple =
↪ ccf_apple_df['stock2'][(ccf_apple_df['max_ccf'] >= 0.5) &
↪ (ccf_apple_df['max_ccf_lag'] <= 10)]

```

```

[257]: len(stock_list_greater_point_five_corr_apple)

```

```

[257]: 761

```

The list I created has 761 stocks that have a correlation value greater than 0.5 within 10 lag values of apple stock price.

```

[258]: stock_list_greater_point_five_corr_apple_open = []
[stock_list_greater_point_five_corr_apple_open.append(a + '_Open') for a in
↪ stock_list_greater_point_five_corr_apple.values]

```

```
stock_list_greater_point_five_corr_apple_volume = []
[stock_list_greater_point_five_corr_apple_volume.append(a + '_Volume') for a in
↳stock_list_greater_point_five_corr_apple.values]

stock_list_greater_point_five_corr_apple_low = []
[stock_list_greater_point_five_corr_apple_low.append(a + '_Low') for a in
↳stock_list_greater_point_five_corr_apple.values]

stock_list_greater_point_five_corr_apple_high = []
[stock_list_greater_point_five_corr_apple_high.append(a + '_High') for a in
↳stock_list_greater_point_five_corr_apple.values]

stock_list_greater_point_five_corr_apple_close = []
[stock_list_greater_point_five_corr_apple_close.append(a + '_Close') for a in
↳stock_list_greater_point_five_corr_apple.values]
```

[illegible]

[illegible]

```
[259]: stock_list_greater_point_five_corr_apple_open.append('AAPL_Open')
stock_list_greater_point_five_corr_apple_volume.append('AAPL_Volume')
stock_list_greater_point_five_corr_apple_low.append('AAPL_Low')
stock_list_greater_point_five_corr_apple_high.append('AAPL_High')
stock_list_greater_point_five_corr_apple_close.append('AAPL_Close')
```

8.0.1 Univariate data prediction

```
[260]: def split_sequence(sequence, n_steps):
        X, y = [], []
        for i in range(len(sequence)):
            end_idx = i + n_steps
            if end_idx > len(sequence)-1:
                break
            seq_x, seq_y = sequence[i:end_idx], sequence[end_idx:]
            X.append(seq_x)
            y.append(seq_y)
        return np.array(X), np.array(y)
```

```
[261]: n_steps=10
apple_open_x, apple_open_y = split_sequence(apple_open.to_numpy(), n_steps)
```

```
[262]: apple_open_x_train = apple_open_x[0:1160]
apple_open_x_test = apple_open_x[1160:]

apple_open_y_train = apple_open_y[0:1160]
apple_open_y_test = apple_open_y[1160:]
```

```
[263]: n_features=1
model1 = Sequential([
    LSTM(10, activation='relu', input_shape=(n_steps, n_features)),
    Dense(1, activation='relu')
])

model1.compile(optimizer='adam', loss='mse', metrics=['mse', 'mae'])
```

WARNING:tensorflow:Layer lstm will not use cuDNN kernels since it doesn't meet the criteria. It will use a generic GPU kernel as fallback when running on GPU.

```
[264]: model1.fit(x=apple_open_x_train, y=apple_open_y_train,
    ↪validation_data=[apple_open_x_test,apple_open_y_test], epochs=10)
```

```
Epoch 1/10
37/37 [=====] - 3s 31ms/step - loss: 5243.4097 - mse:
5243.4097 - mae: 60.5568 - val_loss: 175.5148 - val_mse: 175.5148 - val_mae:
12.0686
Epoch 2/10
37/37 [=====] - 1s 25ms/step - loss: 53.1817 - mse:
53.1817 - mae: 5.5286 - val_loss: 34.9974 - val_mse: 34.9974 - val_mae: 4.7566
Epoch 3/10
37/37 [=====] - 1s 23ms/step - loss: 24.9551 - mse:
24.9551 - mae: 3.9741 - val_loss: 33.8295 - val_mse: 33.8295 - val_mae: 4.6600
Epoch 4/10
37/37 [=====] - 1s 24ms/step - loss: 26.0163 - mse:
26.0163 - mae: 3.8885 - val_loss: 32.9300 - val_mse: 32.9300 - val_mae: 4.5800
Epoch 5/10
37/37 [=====] - 1s 24ms/step - loss: 23.1631 - mse:
23.1631 - mae: 3.6596 - val_loss: 33.1983 - val_mse: 33.1983 - val_mae: 4.6068
Epoch 6/10
37/37 [=====] - 1s 25ms/step - loss: 23.0779 - mse:
23.0779 - mae: 3.6229 - val_loss: 32.6675 - val_mse: 32.6675 - val_mae: 4.5641
Epoch 7/10
37/37 [=====] - 1s 26ms/step - loss: 23.0214 - mse:
23.0214 - mae: 3.6568 - val_loss: 32.6421 - val_mse: 32.6421 - val_mae: 4.5628
Epoch 8/10
37/37 [=====] - 1s 25ms/step - loss: 23.1348 - mse:
23.1348 - mae: 3.6231 - val_loss: 33.2085 - val_mse: 33.2085 - val_mae: 4.6162
```

```
Epoch 9/10
37/37 [=====] - 1s 26ms/step - loss: 23.0396 - mse:
23.0396 - mae: 3.6362 - val_loss: 32.4788 - val_mse: 32.4788 - val_mae: 4.5536
Epoch 10/10
37/37 [=====] - 1s 24ms/step - loss: 22.5745 - mse:
22.5745 - mae: 3.5909 - val_loss: 31.7158 - val_mse: 31.7158 - val_mae: 4.5021
```

[264]: <keras.callbacks.History at 0x1c88ed54b0>

model results	model1 - apple only
Lags	10
LSTM	10
epochs	10
train mae	122
test mae	186
train mse	16881
test mse	34747
——	model2 - apple only
Lags	50
LSTM	10
epochs	10
train mae	118
test mae	185
train mse	17124
test mse	185

[]:

[]:

9 Following tensorflow tutorial at <https://www.tensorflow.org/tutorials/stru>

Will be using the difference of the stocks for making the predictions and predicting apple stock difference

```
[265]: n_features_selected = 761 ### This is all features that correlate with apple
        ↪ within 10 lags greater than 0.5 before taking the difference
data_only_apple_corr =
    ↪ data_open[stock_list_greater_point_five_corr_apple_open[0:
    ↪ n_features_selected]]
try:
    data_only_apple_corr = data_only_apple_corr.drop(['AAPL_Open'])
except:
    pass
data_only_apple_corr['AAPL_Open'] = data_open['AAPL_Open'].copy()
```



```
C:\Users\lvden\AppData\Local\Temp\ipykernel_14560\2788200610.py:7:
PerformanceWarning: DataFrame is highly fragmented.  This is usually the result
of calling `frame.insert` many times, which has poor performance.  Consider
joining all columns at once using `pd.concat(axis=1)` instead.  To get a de-
fragmented frame, use `newframe = frame.copy()`
```

```
data_only_apple_corr['AAPL_Open'] = data_open['AAPL_Open'].copy()
```

```
C:\Users\lvden\AppData\Local\Temp\ipykernel_14560\2788200610.py:7:
```

```
SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
```

```
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data_only_apple_corr['AAPL_Open'] = data_open['AAPL_Open'].copy()
```

```
[266]: ### Data Adjustments
differencing_flag = True
standardize_flag = True
day_of_year_flag = True
day_of_week_flag = True

volume_all_stocks_flag = True
low_all_stocks_flag = True
high_all_stocks_flag = True

lstm_units = 80

### Differencing
if differencing_flag == True:
    data_only_apple_corr = data_only_apple_corr.diff().dropna()

if day_of_year_flag == True:
    day_of_year_list = []
    for val in data_only_apple_corr.index:
        day_of_year = pd.Period(val).day_of_year
        day_of_year_list.append(day_of_year)
    data_only_apple_corr['day_of_year'] = day_of_year_list

if day_of_week_flag == True:
    day_of_week_list = []
    for val in data_only_apple_corr.index:
        day_of_week = pd.Period(val).day_of_week
        day_of_week_list.append(day_of_week)
    data_only_apple_corr['day_of_week'] = day_of_week_list

if volume_all_stocks_flag == True:
```

```

data_only_apple_corr = pd.merge(data_only_apple_corr,
                                ↵
                                ↪data_volume[stock_list_greater_point_five_corr_apple_volume[0:
                                ↪n_features_selected]],
                                on='Date')
if low_all_stocks_flag == True:
    data_only_apple_corr = pd.merge(data_only_apple_corr,
                                    ↵
                                    ↪data_low[stock_list_greater_point_five_corr_apple_low[0:
                                    ↪n_features_selected]],
                                    on='Date')
if high_all_stocks_flag == True:
    data_only_apple_corr = pd.merge(data_only_apple_corr,
                                    ↵
                                    ↪data_high[stock_list_greater_point_five_corr_apple_high[0:
                                    ↪n_features_selected]],
                                    on='Date')

```

C:\Users\lvden\AppData\Local\Temp\ipykernel_14560\4281901595.py:23:
PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance. Consider joining all columns at once using `pd.concat(axis=1)` instead. To get a de-fragmented frame, use ``newframe = frame.copy()``

```

data_only_apple_corr['day_of_year'] = day_of_year_list
C:\Users\lvden\AppData\Local\Temp\ipykernel_14560\4281901595.py:30:  

PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
data_only_apple_corr['day_of_week'] = day_of_week_list

```

```

[267]: column_indices = {name: i for i, name in enumerate(data_only_apple_corr.
↪columns)}

```

```

n = len(data_only_apple_corr)
train_df = data_only_apple_corr[0:int(n*0.7)]
val_df = data_only_apple_corr[int(n*0.7):int(n*0.9)]
test_df = data_only_apple_corr[int(n*0.9):]

num_features = data_only_apple_corr.shape[1]

```

```

[268]: if standardize_flag == True:
        train_mean = train_df.mean()
        train_std = train_df.std()
        train_df = (train_df - train_mean) / train_std

```

```
val_df = (val_df - train_mean) / train_std
test_df = (test_df - train_mean) / train_std
```

```
[269]: class WindowGenerator():
    def __init__(self, input_width, label_width, shift,
                 train_df=train_df, val_df=val_df, test_df=test_df,
                 label_columns=None):
        # Store the raw data.
        self.train_df = train_df
        self.val_df = val_df
        self.test_df = test_df

        # Work out the label column indices.
        self.label_columns = label_columns
        if label_columns is not None:
            self.label_columns_indices = {name: i for i, name in
                                         enumerate(label_columns)}
        self.column_indices = {name: i for i, name in
                               enumerate(train_df.columns)}

        # Work out the window parameters.
        self.input_width = input_width
        self.label_width = label_width
        self.shift = shift

        self.total_window_size = input_width + shift

        self.input_slice = slice(0, input_width)
        self.input_indices = np.arange(self.total_window_size)[self.input_slice]

        self.label_start = self.total_window_size - self.label_width
        self.labels_slice = slice(self.label_start, None)
        self.label_indices = np.arange(self.total_window_size)[self.labels_slice]

    def __repr__(self):
        return '\n'.join([
            f'Total window size: {self.total_window_size}',
            f'Input indices: {self.input_indices}',
            f'Label indices: {self.label_indices}',
            f'Label column name(s): {self.label_columns}'])
```

```
[270]: @property
def train(self):
    return self.make_dataset(self.train_df)

@property
def val(self):
```

```

    return self.make_dataset(self.val_df)

@property
def test(self):
    return self.make_dataset(self.test_df)

@property
def example(self):
    """Get and cache an example batch of `inputs, labels` for plotting."""
    result = getattr(self, '_example', None)
    if result is None:
        # No example batch was found, so get one from the `.train` dataset
        result = next(iter(self.train))
        # And cache it for next time
        self._example = result
    return result

WindowGenerator.train = train
WindowGenerator.val = val
WindowGenerator.test = test
WindowGenerator.example = example

```

```

[271]: def plot(self, model=None, plot_col='AAPL_Open', max_subplots=1):
    inputs, labels = self.example
    plt.figure(figsize=(12, 8))
    plot_col_index = self.column_indices[plot_col]
    max_n = min(max_subplots, len(inputs))
    for n in range(max_n):
        plt.subplot(max_n, 1, n+1)
        plt.ylabel(f'{plot_col} [normed]')
        plt.plot(self.input_indices, inputs[n, :, plot_col_index],
                 label='Inputs', marker='.', zorder=-10)

        if self.label_columns:
            label_col_index = self.label_columns_indices.get(plot_col, None)
        else:
            label_col_index = plot_col_index

        if label_col_index is None:
            continue

        plt.scatter(self.label_indices, labels[n, :, label_col_index],
                   edgecolors='k', label='Labels', c='#2ca02c', s=64)
        if model is not None:
            predictions = model(inputs)
            plt.scatter(self.label_indices, predictions[n, :, label_col_index],
                       marker='X', edgecolors='k', label='Predictions',

```

```

        c='#ff7f0e', s=64)

    if n == 0:
        plt.legend()

    plt.xlabel('Time [d]')

WindowGenerator.plot = plot

```

```

[272]: def make_dataset(self, data):
    data = np.array(data, dtype=np.float32)
    ds = tf.keras.utils.timeseries_dataset_from_array(
        data=data,
        targets=None,
        sequence_length=self.total_window_size,
        sequence_stride=1,
        shuffle=True,
        batch_size=32,)

    ds = ds.map(self.split_window)

    return ds

WindowGenerator.make_dataset = make_dataset

```

```

[273]: def split_window(self, features):
    inputs = features[:, self.input_slice, :]
    labels = features[:, self.labels_slice, :]
    if self.label_columns is not None:
        labels = tf.stack(
            [labels[:, :, self.column_indices[name]] for name in self.
↪label_columns],
            axis=-1)

    # Slicing doesn't preserve static shape information, so set the shapes
    # manually. This way the `tf.data.Datasets` are easier to inspect.
    inputs.set_shape([None, self.input_width, None])
    labels.set_shape([None, self.label_width, None])

    return inputs, labels

WindowGenerator.split_window = split_window

```

9.0.1 Creating a naive baseline model that predicts the previous value as the next value

```
[274]: class Baseline(tf.keras.Model):
        def __init__(self, label_index=None):
            super().__init__()
            self.label_index = label_index

        def call(self, inputs):
            if self.label_index is None:
                return inputs
            result = inputs[:, :, self.label_index]
            return result[:, :, tf.newaxis]
```

```
[275]: single_step_window = WindowGenerator(
        input_width=1, label_width=1, shift=1,
        label_columns=['AAPL_Open'])
single_step_window
```

```
[275]: Total window size: 2
Input indices: [0]
Label indices: [1]
Label column name(s): ['AAPL_Open']
```

```
[276]: baseline = Baseline(label_index=column_indices['AAPL_Open'])

baseline.compile(loss=tf.keras.losses.MeanSquaredError(),
                  metrics=[tf.keras.metrics.MeanAbsoluteError(),
                           tf.keras.metrics.MeanAbsolutePercentageError()])

val_performance = {}
performance = {}
val_performance['Baseline'] = baseline.evaluate(single_step_window.val,
        ↪return_dict=True)
performance['Baseline'] = baseline.evaluate(single_step_window.test, verbose=0,
        ↪return_dict=True)
```

```
8/8 [=====] - 0s 3ms/step - loss: 2.4224 -
mean_absolute_error: 1.1933 - mean_absolute_percentage_error: 4096.6313
```

```
[277]: wide_window = WindowGenerator(
        input_width=10, label_width=1, shift=1,
        label_columns=['AAPL_Open'])

wide_window
```

```
[277]: Total window size: 11
Input indices: [0 1 2 3 4 5 6 7 8 9]
```

```
Label indices: [10]
Label column name(s): ['AAPL_Open']
```

9.0.2 Multivariate model first pass through with model parameters printed below performance.

```
[278]: lstm_model = tf.keras.models.Sequential([
    # Shape [batch, time, features] => [batch, time, lstm_units]
    tf.keras.layers.LSTM(32, return_sequences=True),
    # Shape => [batch, time, features]
    tf.keras.layers.Dense(units=1),
])
```

```
[279]: MAX_EPOCHS = 200

def compile_and_fit(model, window, patience=5):
    early_stopping = tf.keras.callbacks.EarlyStopping(monitor='val_loss',
                                                       patience=patience,
                                                       mode='min')

    model.compile(loss=tf.keras.losses.MeanSquaredError(),
                  optimizer=tf.keras.optimizers.Adam(),
                  metrics=[tf.keras.metrics.MeanAbsoluteError(),
                           tf.keras.metrics.MAPE])

    history = model.fit(window.train, epochs=MAX_EPOCHS,
                        validation_data=window.val,
                        callbacks=[early_stopping])

    return history
```

```
[280]: history = compile_and_fit(lstm_model, wide_window)

IPython.display.clear_output()
val_performance['LSTM'] = lstm_model.evaluate(wide_window.val, return_dict=True)
performance['LSTM'] = lstm_model.evaluate(wide_window.test, verbose=0,
    ↪return_dict=True)
print(performance)
```

```
8/8 [=====] - 0s 6ms/step - loss: 1.2930 -
mean_absolute_error: 0.9083 - mean_absolute_percentage_error: 1209.6652
{'Baseline': {'loss': 2.0856449604034424, 'mean_absolute_error':
1.005099892616272, 'mean_absolute_percentage_error': 896.5128173828125}, 'LSTM':
{'loss': 1.0000287294387817, 'mean_absolute_error': 0.752971351146698,
'mean_absolute_percentage_error': 379.5516662597656}}
```

```
[281]: # wide_window.plot(lstm_model)
# plt.suptitle('Three random predictions')
# plt.show
```

9.0.3 Model and data parameters used for this model

```
[282]: for key in performance:
        for key2 in performance[key]:
            performance[key][key2] = np.round(performance[key][key2], 5)
```

```
[283]: print(performance)
print('standardize: ' + str(standardize_flag))
print('differencing: ' + str(differencing_flag))
print('day_of_year: ' + str(day_of_year_flag))
print('day of week: ' + str(day_of_week_flag))
print('volume: ' + str(volume_all_stocks_flag))
print('high: ' + str(high_all_stocks_flag))
print('low: ' + str(low_all_stocks_flag))
print('num features: ' + str(n_features_selected))
print('LSTM units: ' + str(lstm_units))
```

```
{'Baseline': {'loss': 2.08564, 'mean_absolute_error': 1.0051,
'mean_absolute_percentage_error': 896.51282}, 'LSTM': {'loss': 1.00003,
'mean_absolute_error': 0.75297, 'mean_absolute_percentage_error': 379.55167}}
standardize: True
differencing: True
day_of_year: True
day of week: True
volume: True
high: True
low: True
num features: 761
LSTM units: 80
```

```
[293]: aux_data = pd.read_csv('./Aux_Data_luke/AAPL.csv')
```

9.1 Function to conduct a grid search of model parameters

```
[294]: def lstm_apple_modeling_v4(data_open,
    ↪ stock_list_greater_point_five_corr_apple_open,
        stock_list_greater_point_five_corr_apple_volume,
        stock_list_greater_point_five_corr_apple_low,
        stock_list_greater_point_five_corr_apple_high,
        stock_list_greater_point_five_corr_apple_close,
        aux_data,
        n_features_selected,
        differencing_flag, standardize_flag, aux_data_flag,
    ↪ day_of_year_flag, day_of_week_flag,
        volume_all_stocks_flag, low_all_stocks_flag,
    ↪ high_all_stocks_flag, close_all_stocks_flag, lstm_units,
        lag_len = 10):
```



```

'''VERSION 4 INCLUDES THE CLOSED SALE PRICE AND ABILITY TO INCREASE LAG SIZE
    data_open is the data frame data_open
    stock_list_greater_point_five_corr_apple_open is_
↪stock_list_greater_point_five_corr_apple_open
    n_features_selected is between 1 and 761
    all flags are booleans
    lstm_units is the number of units in the lstm'''
### Original data and the number of additional points that we want
n_features_selected = n_features_selected
data_only_apple_corr =_
↪data_open[stock_list_greater_point_five_corr_apple_open[0:
↪n_features_selected]]
    try:
        data_only_apple_corr = data_only_apple_corr.drop(['AAPL_Open'])
    except:
        pass
data_only_apple_corr['AAPL_Open'] = data_open['AAPL_Open'].copy()

### Data Adjustments
differencing_flag = differencing_flag
standardize_flag = standardize_flag
day_of_year_flag = day_of_year_flag
day_of_week_flag = day_of_week_flag

volume_all_stocks_flag = volume_all_stocks_flag
low_all_stocks_flag = low_all_stocks_flag
high_all_stocks_flag = high_all_stocks_flag

lstm_units = lstm_units

### Differencing
if differencing_flag == True:
    data_only_apple_corr = data_only_apple_corr.diff().dropna()

### Introducing new features
if aux_data_flag == True:
    aux_data2 = aux_data.copy()
    ### Need to replace string dollar signs as ints
    sales_list = []
    [sales_list.append(int(float(a.split('$')[1].strip().split(',')[0] +
        a.split('$')[1].strip().split(',')[1])))_
↪for a in aux_data2[' Sales ']]
    freecashflow_list = []
    [freecashflow_list.append(int(float(a.split('$')[1].strip().
↪split(',')[0] +
        a.split('$')[1].strip().
↪split(',')[1])))] for a in aux_data2[' FreeCashFlow ']]

```

```

        cash_list = []
        [cash_list.append(int(float(a.split('$')[1].strip().split(',')[0] +
                                a.split('$')[1].strip().split(',')[1])))
↪for a in aux_data2[' Cash ']]
        debt_list = []
        [debt_list.append(int(float(a.split('$')[1].strip().split(',')[0] +
                                a.split('$')[1].strip().split(',')[1])))
↪for a in aux_data2[' Long Term Debt ']]

        aux_data2[' Sales '] = sales_list
        aux_data2[' FreeCashFlow '] = freecashflow_list
        aux_data2[' Cash '] = cash_list
        aux_data2[' Long Term Debt '] = debt_list

        aux_data2 = aux_data2.rename({' Sales ':'Sales',
                                       list(aux_data2.columns)[2]:
↪'FreeCashFlow', ' Cash ':'Cash',
                                       list(aux_data2.columns)[4]:
↪'LongTermDebt'})
        date_list = []
        [date_list.append(pd.to_datetime(a).date()) for a in aux_data2['Date']]
↪### Need to change the dates to same object to match and
        ### change back the date object in the primary dataframe to continue
↪merging with the rest of the data
        aux_data2['Date'] = date_list
        aux_data2 = aux_data2.set_index('Date')
        aux_data_sub = aux_data2[aux_data2.index >= pd.
↪to_datetime(min(data_only_apple_corr.index)).date()]
        ### aux_data has older entries, so only want the entries on the dates
↪we have stock data for
        date_list = []
        [date_list.append(pd.to_datetime(a).date()) for a in
↪list(data_only_apple_corr.index)]
        old_index = list(data_only_apple_corr.index)
        data_only_apple_corr['Date'] = date_list
        data_only_apple_corr = data_only_apple_corr.set_index('Date')
        data_only_apple_corr = pd.merge(data_only_apple_corr, aux_data_sub,
↪on='Date', how='left').ffill().bfill()
        data_only_apple_corr = data_only_apple_corr.set_index(pd.
↪Series(old_index, name='Date'))

        if day_of_year_flag == True:
            day_of_year_list = []
            for val in data_only_apple_corr.index:
                day_of_year = pd.Period(val).day_of_year

```

```

        day_of_year_list.append(day_of_year)
        data_only_apple_corr['day_of_year'] = day_of_year_list

    if day_of_week_flag == True:
        day_of_week_list = []
        for val in data_only_apple_corr.index:
            day_of_week = pd.Period(val).day_of_week
            day_of_week_list.append(day_of_week)
            data_only_apple_corr['day_of_week'] = day_of_week_list

    if volume_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         □
↳data_volume[stock_list_greater_point_five_corr_apple_volume[0:
↳n_features_selected]],
                                         on='Date')

    if low_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         □
↳data_low[stock_list_greater_point_five_corr_apple_low[0:
↳n_features_selected]],
                                         on='Date')

    if high_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         □
↳data_high[stock_list_greater_point_five_corr_apple_high[0:
↳n_features_selected]],
                                         on='Date')

    if close_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         □
↳data_close[stock_list_greater_point_five_corr_apple_close[0:
↳n_features_selected]],
                                         on='Date')

    column_indices = {name: i for i, name in enumerate(data_only_apple_corr.
↳columns)}
    n = len(data_only_apple_corr)
    train_df = data_only_apple_corr[0:int(n*0.7)]
    val_df = data_only_apple_corr[int(n*0.7):int(n*0.9)]
    test_df = data_only_apple_corr[int(n*0.9):]
    num_features = data_only_apple_corr.shape[1]

    if standardize_flag == True:
        train_mean = train_df.mean()
        train_std = train_df.std()

```

```

train_df = (train_df - train_mean) / train_std
val_df = (val_df - train_mean) / train_std
test_df = (test_df - train_mean) / train_std

@property
def train(self):
    return self.make_dataset(self.train_df)

@property
def val(self):
    return self.make_dataset(self.val_df)

@property
def test(self):
    return self.make_dataset(self.test_df)

@property
def example(self):
    """Get and cache an example batch of `inputs, labels` for plotting."""
    result = getattr(self, '_example', None)
    if result is None:
        # No example batch was found, so get one from the `.train` dataset
        result = next(iter(self.train))
        # And cache it for next time
        self._example = result
    return result

WindowGenerator.train = train
WindowGenerator.val = val
WindowGenerator.test = test
WindowGenerator.example = example

baseline = Baseline(label_index=column_indices['AAPL_Open'])

baseline.compile(loss=tf.keras.losses.MeanSquaredError(),
                 metrics=[tf.keras.metrics.MeanAbsoluteError(),
                         tf.keras.metrics.MeanAbsolutePercentageError()])

val_performance = {}
performance = {}
val_performance['Baseline'] = baseline.evaluate(single_step_window.val,
↪return_dict=True)
performance['Baseline'] = baseline.evaluate(single_step_window.test,
↪verbose=0, return_dict=True)

wide_window = WindowGenerator(

```

```

        input_width=10, label_width=10, shift=1,
        label_columns=['AAPL_Open'])

lstm_model = tf.keras.models.Sequential([
    # Shape [batch, time, features] => [batch, time, lstm_units]
    tf.keras.layers.LSTM(lstm_units, return_sequences=True),
    # Shape => [batch, time, features]
    tf.keras.layers.Dense(units=1),
])

history = compile_and_fit(lstm_model, wide_window)

IPython.display.clear_output()
val_performance['LSTM'] = lstm_model.evaluate(wide_window.val,
↪return_dict=True)
performance['LSTM'] = lstm_model.evaluate(wide_window.test, verbose=0,
↪return_dict=True)

for key in performance:
    for key2 in performance[key]:
        performance[key][key2] = np.round(performance[key][key2], 5)

return [n_features_selected, differencing_flag, standardize_flag,
↪day_of_year_flag, day_of_week_flag,
        volume_all_stocks_flag, low_all_stocks_flag, high_all_stocks_flag,
↪close_all_stocks_flag,
        aux_data_flag, lstm_units, lag_len, performance]

```

```

[443]: output = lstm_apple_modeling_v4(data_open,
        stock_list_greater_point_five_corr_apple_open =
↪stock_list_greater_point_five_corr_apple_open,
        stock_list_greater_point_five_corr_apple_volume =
↪stock_list_greater_point_five_corr_apple_volume,
        stock_list_greater_point_five_corr_apple_low =
↪stock_list_greater_point_five_corr_apple_low,
        stock_list_greater_point_five_corr_apple_high =
↪stock_list_greater_point_five_corr_apple_high,
        stock_list_greater_point_five_corr_apple_close =
↪stock_list_greater_point_five_corr_apple_close,
        aux_data = aux_data,
↪n_features_selected = 761,
        differencing_flag = True,
↪standardize_flag = True,
        aux_data_flag = True,
        day_of_year_flag = True,
↪day_of_week_flag = True,

```

```

                                volume_all_stocks_flag = True,
    ↪low_all_stocks_flag = True,
                                high_all_stocks_flag = True,
    ↪close_all_stocks_flag=True,
                                lstm_units=128, lag_len=10)

```

8/8 [=====] - 0s 6ms/step - loss: 1.5589 -
mean_absolute_error: 0.9905 - mean_absolute_percentage_error: 2254.5645

```

[444]: scores_baseline = output[12]['Baseline']
scores_lstm = output[12]['LSTM']
n_feat = output[0]
diff_f = output[1]
stand_f = output[2]
aux_f = output[9]
year_f = output[3]
week_f = output[4]
vol_f = output[5]
low_f = output[6]
high_f = output[7]
close_f = output[8]
lstm_units = output[10]
lag_len = output[11]

print(f'''Score is as follows: \nBASELINE {scores_baseline} \nLSTM
    ↪{scores_lstm} \n
Number of features selected: {n_feat} \n
Differencing flag: {diff_f} \n
Standardization flag: {stand_f} \n
Auxilliary data flag: {aux_f} \n
Day of the year flag: {year_f} \n
Day of the week flag: {week_f} \n
Volume of stocks flag: {vol_f} \n
Low of stocks flag: {low_f} \n
High of stocks flag: {high_f} \n
Close of all stocks flag: {close_f} \n
LSTM units: {lstm_units} \n
Number of lags: {lag_len}''')

```

Score is as follows:

```

BASELINE {'loss': 2.08564, 'mean_absolute_error': 1.0051,
'mean_absolute_percentage_error': 896.51276}
LSTM      {'loss': 1.39949, 'mean_absolute_error': 0.93672,
'mean_absolute_percentage_error': 688.4267}

```

Number of features selected: 761

Differencing flag: True

Standardization flag: True

Auxilliary data flag: True

Day of the year flag: True

Day of the week flag: True

Volume of stocks flag: True

Low of stocks flag: True

High of stocks flag: True

Close of all stocks flag: True

LSTM units: 128

Number of lags: 10

[]:

[]:

9.2 Grid search of hyperparameters. Best and most consistent performer is with all extra features

- day of week
- day of year
- all stock high, low, close, and volume values
- auxiliary data on stock such as debt and free cash

```
[297]: # if input('Are you sure you want to run this long function?') == 'y':

#     grid_search_df = pd.DataFrame(columns=['n_features_selected',
# #                                         'differencing_flag', 'standardize_flag',
# → 'day_of_year_flag', 'day_of_week_flag',
# #                                         'volume_all_stocks_flag', 'low_all_stocks_flag',
# → 'high_all_stocks_flag', 'close_all_stocks_flag',
# #                                         'aux_data', 'lstm_units', 'lag_length',
# #                                         'baseline--loss', 'LSTM--loss',
# → 'baseline--MAE', 'LSTM--MAE', 'baseline--MAPE', 'LSTM--MAPE'])

#     # flag_range_diff = [False, True]
#     # flag_range_stand = [False, True]
#     # flag_range_doy = [False, True]
```

```

#     flag_range_dow = [False, True]
#     flag_range_vol = [False, True]
#     flag_range_low = [False, True]
#     flag_range_high = [False, True]
#     flag_range_close = [False, True]
#     flag_range_aux = [False, True]

#     n_features_selected_range = 761
#     lstm_units_range = [16,32,64,128,200]
#     lag_length_range = [10,20,30]

#     total_number_search = 2**7*5*3

#     i=0
#     for flag_range_doy_val in flag_range_doy:
#         for flag_range_dow_val in flag_range_dow:
#             for flag_range_vol_val in flag_range_vol:
#                 for flag_range_low_val in flag_range_low:
#                     for flag_range_high_val in flag_range_high:
#                         for flag_range_close_val in flag_range_close:
#                             for flag_range_aux_val in flag_range_aux:
#                                 for lstm_units_range_val in lstm_units_range:
#                                     for lag_length_range_val in lag_length_range:
#                                         ↪lag_length_range:
#                                             output = lstm_apple_modeling_v4(data_open = data_open,
#                                             ↪stock_list_greater_point_five_corr_apple_open = stock_list_greater_point_five_corr_apple_open,
#                                             ↪stock_list_greater_point_five_corr_apple_volume = stock_list_greater_point_five_corr_apple_volume,
#                                             ↪stock_list_greater_point_five_corr_apple_low = stock_list_greater_point_five_corr_apple_low,
#                                             ↪stock_list_greater_point_five_corr_apple_high = stock_list_greater_point_five_corr_apple_high,
#                                             ↪stock_list_greater_point_five_corr_apple_close = stock_list_greater_point_five_corr_apple_close,

```



```

#                                     aux_data = aux_data,
↳n_features_selected = n_features_selected_range,
#                                     differencing_flag = True,
↳standardize_flag = True,
#                                     aux_data_flag =
↳flag_range_aux_val,
#                                     day_of_year_flag =
↳flag_range_doy_val, day_of_week_flag = flag_range_dow_val,
#                                     volume_all_stocks_flag =
↳flag_range_vol_val, low_all_stocks_flag = flag_range_low_val,
#                                     high_all_stocks_flag =
↳flag_range_high_val, close_all_stocks_flag=flag_range_close_val,
#                                    
↳lstm_units=lstm_units_range_val, lag_len=lag_length_range_val)
#                                     i+=1
#                                     print('Percent complete: ' + str(np.
↳round(i/total_number_search*100,3)))
#                                     grid_search_df.
↳loc[len(grid_search_df)] = [output[0],
↳output[1],output[2],output[3],output[4],
#                                     output[5],output[6],
↳output[7],output[8], output[9], output[10],output[11],
#                                     output[12]['Baseline']['loss'],
#                                     output[12]['LSTM']['loss'],
#                                    
↳output[12]['Baseline']['mean_absolute_error'],
#                                    
↳output[12]['LSTM']['mean_absolute_error'],
#                                    
↳output[12]['Baseline']['mean_absolute_percentage_error'],
#                                    
↳output[12]['LSTM']['mean_absolute_percentage_error']]

```

```
[298]: # grid_search_df.shape
```

```
[299]: # grid_search_df.to_csv('LSTM_grid_search_results_v2.csv')
```

```
[ ]:
```

10 Plot predictions with best performing model based upon grid search results

```
[300]: output = lstm_apple_modeling_v4(data_open,
stock_list_greater_point_five_corr_apple_open =
↳stock_list_greater_point_five_corr_apple_open,
```

```

        stock_list_greater_point_five_corr_apple_volume =
↪stock_list_greater_point_five_corr_apple_volume,
        stock_list_greater_point_five_corr_apple_low =
↪stock_list_greater_point_five_corr_apple_low,
        stock_list_greater_point_five_corr_apple_high =
↪stock_list_greater_point_five_corr_apple_high,
        stock_list_greater_point_five_corr_apple_close =
↪stock_list_greater_point_five_corr_apple_close,
                                aux_data = aux_data,
↪n_features_selected = 761,
                                differencing_flag = True,
↪standardize_flag = True,
                                aux_data_flag = True,
                                day_of_year_flag = True,
↪day_of_week_flag = True,
                                volume_all_stocks_flag = True,
↪low_all_stocks_flag = True,
                                high_all_stocks_flag = True,
↪close_all_stocks_flag=True,
                                lstm_units=128, lag_len=10)

```

8/8 [=====] - 0s 7ms/step - loss: 1.6290 -
mean_absolute_error: 0.9937 - mean_absolute_percentage_error: 4181.1831

```
[445]: baseline_performance = baseline.evaluate(wide_window.test, return_dict=True)
```

4/4 [=====] - 0s 5ms/step - loss: 1.7157 -
mean_absolute_error: 0.9958 - mean_absolute_percentage_error: 719.6003

```
[301]: test_performance = lstm_model.evaluate(wide_window.test, return_dict = True)
```

4/4 [=====] - 0s 6ms/step - loss: 1.0000 -
mean_absolute_error: 0.7530 - mean_absolute_percentage_error: 379.5517

```
[302]: labels = test_df['AAPL_Open']
labels.index = pd.to_datetime(labels.index, utc=True)
```

```
[303]: input_test_list = []
label_test_list = []

for input_test, label_test in wide_window.test:
    input_test_list.append(input_test)
    label_test_list.append(label_test)
```

```
[304]: pred_output_list = []
for batch in input_test_list:
    for row in batch:
        row2 = np.asarray(row).reshape(1,10,3047)
```

```
pred_output_list.append(lstm_model.predict(row2))
```

```
1/1 [=====] - 0s 312ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 14ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 18ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
```

```

1/1 [=====] - 0s 18ms/step
1/1 [=====] - 0s 18ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 18ms/step
1/1 [=====] - 0s 19ms/step
1/1 [=====] - 0s 19ms/step
1/1 [=====] - 0s 19ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 17ms/step
1/1 [=====] - 0s 22ms/step
1/1 [=====] - 0s 22ms/step
1/1 [=====] - 0s 23ms/step
1/1 [=====] - 0s 20ms/step
1/1 [=====] - 0s 21ms/step
1/1 [=====] - 0s 21ms/step
1/1 [=====] - 0s 23ms/step
1/1 [=====] - 0s 22ms/step
1/1 [=====] - 0s 20ms/step
1/1 [=====] - 0s 21ms/step
1/1 [=====] - 0s 19ms/step
1/1 [=====] - 0s 19ms/step
1/1 [=====] - 0s 18ms/step

```

```

[305]: pred_output_first_val_list = []
       for a in pred_output_list:
           pred_output_first_val_list.append(a[0][0][0])

       pred_series = pd.Series(pred_output_first_val_list, index = labels[10:].index)

```

```

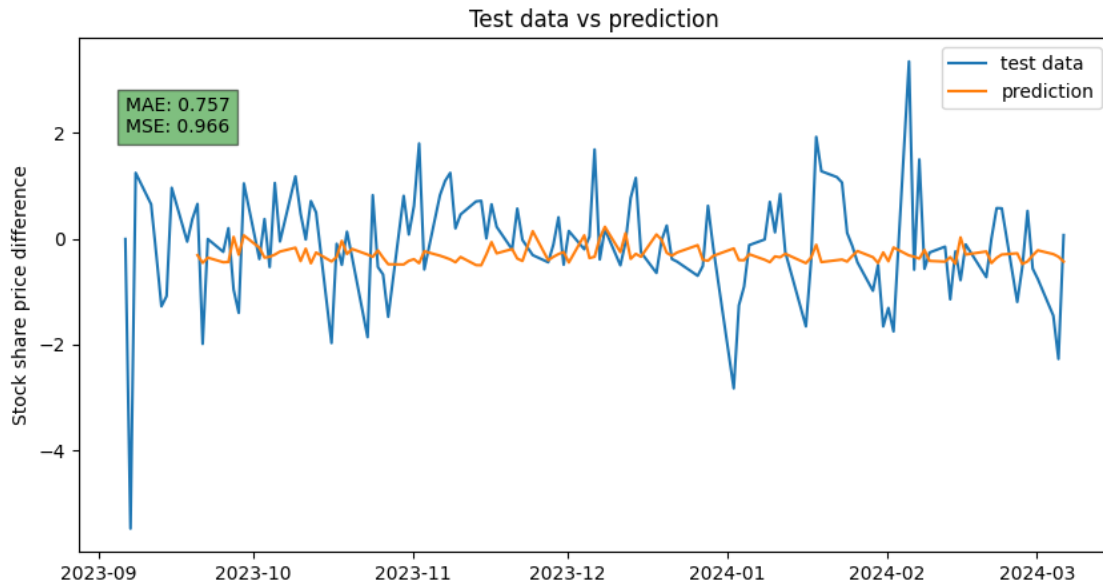
[306]: fig = plt.figure(figsize=(10,5))
       ax = fig.add_subplot(111)

       mae = 'MAE: ' + str(np.round(np.mean(np.abs(pred_series - labels[10:])),3))
       mse = 'MSE: ' + str(np.round(np.mean((pred_series - labels[10:]**2),3))

       ax.text(labels.index[0], 2, mae + '\n' + mse, bbox={'facecolor':'green',
           ↪ 'alpha':0.5})
       ax.plot(labels, label='test data')
       ax.plot(pred_series, label='prediction')
       ax.legend()
       ax.set_title('Test data vs prediction')
       ax.set_ylabel('Stock share price difference')

       plt.show()

```



[]:

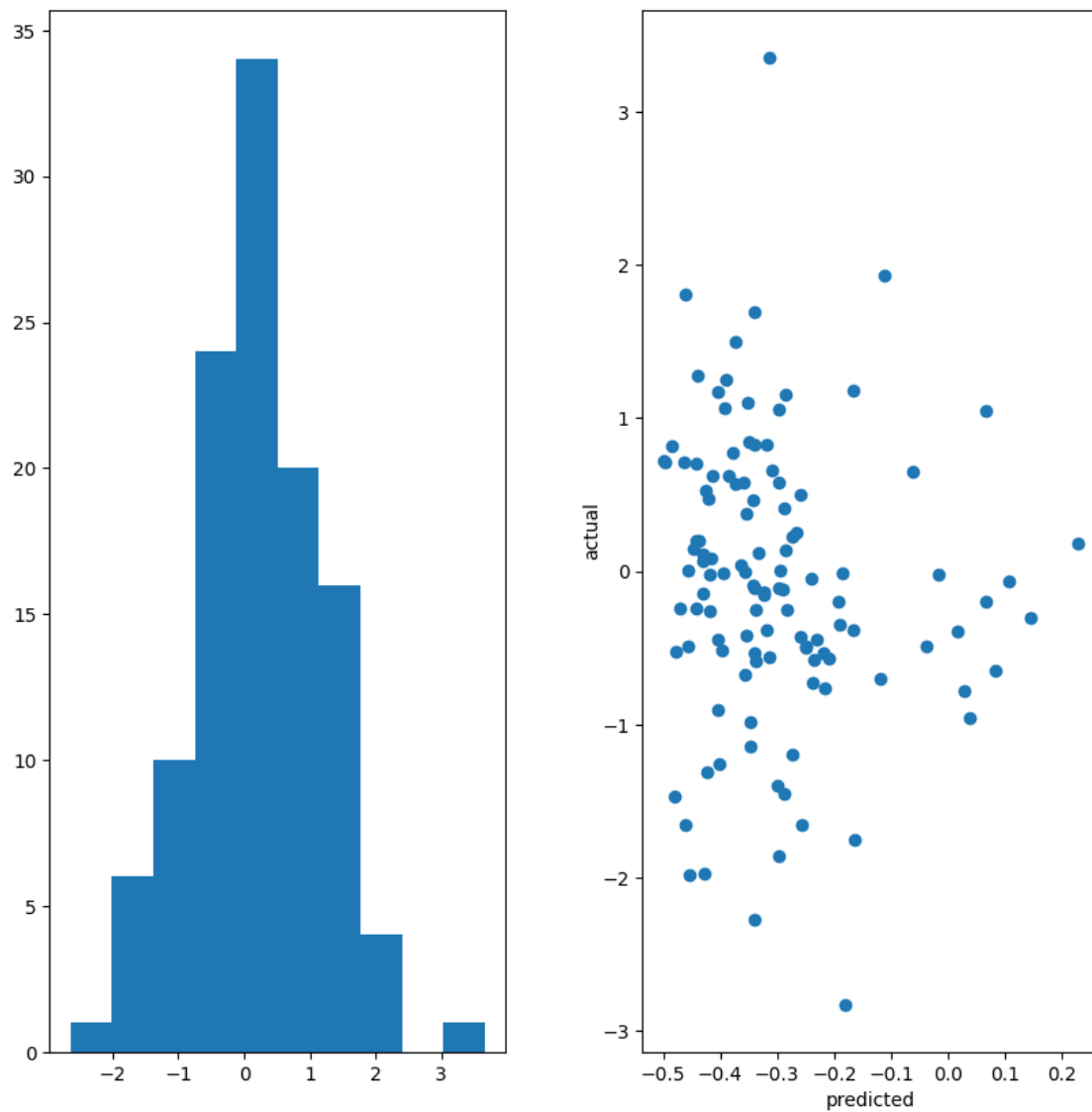
11 Plot the residuals and do the residual analysis

```
[307]: resid = labels[10:] - pred_series
```

```
[308]: fig = plt.figure(figsize=(10,10))
f, (ax1,ax2) = plt.subplots(1,2)
ax1.hist(resid)
ax2.scatter(pred_series,labels[10:])
ax2.set_xlabel('predicted')
ax2.set_ylabel('actual', labelpad=0)
plt.suptitle('Residual plots')
plt.subplots_adjust(wspace=0.3)
plt.show()
```

<Figure size 1000x1000 with 0 Axes>

Residual plots



The histogram of the residuals appears normally distributed, however the scatterplot shows a bias towards a lower prediction of the stock difference value.

[]:

[]:

12 4. Modeling - ARIMA 5. Predicting with ARIMA

13 DURGESH TIWARI

```
[309]: import pyarrow as pa
import pyarrow.parquet as pq

# Read CSV file
df = pd.read_csv('./nasdaq_stock_price_y5_daily/AAPL-Apple Inc. Common Stock.
↳csv')

# covert df to prequet file to save space and time
table = pa.Table.from_pandas(df)

path = 'asset.parquet'

pq.write_table(table, path)

print(f"CSV file '{df.shape[0]}' rows and '{df.shape[1]}' columns converted to_
↳Parquet format.")
print(f"Parquet file saved at '{path}'.")
```

CSV file '1260' rows and '8' columns converted to Parquet format.

Parquet file saved at 'asset.parquet'.

```
[310]: # Specify the path to your Parquet file
path = 'asset.parquet'

df = pd.read_parquet(path)

# Set the index to the DateTimeIndex if applicable
df.set_index('Date', inplace=True) # Assuming 'Date' is the column containing_
↳datetime values

# Drop rows with NaN values
df.dropna(inplace=True)
# first difference

df['Open_diff'] = df['Open'].diff()

df['Open_diff'].fillna(0, inplace=True)

# Print the head of the DataFrame (first few rows)
print(df.head())
```

	Open	High	Low	Close \
Date				
2019-03-06 00:00:00-05:00	42.038652	42.236007	41.862960	42.002552

2019-03-07 00:00:00-05:00	41.846107	41.983294	41.400861	41.516384
2019-03-08 00:00:00-05:00	40.991724	41.653579	40.794369	41.615070
2019-03-11 00:00:00-04:00	42.236007	43.109654	42.202313	43.056705
2019-03-12 00:00:00-04:00	43.321448	43.964049	43.169821	43.540462

Date	Volume	Dividends	Stock Splits	Open_diff
2019-03-06 00:00:00-05:00	83241600	0.0	0.0	0.000000
2019-03-07 00:00:00-05:00	99185600	0.0	0.0	-0.192544
2019-03-08 00:00:00-05:00	95997600	0.0	0.0	-0.854384
2019-03-11 00:00:00-04:00	128044000	0.0	0.0	1.244284
2019-03-12 00:00:00-04:00	129870400	0.0	0.0	1.085440

C:\Users\lvden\AppData\Local\Temp\ipykernel_14560\844830508.py:15:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

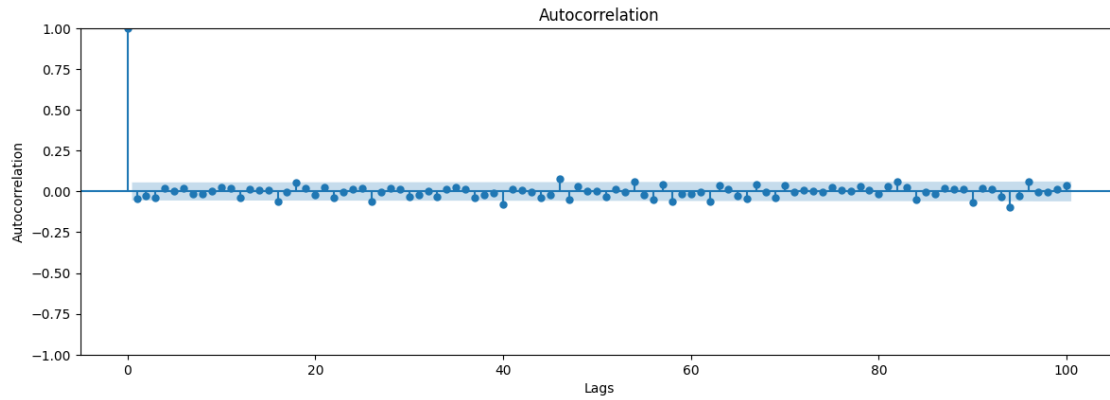
```
df['Open_diff'].fillna(0, inplace=True)
```

```
[311]: # you can see there is no seasonality form the below plot hence we not need to
      ↪ SARIMA for this data

# Plot autocorrelation function (ACF)
plt.figure(figsize=(12, 8))

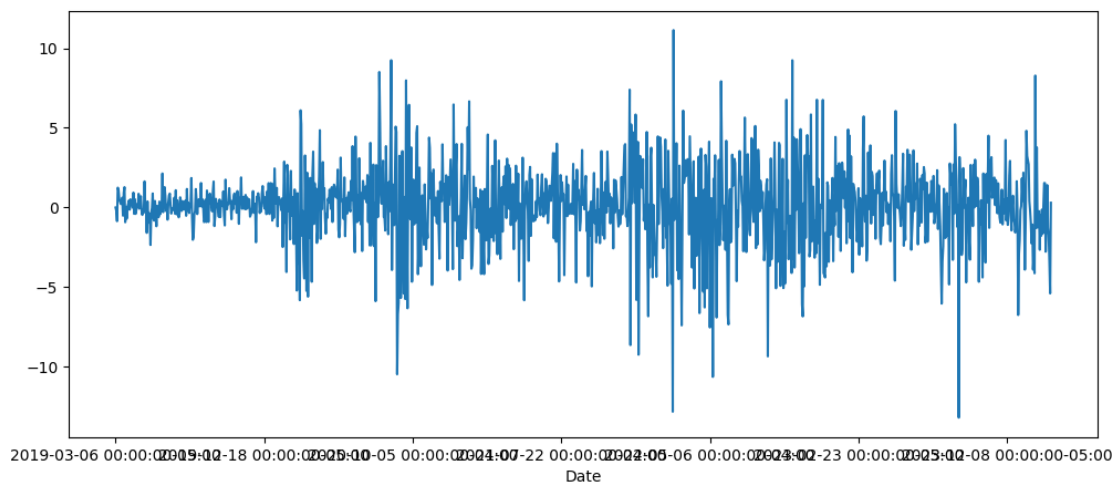
# ACF plot
plt.subplot(2, 1, 1)
plot_acf(df['Open_diff'], lags=100, ax=plt.gca()) # Adjust 'lags' based on
      ↪ your data's frequency
plt.xlabel('Lags')
plt.ylabel('Autocorrelation')

plt.tight_layout()
plt.show()
```

```
[312]: df['Open_diff'].plot(figsize=(12,5))
```

```
[312]: <Axes: xlabel='Date'>
```



```
[313]: result = adfuller(df['Open_diff'])

# p-value obtained from the ADF test is below 0.05 so data is stationary

# Print the results
print('ADF Statistic:', result[0])
print('p-value:', result[1])
print('Critical Values:')
for key, value in result[4].items():
    print(f'    {key}: {value}')
```

```
ADF Statistic: -37.05094702145631
```

p-value: 0.0
Critical Values:
1%: -3.435554672689999
5%: -2.863838402102491
10%: -2.5679936943096284

```
[317]: from pmdarima import auto_arima
import warnings
warnings.filterwarnings("ignore")
step_wise_fit=auto_arima(df['Open_diff'],trace=True,suppress_warnings=True,d=0)
step_wise_fit.summary()
```

Performing stepwise search to minimize aic

```
ARIMA(2,0,2)(0,0,0)[0] intercept : AIC=5866.588, Time=0.59 sec
ARIMA(0,0,0)(0,0,0)[0] intercept : AIC=5863.057, Time=0.09 sec
ARIMA(1,0,0)(0,0,0)[0] intercept : AIC=5862.614, Time=0.14 sec
ARIMA(0,0,1)(0,0,0)[0] intercept : AIC=5862.473, Time=0.17 sec
ARIMA(0,0,0)(0,0,0)[0]          : AIC=5863.213, Time=0.09 sec
ARIMA(1,0,1)(0,0,0)[0] intercept : AIC=5862.968, Time=0.45 sec
ARIMA(0,0,2)(0,0,0)[0] intercept : AIC=5863.511, Time=0.21 sec
ARIMA(1,0,2)(0,0,0)[0] intercept : AIC=5864.870, Time=0.35 sec
ARIMA(0,0,1)(0,0,0)[0]          : AIC=5862.844, Time=0.10 sec
```

Best model: ARIMA(0,0,1)(0,0,0)[0] intercept

Total fit time: 2.197 seconds

[317]:

Dep. Variable:	y	No. Observations:	1260
Model:	SARIMAX(0, 0, 1)	Log Likelihood	-2928.237
Date:	Sun, 31 Mar 2024	AIC	5862.473
Time:	14:32:10	BIC	5877.890
Sample:	0	HQIC	5868.267
	- 1260		
Covariance Type:	opg		

	coef	std err	z	P> z	[0.025	0.975]
intercept	0.1024	0.068	1.505	0.132	-0.031	0.236
ma.L1	-0.0466	0.021	-2.259	0.024	-0.087	-0.006
sigma2	6.1111	0.161	37.905	0.000	5.795	6.427

Ljung-Box (L1) (Q):	0.00	Jarque-Bera (JB):	456.70
Prob(Q):	0.96	Prob(JB):	0.00
Heteroskedasticity (H):	1.70	Skew:	-0.42
Prob(H) (two-sided):	0.00	Kurtosis:	5.83

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
[318]: # Calling Arima model from stats fucntion
from statsmodels.tsa.arima.model import ARIMA
```

```
print(df.shape)

train=df.iloc[:-100]
test=df.iloc[-100:] # last 100 for test
```

(1260, 8)

```
[319]: model=ARIMA(train['Open_diff'],order=(0,0,1))
model=model.fit()
model.summary()
```

```
[319]:
```

Dep. Variable:	Open_diff	No. Observations:	1160
Model:	ARIMA(0, 0, 1)	Log Likelihood	-2701.795
Date:	Sun, 31 Mar 2024	AIC	5409.590
Time:	14:32:13	BIC	5424.758
Sample:	0	HQIC	5415.313
	- 1160		
Covariance Type:	opg		

	coef	std err	z	P> z	[0.025	0.975]
const	0.1170	0.070	1.668	0.095	-0.020	0.254
ma.L1	-0.0643	0.021	-3.023	0.003	-0.106	-0.023
sigma2	6.1742	0.168	36.708	0.000	5.845	6.504

Ljung-Box (L1) (Q):	0.01	Jarque-Bera (JB):	453.48
Prob(Q):	0.93	Prob(JB):	0.00
Heteroskedasticity (H):	2.44	Skew:	-0.46
Prob(H) (two-sided):	0.00	Kurtosis:	5.92

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
[320]: start=len(train)
end=len(train)+len(test)-1
```

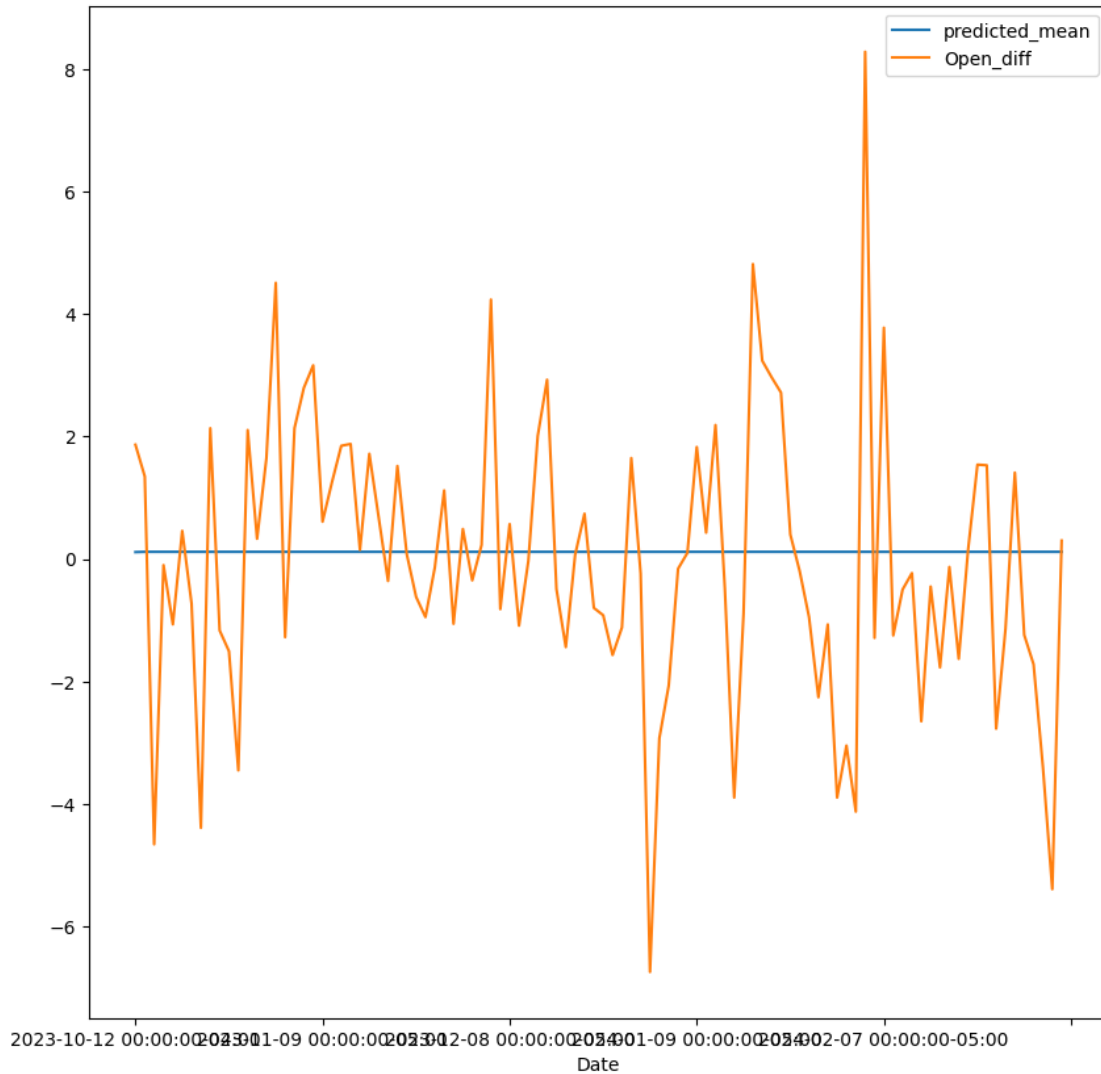
```
[321]: prediction=model.predict(start=start,end=end,type='levels')
prediction.index=df.index[start:end+1]
print(prediction)
```

```
Date
2023-10-12 00:00:00-04:00    0.112480
2023-10-13 00:00:00-04:00    0.116982
2023-10-16 00:00:00-04:00    0.116982
2023-10-17 00:00:00-04:00    0.116982
2023-10-18 00:00:00-04:00    0.116982
...
2024-02-29 00:00:00-05:00    0.116982
2024-03-01 00:00:00-05:00    0.116982
2024-03-04 00:00:00-05:00    0.116982
2024-03-05 00:00:00-05:00    0.116982
```

2024-03-06 00:00:00-05:00 0.116982
Name: predicted_mean, Length: 100, dtype: float64

```
[322]: prediction.plot(legend=True)  
test['Open_diff'].plot(legend=True)
```

[322]: <Axes: xlabel='Date'>



```
[323]: #to check how good model is doing if mean of data set is close to rms then  
        ↳model is not doing good  
        # So accordint to this mean of data set is 185 and rsm is 10 so model is doing  
        ↳great in terms of data we have  
  
test['Open_diff'].mean()
```

[323]: -0.06678732644810736

```
[324]: from sklearn.metrics import mean_squared_error, mean_absolute_error
from math import sqrt
rms=sqrt(mean_squared_error(prediction,test['Open_diff']))
print(f"print mean squared error: {rms}")

mae = mean_absolute_error(prediction, test['Open_diff'])
print(f"mean absolute error: {mae}")
```

print mean squared error: 2.2951756946086985

mean absolute error: 1.7162559868462497

```
[325]: # Train model for the hole data set now:

model1=ARIMA(df['Open_diff'],order=(0,0,1))
model1=model1.fit()
df.tail()
```

```
[325]:
```

	Open	High	Low	Close \
Date				
2024-02-29 00:00:00-05:00	181.270004	182.570007	179.529999	180.750000
2024-03-01 00:00:00-05:00	179.550003	180.529999	177.380005	179.660004
2024-03-04 00:00:00-05:00	176.149994	176.899994	173.789993	175.100006
2024-03-05 00:00:00-05:00	170.759995	172.039993	169.619995	170.119995
2024-03-06 00:00:00-05:00	171.059998	171.210007	168.940002	170.020004

	Volume	Dividends	Stock Splits	Open_diff
Date				
2024-02-29 00:00:00-05:00	136682600	0.0	0.0	-1.239990
2024-03-01 00:00:00-05:00	73488000	0.0	0.0	-1.720001
2024-03-04 00:00:00-05:00	81510100	0.0	0.0	-3.400009
2024-03-05 00:00:00-05:00	94983800	0.0	0.0	-5.389999
2024-03-06 00:00:00-05:00	39173318	0.0	0.0	0.300003

```
[326]: # Future Value Prediction
# Define the start date
start_date = '2024-03-06 00:00:00-05:00'

# Create the date range for 100 days
date_range = pd.date_range(start=start_date, periods=101, freq='D')
# future_index=date_range(start=, end=)

prediction_whole=model1.predict(start=len(df),end=len(df)+100,type='levels')

prediction_whole.index=date_range

print(prediction_whole)
```

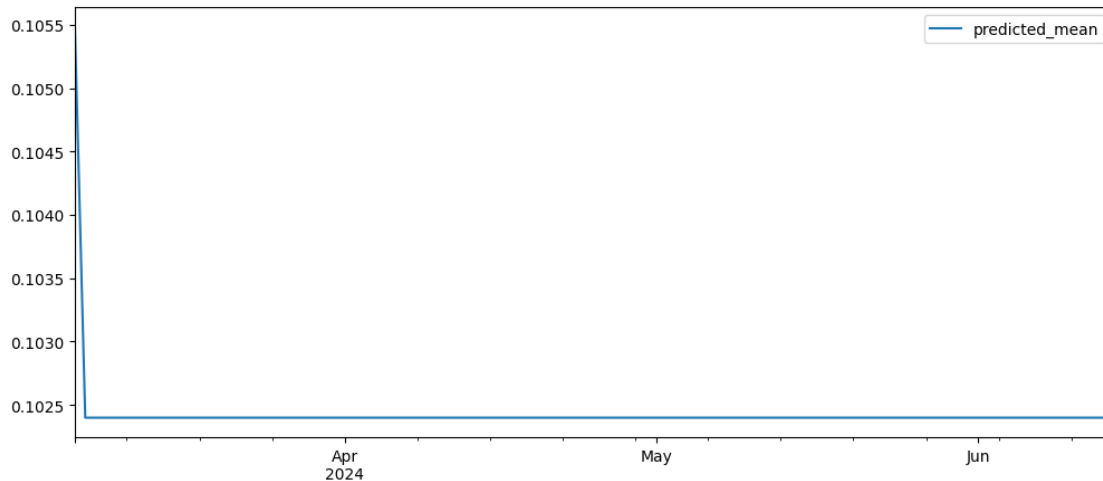
```

2024-03-06 00:00:00-05:00    0.105486
2024-03-07 00:00:00-05:00    0.102399
2024-03-08 00:00:00-05:00    0.102399
2024-03-09 00:00:00-05:00    0.102399
2024-03-10 00:00:00-05:00    0.102399
...
2024-06-10 00:00:00-05:00    0.102399
2024-06-11 00:00:00-05:00    0.102399
2024-06-12 00:00:00-05:00    0.102399
2024-06-13 00:00:00-05:00    0.102399
2024-06-14 00:00:00-05:00    0.102399
Freq: D, Name: predicted_mean, Length: 101, dtype: float64

```

```
[327]: prediction_whole.plot(figsize=(12,5),legend=True)
```

```
[327]: <Axes: >
```



```
[ ]:
```

14 4. Modeling - ELASTIC NET 5. Predicting with ELASTIC NET

15 LUKE DENONCOURT

```
[ ]:
```

```
[392]: def elastic_net_apple_modeling(data_open,
    ↪stock_list_greater_point_five_corr_apple_open,
    stock_list_greater_point_five_corr_apple_volume,
```

```

        stock_list_greater_point_five_corr_apple_low,
        stock_list_greater_point_five_corr_apple_high,
        stock_list_greater_point_five_corr_apple_close,
        aux_data,
        n_features_selected,
        differencing_flag, standardize_flag, aux_data_flag,
↪day_of_year_flag, day_of_week_flag,
        volume_all_stocks_flag, low_all_stocks_flag,
↪high_all_stocks_flag, close_all_stocks_flag,
        alpha, ll_ratio):

    ### Original data and the number of additional points that we want
    n_features_selected = n_features_selected
    data_only_apple_corr =
↪data_open[stock_list_greater_point_five_corr_apple_open[0:
↪n_features_selected]]
    try:
        data_only_apple_corr = data_only_apple_corr.drop(['AAPL_Open'])
    except:
        pass
    data_only_apple_corr['AAPL_Open'] = data_open['AAPL_Open'].copy()

    ### Data Adjustments
    differencing_flag = differencing_flag
    standardize_flag = standardize_flag
    day_of_year_flag = day_of_year_flag
    day_of_week_flag = day_of_week_flag

    volume_all_stocks_flag = volume_all_stocks_flag
    low_all_stocks_flag = low_all_stocks_flag
    high_all_stocks_flag = high_all_stocks_flag

    ### Differencing
    if differencing_flag == True:
        data_only_apple_corr = data_only_apple_corr.diff().dropna()

    ### Introducing new features
    if aux_data_flag == True:
        aux_data2 = aux_data.copy()
        ### Need to replace string dollar signs as ints
        sales_list = []
        [sales_list.append(int(float(a.split('$')[1].strip().split(',')[0] +
        a.split('$')[1].strip().split(',')[1])))
↪for a in aux_data2[' Sales ']]
        freecashflow_list = []
        [freecashflow_list.append(int(float(a.split('$')[1].strip().
↪split(',')[0] +

```

```

                                a.split('$')[1].strip().
↪split(',')[1])) for a in aux_data2[' FreeCashFlow ']]
    cash_list = []
    [cash_list.append(int(float(a.split('$')[1].strip().split(',')[0] +
                                a.split('$')[1].strip().split(',')[1])))
↪for a in aux_data2[' Cash ']]
    debt_list = []
    [debt_list.append(int(float(a.split('$')[1].strip().split(',')[0] +
                                a.split('$')[1].strip().split(',')[1])))
↪for a in aux_data2[' Long Term Debt ']]

    aux_data2[' Sales '] = sales_list
    aux_data2[' FreeCashFlow '] = freecashflow_list
    aux_data2[' Cash '] = cash_list
    aux_data2[' Long Term Debt '] = debt_list

    aux_data2 = aux_data2.rename({' Sales ':'Sales',
                                list(aux_data2.columns)[2]:
↪'FreeCashFlow', ' Cash ':'Cash',
                                list(aux_data2.columns)[4]:
↪'LongTermDebt'})
    date_list = []
    [date_list.append(pd.to_datetime(a).date()) for a in aux_data2['Date']]
↪### Need to change the dates to same object to match and
    ### change back the date object in the primary dataframe to continue
↪merging with the rest of the data
    aux_data2['Date'] = date_list
    aux_data2 = aux_data2.set_index('Date')
    aux_data_sub = aux_data2[aux_data2.index >= pd.
↪to_datetime(min(data_only_apple_corr.index)).date()]
    ### aux_data has older entries, so only want the entries on the dates
↪we have stock data for
    date_list = []
    [date_list.append(pd.to_datetime(a).date()) for a in
↪list(data_only_apple_corr.index)]
    old_index = list(data_only_apple_corr.index)
    data_only_apple_corr['Date'] = date_list
    data_only_apple_corr = data_only_apple_corr.set_index('Date')
    data_only_apple_corr = pd.merge(data_only_apple_corr, aux_data_sub,
↪on='Date', how='left').ffill().bfill()
    data_only_apple_corr = data_only_apple_corr.set_index(pd.
↪Series(old_index, name='Date'))

    if day_of_year_flag == True:
        day_of_year_list = []

```



```

    for val in data_only_apple_corr.index:
        day_of_year = pd.Period(val).day_of_year
        day_of_year_list.append(day_of_year)
    data_only_apple_corr['day_of_year'] = day_of_year_list

    if day_of_week_flag == True:
        day_of_week_list = []
        for val in data_only_apple_corr.index:
            day_of_week = pd.Period(val).day_of_week
            day_of_week_list.append(day_of_week)
        data_only_apple_corr['day_of_week'] = day_of_week_list

    if volume_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         ↵
↪data_volume[stock_list_greater_point_five_corr_apple_volume[0:
↪n_features_selected]],
                                         on='Date')

    if low_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         ↵
↪data_low[stock_list_greater_point_five_corr_apple_low[0:
↪n_features_selected]],
                                         on='Date')

    if high_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         ↵
↪data_high[stock_list_greater_point_five_corr_apple_high[0:
↪n_features_selected]],
                                         on='Date')

    if close_all_stocks_flag == True:
        data_only_apple_corr = pd.merge(data_only_apple_corr,
                                         ↵
↪data_close[stock_list_greater_point_five_corr_apple_close[0:
↪n_features_selected]],
                                         on='Date')

    column_indices = {name: i for i, name in enumerate(data_only_apple_corr.
↪columns)}
    n = len(data_only_apple_corr)
    train_df = data_only_apple_corr[0:int(n*0.9)]
    test_df = data_only_apple_corr[int(n*0.9):]
    num_features = data_only_apple_corr.shape[1]

    if standardize_flag == True:
        train_mean = train_df.mean()

```

```

train_std = train_df.std()
train_df = (train_df - train_mean) / train_std
test_df = (test_df - train_mean) / train_std

train_df_x = train_df.drop(['AAPL_Open'],axis=1)
train_df_y = train_df['AAPL_Open']
test_df_x = test_df.drop(['AAPL_Open'],axis=1)
test_df_y = test_df['AAPL_Open']

# return [train_df_x, train_df_y, test_df_x, test_df_y, train_df]
model = ElasticNet(alpha=alpha, l1_ratio = l1_ratio)
model.fit(X=train_df_x, y=train_df_y)
return model, [test_df_x, test_df_y]

```

```

[405]: elasticnet_grid_search_dict = {}

alpha_range = [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 0.0, 1.0, 10.0, 100.0]
l1_ratio_range = np.arange(0, 1, 0.01)

i=0
for alpha_range_val in alpha_range:
    for l1_ratio_range_val in l1_ratio_range:
        model, test = elastic_net_apple_modeling(data_open,
↪stock_list_greater_point_five_corr_apple_open,
            stock_list_greater_point_five_corr_apple_volume,
            stock_list_greater_point_five_corr_apple_low,
            stock_list_greater_point_five_corr_apple_high,
            stock_list_greater_point_five_corr_apple_close,
            aux_data,
            n_features_selected,
            differencing_flag=True, standardize_flag=True,
↪aux_data_flag=True, day_of_year_flag=True, day_of_week_flag=True,
            volume_all_stocks_flag=True, low_all_stocks_flag=True,
↪high_all_stocks_flag=True, close_all_stocks_flag=True,
            alpha=alpha_range_val,
↪l1_ratio=l1_ratio_range_val)

        test_x = test[0]
        test_y = test[1]
        pred = model.predict(test_x)

        pred_series = pd.Series(pred, index = test_y.index)
        mae = 'MAE: ' + str(np.round(np.mean(np.abs(pred_series - test_y)),3))
        mse = 'MSE: ' + str(np.round(np.mean((pred_series - test_y)**2),3))

```

```

        elasticnet_grid_search_dict[i] = {'alpha':alpha_range_val, 'l1_ratio':
↪l1_ratio_range_val, 'MAE':mae, 'MSE':mse}
        i+=1

```

```
[406]: elasticnet_grid_search_dict
```

```

[406]: {0: {'alpha': 1e-05,
          'l1_ratio': 0.0,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.694'},
       1: {'alpha': 1e-05,
          'l1_ratio': 0.01,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.693'},
       2: {'alpha': 1e-05,
          'l1_ratio': 0.02,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.693'},
       3: {'alpha': 1e-05,
          'l1_ratio': 0.03,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.692'},
       4: {'alpha': 1e-05,
          'l1_ratio': 0.04,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.693'},
       5: {'alpha': 1e-05,
          'l1_ratio': 0.05,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.693'},
       6: {'alpha': 1e-05,
          'l1_ratio': 0.06,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.693'},
       7: {'alpha': 1e-05,
          'l1_ratio': 0.07,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.693'},
       8: {'alpha': 1e-05,
          'l1_ratio': 0.08,
          'MAE': 'MAE: 2.113',
          'MSE': 'MSE: 6.693'},
       9: {'alpha': 1e-05,
          'l1_ratio': 0.09,
          'MAE': 'MAE: 2.113',

```

```
(571, 2.113),
(572, 2.113),
(573, 2.113),
(574, 2.113),
(575, 2.113),
(576, 2.113),
(577, 2.113),
(578, 2.113),
(579, 2.113),
(580, 2.113),
(581, 2.113),
(582, 2.113),
(583, 2.113),
(584, 2.113),
(585, 2.113),
(586, 2.113),
(587, 2.113),
(588, 2.113),
(589, 2.113),
(590, 2.113),
(591, 2.113),
(592, 2.113),
(593, 2.113),
(594, 2.113),
(595, 2.113),
(596, 2.113),
(597, 2.113),
(598, 2.113),
(599, 2.113),
(13, 2.114),
(100, 2.117)]
```

```
[426]: elasticnet_grid_search_dict[612]
```

```
[426]: {'alpha': 1.0, 'l1_ratio': 0.12, 'MAE': 'MAE: 0.615', 'MSE': 'MSE: 0.764'}
```

```
[ ]:
```

```
[427]: model, test = elastic_net_apple_modeling(data_open,
↪stock_list_greater_point_five_corr_apple_open,
        stock_list_greater_point_five_corr_apple_volume,
        stock_list_greater_point_five_corr_apple_low,
        stock_list_greater_point_five_corr_apple_high,
        stock_list_greater_point_five_corr_apple_close,
        aux_data,
        n_features_selected,
```

```

        differencing_flag=True, standardize_flag=True,
        ↪aux_data_flag=True, day_of_year_flag=True, day_of_week_flag=True,
        volume_all_stocks_flag=True, low_all_stocks_flag=True,
        ↪high_all_stocks_flag=True, close_all_stocks_flag=True,
        alpha=1, l1_ratio=0.12)

```

```

[428]: test_x = test[0]
       test_y = test[1]
       pred = model.predict(test_x)

```

```

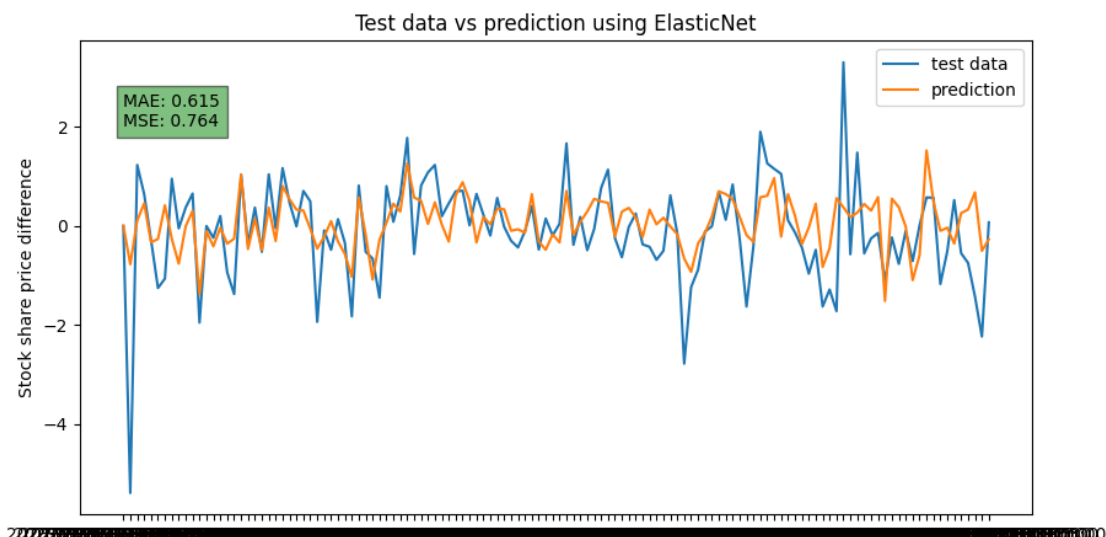
[429]: pred_series = pd.Series(pred, index = test_y.index)

fig = plt.figure(figsize=(10,5))
ax = fig.add_subplot(111)

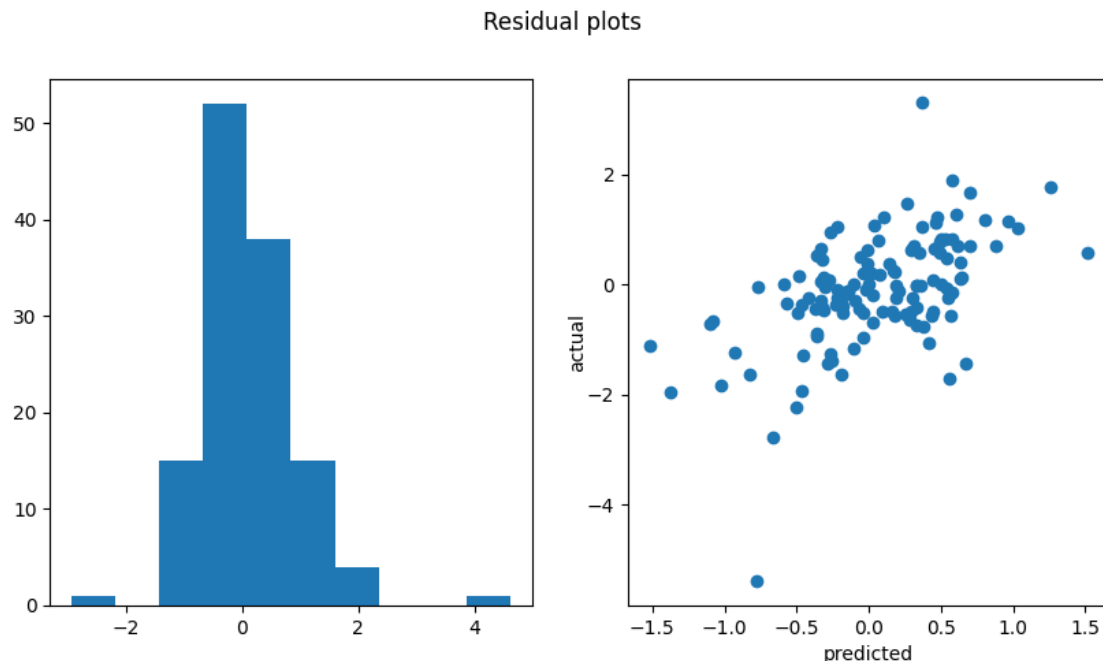
mae = 'MAE: ' + str(np.round(np.mean(np.abs(pred_series - test_y)),3))
mse = 'MSE: ' + str(np.round(np.mean((pred_series - test_y)**2),3))

ax.text(test_y.index[0], 2, mae + '\n' + mse, bbox={'facecolor':'green',
        ↪'alpha':0.5})
ax.plot(test_y, label='test data')
ax.plot(pred_series, label='prediction')
ax.legend()
ax.set_title('Test data vs prediction')
ax.set_ylabel('Stock share price difference')
plt.title('Test data vs prediction using ElasticNet')
plt.show()

```



```
[438]: resid = pred_series - test_y
fig, (ax1,ax2) = plt.subplots(1,2, figsize=(10,5))
ax1.hist(resid)
ax2.scatter(pred_series,test_y)
ax2.set_xlabel('predicted')
ax2.set_ylabel('actual', labelpad=0)
plt.suptitle('Residual plots')
plt.subplots_adjust(wspace=0.2)
plt.show()
```



The residual histogram looks fairly normal with a couple outliers on each side. The scatterplot appears mostly white noise except perhaps a slight upward trend.

```
[ ]:
```

16 Model comparisons

Baseline Naive MAE: 0.996

LSTM MAE: 0.757

ARIMA MAE: 1.716

ElasticNet MAE: 0.615

ElasticNet interestingly was the best performer in predicting the last 100 days of the apple open stock price difference. LSTM came in second with a bit more error than EN and showed a bias

in predicting lower values than the actual outcome. There are certainly more model architectures that could be tried to decrease the MAE of the LSTM due to the sheer amount of hyperparameters that deep learning has to offer. The next best performance was the baseline naive model, which just predicted the previous timestep as the current timestep. Last was the ARIMA model, which I believe was not implemented correctly. The ARIMA only predicted the mean value of the data for each time point and therefore had a large amount of error. The code should be revisited to bring the error below the naive baseline.

[]:

[]:

[]:

[]: